

Stanford CS336: Language Modeling from Scratch (Spring 2026)

中文教材

Based on Stanford CS336 Spring 2026 public videos and course materials

Spring 2026 course-material snapshot; revision slp3-style-textbook-v6-narrative-polish

目录

交付说明	2
课程目录与来源状态	2
本书结构	3
1 01 课程全景与 tokenization	3
2 02 PyTorch、einops 与资源核算	22
3 03 现代 Transformer architecture 与 hyperparameters	41
4 04 Attention alternatives 与 Mixture of Experts	60
5 05 GPU/TPU 硬件模型	80
6 06 Kernel、Triton 与 XLA	99
7 07 多 GPU 并行基础	119
8 08 现代训练并行栈	138
9 09 Scaling laws 基础	157
10 10 推理系统与 decoding	176
11 11 Scaling laws 进阶与实践	195
12 12 Evaluation 方法论	214
13 13 预训练数据：来源与数据集	233

14 14 数据过滤、去重、混合与合成数据	252
15 15 Mid/Post-training: SFT 与 RLHF	271
16 16 RLVR: 可验证奖励强化学习	290
17 17 Alignment 与 multimodality	309
18 18 Guest Lecture: Dan Fu 与推理引擎研究	328
A 资料缺口说明	347

交付说明

本书根据 Stanford CS336 Spring 2026 官方课程页、公开视频 playlist、公开视频字幕和课程资料写成。主体采用中文教材文风，保留必要英文术语、算法名、模型名、benchmark 名称和数学符号。课程页：<https://cs336.stanford.edu/>。公开视频 playlist：<https://www.youtube.com/playlist?list=PLoROMvodv4rMqX0cazWaTUHhq-yembLCV>。教材风格参考 SLP3 开源草稿的章节组织方式：https://web.stanford.edu/~jurafsky/slp3/ed3book_jan26.pdf；本书只借鉴章法，不复制其内容。每章主体先完成问题引入、术语定义、机制展开、公式、伪代码、例题、风险、小结和练习；视频与课程资料的回看路径集中放在章末，便于核对。

课程目录与来源状态

讲次	主题	日期	课程材料
01	课程全景与 tokenization	2026-03-30	公开视频 + 配套资料
02	PyTorch、einops 与资源核算	2026-04-01	公开视频 + 配套资料
03	现代 Transformer architecture 与 hyperparameters	2026-04-06	公开视频 + 配套资料
04	Attention alternatives 与 Mixture of Experts	2026-04-08	公开视频 + 配套资料
05	GPU/TPU 硬件模型	2026-04-13	公开视频 + 配套资料
06	Kernel、Triton 与 XLA	2026-04-15	公开视频 + 配套资料
07	多 GPU 并行基础	2026-04-20	公开视频 + 配套资料
08	现代训练并行栈	2026-04-22	公开视频 + 配套资料
09	Scaling laws 基础	2026-04-27	公开视频 + 配套资料
10	推理系统与 decoding	2026-04-29	公开视频 + 配套资料
11	Scaling laws 进阶与实践	2026-05-04	公开视频 + 配套资料
12	Evaluation 方法论	2026-05-06	公开视频 + 配套资料
13	预训练数据：来源与数据集	2026-05-11	公开视频 + 配套资料
14	数据过滤、去重、混合与合成数据	2026-05-13	公开视频 + 配套资料
15	Mid/Post-training: SFT 与 RLHF	2026-05-18	公开视频 + 配套资料

16	RLVR: 可验证奖励强化学习	2026-05-20	公开视频 + 配套资料
17	Alignment 与 multimodality	2026-05-27	公开视频 + 配套资料
18	Guest Lecture: Dan Fu 与推理引擎研究	2026-06-03	公开视频字幕

本书结构

- 每章先用具体问题或例子引出动机，不从术语列表开始。
- 重要术语首次出现保留中英双语，并解释其在 compute、memory、data、optimization、evaluation 或 deployment 账本中的位置。
- 公式必须有符号解释；算法必须有实现动作解释；方法必须有 caveat 和 failure mode。
- 章末给出本章小结、延伸解释、练习和必要的资料缺口说明。

1 01 课程全景与 tokenization

第 01 讲：课程全景与 tokenization

CS336 Spring 2026 public videos and official course materials

目录

1 本章导读	2
1.1 本章知识路线	2
1.2 结构图	2
2 本章主线	2
2.1 本章要解决的核心问题	3
2.2 从机制到资源账本	3
2.3 从课堂讲解到可复现实验	4
3 术语、符号与问题边界	4
3.1 术语之间的关系	4
4 为什么要从零构建语言模型	4
4.1 课程目标与问题边界	4
4.2 为什么要从零构建语言模型的机制	5
4.3 资源、效率与效果的关系	5
4.4 把课程目标写成检查流程	5
4.5 边界条件与常见误解	5
4.6 与本章其他概念的关系	6
4.7 怎样检验这一课程目标	6
4.8 概念辨析：为什么要从零构建语言模型	6
4.9 为什么要从零构建语言模型的小结	6
5 语言模型历史谱系	6
5.1 为什么要讨论语言模型历史谱系	6
5.2 语言模型历史谱系的机制	7
5.3 语言模型历史谱系的数学表达	7
5.4 从语言模型历史谱系到程序	7
5.5 边界条件与常见误解	7

5.6	与本章其他概念的关系	7
5.7	怎样检验语言模型历史谱系	8
5.8	概念辨析：语言模型历史谱系	8
5.9	语言模型历史谱系的小结	8
6	Tokenization 的问题定义	8
6.1	为什么要讨论 Tokenization 的问题定义	8
6.2	Tokenization 的问题定义的机制	9
6.3	Tokenization 的问题定义的形式化表示	9
6.4	从 Tokenization 的问题定义到程序	9
6.5	边界条件与常见误解	9
6.6	与本章其他概念的关系	10
6.7	怎样检验 Tokenization 的问题定义	10
6.8	概念辨析：Tokenization 的问题定义	10
6.9	Tokenization 的问题定义的小结	10
7	综合讨论：从局部机制到完整系统	10
7.1	为什么要从零构建语言模型在系统中的位置	11
7.2	语言模型历史谱系在系统中的位置	11
7.3	Tokenization 的问题定义在系统中的位置	11
7.4	把本章知识用于读论文和复现实验	11
8	例题：把概念变成可计算检查	12
8.1	BPE merge 如何改变序列长度	12
9	实验设计：从小例子到可复现结论	12
9.1	最小输入	12
9.2	资源账本	12
9.3	单变量消融	13
9.4	失败条件	13
9.5	记录与报告	13
9.6	从小实验到大结论	13
9.7	把本章重新读成一条证据链	13
10	课程资料与回看路径	14
10.1	视频回看路径	14
10.2	课程资料阅读路径	14
11	实践说明、历史位置与风险	15

12 复习要点

12.1 综合自测

13 总结与延伸

13.1 本章小结

13.2 延伸解释

13.3 练习

15

15

16

16

16

16

16

1 本章导读

本章讨论 **课程全景与 tokenization**。CS336 的第一层目标不是把现成大模型当作黑箱使用，而是把 language model 从输入文本、训练目标、模型结构、数据、系统和评估这些部件重新拆开。只有拆开之后，读者才能判断一个设计选择究竟改变了什么。设想你要复现实验室里一个 7B language model（语言模型）的最小训练栈：你不能只下载 tokenizer，也不能只调用框架默认配置，而要解释为什么同一段 Unicode 文本会变成某个 token 序列，为什么这个序列长度会改变训练成本，以及为什么课程要把 tokenizer、architecture、data 和 systems 放在同一个问题里。本章对应 CS336 Spring 2026 第 01 个公开视频，并结合课程页提供的可解析资料。视频和课程页给出事实边界；正文把这些材料改写为可自学的教材章节。阅读本章时，先理解问题为什么存在，再看定义、公式和实现形态，随后通过例题和练习检查自己是否能独立复现这条 reasoning chain。

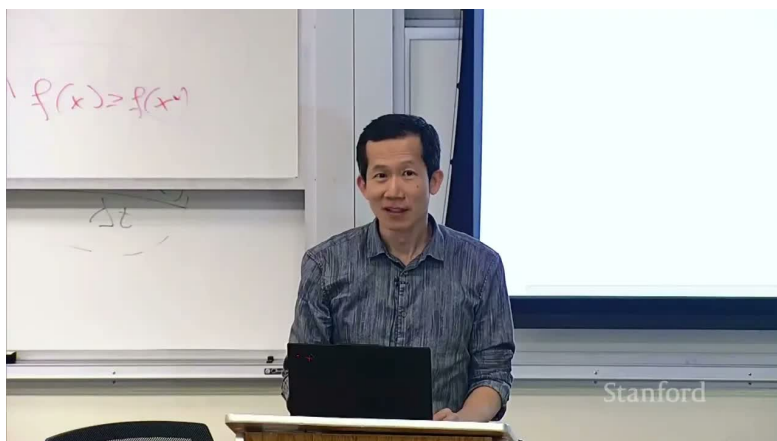


图 1: 第 01 讲的课程图像锚点。

1.1 本章知识路线

- 本章首先说明课程全景与 tokenization 为什么是 language modeling pipeline 中不可跳过的一环。
- 其次定义本章反复使用的术语，并说明这些术语对应的计算对象或系统对象。
- 随后用公式和伪代码表达主要机制，让概念能够落到可计算的变量上。
- 最后给出例题、风险讨论和练习，便于读者检查自己是否真正掌握了机制。

1.2 结构图

下面的图用于帮助读者定位本章的核心结构。先看概念之间的依赖，再回到正文中的公式、伪代码和例题。

2 本章主线

课程全景与 tokenization 的主线是从一个可观察的问题出发，逐步引出表示、目标、实现和实验。读者应把每个机制放回 language modeling pipeline 的位置。

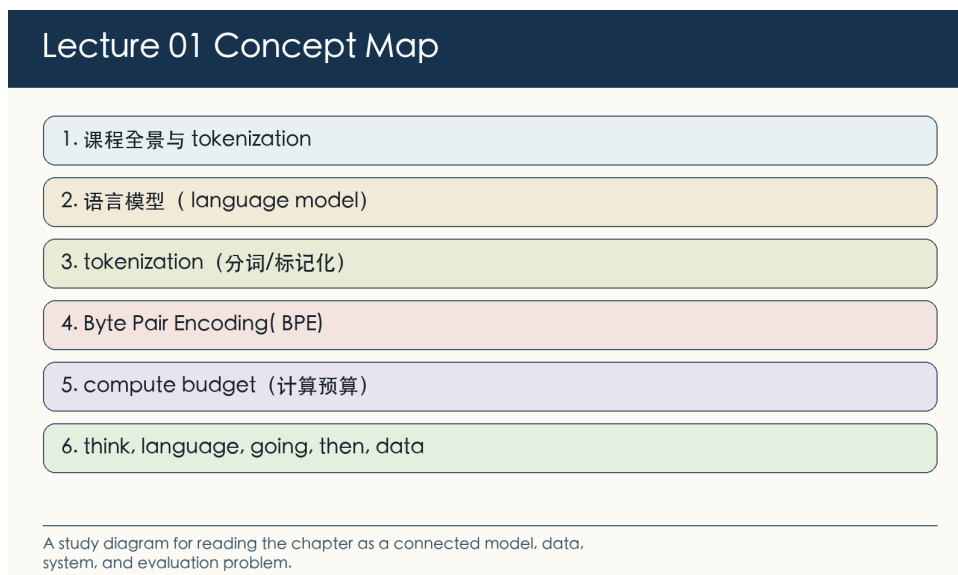


图 2: 第 01 讲概念图：本章问题、术语和机制的关系。

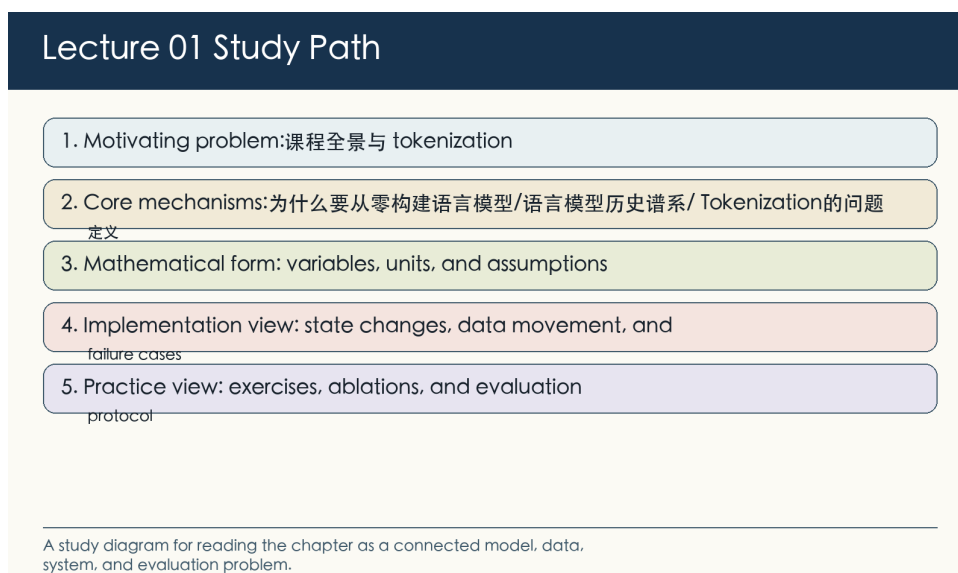


图 3: 第 01 讲学习路径：从问题定义进入公式、实现、实验和练习。

2.1 本章要解决的核心问题

本讲的主线是解释为什么 CS336 要坚持 from scratch。课程不是让学生背诵最新模型名字，而是把 tokenizer、architecture、optimizer、data、systems、evaluation 和 post-training 放在同一个可执行栈里理解。读完这一段后，应能用一两句话说明课程全景与 tokenization 要解决的瓶颈，并指出这个瓶颈发生在数据、模型、优化、系统还是评估层。

2.2 从机制到资源账本

tokenization 是第一个技术入口：任何语言模型都不能直接处理字符串，而要先把 Unicode 文本变成 token 序列。tokenizer 的选择会改变序列长度、训练 token 统计、attention 成本和下游语言覆盖。随后把机制写成账本：输入规模是什么，主要状态是什么，成本或质量指标怎样随变量改变。这个账本让后面的公式不再只是符号。

2.3 从课堂讲解到可复现实验

课程反复强调 mechanics、mindset、intuition 的区别：mechanics 可以通过实现获得，mindset 是计算和数据账本意识，intuition 则依赖实验并且不总能从小模型迁移到 frontier 模型。最后把课堂讲解转成可复现实验：固定协议，改变一个变量，记录中间量，并解释结果为何支持或反驳原来的机制判断。

3 术语、符号与问题边界

本章的术语横跨模型、数据和系统。读者需要先知道每个词指向的对象：有些词描述数学目标，有些词描述张量形状，有些词描述硬件瓶颈，还有一些词描述评估协议。术语边界不清，后面的公式和实现就会变成空洞的缩写。

#	术语	阅读要求
1	语言模型 (language model)	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
2	tokenization (分词/标记化)	说明它如何改变训练样本或 token 分布，并给出一个会改变结论的数据边界条件。
3	Byte Pair Encoding (BPE)	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
4	compute budget (计算预算)	把它放入资源账本：单位是什么，随输入规模怎样变化，最容易被哪类 benchmark 误读。
5	from-scratch philosophy (从零构建哲学)	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。

这些术语之间有依赖关系。例如 tokenization 改变 sequence length，sequence length 又改变 attention FLOPs、KV cache 和 evaluation token budget；parallelism 改变显存占用，也改变通信瓶颈和故障恢复方式。

3.1 术语之间的关系

在本章中，语言模型 (language model) 给出问题入口，tokenization (分词/标记化) 描述主要机制，Byte Pair Encoding (BPE) 则把机制连接到可测量的结果。读教材时应当沿着这个链条追问：输入是什么，中间状态怎样改变，输出指标为什么随之变化。课程材料还会反复出现 data、e.g.、scaling、compute、scale、training 等词。它们不是一组同义标签，而是提示本章的不同层次：有的指对象，有的指操作，有的指约束或指标。因此，术语表不是背诵清单，而是后文阅读的索引。遇到一个术语时，先定位它属于 representation、optimization、systems、data 还是 evaluation，再判断它改变哪一项账本。

4 为什么要从零构建语言模型

本节讨论为什么要从零构建语言模型。它先限定课程全景与 tokenization 要回答的基本问题：哪些现象需要解释，哪些变量可以测量，哪些结论必须回到输入规模、资源使用、指标变化和失败模式中检验。

4.1 课程目标与问题边界

课程把语言模型当作一整套技术栈，而不是只把 API prompt 当作抽象接口；视频开场强调研究者需要理解底层 mechanics、mindset 和 intuition。课程材料把课程目标表述为 understanding via building，并把数据、

architecture、training、systems、evaluation、post-training 串成一条端到端路径。学习这门课时应把每个组件放回 budget 约束：同样资源下，算法效率、数据质量和系统效率共同决定最终模型。为什么要从零构建语言模型为课程全景与 tokenization 提供一个局部观察点。接下来要把定义、公式、实现和实验条件连成同一条解释链。

4.2 为什么要从零构建语言模型的机制

课程把语言模型当作一整套技术栈，而不是只把 API prompt 当作抽象接口；视频开场强调研究者需要理解底层 mechanics、mindset 和 intuition。在为什么要从零构建语言模型中，核心对象是模型、数据或实验状态。本节把这些对象从口头描述转成可定义、可实现和可检查的机制。课程材料把课程目标表述为 understanding via building，并把数据、architecture、training、systems、evaluation、post-training 串成一条端到端路径。因而同一个方法在不同规模下可能改变不同的变量，尤其是输入规模、资源使用、指标变化和失败模式。比较方法时，必须同时给出问题规模和实验条件。学习这门课时应把每个组件放回 budget 约束：同样资源下，算法效率、数据质量和系统效率共同决定最终模型。可检验性来自清楚的变量、可复现的程序和能被反例挑战的结论。因此，为什么要从零构建语言模型的学习目标是把课程全景与 tokenization 中的局部机制讲到可以实现、测试和反驳的程度。

4.3 资源、效率与效果的关系

$$\text{accuracy} \approx \text{algorithmic efficiency} \times \text{resources}$$

符号说明：resources 表示可用算力、数据和工程时间；algorithmic efficiency 表示在相同资源下把 loss 或 benchmark score 推向更优的能力。这个关系式给出为什么要从零构建语言模型的最小数学结构。读者应检查变量单位、取值范围和可观测性；如果某个量只能间接估计，就必须说明 proxy。任何 proxy 都只测量目标的一部分。使用它时要说明观测对象、协议边界和可能失真的方向。

4.4 把课程目标写成检查流程

把为什么要从零构建语言模型写成程序后，抽象机制会变成输入、状态和输出之间的变换。下面的伪代码保留核心计算顺序。

```
for layer in lm_stack:
    identify_mechanics(layer)
    account_for_compute_and_memory(layer)
    test_small_before_scaling(layer)
```

阅读这段程序时，先确定输入、状态和输出，再记录输入规模、资源使用、指标变化和失败模式。可复现实验依赖这些状态被明确保存，而不是依赖运行后的主观描述。

4.5 边界条件与常见误解

- 课程会讨论前沿模型，但公开材料并不披露闭源 frontier model 的完整 recipe。
- 小模型上得到的 intuition 只有一部分可迁移到 frontier scale。
- 小实验中的局部结论不能自动外推到 frontier-scale；scale、数据分布和硬件拓扑都会改变结论。
- benchmark 或 profile 的单个数字不能替代机制解释；必须记录实验设置、输入形状、版本和评估口径。

4.6 与本章其他概念的关系

为什么要从零构建语言模型在本章中承担桥梁作用：它把课程全景与 tokenization 的术语、公式、程序和例题连成一条可复核的推理链。

4.7 怎样检验这一课程目标

检验这一课程目标时，先把抽象说法落到模型、数据或实验状态上。与 mechanics、mindset、efficiency、frontier、building 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让输入规模、资源使用、指标变化和失败模式中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明：resources 表示可用算力、数据和工程时间；algorithmic efficiency 表示在相同资源下把 loss 或 benchmark score 推向更优的能力。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。对照实验一次只改变一个主要因素。多个因素同时变化时，实验最多说明系统表现不同，不能说明是哪一个机制在起作用。最后检查失败条件。工程判断往往在反例中变清楚：课程会讨论前沿模型，但公开材料并不披露闭源 frontier model 的完整 recipe；小模型上得到的 intuition 只有一部分可迁移到 frontier scale。复习时可以把这些限制改写成反例测试，观察它们怎样改变输入规模、资源使用、指标变化和失败模式。把实验结论放回课程全景与 tokenization 时，要说明为什么要从零构建语言模型改变了哪一个可观察变量，以及这个变量为什么支持原来的机制判断。

4.8 概念辨析：为什么要从零构建语言模型

mechanics、mindset、efficiency、frontier、building 在课程材料中相邻出现时，不应被自动当成同义词。读者应先区分对象、操作、约束和指标。区分这些词时，可以先问它们是否改变输入规模、资源使用、指标变化和失败模式。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕为什么要从零构建语言模型复习时，应写出一个最小例子：输入是什么，哪一步使用这个概念，输出或指标怎样变化，失败条件在哪里出现。

4.9 为什么要从零构建语言模型的小结

- 讨论为什么要从零构建语言模型时，应同时报告问题规模、实验设置和主要观测变量，尤其是输入规模、资源使用、指标变化和失败模式。
- 如果一个方法声称降低主要成本，要追问成本是否转移到了显存、通信、数据质量、训练稳定性或评估复杂度。
- 结论应同时能在公式、伪代码和至少一个可复现实验中找到对应证据。

5 语言模型历史谱系

现在进入语言模型历史谱系。这一节把模型大小、训练 token、compute budget 和 loss 放在同一坐标系里，讨论何时可以从小实验推断大训练。

5.1 为什么要讨论语言模型历史谱系

课程材料从 Shannon、n-gram、LSTM、seq2seq、attention、Transformer、GPT、BERT、T5、GPT-3、Chinchilla、Llama、DeepSeek、Qwen、OLMo 等线索组织历史。这条谱系的核心不是模型名堆叠，而是越来

越强的统一接口：从估计文本概率，到 fine-tuning，到 prompting，再到 agents。开放模型与开放数据的价值在于让课程可被复现；否则只能学习接口，不能学习机制。这条历史线索的重点不是模型名称本身，而是接口的变化：从估计文本序列的概率，到给定条件生成输出，再到把模型嵌入 fine-tuning、prompting 和 agent workflow。理解这条线索，才能看出后面各章为什么反复回到数据、计算和评估。

5.2 语言模型历史谱系的机制

Shannon 和 n-gram 代表了最早的概率视角：语言可以被看成一个序列预测问题。这个视角把“理解语言”降解为估计条件概率，虽然表达能力有限，但它给出了可训练、可评估的目标。LSTM、seq2seq、attention 和 Transformer 改变的是上下文接口。模型不再只依赖固定窗口或压缩状态，而是逐渐获得更灵活的条件化能力；这为后来的 pretraining、fine-tuning 和 prompting 打开了统一入口。GPT、BERT、T5、GPT-3、Chinchilla、Llama、DeepSeek、Qwen 和 OLMo 体现了另一条主线：规模、数据和开放性成为模型能力的一部分。课程强调这些名字，不是为了背年表，而是为了说明现代 language model 同时是算法、数据和系统的产物。

5.3 语言模型历史谱系的数学表达

$$p_{\theta}(x_1, \dots, x_T) = \prod_{t=1}^T p_{\theta}(x_t | x_{<t})$$

符号说明：序列 $x_{1:T}$ 被分解为 next-token 条件概率； θ 是模型参数；训练目标通常是最大化这些条件概率。这个关系式给出语言模型历史谱系的缩放假设。它把参数量、训练 token 数和 compute budget 放到同一张账本中，重点是判断小实验能否可靠外推到大训练。在 scaling 实验中，validation loss、benchmark transfer 和训练吞吐都只是不同侧面的 proxy。外推时要说明哪个 proxy 被拟合，哪个 proxy 只是辅助诊断。

5.4 从语言模型历史谱系到程序

把语言模型历史谱系写成程序后，抽象机制会变成输入、状态和输出之间的变换。下面的伪代码保留核心计算顺序。

```
tokens = tokenizer.encode(document)
for t in range(1, len(tokens)):
    loss += -log_prob(model(tokens[:t]), tokens[t])
```

阅读这段流程时，重点看小规模实验怎样产生可外推曲线。参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系需要在每个实验点上同时记录，缺少其中一项都会改变 scaling 解释。

5.5 边界条件与常见误解

- 历史时间线不能替代因果解释；同一个 benchmark 提升可能来自数据、算力、architecture 或训练 recipe。
- 模型开放程度不同：open weights、open code、open data 的可复现性差别很大。
- 小规模 scaling 曲线不一定覆盖大训练的优化不稳定、数据瓶颈和系统瓶颈。
- compute-optimal 结论依赖目标指标；以 loss 最优并不自动意味着 benchmark、latency 或成本最优。

5.6 与本章其他概念的关系

语言模型历史谱系连接本章的小实验和大规模决策：术语表标出预算变量，公式给出外推结构，伪代码组织实验点，例题检查 compute-optimal 判断是否稳健。

5.7 怎样检验语言模型历史谱系

检验语言模型历史谱系时，先把抽象说法落到模型规模、token 数和训练预算上。与 Shannon、n-gram、Transformer、GPT、Llama 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明：序列 $(x_{1:T})$ 被分解为 next-token 条件概率； (θ) 是模型参数；训练目标通常是最大化这些条件概率。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。scaling 消融通常一次只改变参数量、token 数、数据 pipeline 或优化 schedule。若多个轴同时移动，曲线拟合会混合不同机制。最后检查失败条件。工程判断往往在反例中变清楚：历史时间线不能替代因果解释；同一个 benchmark 提升可能来自数据、算力、architecture 或训练 recipe；模型开放程度不同：open weights、open code、open data 的可复现性差别很大。复习时可以把这些限制改写成反例测试，观察它们怎样改变参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。把实验结论放回课程全景与 tokenization 时，要说明语言模型历史谱系改变的是预算轴、拟合曲线、外推误差还是目标指标。能拆开这些因素，才说明读者真正理解 scaling。

5.8 概念辨析：语言模型历史谱系

Shannon、n-gram、Transformer、GPT、Llama、open 常一起出现在 scaling 讨论中，但它们分别指预算轴、拟合对象、外推假设或风险来源。读者应先确定曲线中的自变量和因变量。其中，Transformer 是现代 LM 的基本计算骨架。区分这些词时，可以先问它们是否改变参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕语言模型历史谱系复习时，可以画三到五个小实验点：记录参数量、token 数、预算和 loss，再检查外推是否合理。

5.9 语言模型历史谱系的小结

- 讨论语言模型历史谱系时，应同时报告问题规模、实验设置和主要观测变量，尤其是参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。
- 如果方法声称可以外推，要说明实验点覆盖范围、拟合形式和数据管线是否一致；曲线本身不是自然定律。
- scaling 结论需要把 loss、benchmark 和成本之间的转换关系说清楚，否则 compute-optimal 只是一条局部经验曲线。

6 Tokenization 的问题定义

现在进入 Tokenization 的问题定义。这一节把模型结构还原成表示、参数、activation 和训练动态之间的关系，而不是停留在结构名称。

6.1 为什么要讨论 Tokenization 的问题定义

tokenization 把原始字符串映射成离散 ID，是语言模型与文本世界之间的压缩接口。视频和课程材料强调 Unicode、byte-level 表示、vocabulary size、rare words、multilingual text 与 byte fallback 的工程取舍。好的 tokenizer 不是让文本最短，而是在压缩率、可逆性、训练稳定性、跨语言公平性和实现复杂度之间取平

衡。这里的核心问题是表示单位。文本被切成什么 token，会直接改变序列长度、词表大小、训练 token 统计、attention 成本和多语言覆盖。因此 tokenizer 不是附属预处理，而是语言模型的第一层建模选择。

6.2 Tokenization 的问题定义的机制

tokenization 把原始字符串映射成离散 ID，是语言模型与文本世界之间的压缩接口。在 Tokenization 的问题定义中，核心对象是张量、token 和计算图状态：模型机制必须同时说明表示形态、张量形状和计算代价。视频和课程材料强调 Unicode、byte-level 表示、vocabulary size、rare words、multilingual text 与 byte fallback 的工程取舍。因而同一个方法在不同规模下可能改变不同的变量，尤其是 sequence length、hidden size、head 数、activation size 和训练稳定性。比较方法时，必须同时给出问题规模和实验条件。好的 tokenizer 不是让文本最短，而是在压缩率、可逆性、训练稳定性、跨语言公平性和实现复杂度之间取平衡。可检验性来自形状推导、训练稳定性曲线和结构消融；只画 block diagram 不能说明机制已经成立。因此，Tokenization 的问题定义的学习目标是建立一条计算图解释：读者应能从输入 token 追到 activation、loss 和更新，而不是只记住结构名称。

6.3 Tokenization 的问题定义的形式化表示

$$\text{BPE}(s) = \arg \min_{z_1, \dots, z_m} m \quad \text{s.t.} \quad \text{decode}(z_{1:m}) = s$$

符号说明： s 是原字符串； z_i 是 token；公式表达的是压缩直觉而不是 BPE 的完整优化算法。这个关系式把 Tokenization 的问题定义放回模型计算图。它帮助读者追踪 token、张量形状和参数规模如何进入训练或推理成本。在模型结构实验中，loss、梯度范数、activation memory 和 ablation delta 都是 proxy。它们帮助定位机制，但不能单独证明某个结构在所有规模上更优。

6.4 从 Tokenization 的问题定义到程序

把 Tokenization 的问题定义写成程序后，抽象机制会变成输入、状态和输出之间的变换。下面的伪代码保留核心计算顺序。

```
vocab = bytes()
while len(vocab) < target_size:
    pair = most_frequent_adjacent_pair(corpus)
    merge(pair)
return vocab, merge_rules
```

阅读这段实现时，重点追踪 token、activation 和参数如何穿过计算图。sequence length、hidden size、head 数、activation size 和训练稳定性不是附加统计，而是解释训练稳定性和推理成本的必要坐标。

6.5 边界条件与常见误解

- BPE merge 是贪心过程，不保证全局最优压缩。
- tokenizer 改变 sequence length，从而改变 attention cost、KV cache size 和 training tokens 的统计口径。
- 结构消融常与训练 recipe 耦合；一个 block 在小模型上有效，不代表在长上下文或不同数据分布下仍有效。
- 张量形状和数值稳定性是模型机制的一部分，忽略它们会把实现问题误读成算法问题。

6.6 与本章其他概念的关系

Tokenization 的问题定义连接本章的表示选择和训练行为：定义给出张量对象，公式描述计算关系，伪代码说明实现路径，例题检验形状、稳定性和成本之间的权衡。

6.7 怎样检验 Tokenization 的问题定义

检验 Tokenization 的问题定义时，先把抽象说法落到张量、token 和计算图状态上。与 tokenization、Unicode、bytes、BPE、vocabulary 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让 sequence length、hidden size、head 数、activation size 和训练稳定性中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： $\backslash(s\backslash)$ 是原字符串； $\backslash(z_i\backslash)$ 是 token；公式表达的是压缩直觉而不是 BPE 的完整优化算法。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。模型消融通常一次只改变 normalization、activation、position encoding、head layout 或学习率 schedule。若结构和训练 recipe 同时改变，结果只能作为候选经验。最后检查失败条件。工程判断往往在反例中变清楚：BPE merge 是贪心过程，不保证全局最优压缩；tokenizer 改变 sequence length，从而改变 attention cost、KV cache size 和 training tokens 的统计口径。复习时可以把这些限制改写成反例测试，观察它们怎样改变 sequence length、hidden size、head 数、activation size 和训练稳定性。把实验结论放回课程全景与 tokenization 时，要说明 Tokenization 的问题定义改变的是表示、计算图、梯度路径还是 activation memory。能追踪这些对象，才说明读者真正理解模型机制。

6.8 概念辨析：Tokenization 的问题定义

tokenization、Unicode、bytes、BPE、vocabulary 常一起出现在模型结构讨论中，但它们分别指表示、算子、参数或状态。读者应先把它们放回计算图。其中，tokenization（分词/标记化）决定文本如何变成模型可处理的离散序列；Byte Pair Encoding (BPE) 是用贪心 merge 规则压缩文本的常见 tokenizer 方法。区分这些词时，可以先问它们是否改变 sequence length、hidden size、head 数、activation size 和训练稳定性。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕 Tokenization 的问题定义复习时，可以写一个最小 forward pass：标出每个张量形状、状态更新和可能的数值问题。

6.9 Tokenization 的问题定义的小结

- 讨论 Tokenization 的问题定义时，应同时报告问题规模、实验设置和主要观测变量，尤其是 sequence length、hidden size、head 数、activation size 和训练稳定性。
- 如果方法声称结构更优，要说明它改变的是表达能力、梯度路径、activation memory、KV cache 还是训练稳定性。
- 模型机制需要通过形状推导、训练曲线和消融实验共同验证，不能只依赖 block diagram。

7 综合讨论：从局部机制到完整系统

本章的主题是课程全景与 tokenization，但它不应被读成几个相邻知识点的拼接。为什么要从零构建语言模型、语言模型历史谱系、Tokenization 的问题定义共同回答同一个问题：语言模型系统中哪些对象可以被定义，哪些成本可以被估算，哪些结论必须通过实验或评估来确认。这些对象包括语言模型 (language model)、tokenization (分词/标记化)、Byte Pair Encoding (BPE)、compute budget (计算预算)、from-scratch philosophy

(从零构建哲学)。读者可以把它们看成一组接口：有的接口连接文本和 token，有的连接模型结构和张量计算，有的连接数据样本和训练目标，有的连接离线评估和在线服务。接口一旦改变，后面的成本、错误类型和优化空间也会随之改变。

7.1 为什么要从零构建语言模型在系统中的位置

为什么要从零构建语言模型首先提供一个局部解释：课程把语言模型当作一整套技术栈，而不是只把 API prompt 当作抽象接口；视频开场强调研究者需要理解底层 mechanics、mindset 和 intuition。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，为什么要从零构建语言模型会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，为什么要从零构建语言模型不能脱离边界条件理解。一个关键 caveat 是：课程会讨论前沿模型，但公开材料并不披露闭源 frontier model 的完整 recipe。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.2 语言模型历史谱系在系统中的位置

语言模型历史谱系首先提供一个局部解释：课程材料从 Shannon、n-gram、LSTM、seq2seq、attention、Transformer、GPT、BERT、T5、GPT-3、Chinchilla、Llama、DeepSeek、Qwen、OLMo 等线索组织历史。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，语言模型历史谱系会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，语言模型历史谱系不能脱离边界条件理解。一个关键 caveat 是：历史时间线不能替代因果解释；同一个 benchmark 提升可能来自数据、算力、architecture 或训练 recipe。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.3 Tokenization 的问题定义在系统中的位置

Tokenization 的问题定义首先提供一个局部解释：tokenization 把原始字符串映射成离散 ID，是语言模型与文本世界之间的压缩接口。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，Tokenization 的问题定义会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，Tokenization 的问题定义不能脱离边界条件理解。一个关键 caveat 是：BPE merge 是贪心过程，不保证全局最优压缩。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.4 把本章知识用于读论文和复现实验

读相关论文时，可以把本章变成一个检查表。第一，论文是否清楚定义了输入规模、模型规模、数据来源和评估协议；第二，论文声称的改进落在哪个变量上；第三，作者是否报告了会让结论失效的设置；第四，实验是否能分离模型结构、数据质量、优化设置和系统实现的影响。复现实验时，也应避免直接追求大规模。先用最小程序复现一个公式、一个伪代码分支或一个 profile 现象，再扩大输入规模。这样做的原因很简单：如果小规模程序无法解释中间量，大规模训练只会把错误隐藏在日志和随机波动里。把这些检查合在一起，本章

给出的不是一个固定答案，而是一种阅读 CS336 的方法：每个模型机制都要落到变量，每个变量都要落到测量，每个测量都要说明协议，每个协议都要承认边界。因此，本章中的任何一句结论都可以被改写成一个可引用的完整句式：在给定数据、模型、硬件和评估协议下，某个机制改变了某个变量，并通过某个观测指标表现出来。这样的句子比单独背诵术语更长，但它包含了自学、复现和引用时真正需要的信息。把本章用于自学时，可以按三遍阅读。第一遍只追踪对象和定义，确认每个术语到底指向文本、token、张量、样本、请求还是评估记录。第二遍追踪公式和伪代码，确认每一步改变了哪些状态。第三遍追踪实验和 caveat，确认哪些结论只在特定规模、数据或硬件条件下成立。把本章用于复习时，则应反过来操作：先从一个失败案例或异常指标出发，再回到相应机制。loss 曲线异常可能指向数据、优化或模型容量；吞吐异常可能指向 kernel、通信或调度；benchmark 异常可能指向 contamination、prompt protocol 或采样设置。能够从异常反推机制，是掌握本章的实用标准。

8 例题：把概念变成可计算检查

8.1 BPE merge 如何改变序列长度

给定训练字符串 ‘low lower newest wider’，先把单词拆成字符，再反复合并最频繁的相邻 pair。

- 第一次合并可能把常见 pair ‘l o’ 合成 ‘lo’，使所有含 ‘lo’ 的词减少一个 token。
- 后续合并可能得到 ‘low’、‘er’、‘new’ 等子词；未见过的 ‘lower’ 仍可由 ‘low’ 和 ‘er’ 表示。
- 这种压缩会减少 attention 序列长度，但 vocabulary 变大也会增加 embedding/LM head 的参数和内存。

结论。 tokenizer 是压缩算法、语言覆盖策略和系统成本之间的接口，不只是预处理函数。这个例题的目的不是替代完整工程实现，而是给出一种最小推理形式：把问题转成变量，写出近似公式或伪代码，再说明近似何时失效。

9 实验设计：从小例子到可复现结论

学习课程全景与 tokenization 不能停在概念层。一个教材化结论至少需要四件事：可构造的输入、可观测的中间量、可比较的指标，以及清楚的失败条件。下面的实验设计不是要求训练大模型，而是要求读者用小规模程序复现本章机制。

9.1 最小输入

先构造只触发本章核心机制的小输入。例如 tokenizer 章节可以用几个包含 rare words 和 Unicode 字符的字符串；GPU 章节可以用一个 elementwise kernel 和一个 GEMM；data 章节可以用十几篇近重复文档。小输入的价值在于它让错误可见。

9.2 资源账本

对同一输入写下 FLOPs、bytes moved、显存峰值、通信量或样本数量的估算。估算不需要一开始就精确，但必须说明单位和近似假设。随后用 profiler、benchmark、validation loss 或人工检查去验证估算是否同量级。

9.3 单变量消融

一次只改变一个因素，例如 vocabulary size、head 数、batch size、filter threshold、reward scale、decoding temperature 或 GPU 数量。若多个变量同时变化，实验只能说明系统变了，不能说明机制是什么。

9.4 失败条件

最后要刻意触发失败：极端 context length、错误 dtype、过小 batch、污染 benchmark、低质量数据或不可靠 verifier。失败条件常常比成功曲线更能说明一个方法的边界。

9.5 记录与报告

实验记录应能让另一个读者复现同一结论。最少要写清楚输入构造、代码版本、随机种子、硬件、batch shape、数据版本、评价指标和异常处理方式。报告结论时不要只写“更快”或“更好”，而要说明快在哪里、好在哪个指标上，以及是否牺牲了内存、稳定性、数据质量或可解释性。常见报告错误是把局部现象写成普遍规律。例如一次小模型实验里的 loss 改善，不等于更大模型或不同数据分布上也会改善；一次 kernel benchmark 的吞吐提升，不等于端到端 serving latency 一定降低。教材中的实验报告应始终把结论限定在可复现条件内。

9.6 从小实验到大结论

小实验的作用不是替代课程中的完整训练或系统实现，而是建立一条可检查的推理链。先在小输入上确认变量方向，再在中等规模上确认数量级，最后才讨论是否能外推到更大模型、更多 token 或更复杂 workload。缺少中间层级时，大结论通常只是在复述直觉。因此，实验报告最好同时包含正例和反例：正例说明机制在受控条件下怎样工作，反例说明条件稍变时哪里失效。对 CS336 这样的课程，反例尤其重要，因为 language model 的错误常来自多个层面叠加：数据分布、优化稳定性、硬件瓶颈、评估协议和服务负载都可能同时改变观察结果。还要记录资料版本与复现边界。公开视频、课程页、配套代码和 PDF 可能在不同时间更新；复现实验应写明使用的材料版本，并说明哪些结论来自课程事实，哪些是为了帮助理解而加入的解释性推导。这样才能把学习笔记变成可检查的技术记录。若复现实验与课程结论不一致，首先检查输入、版本和评估协议，而不是立即修改模型假设。

9.7 把本章重新读成一条证据链

完成小实验之后，可以把课程全景与 tokenization 重新读成一条证据链：先用“为什么要从零构建语言模型”确定问题入口，再用“语言模型历史谱系”解释主要机制，最后用“Tokenization 的问题定义”检查边界或外推条件。这种读法比按页背诵更接近教材训练，因为它要求每个结论都能说明前提、变量和证据。本章反复使用的术语包括语言模型 (language model)、tokenization (分词/标记化)、Byte Pair Encoding (BPE)、compute budget (计算预算)、from-scratch philosophy (从零构建哲学)。复习时不要只写定义，而要为每个术语补上一句操作性说明：它在课程代码、公式、数据记录或评估协议中对应什么对象；如果这个对象被删除、替换或缩放，哪一个观测量会变化。最后，用一个反例结束复习。反例可以是资源估算不准、数据污染、reward 失真、benchmark 解析失败、长上下文显存爆炸或 scaling 外推失效。能解释反例的章节，才真正具备自学和引用价值；不能解释反例的章节，只是把课程材料换了一种排版。引用本章结论时，还应保留来源边界：哪些说法直接来自视频、课程页、slides 或代码，哪些是为了串联教材叙述而加入的解释性推导。这个边界不会削弱结论，反而让读者知道何处可以复核，何处需要在自己的实验中重新验证。如果后续课程材料更新，也可以沿着同一证据链替换来源，而不必重写整章结构。

10 课程资料与回看路径

教材正文已经把主要概念、公式和实现逻辑组织成连续叙述；本节只提供回到课程材料的阅读路径。读者复习时可以先完成前文练习，再按下面的时间段或资料组核对细节。

10.1 视频回看路径

时间	关键词	复习任务
00:00:04-00:05:46	language, think, years, time, going, year, we're	回看定义、例子、推导或 caveat 在该段中的位置。
00:18:20-00:24:25	think, your, don't, probably, class, talk, good	回看定义、例子、推导或 caveat 在该段中的位置。
00:37:55-00:44:45	then, there's, data, inference, compute, write, gpus	回看定义、例子、推导或 caveat 在该段中的位置。
00:51:37-00:58:24	data, it's, going, your, there's, then, think	回看定义、例子、推导或 caveat 在该段中的位置。
01:11:55-01:18:56	going, we're, then, time, merges, think, pair	回看定义、例子、推导或 caveat 在该段中的位置。

这些时间段用于定位视频中的讲解顺序。严肃引用时，应回到视频片段确认讲者表述、板书或幻灯片内容。回看视频时，不必逐字抄录字幕。更有效的方法是把每个时间段判定为定义、例子、推导、代码解释或 caveat，再把它放回本章对应小节。

10.2 课程资料阅读路径

资料组	标题/页块	关键词
official_01	Next time: resource accounting	researchers, frontier, language, intuitions, efficiency, offering, youtube
official_03	Remember it's all about efficiency:	compute, memory, efficiency, data, hardware, training, train
official_05	Now you can:	data, scaling, loss, flops, quality, scale, laws
official_08	Example: Noam Shazeer paper that introduced SwiGLU	machine, translation, language, transformer, scaling, swiglu, first
official_10	Function: data	official, reading, markers, function, data, alignment, tokenization

课程资料通常给出更稳定的标题、公式、图或代码结构；视频则补充动机和口头解释。两者合起来构成本章的事实边界。阅读资料时，可以把每个资料组拆成三个对象：一个定义，一个公式或伪代码动作，一个边界条件。这样比逐字翻译页面或脚本更容易形成可用知识。

11 实践说明、历史位置与风险

课程全景与 tokenization 在 CS336 的课程结构中连接基础机制和规模化问题。基础机制解释方法为什么成立；规模化问题检验它在更大模型、更长上下文、更复杂数据或真实 serving workload 下是否仍成立。历史位置不等于模型年表。更重要的是识别问题如何迁移：早期方法解决小规模建模问题，现代方法常常解决同一问题在大规模数据、GPU 集群、长上下文或人类偏好约束下的新形态。

- 资料版本限制 (version risk): 课程页、公开视频和配套资料可能在学期中更新；引用时应注明访问日期或资料版本。
- 测量口径限制 (measurement risk): FLOPs、tokens/s、benchmark score、reward 等数字只有在协议完整时可比较。
- 规模外推限制 (scaling risk): 小模型或小 batch 的机制不必然外推到 frontier-scale。
- 数据分布限制 (data risk): 数据来源、过滤和去重策略会改变模型行为，也会改变 evaluation contamination 风险。

12 复习要点

下面的问题把本章内容压缩为可反复检查的条目。每个条目都应能回到前文的解释、公式、伪代码或例题。

条目	对象	检查动作
条目 1	语言模型 (language model)	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。
条目 2	tokenization (分词/标记化)	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。
条目 3	Byte Pair Encoding (BPE)	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。
条目 4	compute budget (计算预算)	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。
条目 5	from-scratch philosophy (从零构建哲学)	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。

12.1 综合自测

- 我能否不用课程原句，独立解释本章中心问题？
- 我能否把本章至少一个公式改写成可运行的估算函数？
- 我能否指出一个看似改进但实际转移成本的方法？
- 我能否回到视频和课程资料，找到支持本章关键论断的位置？
- 我能否设计一个最小 ablation，验证本章一个 caveat？

13 总结与延伸

13.1 本章小结

- 课程全景与 tokenization 的核心不是记忆术语，而是理解对象、变量和机制之间的关系。
- 视频给出讲解顺序，课程资料提供可核对的公式、图、代码或页面结构。
- 复习时应把每个术语连接到至少一个变量、一个实现动作和一个失败模式。

13.2 延伸解释

本章中的延伸解释用于连接课程材料中的概念，例如说明公式里的 proxy、解释系统 tradeoff，或把多个材料片段串成一个完整推理链。若需要严肃引用，应回到课程视频和配套资料核对原始表述。

13.3 练习

- 定义题：用中英双语解释本章任意五个重要术语，并说明它们属于 compute、memory、data、optimization、evaluation 还是 deployment 账本。
- 推导题：选择本章一个公式，逐个解释符号、单位和近似假设，并说明哪个变量最难在真实系统中测量。
- 实现题：把本章伪代码改写成一个最小 Python sanity check，不要求训练大模型，但要输出可检查的中间量。
- 资料题：从“课程资料与回看路径”中选择一个视频时间段，回到原始视频核对讲者例子，并记录是否存在字幕误差。
- 评估题：为本章方法设计一个 ablation 或 benchmark，说明控制变量、数据版本、指标和 failure criteria。
- 边界题：说明本章一个结论在更长 context、更大 batch、更多 GPU 或更复杂数据分布下可能如何失效。
- 综合题：把本章内容和前一章或后一章连接起来，写出一个端到端训练/推理 pipeline 中的依赖关系。
- 批判题：指出一个容易被营销材料夸大的说法，并用本章的资源账本或评估协议反驳它。

2 02 PyTorch、einops 与资源核算

第 02 讲: PyTorch、einops 与资源核算

CS336 Spring 2026 public videos and official course materials

目录

1 本章导读	2
1.1 本章知识路线	2
1.2 结构图	2
2 本章主线	2
2.1 本章要解决的核心问题	3
2.2 从机制到资源账本	3
2.3 从课堂讲解到可复现实验	4
3 术语、符号与问题边界	4
3.1 术语之间的关系	4
4 张量、dtype 与显存账本	4
4.1 为什么要讨论张量、dtype 与显存账本	5
4.2 张量、dtype 与显存账本的机制	5
4.3 张量、dtype 与显存账本的数学表达	5
4.4 从张量、dtype 与显存账本到程序	5
4.5 边界条件与常见误解	5
4.6 与本章其他概念的关系	6
4.7 怎样检验张量、dtype 与显存账本	6
4.8 概念辨析: 张量、dtype 与显存账本	6
4.9 张量、dtype 与显存账本的小结	6
5 FLOPs 与训练时间的 napkin math	6
5.1 为什么要讨论 FLOPs 与训练时间的 napkin math	7
5.2 FLOPs 与训练时间的 napkin math 的机制	7
5.3 FLOPs 与训练时间的 napkin math 的数学表达	7
5.4 从 FLOPs 与训练时间的 napkin math 到程序	7
5.5 边界条件与常见误解	7

5.6	与本章其他概念的关系	8
5.7	怎样检验 FLOPs 与训练时间的 napkin math	8
5.8	概念辨析: FLOPs 与训练时间的 napkin math	8
5.9	FLOPs 与训练时间的 napkin math 的小结	8
6	einops、算术强度与 bottleneck 判断	9
6.1	为什么要讨论 einops、算术强度与 bottleneck 判断	9
6.2	einops、算术强度与 bottleneck 判断的机制	9
6.3	einops、算术强度与 bottleneck 判断的数学表达	9
6.4	从 einops、算术强度与 bottleneck 判断到程序	9
6.5	边界条件与常见误解	10
6.6	与本章其他概念的关系	10
6.7	怎样检验 einops、算术强度与 bottleneck 判断	10
6.8	概念辨析: einops、算术强度与 bottleneck 判断	10
6.9	einops、算术强度与 bottleneck 判断的小结	10
7	综合讨论: 从局部机制到完整系统	11
7.1	张量、dtype 与显存账本在系统中的位置	11
7.2	FLOPs 与训练时间的 napkin math 在系统中的位置	11
7.3	einops、算术强度与 bottleneck 判断在系统中的位置	11
7.4	把本章知识用于读论文和复现实验	12
8	例题: 把概念变成可计算检查	12
8.1	70B/15T 训练时间的数量级估算	12
9	实验设计: 从小例子到可复现结论	12
9.1	最小输入	12
9.2	资源账本	13
9.3	单变量消融	13
9.4	失败条件	13
9.5	记录与报告	13
9.6	从小实验到大结论	13
9.7	把本章重新读成一条证据链	13
10	课程资料与回看路径	14
10.1	视频回看路径	14
10.2	课程资料阅读路径	14
11	实践说明、历史位置与风险	15

12 复习要点	15
12.1 综合自测	16
13 总结与延伸	16
13.1 本章小结	16
13.2 延伸解释	16
13.3 练习	16

1 本章导读

本章讨论 **PyTorch、einops 与资源核算**。CS336 的第一层目标不是把现成大模型当作黑箱使用，而是把 language model 从输入文本、训练目标、模型结构、数据、系统和评估这些部件重新拆开。只有拆开之后，读者才能判断一个设计选择究竟改变了什么。设想你看到一个训练脚本声称能在若干 H100 上训练 70B 模型。判断它是否可信，第一步不是读模型名字，而是问：参数、梯度、optimizer state 和 activation 各占多少显存？每一步大约多少 FLOPs？瓶颈是 tensor core 还是 HBM bandwidth？本章对应 CS336 Spring 2026 第 02 个公开视频，并结合课程页提供的可解析资料。视频和课程页给出事实边界；正文把这些材料改写为可自学的教材章节。阅读本章时，先理解问题为什么存在，再看定义、公式和实现形态，随后通过例题和练习检查自己是否能独立复现这条 reasoning chain。

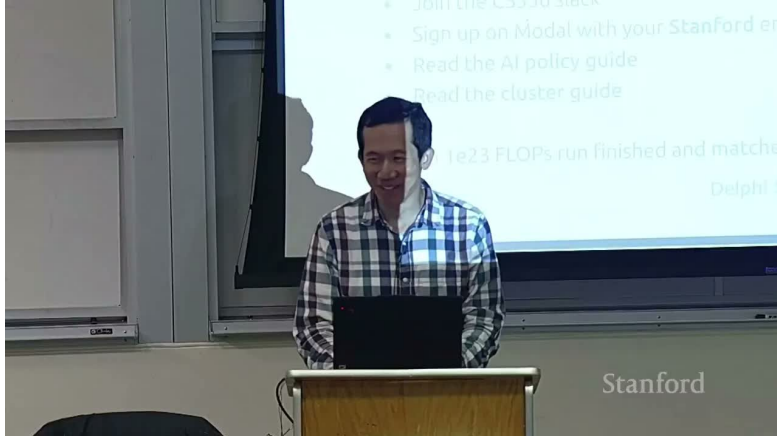


图 1: 第 02 讲的课程图像锚点。

1.1 本章知识路线

- 本章首先说明 PyTorch、einops 与资源核算为什么是 language modeling pipeline 中不可跳过的一环。
- 其次定义本章反复使用的术语，并说明这些术语对应的计算对象或系统对象。
- 随后用公式和伪代码表达主要机制，让概念能够落到可计算的变量上。
- 最后给出例题、风险讨论和练习，便于读者检查自己是否真正掌握了机制。

1.2 结构图

下面的图用于帮助读者定位本章的核心结构。先看概念之间的依赖，再回到正文中的公式、伪代码和例题。

2 本章主线

PyTorch、einops 与资源核算的主线是从一个可观察的问题出发，逐步引出表示、目标、实现和实验。读者应把每个机制放回 language modeling pipeline 的位置。

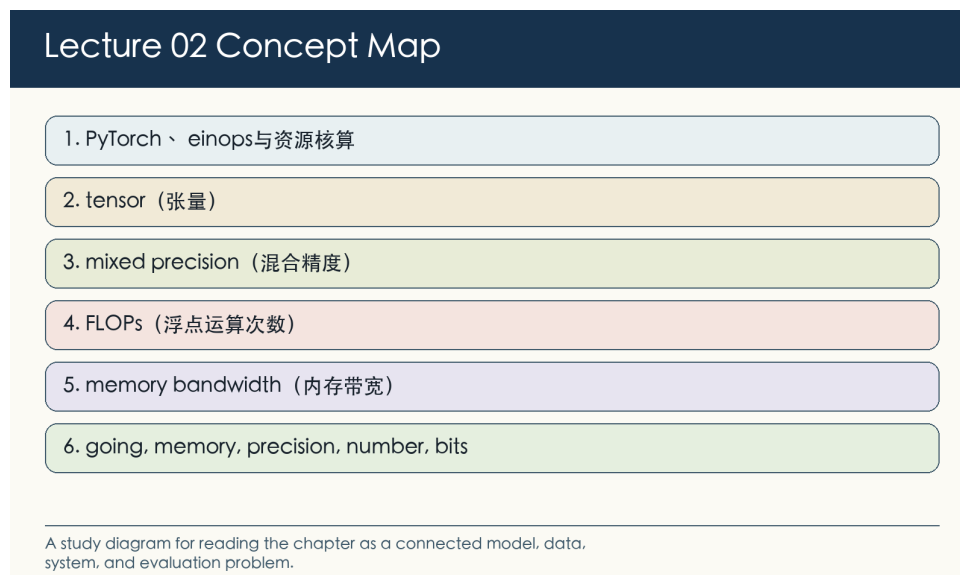


图 2: 第 02 讲概念图: 本章问题、术语和机制的关系。

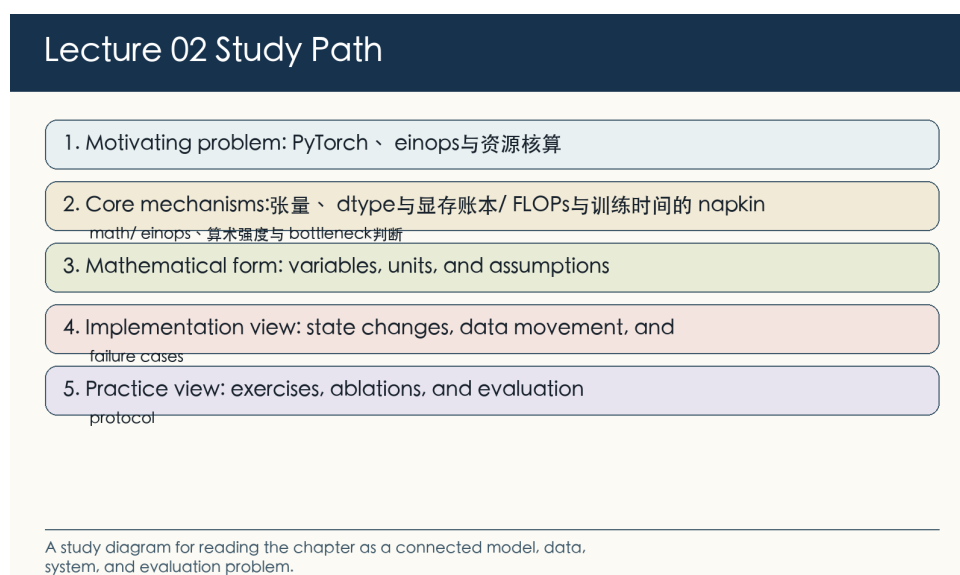


图 3: 第 02 讲学习路径: 从问题定义进入公式、实现、实验和练习。

2.1 本章要解决的核心问题

本讲把 PyTorch tensor 看作所有训练对象的统一表示: 参数、梯度、optimizer state、activation 和数据 batch 都是带 shape、dtype、device 的张量。读完这一段后, 应能用一两句话说明 PyTorch、einops 与资源核算要解决的瓶颈, 并指出这个瓶颈发生在数据、模型、优化、系统还是评估层。

2.2 从机制到资源账本

资源核算不是附属技能。FLOPs 决定训练时间下界, 显存决定模型和 batch 是否放得下, 内存带宽决定很多 elementwise 或 decode workload 是否会被 HBM 限制。随后把机制写成账本: 输入规模是什么, 主要状态是什么, 成本或质量指标怎样随变量改变。这个账本让后面的公式不再只是符号。

2.3 从课堂讲解到可复现实验

einops 的价值不是语法糖，而是让维度语义显式化。把 batch、sequence、head、hidden 等维度写清楚，可以减少实现错误，也方便做 FLOPs/bytes 的手算。最后把课堂讲解转成可复现实验：固定协议，改变一个变量，记录中间量，并解释结果为何支持或反驳原来的机制判断。

3 术语、符号与问题边界

本章的术语横跨模型、数据和系统。读者需要先知道每个词指向的对象：有些词描述数学目标，有些词描述张量形状，有些词描述硬件瓶颈，还有一些词描述评估协议。术语边界不清，后面的公式和实现就会变成空洞的缩写。

#	术语	阅读要求
1	tensor（张量）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
2	mixed precision（混合精度）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
3	FLOPs（浮点运算次数）	把它放入资源账本：单位是什么，随输入规模怎样变化，最容易被哪类 benchmark 误读。
4	memory bandwidth（内存带宽）	把它放入资源账本：单位是什么，随输入规模怎样变化，最容易被哪类 benchmark 误读。
5	arithmetic intensity（算术强度）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
6	roofline analysis（屋顶线分析）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。

这些术语之间有依赖关系。例如 tokenization 改变 sequence length，sequence length 又改变 attention FLOPs、KV cache 和 evaluation token budget；parallelism 改变显存占用，也改变通信瓶颈和故障恢复方式。

3.1 术语之间的关系

在本章中，tensor（张量）给出问题入口，mixed precision（混合精度）描述主要机制，FLOPs（浮点运算次数）则把机制连接到可测量的结果。读教材时应当沿着这个链条追问：输入是什么，中间状态怎样改变，输出指标为什么随之变化。课程材料还会反复出现 memory、flops、compute、training、intensity、parameters 等词。它们不是一组同义标签，而是提示本章的不同层次：有的指对象，有的指操作，有的指约束或指标。因此，术语表不是背诵清单，而是后文阅读的索引。遇到一个术语时，先定位它属于 representation、optimization、systems、data 还是 evaluation，再判断它改变哪一项账本。

4 张量、dtype 与显存账本

现在进入张量、dtype 与显存账本。这一节把算法放到硬件和运行时中考察：同一个数学表达式在不同 batch、shape、GPU 拓扑和 kernel 实现下会得到不同的成本账本。

4.1 为什么要讨论张量、dtype 与显存账本

课程材料用 rank、shape、dtype、device 解释 PyTorch tensor；参数、梯度、optimizer state、activation 和 data 都是 tensor。fp32、fp16、bf16、fp8、fp4 的差别首先体现在每个元素的字节数和动态范围，而不是抽象精度标签。显存估算的基本动作是：元素个数乘以 element size，再乘以副本数量。AdamW 通常还要记参数、梯度和两个 fp32 optimizer states。张量、dtype 与显存账本把 PyTorch、einops 与资源核算放进资源账本。接下来要看的不是单个 API，而是张量形状、内存层级、通信和调度怎样共同决定成本。

4.2 张量、dtype 与显存账本的机制

课程材料用 rank、shape、dtype、device 解释 PyTorch tensor；参数、梯度、optimizer state、activation 和 data 都是 tensor。在张量、dtype 与显存账本中，核心对象是硬件和系统状态：同一算法会因为 memory hierarchy、通信拓扑或 kernel shape 不同而表现出完全不同的速度。fp32、fp16、bf16、fp8、fp4 的差别首先体现在每个元素的字节数和动态范围，而不是抽象精度标签。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。显存估算的基本动作是：元素个数乘以 element size，再乘以副本数量。AdamW 通常还要记参数、梯度和两个 fp32 optimizer states。可检验性来自 profile trace、roofline 估算和端到端延迟分解；如果这些证据互相矛盾，就要先解释 measurement setup。因此，张量、dtype 与显存账本的学习目标是建立一张成本地图：读者应能指出瓶颈在计算、内存、通信还是调度，并能设计一个小 benchmark 去验证。

4.3 张量、dtype 与显存账本的数学表达

$$\text{bytes} = \# \text{elements} \times \text{bytes/element} \times \# \text{copies}$$

符号说明：copies 可以来自参数、梯度、momentum、variance、activation checkpoint 或并行副本；它是系统设计中最容易漏算的一项。这个关系式应被读成张量、dtype 与显存账本的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。在系统实验中，tokens/s、TTFT、显存峰值和 kernel time 都是 proxy。它们各自只观察一部分系统，因此报告时要同时写清 workload shape、warmup、同步方式和硬件型号。

4.4 从张量、dtype 与显存账本到程序

把张量、dtype 与显存账本写成程序时，最重要的是暴露状态变化和数据移动。下面的伪代码保留执行顺序，省略框架样板。

```
def tensor_bytes(shape, bytes_per_element, copies=1):  
    return prod(shape) * bytes_per_element * copies
```

阅读这段实现时，重点看张量、dtype 与显存账本怎样移动张量、占用显存并触发通信。实验记录至少要包含吞吐、延迟、显存峰值、通信量和 kernel 利用率，否则很难判断瓶颈来自算子、内存层级还是调度。

4.5 边界条件与常见误解

- activation memory 依赖 batch size、sequence length 和 checkpointing 策略，不能只看参数量。
- 低精度 dtype 节省显存和带宽，但 dynamic range 与 accumulation 精度会影响训练稳定性。
- 小规模 kernel benchmark 不一定预测端到端训练或 serving latency，因为调度、通信和请求形状会改变瓶颈。

- 硬件规格表和框架 profiler 都需要结合 workload 解释，不能把单个峰值数字当作机制证明。

4.6 与本章其他概念的关系

张量、dtype 与显存账本连接本章的数学机制和工程成本：术语表说明对象，公式估算资源，伪代码暴露状态变化，例题则让读者检查一次完整的成本推理。

4.7 怎样检验张量、dtype 与显存账本

检验张量、dtype 与显存账本时，先把抽象说法落到硬件和系统状态上。与 tensor、dtype、fp32、bf16、optimizer 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明：copies 可以来自参数、梯度、momentum、variance、activation checkpoint 或并行副本；它是系统设计中最容易漏算的一项。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。系统消融通常一次只改变 batch shape、sequence length、GPU 数、kernel 实现或调度策略。若同时改变硬件和 workload，实验只能说明端到端表现不同，不能定位瓶颈。最后检查失败条件。工程判断往往在反例中变清楚：activation memory 依赖 batch size、sequence length 和 checkpointing 策略，不能只看参数量；低精度 dtype 节省显存和带宽，但 dynamic range 与 accumulation 精度会影响训练稳定性。复习时可以把这些限制改写成反例测试，观察它们怎样改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。把实验结论放回 PyTorch、einops 与资源核算时，要说明张量、dtype 与显存账本改变的是训练吞吐、推理延迟、显存峰值还是通信开销。能做出这种归因，才说明读者已经从接口使用进入系统解释。

4.8 概念辨析：张量、dtype 与显存账本

tensor、dtype、fp32、bf16、optimizer、activation 常一起出现在系统讲解中，但它们分别指硬件对象、程序操作、瓶颈来源或测量指标。读者应先定位每个词在执行路径中的位置。其中，tensor（张量）是参数、激活、梯度和数据的共同表示。区分这些词时，可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕张量、dtype 与显存账本复习时，可以写一个小 benchmark：固定输入形状，改变一个系统变量，记录 profile trace 和端到端时间。

4.9 张量、dtype 与显存账本的小结

- 讨论张量、dtype 与显存账本时，应同时报告问题规模、实验设置和主要观测变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。
- 如果方法声称更快，要区分计算减少、内存访问减少、通信隐藏和调度改善；这些机制不能用一个 tokens/s 数字互相替代。
- profile trace、roofline 估算和端到端 benchmark 应互相校验，单独一个指标不足以解释系统瓶颈。

5 FLOPs 与训练时间的 napkin math

现在进入 FLOPs 与训练时间的 napkin math。这一节把算法放到硬件和运行时中考察：同一个数学表达式在不同 batch、shape、GPU 拓扑和 kernel 实现下会得到不同的成本账本。

5.1 为什么要讨论 FLOPs 与训练时间的 napkin math

视频用 70B 参数、15T tokens、1024 H100 这类例子展示快速估算训练时间的方式。经验公式 $\backslash(6ND\backslash)$ 把 forward、backward 和 gradient computation 合并成每 token 每参数约 6 FLOPs 的估计。模型 FLOPs 利用率 (MFU) 把理论峰值和真实训练吞吐连接起来，是判断 kernel、parallelism 和 data pipeline 是否有效的指标。FLOPs 与训练时间的 napkin math 把 PyTorch、einops 与资源核算放进资源账本。接下来要看的不是单个 API，而是张量形状、内存层级、通信和调度怎样共同决定成本。

5.2 FLOPs 与训练时间的 napkin math 的机制

视频用 70B 参数、15T tokens、1024 H100 这类例子展示快速估算训练时间的方式。在 FLOPs 与训练时间的 napkin math 中，核心对象是硬件和系统状态：同一算法会因为 memory hierarchy、通信拓扑或 kernel shape 不同而表现出完全不同的速度。经验公式 $\backslash(6ND\backslash)$ 把 forward、backward 和 gradient computation 合并成每 token 每参数约 6 FLOPs 的估计。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。模型 FLOPs 利用率 (MFU) 把理论峰值和真实训练吞吐连接起来，是判断 kernel、parallelism 和 data pipeline 是否有效的指标。可检验性来自 profile trace、roofline 估算和端到端延迟分解；如果这些证据互相矛盾，就要先解释 measurement setup。因此，FLOPs 与训练时间的 napkin math 的学习目标是建立一张成本地图：读者应能指出瓶颈在计算、内存、通信还是调度，并能设计一个小 benchmark 去验证。

5.3 FLOPs 与训练时间的 napkin math 的数学表达

$$\text{training FLOPs} \approx 6ND, \quad \text{days} \approx \frac{6ND}{F_{\text{gpu}} \cdot n_{\text{gpu}} \cdot \text{MFU} \cdot 86400}$$

符号说明： N 是参数量， D 是训练 token 数， F_{gpu} 是单 GPU 理论 FLOPs/s，MFU 是有效利用率。这个关系式应被读成 FLOPs 与训练时间的 napkin math 的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。在系统实验中，tokens/s、TTFT、显存峰值和 kernel time 都是 proxy。它们各自只观察一部分系统，因此报告时要同时写清 workload shape、warmup、同步方式和硬件型号。

5.4 从 FLOPs 与训练时间的 napkin math 到程序

把 FLOPs 与训练时间的 napkin math 写成程序时，最重要的是暴露状态变化和数据移动。下面的伪代码保留执行顺序，省略框架样板。

```
total_flops = 6 * num_parameters * num_tokens
seconds = total_flops / (gpu_flops * num_gpus * mfu)
```

阅读这段实现时，重点看 FLOPs 与训练时间的 napkin math 怎样移动张量、占用显存并触发通信。实验记录至少要包含吞吐、延迟、显存峰值、通信量和 kernel 利用率，否则很难判断瓶颈来自算子、内存层级还是调度。

5.5 边界条件与常见误解

- 这个估算忽略 optimizer、embedding、attention/MLP 比例、通信和 data loading，但足以暴露数量级错误。
- 不要把 peak FLOPs 当作可持续吞吐；实际 MFU 需要测量。

- 小规模 kernel benchmark 不一定预测端到端训练或 serving latency，因为调度、通信和请求形状会改变瓶颈。
- 硬件规格表和框架 profiler 都需要结合 workload 解释，不能把单个峰值数字当作机制证明。

5.6 与本章其他概念的关系

FLOPs 与训练时间的 napkin math 连接本章的数学机制和工程成本：术语表说明对象，公式估算资源，伪代码暴露状态变化，例题则让读者检查一次完整的成本推理。

5.7 怎样检验 FLOPs 与训练时间的 napkin math

检验 FLOPs 与训练时间的 napkin math 时，先把抽象说法落到硬件和系统状态上。与 FLOPs、MFU、H100、70B、tokens 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： $\backslash(N\backslash)$ 是参数量， $\backslash(D\backslash)$ 是训练 token 数， $\backslash(F_{\text{ext}\{\text{gpu}\}}\backslash)$ 是单 GPU 理论 FLOPs/s，MFU 是有效利用率。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。系统消融通常一次只改变 batch shape、sequence length、GPU 数、kernel 实现或调度策略。若同时改变硬件和 workload，实验只能说明端到端表现不同，不能定位瓶颈。最后检查失败条件。工程判断往往在反例中变清楚：这个估算忽略 optimizer、embedding、attention/MLP 比例、通信和 data loading，但足以暴露数量级错误；不要把 peak FLOPs 当作可持续吞吐；实际 MFU 需要测量。复习时可以把这些限制改写成反例测试，观察它们怎样改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。把实验结论放回 PyTorch、einops 与资源核算时，要说明 FLOPs 与训练时间的 napkin math 改变的是训练吞吐、推理延迟、显存峰值还是通信开销。能做出这种归因，才说明读者已经从接口使用进入系统解释。

5.8 概念辨析：FLOPs 与训练时间的 napkin math

FLOPs、MFU、H100、70B、tokens 常一起出现在系统讲解中，但它们分别指硬件对象、程序操作、瓶颈来源或测量指标。读者应先定位每个词在执行路径中的位置。其中，FLOPs (floating-point operations) 是训练和推理成本账本的核心单位。区分这些词时，可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕 FLOPs 与训练时间的 napkin math 复习时，可以写一个小 benchmark：固定输入形状，改变一个系统变量，记录 profile trace 和端到端时间。

5.9 FLOPs 与训练时间的 napkin math 的小结

- 讨论 FLOPs 与训练时间的 napkin math 时，应同时报告问题规模、实验设置和主要观测变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。
- 如果方法声称更快，要区分计算减少、内存访问减少、通信隐藏和调度改善；这些机制不能用一个 tokens/s 数字互相替代。
- profile trace、roofline 估算和端到端 benchmark 应互相校验，单独一个指标不足以解释系统瓶颈。

6 einops、算术强度与 bottleneck 判断

现在进入 einops、算术强度与 bottleneck 判断。这一节把算法放到硬件和运行时中考察：同一个数学表达式在不同 batch、shape、GPU 拓扑和 kernel 实现下会得到不同的成本账本。

6.1 为什么要讨论 einops、算术强度与 bottleneck 判断

einops 通过命名维度减少 PyTorch 代码里 transpose、view、reshape 的认知负担。arithmetic intensity 衡量每搬运 1 byte 做多少 FLOPs；矩阵乘法通常 compute-bound，elementwise ops 常常 memory-bound。roofline analysis 给出性能上限：低算术强度受带宽限制，高算术强度受峰值计算限制。einops、算术强度与 bottleneck 判断把 PyTorch、einops 与资源核算放进资源账本。接下来要看的不是单个 API，而是张量形状、内存层级、通信和调度怎样共同决定成本。

6.2 einops、算术强度与 bottleneck 判断的机制

einops 通过命名维度减少 PyTorch 代码里 transpose、view、reshape 的认知负担。在 einops、算术强度与 bottleneck 判断中，核心对象是硬件和系统状态：同一算法会因为 memory hierarchy、通信拓扑或 kernel shape 不同而表现出完全不同的速度。arithmetic intensity 衡量每搬运 1 byte 做多少 FLOPs；矩阵乘法通常 compute-bound，elementwise ops 常常 memory-bound。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。roofline analysis 给出性能上限：低算术强度受带宽限制，高算术强度受峰值计算限制。可检验性来自 profile trace、roofline 估算和端到端延迟分解；如果这些证据互相矛盾，就要先解释 measurement setup。因此，einops、算术强度与 bottleneck 判断的学习目标是建立一张成本地图：读者应能指出瓶颈在计算、内存、通信还是调度，并能设计一个小 benchmark 去验证。

6.3 einops、算术强度与 bottleneck 判断的数学表达

$$\text{arithmetic intensity} = \frac{\text{FLOPs}}{\text{bytes moved}}, \quad P \leq \min(P_{\max}, B_{\max} \cdot I)$$

符号说明： I 是算术强度， $P_{\max}$ 是峰值计算， $B_{\max}$ 是内存带宽， P 是可达到性能。这个关系式应被读成 einops、算术强度与 bottleneck 判断的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。在系统实验中，tokens/s、TTFT、显存峰值和 kernel time 都是 proxy。它们各自只观察一部分系统，因此报告时要同时写清 workload shape、warmup、同步方式和硬件型号。

6.4 从 einops、算术强度与 bottleneck 判断到程序

把 einops、算术强度与 bottleneck 判断写成程序时，最重要的是暴露状态变化和数据移动。下面的伪代码保留执行顺序，省略框架样板。

```
scores = einsum(x, y, 'batch_seq_hidden, batch_cand_hidden->batch_seq_cand')
if flops / bytes_moved < roofline_knee:
    optimize_memory_traffic()
else:
    optimize_compute_utilization()
```

阅读这段实现时，重点看 einops、算术强度与 bottleneck 判断怎样移动张量、占用显存并触发通信。实验记录至少要包含吞吐、延迟、显存峰值、通信量和 kernel 利用率，否则很难判断瓶颈来自算子、内存层级还是调度。

6.5 边界条件与常见误解

- roofline 是上界模型，不会告诉你具体哪个 kernel 写错了。
- 维度语义清晰不等于性能最佳；layout 和 contiguous memory 仍然重要。
- 小规模 kernel benchmark 不一定预测端到端训练或 serving latency，因为调度、通信和请求形状会改变瓶颈。
- 硬件规格表和框架 profiler 都需要结合 workload 解释，不能把单个峰值数字当作机制证明。

6.6 与本章其他概念的关系

einops、算术强度与 bottleneck 判断连接本章的数学机制和工程成本：术语表说明对象，公式估算资源，伪代码暴露状态变化，例题则让读者检查一次完整的成本推理。

6.7 怎样检验 einops、算术强度与 bottleneck 判断

检验 einops、算术强度与 bottleneck 判断时，先把抽象说法落到硬件和系统状态上。与 einops、einsum、arithmetic intensity、roofline、memory-bound 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： $\backslash(I\backslash)$ 是算术强度， $\backslash(P_{\backslash\max}\backslash)$ 是峰值计算， $\backslash(B_{\backslash\max}\backslash)$ 是内存带宽， $\backslash(P\backslash)$ 是可达到性能。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。系统消融通常一次只改变 batch shape、sequence length、GPU 数、kernel 实现或调度策略。若同时改变硬件和 workload，实验只能说明端到端表现不同，不能定位瓶颈。最后检查失败条件。工程判断往往在反例中变清楚：roofline 是上界模型，不会告诉你具体哪个 kernel 写错了；维度语义清晰不等于性能最佳；layout 和 contiguous memory 仍然重要。复习时可以把这些限制改写成反例测试，观察它们怎样改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。把实验结论放回 PyTorch、einops 与资源核算时，要说明 einops、算术强度与 bottleneck 判断改变的是训练吞吐、推理延迟、显存峰值还是通信开销。能做出这种归因，才说明读者已经从接口使用进入系统解释。

6.8 概念辨析：einops、算术强度与 bottleneck 判断

einops、einsum、arithmetic intensity、roofline、memory-bound 常一起出现在系统讲解中，但它们分别指硬件对象、程序操作、瓶颈来源或测量指标。读者应先定位每个词在执行路径中的位置。其中，memory（内存/显存）约束决定 batch、sequence length、optimizer state 和 KV cache 能否放下。区分这些词时，可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕 einops、算术强度与 bottleneck 判断复习时，可以写一个小 benchmark：固定输入形状，改变一个系统变量，记录 profile trace 和端到端时间。

6.9 einops、算术强度与 bottleneck 判断的小结

- 讨论 einops、算术强度与 bottleneck 判断时，应同时报告问题规模、实验设置和主要观测变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。
- 如果方法声称更快，要区分计算减少、内存访问减少、通信隐藏和调度改善；这些机制不能用一个 tokens/s 数字互相替代。
- profile trace、roofline 估算和端到端 benchmark 应互相校验，单独一个指标不足以解释系统瓶颈。

7 综合讨论：从局部机制到完整系统

本章的主题是 PyTorch、einops 与资源核算，但它不应被读成几个相邻知识点的拼接。张量、dtype 与显存账本、FLOPs 与训练时间的 napkin math、einops、算术强度与 bottleneck 判断共同回答同一个问题：语言模型系统中哪些对象可以被定义，哪些成本可以被估算，哪些结论必须通过实验或评估来确认。这些对象包括 tensor（张量）、mixed precision（混合精度）、FLOPs（浮点运算次数）、memory bandwidth（内存带宽）、arithmetic intensity（算术强度）、roofline analysis（屋顶线分析）。读者可以把它们看成一组接口：有的接口连接文本和 token，有的连接模型结构和张量计算，有的连接数据样本和训练目标，有的连接离线评估和在线服务。接口一旦改变，后面的成本、错误类型和优化空间也会随之改变。

7.1 张量、dtype 与显存账本在系统中的位置

张量、dtype 与显存账本首先提供一个局部解释：课程材料用 rank、shape、dtype、device 解释 PyTorch tensor；参数、梯度、optimizer state、activation 和 data 都是 tensor。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，张量、dtype 与显存账本会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，张量、dtype 与显存账本不能脱离边界条件理解。一个关键 caveat 是：activation memory 依赖 batch size、sequence length 和 checkpointing 策略，不能只看参数量。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.2 FLOPs 与训练时间的 napkin math 在系统中的位置

FLOPs 与训练时间的 napkin math 首先提供一个局部解释：视频用 70B 参数、15T tokens、1024 H100 这类例子展示快速估算训练时间的方式。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，FLOPs 与训练时间的 napkin math 会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，FLOPs 与训练时间的 napkin math 不能脱离边界条件理解。一个关键 caveat 是：这个估算忽略 optimizer、embedding、attention/MLP 比例、通信和 data loading，但足以暴露数量级错误。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.3 einops、算术强度与 bottleneck 判断在系统中的位置

einops、算术强度与 bottleneck 判断首先提供一个局部解释：einops 通过命名维度减少 PyTorch 代码里 transpose、view、reshape 的认知负担。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，einops、算术强度与 bottleneck 判断会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，einops、算术强度与 bottleneck 判断不能脱离边界条件理解。一个关键 caveat 是：roofline 是上界模型，不会告诉你具体哪个 kernel 写错了。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.4 把本章知识用于读论文和复现实验

读相关论文时，可以把本章变成一个检查表。第一，论文是否清楚定义了输入规模、模型规模、数据来源和评估协议；第二，论文声称的改进落在哪个变量上；第三，作者是否报告了会让结论失效的设置；第四，实验是否能分离模型结构、数据质量、优化设置和系统实现的影响。复现实验时，也应避免直接追求大规模。先用小程序复现一个公式、一个伪代码分支或一个 profile 现象，再扩大输入规模。这样做的原因很简单：如果小规模程序无法解释中间量，大规模训练只会把错误隐藏在日志和随机波动里。把这些检查合在一起，本章给出的不是一个固定答案，而是一种阅读 CS336 的方法：每个模型机制都要落到变量，每个变量都要落到测量，每个测量都要说明协议，每个协议都要承认边界。因此，本章中的任何一句结论都可以被改写成一个可引用的完整句式：在给定数据、模型、硬件和评估协议下，某个机制改变了某个变量，并通过某个观测指标表现出来。这样的句子比单独背诵术语更长，但它包含了自学、复现和引用时真正需要的信息。把本章用于自学时，可以按三遍阅读。第一遍只追踪对象和定义，确认每个术语到底指向文本、token、张量、样本、请求还是评估记录。第二遍追踪公式和伪代码，确认每一步改变了哪些状态。第三遍追踪实验和 caveat，确认哪些结论只在特定规模、数据或硬件条件下成立。把本章用于复习时，则应反过来操作：先从一个失败案例或异常指标出发，再回到相应机制。loss 曲线异常可能指向数据、优化或模型容量；吞吐异常可能指向 kernel、通信或调度；benchmark 异常可能指向 contamination、prompt protocol 或采样设置。能够从异常反推机制，是掌握本章的实用标准。

8 例题：把概念变成可计算检查

8.1 70B/15T 训练时间的数量级估算

用近似公式 ‘training FLOPs $\sim 6ND$ ’，设 ‘ $N=70B$ ’、‘ $D=15T$ ’、‘1024’ 张 GPU、单卡峰值约为 ‘ $1e15$ FLOPs/s’、MFU 为 ‘0.4’。

- ‘ $6ND \sim 6.3e24$ FLOPs’，这是训练总账本。
- 有效吞吐约为 ‘ $1024 \times 1e15 \times 0.4 = 4.096e17$ FLOPs/s’。
- 训练时间约 ‘ $1.54e7$ ’ 秒，折合约 ‘178’ 天；若假设或 MFU 变化，结论会线性变化。

结论。 napkin math 能快速暴露不可能的训练声明，也能提示优化方向是算力、并行效率还是数据规模。这个例题的目的不是替代完整工程实现，而是给出一种最小推理形式：把问题转成变量，写出近似公式或伪代码，再说明近似何时失效。

9 实验设计：从小例子到可复现结论

学习 PyTorch、einops 与资源核算不能停在概念层。一个教材化结论至少需要四件事：可构造的输入、可观测的中间量、可比较的指标，以及清楚的失败条件。下面的实验设计不是要求训练大模型，而是要求读者用小规模程序复现本章机制。

9.1 最小输入

先构造只触发本章核心机制的小输入。例如 tokenizer 章节可以用几个包含 rare words 和 Unicode 字符的字符串；GPU 章节可以用一个 elementwise kernel 和一个 GEMM；data 章节可以用十几篇近重复文档。小输入的价值在于它让错误可见。

9.2 资源账本

对同一输入写下 FLOPs、bytes moved、显存峰值、通信量或样本数量的估算。估算不需要一开始就精确，但必须说明单位和近似假设。随后用 profiler、benchmark、validation loss 或人工检查去验证估算是否同量级。

9.3 单变量消融

一次只改变一个因素，例如 vocabulary size、head 数、batch size、filter threshold、reward scale、decoding temperature 或 GPU 数量。若多个变量同时变化，实验只能说明系统变了，不能说明机制是什么。

9.4 失败条件

最后要刻意触发失败：极端 context length、错误 dtype、过小 batch、污染 benchmark、低质量数据或不可靠 verifier。失败条件常常比成功曲线更能说明一个方法的边界。

9.5 记录与报告

实验记录应能让另一个读者复现同一结论。最少要写清楚输入构造、代码版本、随机种子、硬件、batch shape、数据版本、评价指标和异常处理方式。报告结论时不要只写“更快”或“更好”，而要说明快在哪里、好在哪个指标上，以及是否牺牲了内存、稳定性、数据质量或可解释性。常见报告错误是把局部现象写成普遍规律。例如一次小模型实验里的 loss 改善，不等于更大模型或不同数据分布上也会改善；一次 kernel benchmark 的吞吐提升，不等于端到端 serving latency 一定降低。教材中的实验报告应始终把结论限定在可复现条件内。

9.6 从小实验到大结论

小实验的作用不是替代课程中的完整训练或系统实现，而是建立一条可检查的推理链。先在小输入上确认变量方向，再在中等规模上确认数量级，最后才讨论是否能外推到更大模型、更多 token 或更复杂 workload。缺少中间层级时，大结论通常只是在复述直觉。因此，实验报告最好同时包含正例和反例：正例说明机制在受控条件下怎样工作，反例说明条件稍变时哪里失效。对 CS336 这样的课程，反例尤其重要，因为 language model 的错误常来自多个层面叠加：数据分布、优化稳定性、硬件瓶颈、评估协议和服务负载都可能同时改变观察结果。还要记录资料版本与复现边界。公开视频、课程页、配套代码和 PDF 可能在不同时间更新；复现实验应写明使用的材料版本，并说明哪些结论来自课程事实，哪些是为了帮助理解而加入的解释性推导。这样才能把学习笔记变成可检查的技术记录。若复现实验与课程结论不一致，首先检查输入、版本和评估协议，而不是立即修改模型假设。

9.7 把本章重新读成一条证据链

完成小实验之后，可以把 PyTorch、einops 与资源核算重新读成一条证据链：先用“张量、dtype 与显存账本”确定问题入口，再用“FLOPs 与训练时间的 napkin math”解释主要机制，最后用“einops、算术强度与 bottleneck 判断”检查边界或外推条件。这种读法比按页背诵更接近教材训练，因为它要求每个结论都能说明前提、变量和证据。本章反复使用的术语包括 tensor（张量）、mixed precision（混合精度）、FLOPs（浮点运算次数）、memory bandwidth（内存带宽）、arithmetic intensity（算术强度）、roofline analysis（屋顶线分析）。复习时不要只写定义，而要为每个术语补上一句操作性说明：它在课程代码、公式、数据记录或评估协议中对应什么对象；如果这个对象被删除、替换或缩放，哪一个观测量会变化。最后，用一个反例结束复习。反例可以是资源估算不准、数据污染、reward 失真、benchmark 解析失败、长上下文显存爆炸或 scaling 外推失效。能解释反例的章节，才真正具备自学和引用价值；不能解释反例的章节，只是把课程材料换了一种排版。

引用本章结论时，还应保留来源边界：哪些说法直接来自视频、课程页、slides 或代码，哪些是为了串联教材叙述而加入的解释性推导。这个边界不会削弱结论，反而让读者知道何处可以复核，何处需要在自己的实验中重新验证。如果后续课程材料更新，也可以沿着同一证据链替换来源，而不必重写整章结构。

10 课程资料与回看路径

教材正文已经把主要概念、公式和实现逻辑组织成连续叙述；本节只提供回到课程材料的阅读路径。读者复习时可以先完成前文练习，再按下面的时间段或资料组核对细节。

10.1 视频回看路径

时间	关键词	复习任务
00:00:05-00:06:37	number, going, talk, tensors, compute, point, loss	回看定义、例子、推导或 caveat 在该段中的位置。
00:19:17-00:25:45	going, matrix, dimensions, then, there's, hidden, think	回看定义、例子、推导或 caveat 在该段中的位置。
00:38:27-00:44:55	going, that's, flops, then, memory, number, your	回看定义、例子、推导或 caveat 在该段中的位置。
00:51:02-00:57:26	arithmetic, intensity, memory, going, your, then, maybe	回看定义、例子、推导或 caveat 在该段中的位置。
01:10:50-01:17:01	memory, store, times, number, your, that's, general	回看定义、例子、推导或 caveat 在该段中的位置。

这些时间段用于定位视频中的讲解顺序。严肃引用时，应回到视频片段确认讲者表述、板书或幻灯片内容。回看视频时，不必逐字抄录字幕。更有效的方法是把每个时间段判定为定义、例子、推导、代码解释或 caveat，再把它放回本章对应小节。

10.2 课程资料阅读路径

资料组	标题/页块	关键词
official_01	Announcements:	https, train, resources, memory, operations, read, guide
official_03	Google Brain developed brain floating point (bf16) in 2018 to address this issue.	bf16, memory, nvidia, range, fp32, training, brain
official_05	Model FLOPs utilization (MFU)	accelerator, intensity, communication, computation, time, flop, memory
official_08	- Adam = RMSProp + momentum	memory, activations, compute, training, gradient, gradients, adam

official_10	Function: einops_motivation	function, einops_motivation, einops_einsum, einops_reduce, einops_rearrange, ten- sor_operations_flops, arith- metic_intensity
-------------	-----------------------------	--

课程资料通常给出更稳定的标题、公式、图或代码结构；视频则补充动机和口头解释。两者合起来构成本章的事实边界。阅读资料时，可以把每个资料组拆成三个对象：一个定义，一个公式或伪代码动作，一个边界条件。这样比逐字翻译页面或脚本更容易形成可用知识。

11 实践说明、历史位置与风险

PyTorch、einops 与资源核算在 CS336 的课程结构中连接基础机制和规模化问题。基础机制解释方法为什么成立；规模化问题检验它在更大模型、更长上下文、更复杂数据或真实 serving workload 下是否仍成立。历史位置不等于模型年表。更重要的是识别问题如何迁移：早期方法解决小规模建模问题，现代方法常常解决同一问题在大规模数据、GPU 集群、长上下文或人类偏好约束下的新形态。

- 资料版本限制 (version risk): 课程页、公开视频和配套资料可能在学期中更新；引用时应注明访问日期或资料版本。
- 测量口径限制 (measurement risk): FLOPs、tokens/s、benchmark score、reward 等数字只有在协议完整时可比较。
- 规模外推限制 (scaling risk): 小模型或小 batch 的机制不必然外推到 frontier-scale。
- 数据分布限制 (data risk): 数据来源、过滤和去重策略会改变模型行为，也会改变 evaluation contamination 风险。

12 复习要点

下面的问题把本章内容压缩为可反复检查的条目。每个条目都应能回到前文的解释、公式、伪代码或例题。

条目	对象	检查动作
条目 1	tensor (张量)	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。
条目 2	mixed precision (混合精度)	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。
条目 3	FLOPs (浮点运算次数)	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。
条目 4	memory bandwidth (内存带宽)	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。

条目 5	arithmetic intensity (算术强度)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 6	roofline analysis (屋顶线分析)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。

12.1 综合自测

- 我能否不用课程原句，独立解释本章中心问题？
- 我能否把本章至少一个公式改写成可运行的估算函数？
- 我能否指出一个看似改进但实际转移成本的方法？
- 我能否回到视频和课程资料，找到支持本章关键论断的位置？
- 我能否设计一个最小 ablation，验证本章一个 caveat？

13 总结与延伸

13.1 本章小结

- PyTorch、einops 与资源核算的核心不是记忆术语，而是理解对象、变量和机制之间的关系。
- 视频给出讲解顺序，课程资料提供可核对的公式、图、代码或页面结构。
- 复习时应把每个术语连接到至少一个变量、一个实现动作和一个失败模式。

13.2 延伸解释

本章中的延伸解释用于连接课程材料中的概念，例如说明公式里的 proxy、解释系统 tradeoff，或把多个材料片段串成一个完整推理链。若需要严肃引用，应回到课程视频和配套资料核对原始表述。

13.3 练习

- 定义题：用中英双语解释本章任意五个重要术语，并说明它们属于 compute、memory、data、optimization、evaluation 还是 deployment 账本。
- 推导题：选择本章一个公式，逐个解释符号、单位和近似假设，并说明哪个变量最难在真实系统中测量。
- 实现题：把本章伪代码改写成一个最小 Python sanity check，不要求训练大模型，但要输出可检查的中间量。
- 资料题：从“课程资料与回看路径”中选择一个视频时间段，回到原始视频核对讲者例子，并记录是否存在字幕误差。
- 评估题：为本章方法设计一个 ablation 或 benchmark，说明控制变量、数据版本、指标和 failure criteria。
- 边界题：说明本章一个结论在更长 context、更大 batch、更多 GPU 或更复杂数据分布下可能如何失效。
- 综合题：把本章内容和前一章或后一章连接起来，写出一个端到端训练/推理 pipeline 中的依赖关系。
- 批判题：指出一个容易被营销材料夸大的说法，并用本章的资源账本或评估协议反驳它。

3 03 现代 Transformer architecture 与 hyperparameters

第 03 讲：现代 Transformer architecture 与 hyperparameters

CS336 Spring 2026 public videos and official course materials

目录

1 本章导读	2
1.1 本章知识路线	2
1.2 结构图	2
2 本章主线	2
2.1 本章要解决的核心问题	3
2.2 从机制到资源账本	3
2.3 从课堂讲解到可复现实验	3
3 术语、符号与问题边界	4
3.1 术语之间的关系	4
4 Decoder-only Transformer 的骨架	4
4.1 为什么要讨论 Decoder-only Transformer 的骨架	4
4.2 Decoder-only Transformer 的骨架的机制	5
4.3 Decoder-only Transformer 的骨架的形式化表示	5
4.4 从 Decoder-only Transformer 的骨架到程序	5
4.5 边界条件与常见误解	5
4.6 与本章其他概念的关系	6
4.7 怎样检验 Decoder-only Transformer 的骨架	6
4.8 概念辨析：Decoder-only Transformer 的骨架	6
4.9 Decoder-only Transformer 的骨架的小结	6
5 Normalization、activation 与 position encoding	6
5.1 为什么要讨论 Normalization、activation 与 position encoding	7
5.2 Normalization、activation 与 position encoding 的机制	7
5.3 Normalization、activation 与 position encoding 的数学表达	7
5.4 从 Normalization、activation 与 position encoding 到程序	7
5.5 边界条件与常见误解	7

5.6	与本章其他概念的关系	8
5.7	怎样检验 Normalization、activation 与 position encoding	8
5.8	概念辨析: Normalization、activation 与 position encoding	8
5.9	Normalization、activation 与 position encoding 的小结	8
6	Hyperparameter 的规模化视角	8
6.1	为什么要讨论 Hyperparameter 的规模化视角	9
6.2	Hyperparameter 的规模化视角的机制	9
6.3	Hyperparameter 的规模化视角的数学表达	9
6.4	从 Hyperparameter 的规模化视角到程序	9
6.5	边界条件与常见误解	9
6.6	与本章其他概念的关系	10
6.7	怎样检验 Hyperparameter 的规模化视角	10
6.8	概念辨析: Hyperparameter 的规模化视角	10
6.9	Hyperparameter 的规模化视角的小结	10
7	综合讨论: 从局部机制到完整系统	11
7.1	Decoder-only Transformer 的骨架在系统中的位置	11
7.2	Normalization、activation 与 position encoding 在系统中的位置	11
7.3	Hyperparameter 的规模化视角在系统中的位置	11
7.4	把本章知识用于读论文和复现实验	12
8	例题: 把概念变成可计算检查	12
8.1	Pre-norm block 的信息路径	12
9	实验设计: 从小例子到可复现结论	12
9.1	最小输入	12
9.2	资源账本	13
9.3	单变量消融	13
9.4	失败条件	13
9.5	记录与报告	13
9.6	从小实验到大结论	13
9.7	把本章重新读成一条证据链	13
10	课程资料与回看路径	14
10.1	视频回看路径	14
10.2	课程资料阅读路径	14
11	实践说明、历史位置与风险	15

12 复习要点	15
12.1 综合自测	16
13 总结与延伸	16
13.1 本章小结	16
13.2 延伸解释	16
13.3 练习	16

1 本章导读

本章讨论 **现代 Transformer architecture 与 hyperparameters**。CS336 的第一层目标不是把现成大模型当作黑箱使用，而是把 language model 从输入文本、训练目标、模型结构、数据、系统和评估这些部件重新拆开。只有拆开之后，读者才能判断一个设计选择究竟改变了什么。设想你要从论文表格里选择一个 decoder-only Transformer recipe。所有候选模型都叫 Transformer，但 pre-norm、RMSNorm、RoPE、SwiGLU、QK norm、head layout 和 learning rate schedule 不同；本章训练你判断哪些是核心机制，哪些是经验性工程选择。本章对应 CS336 Spring 2026 第 03 个公开视频，并结合课程页提供的可解析资料。视频和课程页给出事实边界；正文把这些材料改写为可自学的教材章节。阅读本章时，先理解问题为什么存在，再看定义、公式和实现形态，随后通过例题和练习检查自己是否能独立复现这条 reasoning chain。

Lecture 3

EVERYTHING YOU DIDN'T WANT TO KNOW ABOUT
LM ARCHITECTURE AND HYPERPARAMETERS

CS336
Tatsu H



图 1: 第 03 讲的课程图像锚点。

1.1 本章知识路线

- 本章首先说明现代 Transformer architecture 与 hyperparameters 为什么是 language modeling pipeline 中不可跳过的一环。
- 其次定义本章反复使用的术语，并说明这些术语对应的计算对象或系统对象。
- 随后用公式和伪代码表达主要机制，让概念能够落到可计算的变量上。
- 最后给出例题、风险讨论和练习，便于读者检查自己是否真正掌握了机制。

1.2 结构图

下面的图用于帮助读者定位本章的核心结构。先看概念之间的依赖，再回到正文中的公式、伪代码和例题。

2 本章主线

现代 Transformer architecture 与 hyperparameters 的主线是从一个可观察的问题出发，逐步引出表示、目标、实现和实验。读者应把每个机制放回 language modeling pipeline 的位置。

Outline and goals

- ❖ Quick recap of a modern transformer (what you implement)
- ❖ What do most of the large LMs have in common?
- ❖ What are common variations to the architecture / training process?

Today's theme: the best way to learn is hands-on experience
the second best way is to try to learn from others' experience

图 2: 官方材料图页: 现代 Transformer architecture 与 hyperparameters 的课程页/PDF/script 图像锚点。

Lecture 3

EVERYTHING YOU DIDN'T WANT TO KNOW ABOUT
LM ARCHITECTURE AND HYPERPARAMETERS

CS336
Tatsu H

图 3: 官方材料图页: 现代 Transformer architecture 与 hyperparameters 的课程页/PDF/script 图像锚点。

2.1 本章要解决的核心问题

本讲从现代 decoder-only Transformer 的共同骨架出发, 比较 LLaMA-like 架构里的 pre-norm、RMSNorm、RoPE、SwiGLU、bias removal、QK norm 等设计。读完这一段后, 应能用一两句话说明现代 Transformer architecture 与 hyperparameters 要解决的瓶颈, 并指出这个瓶颈发生在数据、模型、优化、系统还是评估层。

2.2 从机制到资源账本

架构选择不能脱离 scale。小模型 ablation 中看似微弱的差异, 在大模型训练中可能表现为稳定性、吞吐、长上下文或超参数敏感性的差异。随后把机制写成账本: 输入规模是什么, 主要状态是什么, 成本或质量指标怎样随变量改变。这个账本让后面的公式不再只是符号。

2.3 从课堂讲解到可复现实验

hyperparameter 的有效读法是把每个选择对应到 failure mode: 学习率过大导致发散, batch 太小导致噪声过大, context 太长带来 attention 和 KV cache 成本, width/depth 比例影响计算形状。最后把课堂讲解转

成可复现实验：固定协议，改变一个变量，记录中间量，并解释结果为何支持或反驳原来的机制判断。

3 术语、符号与问题边界

本章的术语横跨模型、数据和系统。读者需要先知道每个词指向的对象：有些词描述数学目标，有些词描述张量形状，有些词描述硬件瓶颈，还有一些词描述评估协议。术语边界不清，后面的公式和实现就会变成空洞的缩写。

#	术语	阅读要求
1	Transformer architecture (Transformer 架构)	说明它在计算图中的位置、输入输出形状，以及改变它会影响的训练或推理成本。
2	pre-norm (前归一化)	说明它在计算图中的位置、输入输出形状，以及改变它会影响的训练或推理成本。
3	RMSNorm	说明它在计算图中的位置、输入输出形状，以及改变它会影响的训练或推理成本。
4	RoPE (rotary positional embedding)	说明它在计算图中的位置、输入输出形状，以及改变它会影响的训练或推理成本。
5	SwiGLU	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
6	hyperparameter (超参数)	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。

这些术语之间有依赖关系。例如 tokenization 改变 sequence length，sequence length 又改变 attention FLOPs、KV cache 和 evaluation token budget；parallelism 改变显存占用，也改变通信瓶颈和故障恢复方式。

3.1 术语之间的关系

在本章中，Transformer architecture (Transformer 架构) 给出问题入口，pre-norm (前归一化) 描述主要机制，RMSNorm 则把机制连接到可测量的结果。读教材时应当沿着这个链条追问：输入是什么，中间状态怎样改变，输出指标为什么随之变化。课程材料还会反复出现 attention、gemma、most、embeddings、position、many 等词。它们不是一组同义标签，而是提示本章的不同层次：有的指对象，有的指操作，有的指约束或指标。因此，术语表不是背诵清单，而是后文阅读的索引。遇到一个术语时，先定位它属于 representation、optimization、systems、data 还是 evaluation，再判断它改变哪一项账本。

4 Decoder-only Transformer 的骨架

现在进入 Decoder-only Transformer 的骨架。这一节把模型结构还原成表示、参数、activation 和训练动态之间的关系，而不是停留在结构名称。

4.1 为什么要讨论 Decoder-only Transformer 的骨架

视频把 architecture 讲成现代论文中大量细节的集合，而不是单一的 Transformer block 图。decoder-only LM 的主干通常是 token embedding、若干层 causal self-attention 和 MLP、norm、residual connection、LM head。课程材料强调许多看似小的选择会影响 scale：normalization placement、activation、position encoding、

head layout、optimizer schedule。推理阶段的关键区别在于请求是在线到达的，输出又是逐 token 生成的。训练时可以用大 batch 摊薄成本；服务时还要同时考虑首 token 延迟、吞吐、显存占用和长尾请求。

4.2 Decoder-only Transformer 的骨架的机制

视频把 architecture 讲成现代论文中大量细节的集合，而不是单一的 Transformer block 图。在 Decoder-only Transformer 的骨架中，核心对象是张量、token 和计算图状态：模型机制必须同时说明表示形态、张量形状和计算代价。decoder-only LM 的主干通常是 token embedding、若干层 causal self-attention 和 MLP、norm、residual connection、LM head。因而同一个方法在不同规模下可能改变不同的变量，尤其是 sequence length、hidden size、head 数、activation size 和训练稳定性。比较方法时，必须同时给出问题规模和实验条件。课程材料强调许多看似小的选择会影响 scale：normalization placement、activation、position encoding、head layout、optimizer schedule。可检验性来自形状推导、训练稳定性曲线和结构消融；只画 block diagram 不能说明机制已经成立。因此，Decoder-only Transformer 的骨架的学习目标是建立一条计算图解释：读者应能从输入 token 追到 activation、loss 和更新，而不是只记住结构名称。

4.3 Decoder-only Transformer 的骨架的形式化表示

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V$$

符号说明： Q, K, V 是 query/key/value； d_k 是 head dimension； M 是 causal mask，用于禁止看见未来 token。这个关系式把 Decoder-only Transformer 的骨架放回模型计算图。它帮助读者追踪 token、张量形状和参数规模如何进入训练或推理成本。在模型结构实验中，loss、梯度范数、activation memory 和 ablation delta 都是 proxy。它们帮助定位机制，但不能单独证明某个结构在所有规模上更优。

4.4 从 Decoder-only Transformer 的骨架到程序

把 Decoder-only Transformer 的骨架写成程序后，抽象机制会变成输入、状态和输出之间的变换。下面的伪代码保留核心计算顺序。

```
x = token_embedding(tokens)
for block in transformer_blocks:
    x = x + causal_attention(norm(x))
    x = x + mlp(norm(x))
logits = lm_head(norm(x))
```

阅读这段实现时，重点追踪 token、activation 和参数如何穿过计算图。sequence length、hidden size、head 数、activation size 和训练稳定性不是附加统计，而是解释训练稳定性和推理成本的必要坐标。

4.5 边界条件与常见误解

- 公开 slides 总结的是主流设计，不代表所有 frontier models 的真实 recipe。
- architecture comparison 必须控制参数量、训练 tokens、数据和 optimizer，否则容易把 scaling 效应误判为结构效应。
- 结构消融常与训练 recipe 耦合；一个 block 在小模型上有效，不代表在长上下文或不同数据分布下仍有效。
- 张量形状和数值稳定性是模型机制的一部分，忽略它们会把实现问题误读成算法问题。

4.6 与本章其他概念的关系

Decoder-only Transformer 的骨架连接本章的表示选择和训练行为：定义给出张量对象，公式描述计算关系，伪代码说明实现路径，例题检验形状、稳定性和成本之间的权衡。

4.7 怎样检验 Decoder-only Transformer 的骨架

检验 Decoder-only Transformer 的骨架时，先把抽象说法落到张量、token 和计算图状态上。与 architecture、Transformer、attention、MLP、residual 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让 sequence length、hidden size、head 数、activation size 和训练稳定性中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： $\backslash(Q,K,V\backslash)$ 是 query/key/value； $\backslash(d_k\backslash)$ 是 head dimension； $\backslash(M\backslash)$ 是 causal mask，用于禁止看见未来 token。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。模型消融通常一次只改变 normalization、activation、position encoding、head layout 或学习率 schedule。若结构和训练 recipe 同时改变，结果只能作为候选经验。最后检查失败条件。工程判断往往在反例中变清楚：公开 slides 总结的是主流设计，不代表所有 frontier models 的真实 recipe；architecture comparison 必须控制参数量、训练 tokens、数据和 optimizer，否则容易把 scaling 效应误判为结构效应。复习时可以把这些限制改写成反例测试，观察它们怎样改变 sequence length、hidden size、head 数、activation size 和训练稳定性。把实验结论放回现代 Transformer architecture 与 hyperparameters 时，要说明 Decoder-only Transformer 的骨架改变的是表示、计算图、梯度路径还是 activation memory。能追踪这些对象，才说明读者真正理解模型机制。

4.8 概念辨析：Decoder-only Transformer 的骨架

architecture、Transformer、attention、MLP、residual 常一起出现在模型结构讨论中，但它们分别指表示、算子、参数或状态。读者应先把它们放回计算图。其中，Transformer 是现代 LM 的基本计算骨架；attention（注意力）提供 token 间交互，但长上下文下成本很高。区分这些词时，可以先问它们是否改变 sequence length、hidden size、head 数、activation size 和训练稳定性。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕 Decoder-only Transformer 的骨架复习时，可以写一个最小 forward pass：标出每个张量形状、状态更新和可能的数值问题。

4.9 Decoder-only Transformer 的骨架的小结

- 讨论 Decoder-only Transformer 的骨架时，应同时报告问题规模、实验设置和主要观测变量，尤其是 sequence length、hidden size、head 数、activation size 和训练稳定性。
- 如果方法声称结构更优，要说明它改变的是表达能力、梯度路径、activation memory、KV cache 还是训练稳定性。
- 模型机制需要通过形状推导、训练曲线和消融实验共同验证，不能只依赖 block diagram。

5 Normalization、activation 与 position encoding

现在进入 Normalization、activation 与 position encoding。这一节把模型结构还原成表示、参数、activation 和训练动态之间的关系，而不是停留在结构名称。

5.1 为什么要讨论 Normalization、activation 与 position encoding

现代 LLM 常用 RMSNorm 或类似变体以降低开销并改善稳定性。RoPE 把位置信息编码进 query/key 的旋转关系，支持相对位置信息并常见于长上下文模型。SwiGLU/GeGLU 等 gated MLP 变体体现了课程第一讲的 caveat：有些设计先来自实验，再由经验固化。Normalization、activation 与 position encoding 把现代 Transformer architecture 与 hyperparameters 连接到计算图。接下来要看的不是结构名称，而是 token、activation、参数和梯度如何流动。

5.2 Normalization、activation 与 position encoding 的机制

现代 LLM 常用 RMSNorm 或类似变体以降低开销并改善稳定性。在 Normalization、activation 与 position encoding 中，核心对象是张量、token 和计算图状态：模型机制必须同时说明表示形态、张量形状和计算代价。RoPE 把位置信息编码进 query/key 的旋转关系，支持相对位置信息并常见于长上下文模型。因而同一个方法在不同规模下可能改变不同的变量，尤其是 sequence length、hidden size、head 数、activation size 和训练稳定性。比较方法时，必须同时给出问题规模和实验条件。SwiGLU/GeGLU 等 gated MLP 变体体现了课程第一讲的 caveat：有些设计先来自实验，再由经验固化。可检验性来自形状推导、训练稳定性曲线和结构消融；只画 block diagram 不能说明机制已经成立。因此，Normalization、activation 与 position encoding 的学习目标是建立一条计算图解释：读者应能从输入 token 追到 activation、loss 和更新，而不是只记住结构名称。

5.3 Normalization、activation 与 position encoding 的数学表达

$$\text{RMSNorm}(x) = g \odot \frac{x}{\sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}}$$

符号说明： x 是 hidden vector， g 是可学习缩放， d 是 hidden dimension， ϵ 防止除零。这个关系式把 Normalization、activation 与 position encoding 放回模型计算图。它帮助读者追踪 token、张量形状和参数规模如何进入训练或推理成本。在模型结构实验中，loss、梯度范数、activation memory 和 ablation delta 都是 proxy。它们帮助定位机制，但不能单独证明某个结构在所有规模上更优。

5.4 从 Normalization、activation 与 position encoding 到程序

把 Normalization、activation 与 position encoding 写成程序后，抽象机制会变成输入、状态和输出之间的变换。下面的伪代码保留核心计算顺序。

```
def rms_norm(x, weight, eps=1e-6):
    scale = rsqrt(mean(x * x, dim=-1, keepdim=True) + eps)
    return weight * x * scale
```

阅读这段实现时，重点追踪 token、activation 和参数如何穿过计算图。sequence length、hidden size、head 数、activation size 和训练稳定性不是附加统计，而是解释训练稳定性和推理成本的必要坐标。

5.5 边界条件与常见误解

- 归一化改善优化，但也可能改变表示尺度和 residual stream 的解释方式。
- 长上下文 extrapolation 不能只靠位置编码，还受训练数据、attention implementation 和评估分布影响。
- 结构消融常与训练 recipe 耦合；一个 block 在小模型上有效，不代表在长上下文或不同数据分布下仍有效。
- 张量形状和数值稳定性是模型机制的一部分，忽略它们会把实现问题误读成算法问题。

5.6 与本章其他概念的关系

Normalization、activation 与 position encoding 连接本章的表示选择和训练行为：定义给出张量对象，公式描述计算关系，伪代码说明实现路径，例题检验形状、稳定性和成本之间的权衡。

5.7 怎样检验 Normalization、activation 与 position encoding

检验 Normalization、activation 与 position encoding 时，先把抽象说法落到张量、token 和计算图状态上。与 RMSNorm、RoPE、SwiGLU、pre-norm、position 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让 sequence length、hidden size、head 数、activation size 和训练稳定性中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： $\mathbf{x}$ 是 hidden vector， g 是可学习缩放， d 是 hidden dimension， ϵ 防止除零。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。模型消融通常一次只改变 normalization、activation、position encoding、head layout 或学习率 schedule。若结构和训练 recipe 同时改变，结果只能作为候选经验。最后检查失败条件。工程判断往往在反例中变清楚：归一化改善优化，但也可能改变表示尺度和 residual stream 的解释方式；长上下文 extrapolation 不能只靠位置编码，还受训练数据、attention implementation 和评估分布影响。复习时可以把这些限制改写成反例测试，观察它们怎样改变 sequence length、hidden size、head 数、activation size 和训练稳定性。把实验结论放回现代 Transformer architecture 与 hyperparameters 时，要说明 Normalization、activation 与 position encoding 改变的是表示、计算图、梯度路径还是 activation memory。能追踪这些对象，才说明读者真正理解模型机制。

5.8 概念辨析：Normalization、activation 与 position encoding

RMSNorm、RoPE、SwiGLU、pre-norm、position 常一起出现在模型结构讨论中，但它们分别指表示、算子、参数或状态。读者应先把它们放回计算图。其中，normalization（归一化）影响优化稳定性和 residual path；RoPE（rotary positional embedding）把位置信息编码进 query/key 的旋转关系。区分这些词时，可以先问它们是否改变 sequence length、hidden size、head 数、activation size 和训练稳定性。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕 Normalization、activation 与 position encoding 复习时，可以写一个最小 forward pass：标出每个张量形状、状态更新和可能的数值问题。

5.9 Normalization、activation 与 position encoding 的小结

- 讨论 Normalization、activation 与 position encoding 时，应同时报告问题规模、实验设置和主要观测变量，尤其是 sequence length、hidden size、head 数、activation size 和训练稳定性。
- 如果方法声称结构更优，要说明它改变的是表达能力、梯度路径、activation memory、KV cache 还是训练稳定性。
- 模型机制需要通过形状推导、训练曲线和消融实验共同验证，不能只依赖 block diagram。

6 Hyperparameter 的规模化视角

现在进入 Hyperparameter 的规模化视角。这一节把算法放到硬件和运行时中考察：同一个数学表达式在不同 batch、shape、GPU 拓扑和 kernel 实现下会得到不同的成本账本。

6.1 为什么要讨论 Hyperparameter 的规模化视角

lecture 3 的核心态度是：不要把 hyperparameters 当成小模型实验后的装饰参数，它们决定大规模训练的稳定性 and 资源效率。batch size、learning rate、warmup、decay、weight decay、dropout、context length 和 width/depth ratio 都进入 compute-budget 约束。本书需要把每个超参数和其 failure mode 绑定：divergence、undertraining、overfitting、memory blow-up、communication bottleneck。Hyperparameter 的规模化视角把现代 Transformer architecture 与 hyperparameters 放进资源账本。接下来要看的不是单个 API，而是张量形状、内存层级、通信和调度怎样共同决定成本。

6.2 Hyperparameter 的规模化视角的机制

lecture 3 的核心态度是：不要把 hyperparameters 当成小模型实验后的装饰参数，它们决定大规模训练的稳定性 and 资源效率。在 Hyperparameter 的规模化视角中，核心对象是硬件和系统状态：同一算法会因为 memory hierarchy、通信拓扑或 kernel shape 不同而表现出完全不同的速度。batch size、learning rate、warmup、decay、weight decay、dropout、context length 和 width/depth ratio 都进入 compute-budget 约束。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。本书需要把每个超参数和其 failure mode 绑定：divergence、undertraining、overfitting、memory blow-up、communication bottleneck。可检验性来自 profile trace、roofline 估算和端到端延迟分解；如果这些证据互相矛盾，就要先解释 measurement setup。因此，Hyperparameter 的规模化视角的学习目标是建立一张成本地图：读者应能指出瓶颈在计算、内存、通信还是调度，并能设计一个小 benchmark 去验证。

6.3 Hyperparameter 的规模化视角的数学表达

$$\theta_{t+1} = \theta_t - \eta_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} - \eta_t \lambda \theta_t$$

符号说明：这是 AdamW 风格更新； η_t 是学习率， $\hat{m}_t, \hat{v}_t$ 是矩估计， λ 是 decoupled weight decay。这个关系式应被读成 Hyperparameter 的规模化视角的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。在系统实验中，tokens/s、TTFT、显存峰值和 kernel time 都是 proxy。它们各自只观察一部分系统，因此报告时要同时写清 workload shape、warmup、同步方式和硬件型号。

6.4 从 Hyperparameter 的规模化视角到程序

把 Hyperparameter 的规模化视角写成程序时，最重要的是暴露状态变化和 data 移动。下面的伪代码保留执行顺序，省略框架样板。

```
for run in sweep:
    train_small_proxy(run)
    reject_if_unstable_or_inefficient(run)
    scale_only_after_resource_accounting(run)
```

阅读这段实现时，重点看 Hyperparameter 的规模化视角怎样移动张量、占用显存并触发通信。实验记录至少要包含吞吐、延迟、显存峰值、通信量和 kernel 利用率，否则很难判断瓶颈来自算子、内存层级还是调度。

6.5 边界条件与常见误解

- 小规模最优超参数不一定可直接外推。

- hyperparameter search 的成本本身也是 scaling plan 的一部分。
- 小规模 kernel benchmark 不一定预测端到端训练或 serving latency，因为调度、通信和请求形状会改变瓶颈。
- 硬件规格表和框架 profiler 都需要结合 workload 解释，不能把单个峰值数字当作机制证明。

6.6 与本章其他概念的关系

Hyperparameter 的规模化视角连接本章的数学机制和工程成本：术语表说明对象，公式估算资源，伪代码暴露状态变化，例题则让读者检查一次完整的成本推理。

6.7 怎样检验 Hyperparameter 的规模化视角

检验 Hyperparameter 的规模化视角时，先把抽象说法落到硬件和系统状态上。与 hyperparameter、learning rate、AdamW、warmup、scale 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明：这是 AdamW 风格更新； η_t 是学习率， $\hat{m}_t, \hat{v}_t$ 是矩估计， λ 是 decoupled weight decay。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。系统消融通常一次只改变 batch shape、sequence length、GPU 数、kernel 实现或调度策略。若同时改变硬件和 workload，实验只能说明端到端表现不同，不能定位瓶颈。最后检查失败条件。工程判断往往在反例中变清楚：小规模最优超参数不一定可直接外推；hyperparameter search 的成本本身也是 scaling plan 的一部分。复习时可以把这些限制改写成反例测试，观察它们怎样改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。把实验结论放回现代 Transformer architecture 与 hyperparameters 时，要说明 Hyperparameter 的规模化视角改变的是训练吞吐、推理延迟、显存峰值还是通信开销。能做出这种归因，才说明读者已经从接口使用进入系统解释。

6.8 概念辨析：Hyperparameter 的规模化视角

hyperparameter、learning rate、AdamW、warmup、scale 常一起出现在系统讲解中，但它们分别指硬件对象、程序操作、瓶颈来源或测量指标。读者应先定位每个词在执行路径中的位置。区分这些词时，可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕 Hyperparameter 的规模化视角复习时，可以写一个小 benchmark：固定输入形状，改变一个系统变量，记录 profile trace 和端到端时间。

6.9 Hyperparameter 的规模化视角的小结

- 讨论 Hyperparameter 的规模化视角时，应同时报告问题规模、实验设置和主要观测变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。
- 如果方法声称更快，要区分计算减少、内存访问减少、通信隐藏和调度改善；这些机制不能用一个 tokens/s 数字互相替代。
- profile trace、roofline 估算和端到端 benchmark 应互相校验，单独一个指标不足以解释系统瓶颈。

7 综合讨论：从局部机制到完整系统

本章的主题是现代 Transformer architecture 与 hyperparameters，但它不应被读成几个相邻知识点的拼接。Decoder-only Transformer 的骨架、Normalization、activation 与 position encoding、Hyperparameter 的规模化视角共同回答同一个问题：语言模型系统中哪些对象可以被定义，哪些成本可以被估算，哪些结论必须通过实验或评估来确认。这些对象包括 Transformer architecture (Transformer 架构)、pre-norm (前归一化)、RMSNorm、RoPE (rotary positional embedding)、SwiGLU、hyperparameter (超参数)。读者可以把它们看成一组接口：有的接口连接文本和 token，有的连接模型结构和张量计算，有的连接数据样本和训练目标，有的连接离线评估和在线服务。接口一旦改变，后面的成本、错误类型和优化空间也会随之改变。

7.1 Decoder-only Transformer 的骨架在系统中的位置

Decoder-only Transformer 的骨架首先提供一个局部解释：视频把 architecture 讲成现代论文中大量细节的集合，而不是单一的 Transformer block 图。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，Decoder-only Transformer 的骨架会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，Decoder-only Transformer 的骨架不能脱离边界条件理解。一个关键 caveat 是：公开 slides 总结的是主流设计，不代表所有 frontier models 的真实 recipe。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.2 Normalization、activation 与 position encoding 在系统中的位置

Normalization、activation 与 position encoding 首先提供一个局部解释：现代 LLM 常用 RMSNorm 或类似变体以降低开销并改善稳定性。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，Normalization、activation 与 position encoding 会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，Normalization、activation 与 position encoding 不能脱离边界条件理解。一个关键 caveat 是：归一化改善优化，但也可能改变表示尺度和 residual stream 的解释方式。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.3 Hyperparameter 的规模化视角在系统中的位置

Hyperparameter 的规模化视角首先提供一个局部解释：lecture 3 的核心态度是：不要把 hyperparameters 当成小模型实验后的装饰参数，它们决定大规模训练的稳定性和资源效率。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，Hyperparameter 的规模化视角会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，Hyperparameter 的规模化视角不能脱离边界条件理解。一个关键 caveat 是：小规模最优超参数不一定可直接外推。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.4 把本章知识用于读论文和复现实验

读相关论文时，可以把本章变成一个检查表。第一，论文是否清楚定义了输入规模、模型规模、数据来源和评估协议；第二，论文声称的改进落在哪个变量上；第三，作者是否报告了会让结论失效的设置；第四，实验是否能分离模型结构、数据质量、优化设置和系统实现的影响。复现实验时，也应避免直接追求大规模。先用小程序复现一个公式、一个伪代码分支或一个 profile 现象，再扩大输入规模。这样做的原因很简单：如果小规模程序无法解释中间量，大规模训练只会把错误隐藏在日志和随机波动里。把这些检查合在一起，本章给出的不是一个固定答案，而是一种阅读 CS336 的方法：每个模型机制都要落到变量，每个变量都要落到测量，每个测量都要说明协议，每个协议都要承认边界。因此，本章中的任何一句结论都可以被改写成一个可引用的完整句式：在给定数据、模型、硬件和评估协议下，某个机制改变了某个变量，并通过某个观测指标表现出来。这样的句子比单独背诵术语更长，但它包含了自学、复现和引用时真正需要的信息。把本章用于自学时，可以按三遍阅读。第一遍只追踪对象和定义，确认每个术语到底指向文本、token、张量、样本、请求还是评估记录。第二遍追踪公式和伪代码，确认每一步改变了哪些状态。第三遍追踪实验和 caveat，确认哪些结论只在特定规模、数据或硬件条件下成立。把本章用于复习时，则应反过来操作：先从一个失败案例或异常指标出发，再回到相应机制。loss 曲线异常可能指向数据、优化或模型容量；吞吐异常可能指向 kernel、通信或调度；benchmark 异常可能指向 contamination、prompt protocol 或采样设置。能够从异常反推机制，是掌握本章的实用标准。

8 例题：把概念变成可计算检查

8.1 Pre-norm block 的信息路径

比较 post-norm 和 pre-norm residual block：二者都有 attention/MLP，但 normalization 放置不同。

- pre-norm 写作 ' $x \leftarrow x + f(\text{norm}(x))$ '，主 residual path 不直接穿过 normalization。
- post-norm 写作 ' $x \leftarrow \text{norm}(x + f(x))$ '，每层都会重标定 residual stream。
- 大模型训练中，pre-norm 常被偏好，因为梯度和 residual 信息路径更稳定。

结论。架构细节要从优化稳定性、表示尺度和实现成本一起解释，不能只画 block 图。这个例题的目的不是替代完整工程实现，而是给出一种最小推理形式：把问题转成变量，写出近似公式或伪代码，再说明近似何时失效。

9 实验设计：从小例子到可复现结论

学习现代 Transformer architecture 与 hyperparameters 不能停在概念层。一个教材化结论至少需要四件事：可构造的输入、可观测的中间量、可比较的指标，以及清楚的失败条件。下面的实验设计不是要求训练大模型，而是要求读者用小规模程序复现本章机制。

9.1 最小输入

先构造只触发本章核心机制的小输入。例如 tokenizer 章节可以用几个包含 rare words 和 Unicode 字符的字符串；GPU 章节可以用一个 elementwise kernel 和一个 GEMM；data 章节可以用十几篇近重复文档。小输入的价值在于它让错误可见。

9.2 资源账本

对同一输入写下 FLOPs、bytes moved、显存峰值、通信量或样本数量的估算。估算不需要一开始就精确，但必须说明单位和近似假设。随后用 profiler、benchmark、validation loss 或人工检查去验证估算是否同量级。

9.3 单变量消融

一次只改变一个因素，例如 vocabulary size、head 数、batch size、filter threshold、reward scale、decoding temperature 或 GPU 数量。若多个变量同时变化，实验只能说明系统变了，不能说明机制是什么。

9.4 失败条件

最后要刻意触发失败：极端 context length、错误 dtype、过小 batch、污染 benchmark、低质量数据或不可靠 verifier。失败条件常常比成功曲线更能说明一个方法的边界。

9.5 记录与报告

实验记录应能让另一个读者复现同一结论。最少要写清楚输入构造、代码版本、随机种子、硬件、batch shape、数据版本、评价指标和异常处理方式。报告结论时不要只写“更快”或“更好”，而要说明快在哪里、好在哪个指标上，以及是否牺牲了内存、稳定性、数据质量或可解释性。常见报告错误是把局部现象写成普遍规律。例如一次小模型实验里的 loss 改善，不等于更大模型或不同数据分布上也会改善；一次 kernel benchmark 的吞吐提升，不等于端到端 serving latency 一定降低。教材中的实验报告应始终把结论限定在可复现条件内。

9.6 从小实验到大结论

小实验的作用不是替代课程中的完整训练或系统实现，而是建立一条可检查的推理链。先在小输入上确认变量方向，再在中等规模上确认数量级，最后才讨论是否能外推到更大模型、更多 token 或更复杂 workload。缺少中间层级时，大结论通常只是在复述直觉。因此，实验报告最好同时包含正例和反例：正例说明机制在受控条件下怎样工作，反例说明条件稍变时哪里失效。对 CS336 这样的课程，反例尤其重要，因为 language model 的错误常来自多个层面叠加：数据分布、优化稳定性、硬件瓶颈、评估协议和服务负载都可能同时改变观察结果。还要记录资料版本与复现边界。公开视频、课程页、配套代码和 PDF 可能在不同时间更新；复现实验应写明使用的材料版本，并说明哪些结论来自课程事实，哪些是为了帮助理解而加入的解释性推导。这样才能把学习笔记变成可检查的技术记录。若复现实验与课程结论不一致，首先检查输入、版本和评估协议，而不是立即修改模型假设。

9.7 把本章重新读成一条证据链

完成小实验之后，可以把现代 Transformer architecture 与 hyperparameters 重新读成一条证据链：先用“Decoder-only Transformer 的骨架”确定问题入口，再用“Normalization、activation 与 position encoding”解释主要机制，最后用“Hyperparameter 的规模化视角”检查边界或外推条件。这种读法比按页背诵更接近教材训练，因为它要求每个结论都能说明前提、变量和证据。本章反复使用的术语包括 Transformer architecture (Transformer 架构)、pre-norm (前归一化)、RMSNorm、RoPE (rotary positional embedding)、SwiGLU、hyperparameter (超参数)。复习时不要只写定义，而要为每个术语补上一句操作性说明：它在课程代码、公式、数据记录或评估协议中对应什么对象；如果这个对象被删除、替换或缩放，哪一个观测量会变化。最后，用一个反例结束复习。反例可以是资源估算不准、数据污染、reward 失真、benchmark 解析失败、长上下文显存爆炸或 scaling 外推失效。能解释反例的章节，才真正具备自学和引用价值；不能解释反例的章节，只是把

课程材料换了一种排版。引用本章结论时，还应保留来源边界：哪些说法直接来自视频、课程页、slides 或代码，哪些是为了串联教材叙述而加入的解释性推导。这个边界不会削弱结论，反而让读者知道何处可以复核，何处需要在自己的实验中重新验证。如果后续课程材料更新，也可以沿着同一证据链替换来源，而不必重写整章结构。

10 课程资料与回看路径

教材正文已经把主要概念、公式和实现逻辑组织成连续叙述；本节只提供回到课程材料的阅读路径。读者复习时可以先完成前文练习，再按下面的时间段或资料组核对细节。

10.1 视频回看路径

时间	关键词	复习任务
00:00:05-00:07:01	going, we're, different, there's, then, that's, architecture	回看定义、例子、推导或 caveat 在该段中的位置。
00:21:58-00:29:40	think, very, relu, going, architecture, then, output	回看定义、例子、推导或 caveat 在该段中的位置。
00:43:57-00:51:44	very, your, think, well, ratio, head, heads	回看定义、例子、推导或 caveat 在该段中的位置。
00:59:12-01:06:34	decay, think, very, weight, language, maybe, your	回看定义、例子、推导或 caveat 在该段中的位置。
01:21:33-01:29:08	attention, think, very, your, performance, full, it's	回看定义、例子、推导或 caveat 在该段中的位置。

这些时间段用于定位视频中的讲解顺序。严肃引用时，应回到视频片段确认讲者表述、板书或幻灯片内容。回看视频时，不必逐字抄录字幕。更有效的方法是把每个时间段判定为定义、例子、推导、代码解释或 caveat，再把它放回本章对应小节。

10.2 课程资料阅读路径

资料组	标题/页块	关键词
official_01	Lecture 3 E V E R Y T H I N G Y O U D I D N ' T W A N T T O K N O W A B O U T L M A R C H I T E ...	architecture, many, common, learn, layernorm, architectures, modern
official_02	Pre-vs-post-norm, the data Salazar and Ngyuen 2019 Figure from Xiong 2020	rmsnorm, notable, norm, bias, most, gated, original
official_03	Do gated linear units work? Yes, fairly consistently so. Shazeer 2020	embeddings, position, rope, attention, embedding, relative, layers
official_05	Evidence on aspect ratio scaling [Kaplan et al 2020] [Tay et al 2021]	attention, head, dropout, weight, decay, softmax, total

课程资料通常给出更稳定的标题、公式、图或代码结构；视频则补充动机和口头解释。两者合起来构成本章的事实边界。阅读资料时，可以把每个资料组拆成三个对象：一个定义，一个公式或伪代码动作，一个边界条件。这样比逐字翻译页面或脚本更容易形成可用知识。

11 实践说明、历史位置与风险

现代 Transformer architecture 与 hyperparameters 在 CS336 的课程结构中连接基础机制和规模化问题。基础机制解释方法为什么成立；规模化问题检验它在更大模型、更长上下文、更复杂数据或真实 serving workload 下是否仍成立。历史位置不等于模型年表。更重要的是识别问题如何迁移：早期方法解决小规模建模问题，现代方法常常解决同一问题在大规模数据、GPU 集群、长上下文或人类偏好约束下的新形态。

- 资料版本限制 (version risk): 课程页、公开视频和配套资料可能在学期中更新；引用时应注明访问日期或资料版本。
- 测量口径限制 (measurement risk): FLOPs、tokens/s、benchmark score、reward 等数字只有在协议完整时可比较。
- 规模外推限制 (scaling risk): 小模型或小 batch 的机制不必然外推到 frontier-scale。
- 数据分布限制 (data risk): 数据来源、过滤和去重策略会改变模型行为，也会改变 evaluation contamination 风险。

12 复习要点

下面的问题把本章内容压缩为可反复检查的条目。每个条目都应能回到前文的解释、公式、伪代码或例题。

条目	对象	检查动作
条目 1	Transformer architecture (Transformer 架构)	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。
条目 2	pre-norm (前归一化)	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。
条目 3	RMSNorm	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。
条目 4	RoPE (rotary positional embedding)	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。
条目 5	SwiGLU	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。

12.1 综合自测

- 我能否不用课程原句, 独立解释本章中心问题?
- 我能否把本章至少一个公式改写成可运行的估算函数?
- 我能否指出一个看似改进但实际转移成本的方法?
- 我能否回到视频和课程资料, 找到支持本章关键论断的位置?
- 我能否设计一个最小 ablation, 验证本章一个 caveat?

13 总结与延伸

13.1 本章小结

- 现代 Transformer architecture 与 hyperparameters 的核心不是记忆术语, 而是理解对象、变量和机制之间的关系。
- 视频给出讲解顺序, 课程资料提供可核对的公式、图、代码或页面结构。
- 复习时应把每个术语连接到至少一个变量、一个实现动作和一个失败模式。

13.2 延伸解释

本章中的延伸解释用于连接课程材料中的概念, 例如说明公式里的 proxy、解释系统 tradeoff, 或把多个材料片段串成一个完整推理链。若需要严肃引用, 应回到课程视频和配套资料核对原始表述。

13.3 练习

- 定义题: 用中英双语解释本章任意五个重要术语, 并说明它们属于 compute、memory、data、optimization、evaluation 还是 deployment 账本。
- 推导题: 选择本章一个公式, 逐个解释符号、单位和近似假设, 并说明哪个变量最难在真实系统中测量。
- 实现题: 把本章伪代码改写成一个最小 Python sanity check, 不要求训练大模型, 但要输出可检查的中间量。
- 资料题: 从“课程资料与回看路径”中选择一个视频时间段, 回到原始视频核对讲者例子, 并记录是否存在字幕误差。
- 评估题: 为本章方法设计一个 ablation 或 benchmark, 说明控制变量、数据版本、指标和 failure criteria。
- 边界题: 说明本章一个结论在更长 context、更大 batch、更多 GPU 或更复杂数据分布下可能如何失效。
- 综合题: 把本章内容和前一章或后一章连接起来, 写出一个端到端训练/推理 pipeline 中的依赖关系。
- 批判题: 指出一个容易被营销材料夸大的说法, 并用本章的资源账本或评估协议反驳它。

4 04 Attention alternatives 与 Mixture of Experts

第 04 讲: Attention alternatives 与 Mixture of Experts

CS336 Spring 2026 public videos and official course materials

目录

1 本章导读	2
1.1 本章知识路线	2
1.2 结构图	2
2 本章主线	2
2.1 本章要解决的核心问题	3
2.2 从机制到资源账本	3
2.3 从课堂讲解到可复现实验	3
3 术语、符号与问题边界	4
3.1 术语之间的关系	4
4 为什么要替代标准 attention	4
4.1 课程目标与问题边界	4
4.2 为什么要替代标准 attention 的机制	5
4.3 资源、效率与效果的关系	5
4.4 把课程目标写成检查流程	5
4.5 边界条件与常见误解	5
4.6 与本章其他概念的关系	6
4.7 怎样检验这一课程目标	6
4.8 概念辨析: 为什么要替代标准 attention	6
4.9 为什么要替代标准 attention 的小结	6
5 Linear attention、state-space 与 recurrence	6
5.1 为什么要讨论 Linear attention、state-space 与 recurrence	7
5.2 Linear attention、state-space 与 recurrence 的机制	7
5.3 Linear attention、state-space 与 recurrence 的形式化表示	7
5.4 从 Linear attention、state-space 与 recurrence 到程序	7
5.5 边界条件与常见误解	7

5.6	与本章其他概念的关系	8
5.7	怎样检验 Linear attention、state-space 与 recurrence	8
5.8	概念辨析：Linear attention、state-space 与 recurrence	8
5.9	Linear attention、state-space 与 recurrence 的小结	8
6	MoE 的稀疏计算与路由	9
6.1	为什么要讨论 MoE 的稀疏计算与路由	9
6.2	MoE 的稀疏计算与路由的机制	9
6.3	MoE 的稀疏计算与路由的形式化表示	9
6.4	从 MoE 的稀疏计算与路由到程序	9
6.5	边界条件与常见误解	10
6.6	与本章其他概念的关系	10
6.7	怎样检验 MoE 的稀疏计算与路由	10
6.8	概念辨析：MoE 的稀疏计算与路由	10
6.9	MoE 的稀疏计算与路由的小结	10
7	综合讨论：从局部机制到完整系统	11
7.1	为什么要替代标准 attention 在系统中的位置	11
7.2	Linear attention、state-space 与 recurrence 在系统中的位置	11
7.3	MoE 的稀疏计算与路由在系统中的位置	11
7.4	把本章知识用于读论文和复现实验	12
8	例题：把概念变成可计算检查	12
8.1	KV cache 与 GQA 的显存差异	12
9	实验设计：从小例子到可复现结论	12
9.1	最小输入	13
9.2	资源账本	13
9.3	单变量消融	13
9.4	失败条件	13
9.5	记录与报告	13
9.6	从小实验到大结论	13
9.7	把本章重新读成一条证据链	13
10	课程资料与回看路径	14
10.1	视频回看路径	14
10.2	课程资料阅读路径	14
11	实践说明、历史位置与风险	15

12 复习要点 15

12.1 综合自测 16

13 总结与延伸 16

13.1 本章小结 16

13.2 延伸解释 16

13.3 练习 16

1 本章导读

本章讨论 **Attention alternatives 与 Mixture of Experts**。CS336 的第一层目标不是把现成大模型当作黑箱使用，而是把 language model 从输入文本、训练目标、模型结构、数据、系统和评估这些部件重新拆开。只有拆开之后，读者才能判断一个设计选择究竟改变了什么。设想用户要求模型读入一本书长度的上下文，同时服务系统还要低延迟。标准 attention 的二次成本和 KV cache 很快变成瓶颈；本章的问题是：哪些替代结构真的改变复杂度，哪些只是在另一个地方付账？本章对应 CS336 Spring 2026 第 04 个公开视频，并结合课程页提供的可解析资料。视频和课程页给出事实边界；正文把这些材料改写为可自学的教材章节。阅读本章时，先理解问题为什么存在，再看定义、公式和实现形态，随后通过例题和练习检查自己是否能独立复现这条 reasoning chain。

Lecture 4

ATTENTION ALTERNATIVES AND MIXTURES OF EXPERTS

CS336
Tatsu H



图 1: 第 04 讲的课程图像锚点。

1.1 本章知识路线

- 本章首先说明 Attention alternatives 与 Mixture of Experts 为什么是 language modeling pipeline 中不可跳过的一环。
- 其次定义本章反复使用的术语，并说明这些术语对应的计算对象或系统对象。
- 随后用公式和伪代码表达主要机制，让概念能够落到可计算的变量上。
- 最后给出例题、风险讨论和练习，便于读者检查自己是否真正掌握了机制。

1.2 结构图

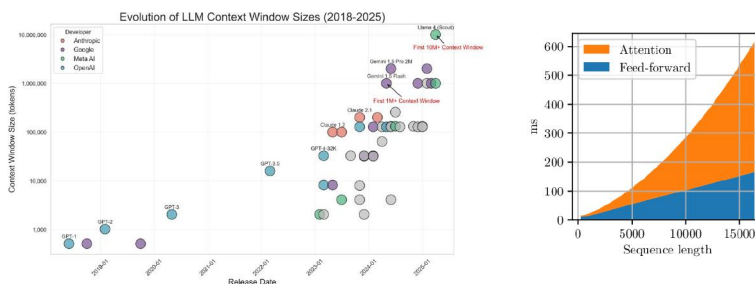
下面的图用于帮助读者定位本章的核心结构。先看概念之间的依赖，再回到正文中的公式、伪代码和例题。

2 本章主线

Attention alternatives 与 Mixture of Experts 的主线是从一个可观察的问题出发，逐步引出表示、目标、实现和实验。读者应把每个机制放回 language modeling pipeline 的位置。

Attention alternatives

Cost of attention rises with large context sizes... how do we control those costs?



<https://www.meibei.ai/post/understanding-the-impact-of-increasing-llm-context-windows>

图 2: 官方材料图页: Attention alternatives 与 Mixture of Experts 的课程页/PDF/script 图像锚点。

Lecture 4

ATTENTION ALTERNATIVES AND MIXTURES OF EXPERTS

CS336

Tatsu H

图 3: 官方材料图页: Attention alternatives 与 Mixture of Experts 的课程页/PDF/script 图像锚点。

2.1 本章要解决的核心问题

本讲围绕两个方向: 降低 attention 长上下文成本, 以及用 MoE 把参数容量和每 token 计算量解耦。读完这一段后, 应能用一两句话说明 Attention alternatives 与 Mixture of Experts 要解决的瓶颈, 并指出这个瓶颈发生在数据、模型、优化、系统还是评估层。

2.2 从机制到资源账本

linear attention、Mamba、Gated Delta Net 等方法的目标是避免完整构造二次复杂度的 attention matrix, 同时保留足够的选择性记忆。随后把机制写成账本: 输入规模是什么, 主要状态是什么, 成本或质量指标怎样随变量改变。这个账本让后面的公式不再只是符号。

2.3 从课堂讲解到可复现实验

MoE 的核心不是简单堆更多参数, 而是 router、expert capacity、load balancing、all-to-all communication 和 active parameters 之间的系统协同。最后把课堂讲解转成可复现实验: 固定协议, 改变一个变量, 记录中

间量，并解释结果为何支持或反驳原来的机制判断。

3 术语、符号与问题边界

本章的术语横跨模型、数据和系统。读者需要先知道每个词指向的对象：有些词描述数学目标，有些词描述张量形状，有些词描述硬件瓶颈，还有一些词描述评估协议。术语边界不清，后面的公式和实现就会变成空洞的缩写。

#	术语	阅读要求
1	attention alternatives（注意力替代机制）	说明它在计算图中的位置、输入输出形状，以及改变它会影响的训练或推理成本。
2	Grouped-Query Attention (GQA)	说明它在计算图中的位置、输入输出形状，以及改变它会影响的训练或推理成本。
3	Multi-Query Attention (MQA)	说明它在计算图中的位置、输入输出形状，以及改变它会影响的训练或推理成本。
4	linear attention（线性注意力）	说明它在计算图中的位置、输入输出形状，以及改变它会影响的训练或推理成本。
5	Mixture of Experts (MoE)	说明它如何改变训练样本或 token 分布，并给出一个会改变结论的数据边界条件。
6	routing（路由）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。

这些术语之间有依赖关系。例如 tokenization 改变 sequence length，sequence length 又改变 attention FLOPs、KV cache 和 evaluation token budget；parallelism 改变显存占用，也改变通信瓶颈和故障恢复方式。

3.1 术语之间的关系

在本章中，attention alternatives（注意力替代机制）给出问题入口，Grouped-Query Attention (GQA) 描述主要机制，Multi-Query Attention (MQA) 则把机制连接到可测量的结果。读教材时应当沿着这个链条追问：输入是什么，中间状态怎样改变，输出指标为什么随之变化。课程材料还会反复出现 moes、deepseek、attention、routing、experts、linear 等词。它们不是一组同义标签，而是提示本章的不同层次：有的指对象，有的指操作，有的指约束或指标。因此，术语表不是背诵清单，而是后文阅读的索引。遇到一个术语时，先定位它属于 representation、optimization、systems、data 还是 evaluation，再判断它改变哪一项账本。

4 为什么要替代标准 attention

本节讨论为什么要替代标准 attention。它先限定 Attention alternatives 与 Mixture of Experts 要回答的基本问题：哪些现象需要解释，哪些变量可以测量，哪些结论必须回到吞吐、延迟、显存峰值、通信量和 kernel 利用率中检验。

4.1 课程目标与问题边界

视频把本讲定位为更高级的 architecture ideas：上一讲讨论主流 Transformer，本讲讨论 attention 的成本和替代机制。标准 attention 的 $\mathcal{O}(T^2)$ 交互在长上下文、prefill、训练显存和推理 KV cache 中都昂贵。

GQA/MQA 减少 key/value head 数量，目标是在质量下降可控的情况下降低推理带宽和 cache 占用。为什么要替代标准 attention 把 Attention alternatives 与 Mixture of Experts 放进资源账本。接下来要看的不是单个 API，而是张量形状、内存层级、通信和调度怎样共同决定成本。

4.2 为什么要替代标准 attention 的机制

视频把本讲定位为更高级的 architecture ideas：上一讲讨论主流 Transformer，本讲讨论 attention 的成本和替代机制。在为什么要替代标准 attention 中，核心对象是硬件和系统状态：同一算法会因为 memory hierarchy、通信拓扑或 kernel shape 不同而表现出完全不同的速度。标准 attention 的 $\mathcal{O}(T^2)$ 交互在长上下文、prefill、训练显存和推理 KV cache 中都昂贵。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。GQA/MQA 减少 key/value head 数量，目标是在质量下降可控的情况下降低推理带宽和 cache 占用。可检验性来自 profile trace、roofline 估算和端到端延迟分解；如果这些证据互相矛盾，就要先解释 measurement setup。因此，为什么要替代标准 attention 的学习目标是建立一张成本地图：读者应能指出瓶颈在计算、内存、通信还是调度，并能设计一个小 benchmark 去验证。

4.3 资源、效率与效果的关系

$$\text{KV cache bytes} \approx 2 \cdot L \cdot T \cdot H_{kv} \cdot d_{\text{head}} \cdot b$$

符号说明： L 是层数， T 是上下文长度， H_{kv} 是 KV heads， d_{head} 是 head 维度， b 是每元素字节数。这个关系式应被读成为什么要替代标准 attention 的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。在系统实验中，tokens/s、TTFT、显存峰值和 kernel time 都是 proxy。它们各自只观察一部分系统，因此报告时要同时写清 workload shape、warmup、同步方式和硬件型号。

4.4 把课程目标写成检查流程

把为什么要替代标准 attention 写成程序时，最重要的是暴露状态变化和数据移动。下面的伪代码保留执行顺序，省略框架样板。

```
if kv_cache_is_bottleneck:
    share_key_value_heads_with_gqa()
else:
    preserve_full_multi_head_attention()
```

阅读这段实现时，重点看为什么要替代标准 attention 怎样移动张量、占用显存并触发通信。实验记录至少要包含吞吐、延迟、显存峰值、通信量和 kernel 利用率，否则很难判断瓶颈来自算子、内存层级还是调度。

4.5 边界条件与常见误解

- KV head 共享降低内存，但可能损失表示能力。
- attention 替代方法要同时看训练吞吐、推理延迟和模型质量。
- 小规模 kernel benchmark 不一定预测端到端训练或 serving latency，因为调度、通信和请求形状会改变瓶颈。
- 硬件规格表和框架 profiler 都需要结合 workload 解释，不能把单个峰值数字当作机制证明。

4.6 与本章其他概念的关系

为什么要替代标准 attention 连接本章的数学机制和工程成本：术语表说明对象，公式估算资源，伪代码暴露状态变化，例题则让读者检查一次完整的成本推理。对这一类主题，最常见的误解是只列方法名。更有用的做法是比较每个方法改变的成本项、依赖的假设和可能的失败模式。

4.7 怎样检验这一课程目标

检验这一课程目标时，先把抽象说法落到硬件和系统状态上。与 attention、GQA、MQA、KV cache、long context 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： $\backslash(L\backslash)$ 是层数， $\backslash(T\backslash)$ 是上下文长度， $\backslash(H_{\{kv\}}\backslash)$ 是 KV heads， $\backslash(d_{\{ \text{ext}\{head\}}\backslash)$ 是 head 维度， $\backslash(b\backslash)$ 是每元素字节数。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。系统消融通常一次只改变 batch shape、sequence length、GPU 数、kernel 实现或调度策略。若同时改变硬件和 workload，实验只能说明端到端表现不同，不能定位瓶颈。最后检查失败条件。工程判断往往在反例中变清楚：KV head 共享降低内存，但可能损失表示能力；attention 替代方法要同时看训练吞吐、推理延迟和模型质量。复习时可以把这些限制改写成反例测试，观察它们怎样改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。把实验结论放回 Attention alternatives 与 Mixture of Experts 时，要说明为什么要替代标准 attention 改变的是训练吞吐、推理延迟、显存峰值还是通信开销。能做出这种归因，才说明读者已经从接口使用进入系统解释。

4.8 概念辨析：为什么要替代标准 attention

attention、GQA、MQA、KV cache、long context 常一起出现在系统讲解中，但它们分别指硬件对象、程序操作、瓶颈来源或测量指标。读者应先定位每个词在执行路径中的位置。其中，attention（注意力）提供 token 间交互，但长上下文下成本很高；KV cache 缓存 key/value，避免重复计算历史上下文。区分这些词时，可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕为什么要替代标准 attention 复习时，可以写一个小 benchmark：固定输入形状，改变一个系统变量，记录 profile trace 和端到端时间。

4.9 为什么要替代标准 attention 的小结

- 讨论为什么要替代标准 attention 时，应同时报告问题规模、实验设置和主要观测变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。
- 如果方法声称更快，要区分计算减少、内存访问减少、通信隐藏和调度改善；这些机制不能用一个 tokens/s 数字互相替代。
- profile trace、roofline 估算和端到端 benchmark 应互相校验，单独一个指标不足以解释系统瓶颈。

5 Linear attention、state-space 与 recurrence

现在进入 Linear attention、state-space 与 recurrence。这一节把模型结构还原成表示、参数、activation 和训练动态之间的关系，而不是停留在结构名称。

5.1 为什么要讨论 Linear attention、state-space 与 recurrence

课程 slides 把 linear attention、Mamba/SSM 等归入 attention alternatives 的讨论范围。核心思想是把全量 pairwise interaction 改写成可递推或可分解的状态更新，从而降低长序列成本。这些方法的关键问题不是复杂度公式本身，而是能否保持语言建模所需的选择性记忆和训练稳定性。Linear attention、state-space 与 recurrence 把 Attention alternatives 与 Mixture of Experts 连接到计算图。接下来要看的不是结构名称，而是 token、activation、参数和梯度如何流动。

5.2 Linear attention、state-space 与 recurrence 的机制

课程 slides 把 linear attention、Mamba/SSM 等归入 attention alternatives 的讨论范围。在 Linear attention、state-space 与 recurrence 中，核心对象是张量、token 和计算图状态：模型机制必须同时说明表示形态、张量形状和计算代价。核心思想是把全量 pairwise interaction 改写成可递推或可分解的状态更新，从而降低长序列成本。因而同一个方法在不同规模下可能改变不同的变量，尤其是 sequence length、hidden size、head 数、activation size 和训练稳定性。比较方法时，必须同时给出问题规模和实验条件。这些方法的关键问题不是复杂度公式本身，而是能否保持语言建模所需的选择性记忆和训练稳定性。可检验性来自形状推导、训练稳定性曲线和结构消融；只画 block diagram 不能说明机制已经成立。因此，Linear attention、state-space 与 recurrence 的学习目标是建立一条计算图解释：读者应能从输入 token 追到 activation、loss 和更新，而不是只记住结构名称。

5.3 Linear attention、state-space 与 recurrence 的形式化表示

$$\text{Attn}(Q, K, V) \approx \phi(Q) (\phi(K)^\top V)$$

符号说明： $\phi(\cdot)$ 是 feature map；近似把 $T \times T$ attention matrix 的构造改成可结合的矩阵乘积。这个关系式把 Linear attention、state-space 与 recurrence 放回模型计算图。它帮助读者追踪 token、张量形状和参数规模如何进入训练或推理成本。在模型结构实验中，loss、梯度范数、activation memory 和 ablation delta 都是 proxy。它们帮助定位机制，但不能单独证明某个结构在所有规模上更优。

5.4 从 Linear attention、state-space 与 recurrence 到程序

把 Linear attention、state-space 与 recurrence 写成程序后，抽象机制会变成输入、状态和输出之间的变换。下面的伪代码保留核心计算顺序。

```
state = zeros()
for token in sequence:
    state = update(state, token)
    output = readout(state, token)
```

阅读这段实现时，重点追踪 token、activation 和参数如何穿过计算图。sequence length、hidden size、head 数、activation size 和训练稳定性不是附加统计，而是解释训练稳定性和推理成本的必要坐标。

5.5 边界条件与常见误解

- 线性复杂度不自动意味着实际更快；kernel、memory access 和 hardware utilization 仍然决定吞吐。
- 递推结构可能更难并行训练，也可能需要特殊初始化或 normalization。
- 结构消融常与训练 recipe 耦合；一个 block 在小模型上有效，不代表在长上下文或不同数据分布下仍有效。

- 张量形状和数值稳定性是模型机制的一部分，忽略它们会把实现问题误读成算法问题。

5.6 与本章其他概念的关系

Linear attention、state-space 与 recurrence 连接本章的表示选择和训练行为：定义给出张量对象，公式描述计算关系，伪代码说明实现路径，例题检验形状、稳定性和成本之间的权衡。对这一类主题，最常见的误解是只列方法名。更有用的做法是比较每个方法改变的成本项、依赖的假设和可能的失败模式。

5.7 怎样检验 Linear attention、state-space 与 recurrence

检验 Linear attention、state-space 与 recurrence 时，先把抽象说法落到张量、token 和计算图状态上。与 linear attention、Mamba、SSM、recurrence、feature map 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让 sequence length、hidden size、head 数、activation size 和训练稳定性中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： $\phi(\cdot)$ 是 feature map；近似把 $(T \times T)$ attention matrix 的构造改成可结合的矩阵乘积。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。模型消融通常一次只改变 normalization、activation、position encoding、head layout 或学习率 schedule。若结构和训练 recipe 同时改变，结果只能作为候选经验。最后检查失败条件。工程判断往往在反例中变清楚：线性复杂度不自动意味着实际更快；kernel、memory access 和 hardware utilization 仍然决定吞吐；递推结构可能更难并行训练，也可能需要特殊初始化或 normalization。复习时可以把这些限制改写成反例测试，观察它们怎样改变 sequence length、hidden size、head 数、activation size 和训练稳定性。把实验结论放回 Attention alternatives 与 Mixture of Experts 时，要说明 Linear attention、state-space 与 recurrence 改变的是表示、计算图、梯度路径还是 activation memory。能追踪这些对象，才说明读者真正理解模型机制。

5.8 概念辨析：Linear attention、state-space 与 recurrence

linear attention、Mamba、SSM、recurrence、feature map 常一起出现在模型结构讨论中，但它们分别指表示、算子、参数或状态。读者应先把它们放回计算图。其中，attention（注意力）提供 token 间交互，但长上下文下成本很高。区分这些词时，可以先问它们是否改变 sequence length、hidden size、head 数、activation size 和训练稳定性。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕 Linear attention、state-space 与 recurrence 复习时，可以写一个最小 forward pass：标出每个张量形状、状态更新和可能的数值问题。

5.9 Linear attention、state-space 与 recurrence 的小结

- 讨论 Linear attention、state-space 与 recurrence 时，应同时报告问题规模、实验设置和主要观测变量，尤其是 sequence length、hidden size、head 数、activation size 和训练稳定性。
- 如果方法声称结构更优，要说明它改变的是表达能力、梯度路径、activation memory、KV cache 还是训练稳定性。
- 模型机制需要通过形状推导、训练曲线和消融实验共同验证，不能只依赖 block diagram。

6 MoE 的稀疏计算与路由

现在进入 MoE 的稀疏计算与路由。这一节把算法放到硬件和运行时中考察：同一个数学表达式在不同 batch、shape、GPU 拓扑和 kernel 实现下会得到不同的成本账本。

6.1 为什么要讨论 MoE 的稀疏计算与路由

MoE 让每个 token 只激活少数 experts，从而在参数量很大时保持每 token compute 相对较低。routing 决定 token 到 expert 的分配；负载均衡、capacity factor、communication 和 expert parallelism 是核心系统问题。MoE 的收益来自参数容量和计算稀疏性的分离，但训练稳定性和 serving 成本会变复杂。MoE 的稀疏计算与路由把 Attention alternatives 与 Mixture of Experts 放进资源账本。接下来要看的不是单个 API，而是张量形状、内存层级、通信和调度怎样共同决定成本。

6.2 MoE 的稀疏计算与路由的机制

MoE 让每个 token 只激活少数 experts，从而在参数量很大时保持每 token compute 相对较低。在 MoE 的稀疏计算与路由中，核心对象是硬件和系统状态：同一算法会因为 memory hierarchy、通信拓扑或 kernel shape 不同而表现出完全不同的速度。routing 决定 token 到 expert 的分配；负载均衡、capacity factor、communication 和 expert parallelism 是核心系统问题。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。MoE 的收益来自参数容量和计算稀疏性的分离，但训练稳定性和 serving 成本会变复杂。可检验性来自 profile trace、roofline 估算和端到端延迟分解；如果这些证据互相矛盾，就要先解释 measurement setup。因此，MoE 的稀疏计算与路由的学习目标是建立一张成本地图：读者应能指出瓶颈在计算、内存、通信还是调度，并能设计一个小 benchmark 去验证。

6.3 MoE 的稀疏计算与路由的形式化表示

$$y = \sum_{e \in \text{TopK}(r(x))} g_e(x) E_e(x)$$

符号说明： $r(x)$ 是 router， E_e 是第 e 个 expert， g_e 是 routing weight，TopK 表示只选少数 experts。这个关系式应被读成 MoE 的稀疏计算与路由的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。在系统实验中，tokens/s、TTFT、显存峰值和 kernel time 都是 proxy。它们各自只观察一部分系统，因此报告时要同时写清 workload shape、warmup、同步方式和硬件型号。

6.4 从 MoE 的稀疏计算与路由到程序

把 MoE 的稀疏计算与路由写成程序时，最重要的是暴露状态变化和数据移动。下面的伪代码保留执行顺序，省略框架样板。

```
scores = router(hidden)
experts = topk(scores, k=2)
for expert in experts:
    dispatch_tokens(expert)
combine_weighted_outputs()
```

阅读这段实现时，重点看 MoE 的稀疏计算与路由怎样移动张量、占用显存并触发通信。实验记录至少要包含吞吐、延迟、显存峰值、通信量和 kernel 利用率，否则很难判断瓶颈来自算子、内存层级还是调度。

6.5 边界条件与常见误解

- MoE 参数量大不等于每 token FLOPs 大；报告模型大小时要区分 total parameters 和 active parameters。
- 路由不均衡会导致部分 experts 过载，吞吐被最慢 shard 拖住。
- 小规模 kernel benchmark 不一定预测端到端训练或 serving latency，因为调度、通信和请求形状会改变瓶颈。
- 硬件规格表和框架 profiler 都需要结合 workload 解释，不能把单个峰值数字当作机制证明。

6.6 与本章其他概念的关系

MoE 的稀疏计算与路由连接本章的数学机制和工程成本：术语表说明对象，公式估算资源，伪代码暴露状态变化，例题则让读者检查一次完整的成本推理。对这一类主题，最常见的误解是只列方法名。更有用的做法是比较每个方法改变的成本项、依赖的假设和可能的失败模式。

6.7 怎样检验 MoE 的稀疏计算与路由

检验 MoE 的稀疏计算与路由时，先把抽象说法落到硬件和系统状态上。与 MoE、expert、router、TopK、capacity 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： $r(x)$ 是 router， E_e 是第 e 个 expert， g_e 是 routing weight，TopK 表示只选少数 experts。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。系统消融通常一次只改变 batch shape、sequence length、GPU 数、kernel 实现或调度策略。若同时改变硬件和 workload，实验只能说明端到端表现不同，不能定位瓶颈。最后检查失败条件。工程判断往往在反例中变清楚：MoE 参数量大不等于每 token FLOPs 大；报告模型大小时要区分 total parameters 和 active parameters；路由不均衡会导致部分 experts 过载，吞吐被最慢 shard 拖住。复习时可以把这些限制改写成反例测试，观察它们怎样改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。把实验结论放回 Attention alternatives 与 Mixture of Experts 时，要说明 MoE 的稀疏计算与路由改变的是训练吞吐、推理延迟、显存峰值还是通信开销。能做出这种归因，才说明读者已经从接口使用进入系统解释。

6.8 概念辨析：MoE 的稀疏计算与路由

MoE、expert、router、TopK、capacity 常一起出现在系统讲解中，但它们分别指硬件对象、程序操作、瓶颈来源或测量指标。读者应先定位每个词在执行路径中的位置。其中，Mixture of Experts (MoE) 用稀疏激活扩展参数容量；expert（专家）是 MoE 中由 router 选择的子网络。区分这些词时，可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕 MoE 的稀疏计算与路由复习时，可以写一个小 benchmark：固定输入形状，改变一个系统变量，记录 profile trace 和端到端时间。

6.9 MoE 的稀疏计算与路由的小结

- 讨论 MoE 的稀疏计算与路由时，应同时报告问题规模、实验设置和主要观测变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。

- 如果方法声称更快，要区分计算减少、内存访问减少、通信隐藏和调度改善；这些机制不能用一个 tokens/s 数字互相替代。
- profile trace、roofline 估算和端到端 benchmark 应互相校验，单独一个指标不足以解释系统瓶颈。

7 综合讨论：从局部机制到完整系统

本章的主题是 Attention alternatives 与 Mixture of Experts，但它不应被读成几个相邻知识点的拼接。为什么要替代标准 attention、Linear attention、state-space 与 recurrence、MoE 的稀疏计算与路由共同回答同一个问题：语言模型系统中哪些对象可以被定义，哪些成本可以被估算，哪些结论必须通过实验或评估来确认。这些对象包括 attention alternatives（注意力替代机制）、Grouped-Query Attention (GQA)、Multi-Query Attention (MQA)、linear attention（线性注意力）、Mixture of Experts (MoE)、routing（路由）。读者可以把它们看成一组接口：有的接口连接文本和 token，有的连接模型结构和张量计算，有的连接数据样本和训练目标，有的连接离线评估和在线服务。接口一旦改变，后面的成本、错误类型和优化空间也会随之改变。

7.1 为什么要替代标准 attention 在系统中的位置

为什么要替代标准 attention 首先提供一个局部解释：视频把本讲定位为更高级的 architecture ideas：上一讲讨论主流 Transformer，本讲讨论 attention 的成本和替代机制。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，为什么要替代标准 attention 会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，为什么要替代标准 attention 不能脱离边界条件理解。一个关键 caveat 是：KV head 共享降低内存，但可能损失表示能力。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.2 Linear attention、state-space 与 recurrence 在系统中的位置

Linear attention、state-space 与 recurrence 首先提供一个局部解释：课程 slides 把 linear attention、Mamba/SSM 等归入 attention alternatives 的讨论范围。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，Linear attention、state-space 与 recurrence 会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，Linear attention、state-space 与 recurrence 不能脱离边界条件理解。一个关键 caveat 是：线性复杂度不自动意味着实际更快；kernel、memory access 和 hardware utilization 仍然决定吞吐。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.3 MoE 的稀疏计算与路由在系统中的位置

MoE 的稀疏计算与路由首先提供一个局部解释：MoE 让每个 token 只激活少数 experts，从而在参数量很大时保持每 token compute 相对较低。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，MoE 的稀疏计算与路由会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实

训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，MoE 的稀疏计算与路由不能脱离边界条件理解。一个关键 caveat 是：MoE 参数量大不等于每 token FLOPs 大；报告模型大小时要区分 total parameters 和 active parameters。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.4 把本章知识用于读论文和复现实验

读相关论文时，可以把本章变成一个检查表。第一，论文是否清楚定义了输入规模、模型规模、数据来源和评估协议；第二，论文声称的改进落在哪个变量上；第三，作者是否报告了会让结论失效的设置；第四，实验是否能分离模型结构、数据质量、优化设置和系统实现的影响。复现实验时，也应避免直接追求大规模。先用最小程序复现一个公式、一个伪代码分支或一个 profile 现象，再扩大输入规模。这样做的原因很简单：如果小规模程序无法解释中间量，大规模训练只会把错误隐藏在日志和随机波动里。把这些检查合在一起，本章给出的不是一个固定答案，而是一种阅读 CS336 的方法：每个模型机制都要落到变量，每个变量都要落到测量，每个测量都要说明协议，每个协议都要承认边界。因此，本章中的任何一句结论都可以被改写成一个可引用的完整句式：在给定数据、模型、硬件和评估协议下，某个机制改变了某个变量，并通过某个观测指标表现出来。这样的句子比单独背诵术语更长，但它包含了自学、复现和引用时真正需要的信息。把本章用于自学时，可以按三遍阅读。第一遍只追踪对象和定义，确认每个术语到底指向文本、token、张量、样本、请求还是评估记录。第二遍追踪公式和伪代码，确认每一步改变了哪些状态。第三遍追踪实验和 caveat，确认哪些结论只在特定规模、数据或硬件条件下成立。把本章用于复习时，则应反过来操作：先从一个失败案例或异常指标出发，再回到相应机制。loss 曲线异常可能指向数据、优化或模型容量；吞吐异常可能指向 kernel、通信或调度；benchmark 异常可能指向 contamination、prompt protocol 或采样设置。能够从异常反推机制，是掌握本章的实用标准。

8 例题：把概念变成可计算检查

8.1 KV cache 与 GQA 的显存差异

比较 full multi-head attention 和 grouped-query attention。假设层数、上下文长度和 head dimension 固定，只减少 KV heads。

- ‘KV cache bytes H_{kv} ’，把 KV heads 从 32 减到 8 会近似把 KV cache 降为四分之一。
- query heads 仍可保持较多，以保留表达能力。
- 质量损失是否可接受必须通过同等训练/推理评估验证。

结论。 attention 变体往往是在表示能力、cache size 和 memory bandwidth 之间交换资源。这个例题的目的不是替代完整工程实现，而是给出一种最小推理形式：把问题转成变量，写出近似公式或伪代码，再说明近似何时失效。

9 实验设计：从小例子到可复现结论

学习 Attention alternatives 与 Mixture of Experts 不能停在概念层。一个教材化结论至少需要四件事：可构造的输入、可观测的中间量、可比较的指标，以及清楚的失败条件。下面的实验设计不是要求训练大模型，而是要求读者用小规模程序复现本章机制。

9.1 最小输入

先构造只触发本章核心机制的小输入。例如 tokenizer 章节可以用几个包含 rare words 和 Unicode 字符的字符串；GPU 章节可以用一个 elementwise kernel 和一个 GEMM；data 章节可以用十几篇近重复文档。小输入的价值在于它让错误可见。

9.2 资源账本

对同一输入写下 FLOPs、bytes moved、显存峰值、通信量或样本数量的估算。估算不需要一开始就精确，但必须说明单位和近似假设。随后用 profiler、benchmark、validation loss 或人工检查去验证估算是否同量级。

9.3 单变量消融

一次只改变一个因素，例如 vocabulary size、head 数、batch size、filter threshold、reward scale、decoding temperature 或 GPU 数量。若多个变量同时变化，实验只能说明系统变了，不能说明机制是什么。

9.4 失败条件

最后要刻意触发失败：极端 context length、错误 dtype、过小 batch、污染 benchmark、低质量数据或不可靠 verifier。失败条件常常比成功曲线更能说明一个方法的边界。

9.5 记录与报告

实验记录应能让另一个读者复现同一结论。最少要写清楚输入构造、代码版本、随机种子、硬件、batch shape、数据版本、评价指标和异常处理方式。报告结论时不要只写“更快”或“更好”，而要说明快在哪里、好在哪个指标上，以及是否牺牲了内存、稳定性、数据质量或可解释性。常见报告错误是把局部现象写成普遍规律。例如一次小模型实验里的 loss 改善，不等于更大模型或不同数据分布上也会改善；一次 kernel benchmark 的吞吐提升，不等于端到端 serving latency 一定降低。教材中的实验报告应始终把结论限定在可复现条件内。

9.6 从小实验到大结论

小实验的作用不是替代课程中的完整训练或系统实现，而是建立一条可检查的推理链。先在小输入上确认变量方向，再在中等规模上确认数量级，最后才讨论是否能外推到更大模型、更多 token 或更复杂 workload。缺少中间层级时，大结论通常只是在复述直觉。因此，实验报告最好同时包含正例和反例：正例说明机制在受控条件下怎样工作，反例说明条件稍变时哪里失效。对 CS336 这样的课程，反例尤其重要，因为 language model 的错误常来自多个层面叠加：数据分布、优化稳定性、硬件瓶颈、评估协议和服务负载都可能同时改变观察结果。还要记录资料版本与复现边界。公开视频、课程页、配套代码和 PDF 可能在不同时间更新；复现实验应写明使用的材料版本，并说明哪些结论来自课程事实，哪些是为了帮助理解而加入的解释性推导。这样才能把学习笔记变成可检查的技术记录。若复现实验与课程结论不一致，首先检查输入、版本和评估协议，而不是立即修改模型假设。

9.7 把本章重新读成一条证据链

完成小实验之后，可以把 Attention alternatives 与 Mixture of Experts 重新读成一条证据链：先用“为什么要替代标准 attention”确定问题入口，再用“Linear attention、state-space 与 recurrence”解释主要机制，最后用“MoE 的稀疏计算与路由”检查边界或外推条件。这种读法比按页背诵更接近教材训练，因为它要

求每个结论都能说明前提、变量和证据。本章反复使用的术语包括 attention alternatives（注意力替代机制）、Grouped-Query Attention (GQA)、Multi-Query Attention (MQA)、linear attention（线性注意力）、Mixture of Experts (MoE)、routing（路由）。复习时不要只写定义，而要为每个术语补上一句操作性说明：它在课程代码、公式、数据记录或评估协议中对应什么对象；如果这个对象被删除、替换或缩放，哪一个观测量会变化。最后，用一个反例结束复习。反例可以是资源估算不准、数据污染、reward 失真、benchmark 解析失败、长上下文显存爆炸或 scaling 外推失效。能解释反例的章节，才真正具备自学和引用价值；不能解释反例的章节，只是把课程材料换了一种排版。引用本章结论时，还应保留来源边界：哪些说法直接来自视频、课程页、slides 或代码，哪些是为了串联教材叙述而加入的解释性推导。这个边界不会削弱结论，反而让读者知道何处可以复核，何处需要在自己的实验中重新验证。如果后续课程材料更新，也可以沿着同一证据链替换来源，而不必重写整章结构。

10 课程资料与回看路径

教材正文已经把主要概念、公式和实现逻辑组织成连续叙述；本节只提供回到课程材料的阅读路径。读者复习时可以先完成前文练习，再按下面的时间段或资料组核对细节。

10.1 视频回看路径

时间	关键词	复习任务
00:00:05-00:07:22	attention, going, it's, very, think, part, sequence	回看定义、例子、推导或 caveat 在该段中的位置。
00:22:10-00:29:16	going, attention, very, indexer, then, it's, another	回看定义、例子、推导或 caveat 在该段中的位置。
00:42:53-00:49:34	going, token, experts, it's, moes, expert, different	回看定义、例子、推导或 caveat 在该段中的位置。
00:56:40-01:03:55	experts, going, shared, expert, that's, very, it's	回看定义、例子、推导或 caveat 在该段中的位置。
01:18:41-01:26:07	think, your, it's, very, model., then, going	回看定义、例子、推导或 caveat 在该段中的位置。

这些时间段用于定位视频中的讲解顺序。严肃引用时，应回到视频片段确认讲者表述、板书或幻灯片内容。回看视频时，不必逐字抄录字幕。更有效的方法是把每个时间段判定为定义、例子、推导、代码解释或 caveat，再把它放回本章对应小节。

10.2 课程资料阅读路径

资料组	标题/页块	关键词
official_01	Lecture 4 A T T E N T I O N A L T E R N A T I V E S A N D M I X T U R E S O F E X P E R ...	attention, linear, hybrid, context, form, duality, performance
official_02	DSA –Deepseek Sparse Attention (v3.2, GLM5)	experts, fedus, feedforward, deepseek, sparse, attention, glm5
official_03	Why are MoEs getting popular? Faster to train MoEs [OIMoE]	moes, getting, popular, results, chinese, quite, earlier

official_05	Example from deepseek (v1-2) Per-expert balancing -same as the switch transformer ...	moes, balancing, per-expert, side, additional, parallelism, deepseek
official_06	Z-loss stability for the router What happens when we remove the z-loss?	deepseek, active, experts, need, moes, upcycling, shared

课程资料通常给出更稳定的标题、公式、图或代码结构；视频则补充动机和口头解释。两者合起来构成本章的事实边界。阅读资料时，可以把每个资料组拆成三个对象：一个定义，一个公式或伪代码动作，一个边界条件。这样比逐字翻译页面或脚本更容易形成可用知识。

11 实践说明、历史位置与风险

Attention alternatives 与 Mixture of Experts 在 CS336 的课程结构中连接基础机制和规模化问题。基础机制解释方法为什么成立；规模化问题检验它在更大模型、更长上下文、更复杂数据或真实 serving workload 下是否仍成立。历史位置不等于模型年表。更重要的是识别问题如何迁移：早期方法解决小规模建模问题，现代方法常常解决同一问题在大规模数据、GPU 集群、长上下文或人类偏好约束下的新形态。

- 资料版本限制 (version risk): 课程页、公开视频和配套资料可能在学期中更新；引用时应注明访问日期或资料版本。
- 测量口径限制 (measurement risk): FLOPs、tokens/s、benchmark score、reward 等数字只有在协议完整时可比较。
- 规模外推限制 (scaling risk): 小模型或小 batch 的机制不必然外推到 frontier-scale。
- 数据分布限制 (data risk): 数据来源、过滤和去重策略会改变模型行为，也会改变 evaluation contamination 风险。

12 复习要点

下面的问题把本章内容压缩为可反复检查的条目。每个条目都应能回到前文的解释、公式、伪代码或例题。

条目	对象	检查动作
条目 1	attention alternatives (注意力替代机制)	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。
条目 2	Grouped-Query Attention (GQA)	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。
条目 3	Multi-Query Attention (MQA)	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。
条目 4	linear attention (线性注意力)	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。

条目 5	Mixture of Experts (MoE)	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。
条目 6	routing（路由）	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。

12.1 综合自测

- 我能否不用课程原句，独立解释本章中心问题？
- 我能否把本章至少一个公式改写成可运行的估算函数？
- 我能否指出一个看似改进但实际转移成本的方法？
- 我能否回到视频和课程资料，找到支持本章关键论断的位置？
- 我能否设计一个最小 ablation，验证本章一个 caveat？

13 总结与延伸

13.1 本章小结

- Attention alternatives 与 Mixture of Experts 的核心不是记忆术语，而是理解对象、变量和机制之间的关系。
- 视频给出讲解顺序，课程资料提供可核对的公式、图、代码或页面结构。
- 复习时应把每个术语连接到至少一个变量、一个实现动作和一个失败模式。

13.2 延伸解释

本章中的延伸解释用于连接课程材料中的概念，例如说明公式里的 proxy、解释系统 tradeoff，或把多个材料片段串成一个完整推理链。若需要严肃引用，应回到课程视频和配套资料核对原始表述。

13.3 练习

- 定义题：用中英双语解释本章任意五个重要术语，并说明它们属于 compute、memory、data、optimization、evaluation 还是 deployment 账本。
- 推导题：选择本章一个公式，逐个解释符号、单位和近似假设，并说明哪个变量最难在真实系统中测量。
- 实现题：把本章伪代码改写成最小 Python sanity check，不要求训练大模型，但要输出可检查的中间量。
- 资料题：从“课程资料与回看路径”中选择一个视频时间段，回到原始视频核对讲者例子，并记录是否存在字幕误差。
- 评估题：为本章方法设计一个 ablation 或 benchmark，说明控制变量、数据版本、指标和 failure criteria。
- 边界题：说明本章一个结论在更长 context、更大 batch、更多 GPU 或更复杂数据分布下可能如何失效。
- 综合题：把本章内容和前一章或后一章连接起来，写出一个端到端训练/推理 pipeline 中的依赖关系。

- 批判题：指出一个容易被营销材料夸大的说法，并用本章的资源账本或评估协议反驳它。

5 05 GPU/TPU 硬件模型

第 05 讲: GPU/TPU 硬件模型

CS336 Spring 2026 public videos and official course materials

目录

1 本章导读	2
1.1 本章知识路线	2
1.2 结构图	2
2 本章主线	2
2.1 本章要解决的核心问题	2
2.2 从机制到资源账本	3
2.3 从课堂讲解到可复现实验	3
3 术语、符号与问题边界	4
3.1 术语之间的关系	4
4 硬件层级: 从 HBM 到 registers	4
4.1 为什么要讨论硬件层级: 从 HBM 到 registers	4
4.2 硬件层级: 从 HBM 到 registers 的机制	5
4.3 硬件层级: 从 HBM 到 registers 的数学表达	5
4.4 从硬件层级: 从 HBM 到 registers 到程序	5
4.5 边界条件与常见误解	5
4.6 与本章其他概念的关系	5
4.7 怎样检验硬件层级: 从 HBM 到 registers	6
4.8 概念辨析: 硬件层级: 从 HBM 到 registers	6
4.9 硬件层级: 从 HBM 到 registers 的小结	6
5 Warp、thread block 与 occupancy	6
5.1 为什么要讨论 Warp、thread block 与 occupancy	6
5.2 Warp、thread block 与 occupancy 的机制	7
5.3 Warp、thread block 与 occupancy 的数学表达	7
5.4 从 Warp、thread block 与 occupancy 到程序	7
5.5 边界条件与常见误解	7

5.6	与本章其他概念的关系	7
5.7	怎样检验 Warp、thread block 与 occupancy	8
5.8	概念辨析：Warp、thread block 与 occupancy	8
5.9	Warp、thread block 与 occupancy 的小结	8
6	用硬件模型解释 LM 工作负载	8
6.1	为什么要讨论用硬件模型解释 LM 工作负载	8
6.2	用硬件模型解释 LM 工作负载的机制	9
6.3	用硬件模型解释 LM 工作负载的数学表达	9
6.4	从用硬件模型解释 LM 工作负载到程序	9
6.5	边界条件与常见误解	9
6.6	与本章其他概念的关系	9
6.7	怎样检验用硬件模型解释 LM 工作负载	10
6.8	概念辨析：用硬件模型解释 LM 工作负载	10
6.9	用硬件模型解释 LM 工作负载的小结	10
7	综合讨论：从局部机制到完整系统	10
7.1	硬件层级：从 HBM 到 registers 在系统中的位置	11
7.2	Warp、thread block 与 occupancy 在系统中的位置	11
7.3	用硬件模型解释 LM 工作负载在系统中的位置	11
7.4	把本章知识用于读论文和复现实验	11
8	例题：把概念变成可计算检查	12
8.1	判断一个算子是 compute-bound 还是 memory-bound	12
9	实验设计：从小例子到可复现结论	12
9.1	最小输入	12
9.2	资源账本	12
9.3	单变量消融	12
9.4	失败条件	13
9.5	记录与报告	13
9.6	从小实验到大结论	13
9.7	把本章重新读成一条证据链	13
10	课程资料与回看路径	13
10.1	视频回看路径	14
10.2	课程资料阅读路径	14
11	实践说明、历史位置与风险	14

12 复习要点	15
12.1 综合自测	15
13 总结与延伸	16
13.1 本章小结	16
13.2 延伸解释	16
13.3 练习	16

1 本章导读

本章讨论 **GPU/TPU 硬件模型**。CS336 的第一层目标不是把现成大模型当作黑箱使用，而是把 language model 从输入文本、训练目标、模型结构、数据、系统和评估这些部件重新拆开。只有拆开之后，读者才能判断一个设计选择究竟改变了什么。设想 profiler 告诉你 GPU 利用率很低，但代码里全部是 PyTorch tensor 操作。要解释这个现象，必须下沉到 SM、warp、register、shared memory、L2、HBM 和 tensor core 的硬件层级。本章对应 CS336 Spring 2026 第 05 个公开视频，并结合课程页提供的可解析资料。视频和课程页给出事实边界；正文把这些材料改写为可自学的教材章节。阅读本章时，先理解问题为什么存在，再看定义、公式和实现形态，随后通过例题和练习检查自己是否能独立复现这条 reasoning chain。

Lecture 5

GPUs

CS336

Tatsu H



图 1: 第 05 讲的课程图像锚点。

1.1 本章知识路线

- 本章首先说明 GPU/TPU 硬件模型为什么是 language modeling pipeline 中不可跳过的一环。
- 其次定义本章反复使用的术语，并说明这些术语对应的计算对象或系统对象。
- 随后用公式和伪代码表达主要机制，让概念能够落到可计算的变量上。
- 最后给出例题、风险讨论和练习，便于读者检查自己是否真正掌握了机制。

1.2 结构图

下面的图用于帮助读者定位本章的核心结构。先看概念之间的依赖，再回到正文中的公式、伪代码和例题。

2 本章主线

GPU/TPU 硬件模型的主线是把 language model 的抽象计算放回真实机器：哪些张量要移动，哪些状态要保存，哪些瓶颈会在训练或服务时显现。

2.1 本章要解决的核心问题

本讲把 GPU/TPU 从黑箱变成可推理的硬件层级：SM、warp、thread block、register、shared memory、L2、HBM、tensor core 都会影响语言模型训练吞吐。读完这一段后，应能用一两句话说明 GPU/TPU 硬件模

Outline and goals

❖ Make CUDA and GPUs less magic

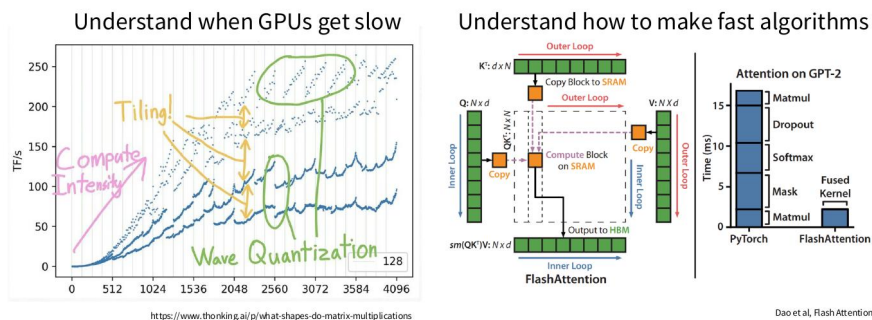


图 2: 官方材料图页: GPU/TPU 硬件模型的课程页/PDF/script 图像锚点。

Lecture 5

GPUs

CS336

Tatsu H

图 3: 官方材料图页: GPU/TPU 硬件模型的课程页/PDF/script 图像锚点。

型要解决的瓶颈，并指出这个瓶颈发生在数据、模型、优化、系统还是评估层。

2.2 从机制到资源账本

GPU 和 CPU 的差异在于吞吐优先而非单线程延迟优先。大规模矩阵乘法可以吃满 tensor cores，但不规则访存、分支、bank conflict 和低 occupancy 会显著拖慢。随后把机制写成账本：输入规模是什么，主要状态是什么，成本或质量指标怎样随变量改变。这个账本让后面的公式不再只是符号。

2.3 从课堂讲解到可复现实验

理解硬件的目标是解释算法为什么快或慢。FlashAttention 之类方法之所以重要，是因为它把 attention 的数学等价变换和 HBM traffic 的减少结合起来。最后把课堂讲解转成可复现实验：固定协议，改变一个变量，记录中间量，并解释结果为何支持或反驳原来的机制判断。

3 术语、符号与问题边界

本章的术语横跨模型、数据和系统。读者需要先知道每个词指向的对象：有些词描述数学目标，有些词描述张量形状，有些词描述硬件瓶颈，还有一些词描述评估协议。术语边界不清，后面的公式和实现就会变成空洞的缩写。

#	术语	阅读要求
1	GPU（图形处理器）	把它放入资源账本：单位是什么，随输入规模怎样变化，最容易被哪类 benchmark 误读。
2	TPU（张量处理器）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
3	SM（streaming multiprocessor）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
4	HBM（high-bandwidth memory）	把它放入资源账本：单位是什么，随输入规模怎样变化，最容易被哪类 benchmark 误读。
5	tensor core（张量核心）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
6	occupancy（占用率）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。

这些术语之间有依赖关系。例如 tokenization 改变 sequence length，sequence length 又改变 attention FLOPs、KV cache 和 evaluation token budget；parallelism 改变显存占用，也改变通信瓶颈和故障恢复方式。

3.1 术语之间的关系

在本章中，GPU（图形处理器）给出问题入口，TPU（张量处理器）描述主要机制，SM（streaming multiprocessor）则把机制连接到可测量的结果。读教材时应当沿着这个链条追问：输入是什么，中间状态怎样改变，输出指标为什么随之变化。课程材料还会反复出现 memory、gpus、https、matrix、compute、tiling 等词。它们不是一组同义标签，而是提示本章的不同层次：有的指对象，有的指操作，有的指约束或指标。因此，术语表不是背诵清单，而是后文阅读的索引。遇到一个术语时，先定位它属于 representation、optimization、systems、data 还是 evaluation，再判断它改变哪一项账本。

4 硬件层级：从 HBM 到 registers

现在进入硬件层级：从 HBM 到 registers。这一节把算法放到硬件和运行时中考察：同一个数学表达式在不同 batch、shape、GPU 拓扑和 kernel 实现下会得到不同的成本账本。

4.1 为什么要讨论硬件层级：从 HBM 到 registers

视频宣布进入 systems portion，目标是理解 GPU、parallelization 和 inference，而不是把硬件当黑箱。GPU performance 取决于 HBM、L2、shared memory/L1、registers、SM、warps 和 tensor cores 的层级关系。语言模型训练中最常见的瓶颈是计算峰值、内存带宽、通信带宽和 kernel launch/调度开销。硬件层级：从 HBM 到 registers 把 GPU/TPU 硬件模型放进资源账本。接下来要看的不是单个 API，而是张量形状、内存层级、通信和调度怎样共同决定成本。

4.2 硬件层级：从 HBM 到 registers 的机制

视频宣布进入 systems portion，目标是理解 GPU、parallelization 和 inference，而不是把硬件当黑箱。在硬件层级：从 HBM 到 registers 中，核心对象是硬件和系统状态：同一算法会因为 memory hierarchy、通信拓扑或 kernel shape 不同而表现出完全不同的速度。GPU performance 取决于 HBM、L2、shared memory/L1、registers、SM、warps 和 tensor cores 的层级关系。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。语言模型训练中最常见的瓶颈是计算峰值、内存带宽、通信带宽和 kernel launch/调度开销。可检验性来自 profile trace、roofline 估算和端到端延迟分解；如果这些证据互相矛盾，就要先解释 measurement setup。因此，硬件层级：从 HBM 到 registers 的学习目标是建立一张成本地图：读者应能指出瓶颈在计算、内存、通信还是调度，并能设计一个小 benchmark 去验证。

4.3 硬件层级：从 HBM 到 registers 的数学表达

$$t_{\text{op}} \geq \max\left(\frac{\text{FLOPs}}{P_{\text{max}}}, \frac{\text{bytes}}{B_{\text{max}}}\right)$$

符号说明： P_{max} 是硬件峰值算力， B_{max} 是可持续内存带宽；实际时间至少受二者较慢者限制。这个关系式应被读成硬件层级：从 HBM 到 registers 的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。在系统实验中，tokens/s、TTFT、显存峰值和 kernel time 都是 proxy。它们各自只观察一部分系统，因此报告时要同时写清 workload shape、warmup、同步方式和硬件型号。

4.4 从硬件层级：从 HBM 到 registers 到程序

把硬件层级：从 HBM 到 registers 写成程序时，最重要的是暴露状态变化和数据移动。下面的伪代码保留执行顺序，省略框架样板。

```
if operation_is_matmul:
    use_tensor_cores()
elif operation_is_elementwise:
    reduce_hbm_round_trips()
```

阅读这段实现时，重点看硬件层级：从 HBM 到 registers 怎样移动张量、占用显存并触发通信。实验记录至少要包含吞吐、延迟、显存峰值、通信量和 kernel 利用率，否则很难判断瓶颈来自算子、内存层级还是调度。

4.5 边界条件与常见误解

- 硬件规格表给的是上限，不是可直接达到的性能。
- TPU 与 GPU 编程模型不同，但同样要关注矩阵单元、片上内存和通信拓扑。
- 小规模 kernel benchmark 不一定预测端到端训练或 serving latency，因为调度、通信和请求形状会改变瓶颈。
- 硬件规格表和框架 profiler 都需要结合 workload 解释，不能把单个峰值数字当作机制证明。

4.6 与本章其他概念的关系

硬件层级：从 HBM 到 registers 连接本章的数学机制和工程成本：术语表说明对象，公式估算资源，伪代码暴露状态变化，例题则让读者检查一次完整的成本推理。

4.7 怎样检验硬件层级：从 HBM 到 registers

检验硬件层级：从 HBM 到 registers 时，先把抽象说法落到硬件和系统状态上。与 GPU、TPU、HBM、SM、tensor core 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： $(P_{\max})$ 是硬件峰值算力， $(B_{\max})$ 是可持续内存带宽；实际时间至少受二者较慢者限制。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。系统消融通常一次只改变 batch shape、sequence length、GPU 数、kernel 实现或调度策略。若同时改变硬件和 workload，实验只能说明端到端表现不同，不能定位瓶颈。最后检查失败条件。工程判断往往在反例中变清楚：硬件规格表给的是上限，不是可直接达到的性能；TPU 与 GPU 编程模型不同，但同样要关注矩阵单元、片上内存和通信拓扑。复习时可以把这些限制改写成反例测试，观察它们怎样改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。把实验结论放回 GPU/TPU 硬件模型时，要说明硬件层级：从 HBM 到 registers 改变的是训练吞吐、推理延迟、显存峰值还是通信开销。能做出这种归因，才说明读者已经从接口使用进入系统解释。

4.8 概念辨析：硬件层级：从 HBM 到 registers

GPU、TPU、HBM、SM、tensor core 常一起出现在系统讲解中，但它们分别指硬件对象、程序操作、瓶颈来源或测量指标。读者应先定位每个词在执行路径中的位置。其中，GPU 是 CS336 systems 部分的主要硬件对象；HBM (high-bandwidth memory) 常是推理和非融合算子的瓶颈。区分这些词时，可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕硬件层级：从 HBM 到 registers 复习时，可以写一个小 benchmark：固定输入形状，改变一个系统变量，记录 profile trace 和端到端时间。

4.9 硬件层级：从 HBM 到 registers 的小结

- 讨论硬件层级：从 HBM 到 registers 时，应同时报告问题规模、实验设置和主要观测变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。
- 如果方法声称更快，要区分计算减少、内存访问减少、通信隐藏和调度改善；这些机制不能用一个 tokens/s 数字互相替代。
- profile trace、roofline 估算和端到端 benchmark 应互相校验，单独一个指标不足以解释系统瓶颈。

5 Warp、thread block 与 occupancy

现在进入 Warp、thread block 与 occupancy。这一节把算法放到硬件和运行时中考察：同一个数学表达式在不同 batch、shape、GPU 拓扑和 kernel 实现下会得到不同的成本账本。

5.1 为什么要讨论 Warp、thread block 与 occupancy

CUDA 编程模型通过 thread、warp、thread block、grid 抽象硬件并行性。occupancy 不是越高越好；如果单线程更多寄存器带来更少 HBM 访问，低 occupancy 也可能更快。bank conflict、memory coalescing 和 wave quantization 是从硬件细节到 kernel 性能的典型桥梁。Warp、thread block 与 occupancy 把 GPU/TPU 硬件模型放进资源账本。接下来要看的不是单个 API，而是张量形状、内存层级、通信和调度怎样共同决定成本。

5.2 Warp、thread block 与 occupancy 的机制

CUDA 编程模型通过 thread、warp、thread block、grid 抽象硬件并行性。在 Warp、thread block 与 occupancy 中，核心对象是硬件和系统状态：同一算法会因为 memory hierarchy、通信拓扑或 kernel shape 不同而表现出完全不同的速度。occupancy 不是越高越好；如果单线程更多寄存器带来更少 HBM 访问，低 occupancy 也可能更快。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。bank conflict、memory coalescing 和 wave quantization 是从硬件细节到 kernel 性能的典型桥梁。可检验性来自 profile trace、roofline 估算和端到端延迟分解；如果这些证据互相矛盾，就要先解释 measurement setup。因此，Warp、thread block 与 occupancy 的学习目标是建立一张成本地图：读者应能指出瓶颈在计算、内存、通信还是调度，并能设计一个小 benchmark 去验证。

5.3 Warp、thread block 与 occupancy 的数学表达

$$\text{occupancy} = \frac{\text{active warps per SM}}{\text{max warps per SM}}$$

符号说明：active warps 受 registers、shared memory、threads per block 和硬件限制共同约束。这个关系式应被读成 Warp、thread block 与 occupancy 的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。在系统实验中，tokens/s、TTFT、显存峰值和 kernel time 都是 proxy。它们各自只观察一部分系统，因此报告时要同时写清 workload shape、warmup、同步方式和硬件型号。

5.4 从 Warp、thread block 与 occupancy 到程序

把 Warp、thread block 与 occupancy 写成程序时，最重要的是暴露状态变化和数据移动。下面的伪代码保留执行顺序，省略框架样板。

```
blocks_per_sm = min(register_limit, shared_memory_limit, warp_limit)
occupancy = active_warps(blocks_per_sm) / max_warps
```

阅读这段实现时，重点看 Warp、thread block 与 occupancy 怎样移动张量、占用显存并触发通信。实验记录至少要包含吞吐、延迟、显存峰值、通信量和 kernel 利用率，否则很难判断瓶颈来自算子、内存层级还是调度。

5.5 边界条件与常见误解

- 高 occupancy 无法弥补非合并访存或严重 bank conflicts。
- 小矩阵或小 batch 会受到 launch overhead 和 wave quantization 的影响。
- 小规模 kernel benchmark 不一定预测端到端训练或 serving latency，因为调度、通信和请求形状会改变瓶颈。
- 硬件规格表和框架 profiler 都需要结合 workload 解释，不能把单个峰值数字当作机制证明。

5.6 与本章其他概念的关系

Warp、thread block 与 occupancy 连接本章的数学机制和工程成本：术语表说明对象，公式估算资源，伪代码暴露状态变化，例题则让读者检查一次完整的成本推理。

5.7 怎样检验 Warp、thread block 与 occupancy

检验 Warp、thread block 与 occupancy 时，先把抽象说法落到硬件和系统状态上。与 warp、thread block、occupancy、bank conflict、coalescing 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明：active warps 受 registers、shared memory、threads per block 和硬件限制共同约束。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。系统消融通常一次只改变 batch shape、sequence length、GPU 数、kernel 实现或调度策略。若同时改变硬件和 workload，实验只能说明端到端表现不同，不能定位瓶颈。最后检查失败条件。工程判断往往在反例中变清楚：高 occupancy 无法弥补非合并访存或严重 bank conflicts；小矩阵或小 batch 会受到 launch overhead 和 wave quantization 的影响。复习时可以把这些限制改写成反例测试，观察它们怎样改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。把实验结论放回 GPU/TPU 硬件模型时，要说明 Warp、thread block 与 occupancy 改变的是训练吞吐、推理延迟、显存峰值还是通信开销。能做出这种归因，才说明读者已经从接口使用进入系统解释。

5.8 概念辨析：Warp、thread block 与 occupancy

warp、thread block、occupancy、bank conflict、coalescing 常一起出现在系统讲解中，但它们分别指硬件对象、程序操作、瓶颈来源或测量指标。读者应先定位每个词在执行路径中的位置。区分这些词时，可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕 Warp、thread block 与 occupancy 复习时，可以写一个小 benchmark：固定输入形状，改变一个系统变量，记录 profile trace 和端到端时间。

5.9 Warp、thread block 与 occupancy 的小结

- 讨论 Warp、thread block 与 occupancy 时，应同时报告问题规模、实验设置和主要观测变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。
- 如果方法声称更快，要区分计算减少、内存访问减少、通信隐藏和调度改善；这些机制不能用一个 tokens/s 数字互相替代。
- profile trace、roofline 估算和端到端 benchmark 应互相校验，单独一个指标不足以解释系统瓶颈。

6 用硬件模型解释 LM 工作负载

现在进入用硬件模型解释 LM 工作负载。这一节把算法放到硬件和运行时中考察：同一个数学表达式在不同 batch、shape、GPU 拓扑和 kernel 实现下会得到不同的成本账本。

6.1 为什么要讨论用硬件模型解释 LM 工作负载

Transformer 训练由大 GEMM、attention、normalization、softmax、optimizer step 和通信构成。大 GEMM 更容易吃满 tensor cores；softmax、LayerNorm、dropout、optimizer 等更容易 memory-bound。系统优化的目标是把 high-level PyTorch graph 映射成尽量少、尽量高效、尽量少搬数据的硬件操作。用硬件模型解释 LM 工作负载把 GPU/TPU 硬件模型放进资源账本。接下来要看的不是单个 API，而是张量形状、内存层级、通信和调度怎样共同决定成本。

6.2 用硬件模型解释 LM 工作负载的机制

Transformer 训练由大 GEMM、attention、normalization、softmax、optimizer step 和通信构成。在用硬件模型解释 LM 工作负载中，核心对象是硬件和系统状态：同一算法会因为 memory hierarchy、通信拓扑或 kernel shape 不同而表现出完全不同的速度。大 GEMM 更容易吃满 tensor cores；softmax、LayerNorm、dropout、optimizer 等更容易 memory-bound。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。系统优化的目标是把 high-level PyTorch graph 映射成尽量少、尽量高效、尽量少搬数据的硬件操作。可检验性来自 profile trace、roofline 估算和端到端延迟分解；如果这些证据互相矛盾，就要先解释 measurement setup。因此，用硬件模型解释 LM 工作负载的学习目标是建立一张成本地图：读者应能指出瓶颈在计算、内存、通信还是调度，并能设计一个小 benchmark 去验证。

6.3 用硬件模型解释 LM 工作负载的数学表达

$$\text{tokens/s} = \frac{\text{batch} \times \text{sequence length}}{\text{step time}}$$

符号说明：tokens/s 是训练吞吐的直接指标；step time 又由 compute、memory、communication 和 synchronization 构成。这个关系式应被读成用硬件模型解释 LM 工作负载的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。在系统实验中，tokens/s、TTFT、显存峰值和 kernel time 都是 proxy。它们各自只观察一部分系统，因此报告时要同时写清 workload shape、warmup、同步方式和硬件型号。

6.4 从用硬件模型解释 LM 工作负载到程序

把用硬件模型解释 LM 工作负载写成程序时，最重要的是暴露状态变化和数据移动。下面的伪代码保留执行顺序，省略框架样板。

```
profile_step()
for kernel in hot_kernels:
    classify_compute_or_memory_bound(kernel)
    optimize_layout_or_fusion(kernel)
```

阅读这段实现时，重点看用硬件模型解释 LM 工作负载怎样移动张量、占用显存并触发通信。实验记录至少要包含吞吐、延迟、显存峰值、通信量和 kernel 利用率，否则很难判断瓶颈来自算子、内存层级还是调度。

6.5 边界条件与常见误解

- tokens/s 不能单独比较模型质量；更合理的是把 loss、tokens、FLOPs 和成本一起报告。
- benchmark 要包含 warmup 和同步，否则 GPU 异步执行会误导计时。
- 小规模 kernel benchmark 不一定预测端到端训练或 serving latency，因为调度、通信和请求形状会改变瓶颈。
- 硬件规格表和框架 profiler 都需要结合 workload 解释，不能把单个峰值数字当作机制证明。

6.6 与本章其他概念的关系

用硬件模型解释 LM 工作负载连接本章的数学机制和工程成本：术语表说明对象，公式估算资源，伪代码暴露状态变化，例题则让读者检查一次完整的成本推理。

6.7 怎样检验用硬件模型解释 LM 工作负载

检验用硬件模型解释 LM 工作负载时，先把抽象说法落到硬件和系统状态上。与 GEMM、LayerNorm、softmax、optimizer、tokens/s 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明：tokens/s 是训练吞吐的直接指标；step time 又由 compute、memory、communication 和 synchronization 构成。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。系统消融通常一次只改变 batch shape、sequence length、GPU 数、kernel 实现或调度策略。若同时改变硬件和 workload，实验只能说明端到端表现不同，不能定位瓶颈。最后检查失败条件。工程判断往往在反例中变清楚：tokens/s 不能单独比较模型质量；更合理的是把 loss、tokens、FLOPs 和成本一起报告；benchmark 要包含 warmup 和同步，否则 GPU 异步执行会误导计时。复习时可以把这些限制改写成反例测试，观察它们怎样改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。把实验结论放回 GPU/TPU 硬件模型时，要说明用硬件模型解释 LM 工作负载改变的是训练吞吐、推理延迟、显存峰值还是通信开销。能做出这种归因，才说明读者已经从接口使用进入系统解释。

6.8 概念辨析：用硬件模型解释 LM 工作负载

GEMM、LayerNorm、softmax、optimizer、tokens/s 常一起出现在系统讲解中，但它们分别指硬件对象、程序操作、瓶颈来源或测量指标。读者应先定位每个词在执行路径中的位置。其中，normalization（归一化）影响优化稳定性和 residual path。区分这些词时，可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕用硬件模型解释 LM 工作负载复习时，可以写一个小 benchmark：固定输入形状，改变一个系统变量，记录 profile trace 和端到端时间。

6.9 用硬件模型解释 LM 工作负载的小结

- 讨论用硬件模型解释 LM 工作负载时，应同时报告问题规模、实验设置和主要观测变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。
- 如果方法声称更快，要区分计算减少、内存访问减少、通信隐藏和调度改善；这些机制不能用一个 tokens/s 数字互相替代。
- profile trace、roofline 估算和端到端 benchmark 应互相校验，单独一个指标不足以解释系统瓶颈。

7 综合讨论：从局部机制到完整系统

本章的主题是 GPU/TPU 硬件模型，但它不应被读成几个相邻知识点的拼接。硬件层级：从 HBM 到 registers、Warp、thread block 与 occupancy、用硬件模型解释 LM 工作负载共同回答同一个问题：语言模型系统中哪些对象可以被定义，哪些成本可以被估算，哪些结论必须通过实验或评估来确认。这些对象包括 GPU（图形处理器）、TPU（张量处理器）、SM（streaming multiprocessor）、HBM（high-bandwidth memory）、tensor core（张量核心）、occupancy（占用率）。读者可以把它们看成一组接口：有的接口连接文本和 token，有的连接模型结构和张量计算，有的连接数据样本和训练目标，有的连接离线评估和在线服务。接口一旦改变，后面的成本、错误类型和优化空间也会随之改变。

7.1 硬件层级：从 HBM 到 registers 在系统中的位置

硬件层级：从 HBM 到 registers 首先提供一个局部解释：视频宣布进入 systems portion，目标是理解 GPU、parallelization 和 inference，而不是把硬件当黑箱。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，硬件层级：从 HBM 到 registers 会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，硬件层级：从 HBM 到 registers 不能脱离边界条件理解。一个关键 caveat 是：硬件规格表给的是上限，不是可直接达到的性能。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.2 Warp、thread block 与 occupancy 在系统中的位置

Warp、thread block 与 occupancy 首先提供一个局部解释：CUDA 编程模型通过 thread、warp、thread block、grid 抽象硬件并行性。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，Warp、thread block 与 occupancy 会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，Warp、thread block 与 occupancy 不能脱离边界条件理解。一个关键 caveat 是：高 occupancy 无法弥补非合并访存或严重 bank conflicts。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.3 用硬件模型解释 LM 工作负载在系统中的位置

用硬件模型解释 LM 工作负载首先提供一个局部解释：Transformer 训练由大 GEMM、attention、normalization、softmax、optimizer step 和通信构成。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，用硬件模型解释 LM 工作负载会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，用硬件模型解释 LM 工作负载不能脱离边界条件理解。一个关键 caveat 是：tokens/s 不能单独比较模型质量；更合理的是把 loss、tokens、FLOPs 和成本一起报告。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.4 把本章知识用于读论文和复现实验

读相关论文时，可以把本章变成一个检查表。第一，论文是否清楚定义了输入规模、模型规模、数据来源和评估协议；第二，论文声称的改进落在哪个变量上；第三，作者是否报告了会让结论失效的设置；第四，实验是否能分离模型结构、数据质量、优化设置和系统实现的影响。复现实验时，也应避免直接追求大规模。先用最小程序复现一个公式、一个伪代码分支或一个 profile 现象，再扩大输入规模。这样做的原因很简单：如果小规模程序无法解释中间量，大规模训练只会把错误隐藏在日志和随机波动里。把这些检查合在一起，本章给出的不是一个固定答案，而是一种阅读 CS336 的方法：每个模型机制都要落到变量，每个变量都要落到测量，每个测量都要说明协议，每个协议都要承认边界。因此，本章中的任何一句结论都可以被改写成一个可引用的完整句式：在给定数据、模型、硬件和评估协议下，某个机制改变了某个变量，并通过某个观测指标表现出来。这样的句子比单独背诵术语更长，但它包含了自学、复现和引用时真正需要的信息。把本章用于自学时，

可以按三遍阅读。第一遍只追踪对象和定义，确认每个术语到底指向文本、token、张量、样本、请求还是评估记录。第二遍追踪公式和伪代码，确认每一步改变了哪些状态。第三遍追踪实验和 caveat，确认哪些结论只在特定规模、数据或硬件条件下成立。把本章用于复习时，则应反过来操作：先从一个失败案例或异常指标出发，再回到相应机制。loss 曲线异常可能指向数据、优化或模型容量；吞吐异常可能指向 kernel、通信或调度；benchmark 异常可能指向 contamination、prompt protocol 或采样设置。能够从异常反推机制，是掌握本章的实用标准。

8 例题：把概念变成可计算检查

8.1 判断一个算子是 compute-bound 还是 memory-bound

对一个 elementwise activation 和一个大矩阵乘分别估算 arithmetic intensity。

- elementwise activation 每个元素只做少量 FLOPs，却要读写至少一次 HBM，通常 memory-bound。
- 大 GEMM 会重复使用 tile 中的数据，FLOPs/byte 高，通常更接近 compute-bound。
- 优化前先放到 roofline 模型里，避免把时间浪费在错误瓶颈上。

结论。 GPU 优化不是“多用并行”四个字，而是让数据移动和计算形状匹配硬件。这个例题的目的不是替代完整工程实现，而是给出一种最小推理形式：把问题转成变量，写出近似公式或伪代码，再说明近似何时失效。

9 实验设计：从小例子到可复现结论

学习 GPU/TPU 硬件模型不能停在概念层。一个教材化结论至少需要四件事：可构造的输入、可观测的中间量、可比较的指标，以及清楚的失败条件。下面的实验设计不是要求训练大模型，而是要求读者用小规模程序复现本章机制。

9.1 最小输入

先构造只触发本章核心机制的小输入。例如 tokenizer 章节可以用几个包含 rare words 和 Unicode 字符的字符串；GPU 章节可以用一个 elementwise kernel 和一个 GEMM；data 章节可以用十几篇近重复文档。小输入的价值在于它让错误可见。

9.2 资源账本

对同一输入写下 FLOPs、bytes moved、显存峰值、通信量或样本数量的估算。估算不需要一开始就精确，但必须说明单位和近似假设。随后用 profiler、benchmark、validation loss 或人工检查去验证估算是否同量级。

9.3 单变量消融

一次只改变一个因素，例如 vocabulary size、head 数、batch size、filter threshold、reward scale、decoding temperature 或 GPU 数量。若多个变量同时变化，实验只能说明系统变了，不能说明机制是什么。

9.4 失败条件

最后要刻意触发失败：极端 context length、错误 dtype、过小 batch、污染 benchmark、低质量数据或不可靠 verifier。失败条件常常比成功曲线更能说明一个方法的边界。

9.5 记录与报告

实验记录应能让另一个读者复现同一结论。最少要写清楚输入构造、代码版本、随机种子、硬件、batch shape、数据版本、评价指标和异常处理方式。报告结论时不要只写“更快”或“更好”，而要说明快在哪里、好在哪个指标上，以及是否牺牲了内存、稳定性、数据质量或可解释性。常见报告错误是把局部现象写成普遍规律。例如一次小模型实验里的 loss 改善，不等于更大模型或不同数据分布上也会改善；一次 kernel benchmark 的吞吐提升，不等于端到端 serving latency 一定降低。教材中的实验报告应始终把结论限定在可复现条件内。

9.6 从小实验到大结论

小实验的作用不是替代课程中的完整训练或系统实现，而是建立一条可检查的推理链。先在小输入上确认变量方向，再在中等规模上确认数量级，最后才讨论是否能外推到更大模型、更多 token 或更复杂 workload。缺少中间层级时，大结论通常只是在复述直觉。因此，实验报告最好同时包含正例和反例：正例说明机制在受控条件下怎样工作，反例说明条件稍变时哪里失效。对 CS336 这样的课程，反例尤其重要，因为 language model 的错误常来自多个层面叠加：数据分布、优化稳定性、硬件瓶颈、评估协议和服务负载都可能同时改变观察结果。还要记录资料版本与复现边界。公开视频、课程页、配套代码和 PDF 可能在不同时间更新；复现实验应写明使用的材料版本，并说明哪些结论来自课程事实，哪些是为了帮助理解而加入的解释性推导。这样才能把学习笔记变成可检查的技术记录。若复现实验与课程结论不一致，首先检查输入、版本和评估协议，而不是立即修改模型假设。

9.7 把本章重新读成一条证据链

完成小实验之后，可以把 GPU/TPU 硬件模型重新读成一条证据链：先用“硬件层级：从 HBM 到 registers”确定问题入口，再用“Warp、thread block 与 occupancy”解释主要机制，最后用“用硬件模型解释 LM 工作负载”检查边界或外推条件。这种读法比按页背诵更接近教材训练，因为它要求每个结论都能说明前提、变量和证据。本章反复使用的术语包括 GPU（图形处理器）、TPU（张量处理器）、SM（streaming multiprocessor）、HBM（high-bandwidth memory）、tensor core（张量核心）、occupancy（占用率）。复习时不要只写定义，而要为每个术语补上一句操作性说明：它在课程代码、公式、数据记录或评估协议中对应什么对象；如果这个对象被删除、替换或缩放，哪一个观测量会变化。最后，用一个反例结束复习。反例可以是资源估算不准、数据污染、reward 失真、benchmark 解析失败、长上下文显存爆炸或 scaling 外推失效。能解释反例的章节，才真正具备自学和引用价值；不能解释反例的章节，只是把课程材料换了一种排版。引用本章结论时，还应保留来源边界：哪些说法直接来自视频、课程页、slides 或代码，哪些是为了串联教材叙述而加入的解释性推导。这个边界不会削弱结论，反而让读者知道何处可以复核，何处需要在自己的实验中重新验证。如果后续课程材料更新，也可以沿着同一证据链替换来源，而不必重写整章结构。

10 课程资料与回看路径

教材正文已经把主要概念、公式和实现逻辑组织成连续叙述；本节只提供回到课程材料的阅读路径。读者复习时可以先完成前文练习，再按下面的时间段或资料组核对细节。

10.1 视频回看路径

时间	关键词	复习任务
00:00:05-00:06:39	going, think, your, faster, kind, systems, under-stand	回看定义、例子、推导或 caveat 在该段中的位置。
00:20:19-00:26:48	matrix, that's, multiply, it's, going, very, memory	回看定义、例子、推导或 caveat 在该段中的位置。
00:39:39-00:46:02	going, scaling, think, quantize, matrix, it's, factor	回看定义、例子、推导或 caveat 在该段中的位置。
00:52:04-00:58:54	memory, read, going, threads, matrix, think, it's	回看定义、例子、推导或 caveat 在该段中的位置。
01:11:58-01:18:25	going, then, think, attention, compute, very, soft-max	回看定义、例子、推导或 caveat 在该段中的位置。

这些时间段用于定位视频中的讲解顺序。严肃引用时，应回到视频片段确认讲者表述、板书或幻灯片内容。回看视频时，不必逐字抄录字幕。更有效的方法是把每个时间段判定为定义、例子、推导、代码解释或 caveat，再把它放回本章对应小节。

10.2 课程资料阅读路径

资料组	标题/页块	关键词
official_01	Lecture 5 GPUS CS336 Tatsu H	memory, gpus, scaling, threads, fast, compute, https
official_02	Recap: GPUs –what are they and how do they work GPUs are massively parallel –same ...	memory, gpus, fast, same, make, workloads, control
official_03	Trick 1: Low precision computation If you have fewer bits, you have fewer bits to move	memory, https, precision, access, operations, trick, read
official_04	Tiling math Non-tiled matrix multiply: each input is read times from global memory ...	tiling, matrix, memory, softmax, part, attention, computation

课程资料通常给出更稳定的标题、公式、图或代码结构；视频则补充动机和口头解释。两者合起来构成本章的事实边界。阅读资料时，可以把每个资料组拆成三个对象：一个定义，一个公式或伪代码动作，一个边界条件。这样比逐字翻译页面或脚本更容易形成可用知识。

11 实践说明、历史位置与风险

GPU/TPU 硬件模型在 CS336 的课程结构中连接基础机制和规模化问题。基础机制解释方法为什么成立；规模化问题检验它在更大模型、更长上下文、更复杂数据或真实 serving workload 下是否仍成立。历史位置不等于模型年表。更重要的是识别问题如何迁移：早期方法解决小规模建模问题，现代方法常常解决同一问题在大规模数据、GPU 集群、长上下文或人类偏好约束下的新形态。

- 资料版本限制 (version risk)：课程页、公开视频和配套资料可能在学期中更新；引用时应注明访问日期或资料版本。

- 测量口径限制 (measurement risk): FLOPs、tokens/s、benchmark score、reward 等数字只有在协议完整时可比较。
- 规模外推限制 (scaling risk): 小模型或小 batch 的机制不必然外推到 frontier-scale。
- 数据分布限制 (data risk): 数据来源、过滤和去重策略会改变模型行为, 也会改变 evaluation contamination 风险。
- 系统风险 (systems risk): 性能剖析结果依赖硬件、driver、kernel、shape 和 batch distribution; 脱离 workload 的性能数字不可直接复用。

12 复习要点

下面的问题把本章内容压缩为可反复检查的条目。每个条目都应能回到前文的解释、公式、伪代码或例题。

条目	对象	检查动作
条目 1	GPU (图形处理器)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 2	TPU (张量处理器)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 3	SM (streaming multiprocessor)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 4	HBM (high-bandwidth memory)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 5	tensor core (张量核心)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 6	occupancy (占用率)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。

12.1 综合自测

- 我能否不用课程原句, 独立解释本章中心问题?
- 我能否把本章至少一个公式改写成可运行的估算函数?
- 我能否指出一个看似改进但实际转移成本的方法?
- 我能否回到视频和课程资料, 找到支持本章关键论断的位置?
- 我能否设计一个最小 ablation, 验证本章一个 caveat?

13 总结与延伸

13.1 本章小结

- GPU/TPU 硬件模型的核心不是记忆术语，而是理解对象、变量和机制之间的关系。
- 视频给出讲解顺序，课程资料提供可核对的公式、图、代码或页面结构。
- 复习时应把每个术语连接到至少一个变量、一个实现动作和一个失败模式。

13.2 延伸解释

本章中的延伸解释用于连接课程材料中的概念，例如说明公式里的 proxy、解释系统 tradeoff，或把多个材料片段串成一个完整推理链。若需要严肃引用，应回到课程视频和配套资料核对原始表述。

13.3 练习

- 定义题：用中英双语解释本章任意五个重要术语，并说明它们属于 compute、memory、data、optimization、evaluation 还是 deployment 账本。
- 推导题：选择本章一个公式，逐个解释符号、单位和近似假设，并说明哪个变量最难在真实系统中测量。
- 实现题：把本章伪代码改写成一个最小 Python sanity check，不要求训练大模型，但要输出可检查的中间量。
- 资料题：从“课程资料与回看路径”中选择一个视频时间段，回到原始视频核对讲者例子，并记录是否存在字幕误差。
- 评估题：为本章方法设计一个 ablation 或 benchmark，说明控制变量、数据版本、指标和 failure criteria。
- 边界题：说明本章一个结论在更长 context、更大 batch、更多 GPU 或更复杂数据分布下可能如何失效。
- 综合题：把本章内容和前一章或后一章连接起来，写出一个端到端训练/推理 pipeline 中的依赖关系。
- 批判题：指出一个容易被营销材料夸大的说法，并用本章的资源账本或评估协议反驳它。

6 06 Kernel、Triton 与 XLA

第 06 讲: Kernel、Triton 与 XLA

CS336 Spring 2026 public videos and official course materials

目录

1 本章导读	2
1.1 本章知识路线	2
1.2 结构图	2
2 本章主线	2
2.1 本章要解决的核心问题	3
2.2 从机制到资源账本	3
2.3 从课堂讲解到可复现实验	4
3 术语、符号与问题边界	4
3.1 术语之间的关系	4
4 Benchmark/profiling 的基本闭环	4
4.1 为什么要讨论 Benchmark/profiling 的基本闭环	5
4.2 Benchmark/profiling 的基本闭环的机制	5
4.3 Benchmark/profiling 的基本闭环的数学表达	5
4.4 从 Benchmark/profiling 的基本闭环到程序	5
4.5 边界条件与常见误解	5
4.6 与本章其他概念的关系	6
4.7 怎样检验 Benchmark/profiling 的基本闭环	6
4.8 概念辨析: Benchmark/profiling 的基本闭环	6
4.9 Benchmark/profiling 的基本闭环的小结	6
5 Triton 的 programming model	7
5.1 为什么要讨论 Triton 的 programming model	7
5.2 Triton 的 programming model 的机制	7
5.3 Triton 的 programming model 的数学表达	7
5.4 从 Triton 的 programming model 到程序	7
5.5 边界条件与常见误解	8

5.6	与本章其他概念的关系	8
5.7	怎样检验 Triton 的 programming model	8
5.8	概念辨析: Triton 的 programming model	8
5.9	Triton 的 programming model 的小结	8
6	Fusion、tiling 与 compiler	9
6.1	为什么要讨论 Fusion、tiling 与 compiler	9
6.2	Fusion、tiling 与 compiler 的机制	9
6.3	Fusion、tiling 与 compiler 的数学表达	9
6.4	从 Fusion、tiling 与 compiler 到程序	9
6.5	边界条件与常见误解	10
6.6	与本章其他概念的关系	10
6.7	怎样检验 Fusion、tiling 与 compiler	10
6.8	概念辨析: Fusion、tiling 与 compiler	10
6.9	Fusion、tiling 与 compiler 的小结	11
7	综合讨论: 从局部机制到完整系统	11
7.1	Benchmark/profiling 的基本闭环在系统中的位置	11
7.2	Triton 的 programming model 在系统中的位置	11
7.3	Fusion、tiling 与 compiler 在系统中的位置	11
7.4	把本章知识用于读论文和复现实验	12
8	例题: 把概念变成可计算检查	12
8.1	Triton row-wise softmax 的稳定实现	12
9	实验设计: 从小例子到可复现结论	12
9.1	最小输入	13
9.2	资源账本	13
9.3	单变量消融	13
9.4	失败条件	13
9.5	记录与报告	13
9.6	从小实验到大结论	13
9.7	把本章重新读成一条证据链	13
10	课程资料与回看路径	14
10.1	视频回看路径	14
10.2	课程资料阅读路径	14
11	实践说明、历史位置与风险	15

12 复习要点	15
12.1 综合自测	16
13 总结与延伸	16
13.1 本章小结	16
13.2 延伸解释	16
13.3 练习	16

1 本章导读

本章讨论 **Kernel、Triton 与 XLA**。CS336 的第一层目标不是把现成大模型当作黑箱使用，而是把 language model 从输入文本、训练目标、模型结构、数据、系统和评估这些部件重新拆开。只有拆开之后，读者才能判断一个设计选择究竟改变了什么。设想一个 row-wise softmax kernel 比 PyTorch baseline 慢。真正的修复流程不是凭直觉改代码，而是先做 warmup 和同步计时，再用 profiler 找瓶颈，最后通过 tiling、fusion 或 Triton block program 控制数据移动。本章对应 CS336 Spring 2026 第 06 个公开视频，并结合课程页提供的可解析资料。视频和课程页给出事实边界；正文把这些材料改写为可自学的教材章节。阅读本章时，先理解问题为什么存在，再看定义、公式和实现形态，随后通过例题和练习检查自己是否能独立复现这条 reasoning chain。

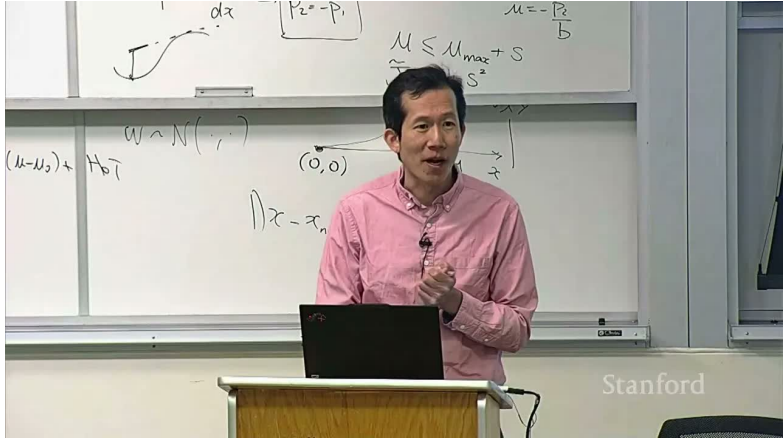


图 1: 第 06 讲的课程图像锚点。

1.1 本章知识路线

- 本章首先说明 Kernel、Triton 与 XLA 为什么是 language modeling pipeline 中不可跳过的一环。
- 其次定义本章反复使用的术语，并说明这些术语对应的计算对象或系统对象。
- 随后用公式和伪代码表达主要机制，让概念能够落到可计算的变量上。
- 最后给出例题、风险讨论和练习，便于读者检查自己是否真正掌握了机制。

1.2 结构图

下面的图用于帮助读者定位本章的核心结构。先看概念之间的依赖，再回到正文中的公式、伪代码和例题。

2 本章主线

Kernel、Triton 与 XLA 的主线是把 language model 的抽象计算放回真实机器：哪些张量要移动，哪些状态要保存，哪些瓶颈会在训练或服务时显现。

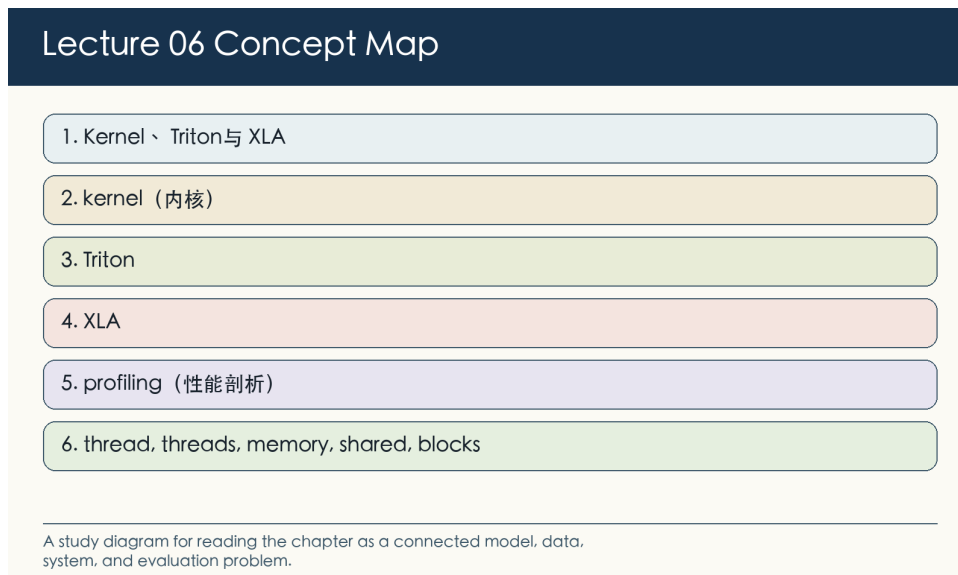


图 2: 第 06 讲概念图: 本章问题、术语和机制的关系。

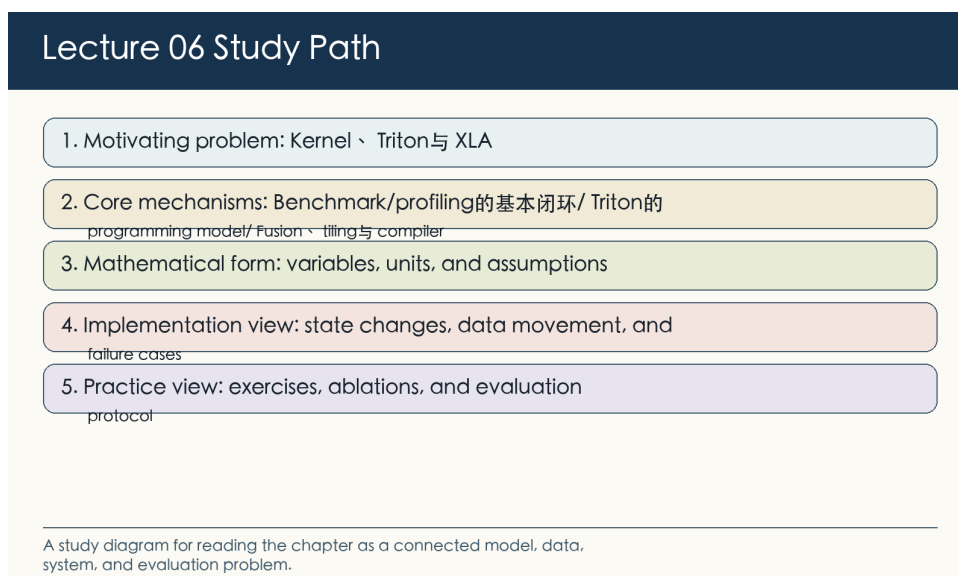


图 3: 第 06 讲学习路径: 从问题定义进入公式、实现、实验和练习。

2.1 本章要解决的核心问题

本讲从 profiling 和 benchmarking 出发, 要求先测量再优化。GPU 异步执行意味着计时必须 warmup、同步, 并用 profiler 找到真正热点。读完这一段后, 应能用一两句话说明 Kernel、Triton 与 XLA 要解决的瓶颈, 并指出这个瓶颈发生在数据、模型、优化、系统还是评估层。

2.2 从机制到资源账本

Triton 的教学价值在于把 GPU kernel 写作提升到 block-level 抽象: 显式处理 offsets、mask、load、compute、store, 同时仍能思考 coalescing、shared memory 和 tiling。随后把机制写成账本: 输入规模是什么, 主要状态是什么, 成本或质量指标怎样随变量改变。这个账本让后面的公式不再只是符号。

2.3 从课堂讲解到可复现实验

kernel fusion 和 tiling 是减少 HBM 往返的基本策略。它们提高性能的前提是没有引入过高寄存器压力、过低 occupancy 或数值错误。最后把课堂讲解转成可复现实验：固定协议，改变一个变量，记录中间量，并解释结果为何支持或反驳原来的机制判断。

3 术语、符号与问题边界

本章的术语横跨模型、数据和系统。读者需要先知道每个词指向的对象：有些词描述数学目标，有些词描述张量形状，有些词描述硬件瓶颈，还有一些词描述评估协议。术语边界不清，后面的公式和实现就会变成空洞的缩写。

#	术语	阅读要求
1	kernel（内核）	把它放入资源账本：单位是什么，随输入规模怎样变化，最容易被哪类 benchmark 误读。
2	Triton	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
3	XLA	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
4	profiling（性能剖析）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
5	fusion（算子融合）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
6	tiling（分块）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。

这些术语之间有依赖关系。例如 tokenization 改变 sequence length，sequence length 又改变 attention FLOPs、KV cache 和 evaluation token budget；parallelism 改变显存占用，也改变通信瓶颈和故障恢复方式。

3.1 术语之间的关系

在本章中，kernel（内核）给出问题入口，Triton 描述主要机制，XLA 则把机制连接到可测量的结果。读教材时应当沿着这个链条追问：输入是什么，中间状态怎样改变，输出指标为什么随之变化。课程材料还会反复出现 thread、memory、shared、threads、block、triton 等词。它们不是一组同义标签，而是提示本章的不同层次：有的指对象，有的指操作，有的指约束或指标。因此，术语表不是背诵清单，而是后文阅读的索引。遇到一个术语时，先定位它属于 representation、optimization、systems、data 还是 evaluation，再判断它改变哪一项账本。

4 Benchmark/profiling 的基本闭环

现在进入 Benchmark/profiling 的基本闭环。这一节把算法放到硬件和运行时中考察：同一个数学表达式在不同 batch、shape、GPU 拓扑和 kernel 实现下会得到不同的成本账本。

4.1 为什么要讨论 Benchmark/profiling 的基本闭环

课程材料明确把本讲定位为 GPU 高层概览后的代码层深入：写 Triton kernels、benchmark 和 profiling。benchmark 回答端到端多快，profiling 回答时间花在哪里；两者必须迭代使用。正确计时 GPU 代码需要 warmup、CUDA events 或同步，否则测到的可能只是 CPU launch time。Benchmark/profiling 的基本闭环把 Kernel、Triton 与 XLA 放进资源账本。接下来要看的不是单个 API，而是张量形状、内存层级、通信和调度怎样共同决定成本。

4.2 Benchmark/profiling 的基本闭环的机制

课程材料明确把本讲定位为 GPU 高层概览后的代码层深入：写 Triton kernels、benchmark 和 profiling。在 Benchmark/profiling 的基本闭环中，核心对象是硬件和系统状态：同一算法会因为 memory hierarchy、通信拓扑或 kernel shape 不同而表现出完全不同的速度。benchmark 回答端到端多快，profiling 回答时间花在哪里；两者必须迭代使用。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。正确计时 GPU 代码需要 warmup、CUDA events 或同步，否则测到的可能只是 CPU launch time。可检验性来自 profile trace、roofline 估算和端到端延迟分解；如果这些证据互相矛盾，就要先解释 measurement setup。因此，Benchmark/profiling 的基本闭环的学习目标是建立一张成本地图：读者应能指出瓶颈在计算、内存、通信还是调度，并能设计一个小 benchmark 去验证。

4.3 Benchmark/profiling 的基本闭环的数学表达

$$\bar{t} = \frac{1}{n} \sum_{i=1}^n t_i, \quad \text{speedup} = \frac{t_{\text{baseline}}}{t_{\text{optimized}}}$$

符号说明： $\bar{t}$ 是多次 trial 的平均时间；speedup 要在同一输入、同一硬件、同一 dtype 下比较。这个关系式应被读成 Benchmark/profiling 的基本闭环的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。在系统实验中，tokens/s、TTFT、显存峰值和 kernel time 都是 proxy。它们各自只观察一部分系统，因此报告时要同时写清 workload shape、warmup、同步方式和硬件型号。

4.4 从 Benchmark/profiling 的基本闭环到程序

把 Benchmark/profiling 的基本闭环写成程序时，最重要的是暴露状态变化和数据移动。下面的伪代码保留执行顺序，省略框架样板。

```
for _ in range(warmups): run()
synchronize()
for trial in trials:
    start_event.record(); run(); end_event.record(); synchronize()
```

阅读这段实现时，重点看 Benchmark/profiling 的基本闭环怎样移动张量、占用显存并触发通信。实验记录至少要包含吞吐、延迟、显存峰值、通信量和 kernel 利用率，否则很难判断瓶颈来自算子、内存层级还是调度。

4.5 边界条件与常见误解

- 一次 benchmark 不足以说明 scaling；要扫维度、batch 和 dtype。
- profiling 结果依赖输入 shape 和编译状态。

- 小规模 kernel benchmark 不一定预测端到端训练或 serving latency，因为调度、通信和请求形状会改变瓶颈。
- 硬件规格表和框架 profiler 都需要结合 workload 解释，不能把单个峰值数字当作机制证明。

4.6 与本章其他概念的关系

Benchmark/profiling 的基本闭环连接本章的数学机制和工程成本：术语表说明对象，公式估算资源，伪代码暴露状态变化，例题则让读者检查一次完整的成本推理。

4.7 怎样检验 Benchmark/profiling 的基本闭环

检验 Benchmark/profiling 的基本闭环时，先把抽象说法落到硬件和系统状态上。与 benchmark、profiling、CUDA event、warmup、speedup 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： $\bar{t}$ 是多次 trial 的平均时间；speedup 要在同一输入、同一硬件、同一 dtype 下比较。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。系统消融通常一次只改变 batch shape、sequence length、GPU 数、kernel 实现或调度策略。若同时改变硬件和 workload，实验只能说明端到端表现不同，不能定位瓶颈。最后检查失败条件。工程判断往往在反例中变清楚：一次 benchmark 不足以说明 scaling；要扫维度、batch 和 dtype；profiling 结果依赖输入 shape 和编译状态。复习时可以把这些限制改写成反例测试，观察它们怎样改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。把实验结论放回 Kernel、Triton 与 XLA 时，要说明 Benchmark/profiling 的基本闭环改变的是训练吞吐、推理延迟、显存峰值还是通信开销。能做出这种归因，才说明读者已经从接口使用进入系统解释。

4.8 概念辨析：Benchmark/profiling 的基本闭环

benchmark、profiling、CUDA event、warmup、speedup 常一起出现在系统讲解中，但它们分别指硬件对象、程序操作、瓶颈来源或测量指标。读者应先定位每个词在执行路径中的位置。区分这些词时，可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕 Benchmark/profiling 的基本闭环复习时，可以写一个小 benchmark：固定输入形状，改变一个系统变量，记录 profile trace 和端到端时间。

4.9 Benchmark/profiling 的基本闭环的小结

- 讨论 Benchmark/profiling 的基本闭环时，应同时报告问题规模、实验设置和主要观测变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。
- 如果方法声称更快，要区分计算减少、内存访问减少、通信隐藏和调度改善；这些机制不能用一个 tokens/s 数字互相替代。
- profile trace、roofline 估算和端到端 benchmark 应互相校验，单独一个指标不足以解释系统瓶颈。

5 Triton 的 programming model

现在进入 Triton 的 programming model。这一节把算法放到硬件和运行时中考察：同一个数学表达式在不同 batch、shape、GPU 拓扑和 kernel 实现下会得到不同的成本账本。

5.1 为什么要讨论 Triton 的 programming model

Triton 让用户以 block 为单位写 kernel，显式处理 offsets、mask、load、compute、store。elementwise GeLU、row-wise softmax、row sum 和 matmul+ReLU 是课程材料中的核心例子。高性能 kernel 的关键是减少 HBM 往返、利用 shared/on-chip memory、控制 block size 并让访存合并。Triton 的 programming model 把 Kernel、Triton 与 XLA 放进资源账本。接下来要看的不是单个 API，而是张量形状、内存层级、通信和调度怎样共同决定成本。

5.2 Triton 的 programming model 的机制

Triton 让用户以 block 为单位写 kernel，显式处理 offsets、mask、load、compute、store。在 Triton 的 programming model 中，核心对象是硬件和系统状态：同一算法会因为 memory hierarchy、通信拓扑或 kernel shape 不同而表现出完全不同的速度。elementwise GeLU、row-wise softmax、row sum 和 matmul+ReLU 是课程材料中的核心例子。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。高性能 kernel 的关键是减少 HBM 往返、利用 shared/on-chip memory、控制 block size 并让访存合并。可检验性来自 profile trace、roofline 估算和端到端延迟分解；如果这些证据互相矛盾，就要先解释 measurement setup。因此，Triton 的 programming model 的学习目标是建立一张成本地图：读者应能指出瓶颈在计算、内存、通信还是调度，并能设计一个小 benchmark 去验证。

5.3 Triton 的 programming model 的数学表达

$$\text{softmax}(x_i) = \frac{\exp(x_i - \max_j x_j)}{\sum_j \exp(x_j - \max_k x_k)}$$

符号说明：减去最大值是数值稳定技巧；softmax kernel 通常需要 reduction 和 row-wise normalization。这个关系式应被读成 Triton 的 programming model 的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。在系统实验中，tokens/s、TTFT、显存峰值和 kernel time 都是 proxy。它们各自只观察一部分系统，因此报告时要同时写清 workload shape、warmup、同步方式和硬件型号。

5.4 从 Triton 的 programming model 到程序

把 Triton 的 programming model 写成程序时，最重要的是暴露状态变化和数据移动。下面的伪代码保留执行顺序，省略框架样板。

```
pid = program_id(0)
offsets = pid * BLOCK + arange(0, BLOCK)
x = load(ptr + offsets, mask=offsets < n)
y = gelu(x)
store(out + offsets, y, mask=offsets < n)
```

阅读这段实现时，重点看 Triton 的 programming model 怎样移动张量、占用显存并触发通信。实验记录至少要包含吞吐、延迟、显存峰值、通信量和 kernel 利用率，否则很难判断瓶颈来自算子、内存层级还是调度。

5.5 边界条件与常见误解

- Triton 代码可读性高于手写 PTX/CUDA，但仍需要理解硬件。
- mask 处理错误会带来 silent correctness bugs。
- 小规模 kernel benchmark 不一定预测端到端训练或 serving latency，因为调度、通信和请求形状会改变瓶颈。
- 硬件规格表和框架 profiler 都需要结合 workload 解释，不能把单个峰值数字当作机制证明。

5.6 与本章其他概念的关系

Triton 的 programming model 连接本章的数学机制和工程成本：术语表说明对象，公式估算资源，伪代码暴露状态变化，例题则让读者检查一次完整的成本推理。

5.7 怎样检验 Triton 的 programming model

检验 Triton 的 programming model 时，先把抽象说法落到硬件和系统状态上。与 Triton、GeLU、softmax、mask、block 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明：减去最大值是数值稳定技巧；softmax kernel 通常需要 reduction 和 row-wise normalization。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。系统消融通常一次只改变 batch shape、sequence length、GPU 数、kernel 实现或调度策略。若同时改变硬件和 workload，实验只能说明端到端表现不同，不能定位瓶颈。最后检查失败条件。工程判断往往在反例中变清楚：Triton 代码可读性高于手写 PTX/CUDA，但仍需要理解硬件；mask 处理错误会带来 silent correctness bugs。复习时可以把这些限制改写成反例测试，观察它们怎样改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。把实验结论放回 Kernel、Triton 与 XLA 时，要说明 Triton 的 programming model 改变的是训练吞吐、推理延迟、显存峰值还是通信开销。能做出这种归因，才说明读者已经从接口使用进入系统解释。

5.8 概念辨析：Triton 的 programming model

Triton、GeLU、softmax、mask、block 常一起出现在系统讲解中，但它们分别指硬件对象、程序操作、瓶颈来源或测量指标。读者应先定位每个词在执行路径中的位置。其中，Triton 让课程把 kernel 优化写成可读的 block-level 程序。区分这些词时，可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕 Triton 的 programming model 复习时，可以写一个小 benchmark：固定输入形状，改变一个系统变量，记录 profile trace 和端到端时间。

5.9 Triton 的 programming model 的小结

- 讨论 Triton 的 programming model 时，应同时报告问题规模、实验设置和主要观测变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。

- 如果方法声称更快，要区分计算减少、内存访问减少、通信隐藏和调度改善；这些机制不能用一个 tokens/s 数字互相替代。
- profile trace、roofline 估算和端到端 benchmark 应互相校验，单独一个指标不足以解释系统瓶颈。

6 Fusion、tiling 与 compiler

现在进入 Fusion、tiling 与 compiler。这一节把算法放到硬件和运行时中考察：同一个数学表达式在不同 batch、shape、GPU 拓扑和 kernel 实现下会得到不同的成本账本。

6.1 为什么要讨论 Fusion、tiling 与 compiler

kernel fusion 把多个 elementwise/reduction 操作合并，减少中间张量写回 HBM。tiling 让 matmul 或 attention 在片上内存中复用数据，是 FlashAttention 等算法的核心系统思想。XLA/torch.compile 等 compiler 可以自动做一部分融合，但手写 kernel 仍常用于热点路径。Fusion、tiling 与 compiler 把 Kernel、Triton 与 XLA 放进资源账本。接下来要看的不是单个 API，而是张量形状、内存层级、通信和调度怎样共同决定成本。

6.2 Fusion、tiling 与 compiler 的机制

kernel fusion 把多个 elementwise/reduction 操作合并，减少中间张量写回 HBM。在 Fusion、tiling 与 compiler 中，核心对象是硬件和系统状态：同一算法会因为 memory hierarchy、通信拓扑或 kernel shape 不同而表现出完全不同的速度。tiling 让 matmul 或 attention 在片上内存中复用数据，是 FlashAttention 等算法的核心系统思想。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。XLA/torch.compile 等 compiler 可以自动做一部分融合，但手写 kernel 仍常用于热点路径。可检验性来自 profile trace、roofline 估算和端到端延迟分解；如果这些证据互相矛盾，就要先解释 measurement setup。因此，Fusion、tiling 与 compiler 的学习目标是建立一张成本地图：读者应能指出瓶颈在计算、内存、通信还是调度，并能设计一个小 benchmark 去验证。

6.3 Fusion、tiling 与 compiler 的数学表达

$$\text{HBM traffic}_{\text{fused}} < \sum_i \text{HBM traffic}_i$$

符号说明：融合的收益来自减少中间结果读写；但如果融合降低 occupancy 或增加寄存器压力，也可能不划算。这个关系式应被读成 Fusion、tiling 与 compiler 的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。在系统实验中，tokens/s、TTFT、显存峰值和 kernel time 都是 proxy。它们各自只观察一部分系统，因此报告时要同时写清 workload shape、warmup、同步方式和硬件型号。

6.4 从 Fusion、tiling 与 compiler 到程序

把 Fusion、tiling 与 compiler 写成程序时，最重要的是暴露状态变化和数据移动。下面的伪代码保留执行顺序，省略框架样板。

```
for q_tile in Q:
    for k_tile, v_tile in KV:
        update_online_softmax(q_tile, k_tile, v_tile)
```

`write_output_once()`

阅读这段实现时，重点看 Fusion、tiling 与 compiler 怎样移动张量、占用显存并触发通信。实验记录至少要包含吞吐、延迟、显存峰值、通信量和 kernel 利用率，否则很难判断瓶颈来自算子、内存层级还是调度。

6.5 边界条件与常见误解

- fusion 会增加 kernel 复杂度，debug 和数值验证成本上升。
- compiler 生成的 kernel 需要 profile 验证，不能默认最优。
- 小规模 kernel benchmark 不一定预测端到端训练或 serving latency，因为调度、通信和请求形状会改变瓶颈。
- 硬件规格表和框架 profiler 都需要结合 workload 解释，不能把单个峰值数字当作机制证明。

6.6 与本章其他概念的关系

Fusion、tiling 与 compiler 连接本章的数学机制和工程成本：术语表说明对象，公式估算资源，伪代码暴露状态变化，例题则让读者检查一次完整的成本推理。

6.7 怎样检验 Fusion、tiling 与 compiler

检验 Fusion、tiling 与 compiler 时，先把抽象说法落到硬件和系统状态上。与 fusion、tiling、FlashAttention、XLA、compiler 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明：融合的收益来自减少中间结果读写；但如果融合降低 occupancy 或增加寄存器压力，也可能不划算。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。系统消融通常一次只改变 batch shape、sequence length、GPU 数、kernel 实现或调度策略。若同时改变硬件和 workload，实验只能说明端到端表现不同，不能定位瓶颈。最后检查失败条件。工程判断往往在反例中变清楚：fusion 会增加 kernel 复杂度，debug 和数值验证成本上升；compiler 生成的 kernel 需要 profile 验证，不能默认最优。复习时可以把这些限制改写成反例测试，观察它们怎样改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。把实验结论放回 Kernel、Triton 与 XLA 时，要说明 Fusion、tiling 与 compiler 改变的是训练吞吐、推理延迟、显存峰值还是通信开销。能做出这种归因，才说明读者已经从接口使用进入系统解释。

6.8 概念辨析：Fusion、tiling 与 compiler

fusion、tiling、FlashAttention、XLA、compiler 常一起出现在系统讲解中，但它们分别指硬件对象、程序操作、瓶颈来源或测量指标。读者应先定位每个词在执行路径中的位置。其中，attention（注意力）提供 token 间交互，但长上下文下成本很高。区分这些词时，可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕 Fusion、tiling 与 compiler 复习时，可以写一个小 benchmark：固定输入形状，改变一个系统变量，记录 profile trace 和端到端时间。

6.9 Fusion、tiling 与 compiler 的小结

- 讨论 Fusion、tiling 与 compiler 时，应同时报告问题规模、实验设置和主要观测变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。
- 如果方法声称更快，要区分计算减少、内存访问减少、通信隐藏和调度改善；这些机制不能用一个 tokens/s 数字互相替代。
- profile trace、roofline 估算和端到端 benchmark 应互相校验，单独一个指标不足以解释系统瓶颈。

7 综合讨论：从局部机制到完整系统

本章的主题是 Kernel、Triton 与 XLA，但它不应被读成几个相邻知识点的拼接。Benchmark/profiling 的基本闭环、Triton 的 programming model、Fusion、tiling 与 compiler 共同回答同一个问题：语言模型系统中哪些对象可以被定义，哪些成本可以被估算，哪些结论必须通过实验或评估来确认。这些对象包括 kernel（内核）、Triton、XLA、profiling（性能剖析）、fusion（算子融合）、tiling（分块）。读者可以把它们看成一组接口：有的接口连接文本和 token，有的连接模型结构和张量计算，有的连接数据样本和训练目标，有的连接离线评估和在线服务。接口一旦改变，后面的成本、错误类型和优化空间也会随之改变。

7.1 Benchmark/profiling 的基本闭环在系统中的位置

Benchmark/profiling 的基本闭环首先提供一个局部解释：课程材料明确把本讲定位为 GPU 高层概览后的代码层深入：写 Triton kernels、benchmark 和 profiling。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，Benchmark/profiling 的基本闭环会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，Benchmark/profiling 的基本闭环不能脱离边界条件理解。一个关键 caveat 是：一次 benchmark 不足以说明 scaling；要扫维度、batch 和 dtype。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.2 Triton 的 programming model 在系统中的位置

Triton 的 programming model 首先提供一个局部解释：Triton 让用户以 block 为单位写 kernel，显式处理 offsets、mask、load、compute、store。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，Triton 的 programming model 会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，Triton 的 programming model 不能脱离边界条件理解。一个关键 caveat 是：Triton 代码可读性高于手写 PTX/CUDA，但仍需要理解硬件。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.3 Fusion、tiling 与 compiler 在系统中的位置

Fusion、tiling 与 compiler 首先提供一个局部解释：kernel fusion 把多个 elementwise/reduction 操作合并，减少中间张量写回 HBM。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，Fusion、tiling 与 compiler 会改变至

少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，Fusion、tiling 与 compiler 不能脱离边界条件理解。一个关键 caveat 是：fusion 会增加 kernel 复杂度，debug 和数值验证成本上升。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.4 把本章知识用于读论文和复现实验

读相关论文时，可以把本章变成一个检查表。第一，论文是否清楚定义了输入规模、模型规模、数据来源和评估协议；第二，论文声称的改进落在哪个变量上；第三，作者是否报告了会让结论失效的设置；第四，实验是否能分离模型结构、数据质量、优化设置和系统实现的影响。复现实验时，也应避免直接追求大规模。先用最小程序复现一个公式、一个伪代码分支或一个 profile 现象，再扩大输入规模。这样做的原因很简单：如果小规模程序无法解释中间量，大规模训练只会把错误隐藏在日志和随机波动里。把这些检查合在一起，本章给出的不是一个固定答案，而是一种阅读 CS336 的方法：每个模型机制都要落到变量，每个变量都要落到测量，每个测量都要说明协议，每个协议都要承认边界。因此，本章中的任何一句结论都可以被改写成一个可引用的完整句式：在给定数据、模型、硬件和评估协议下，某个机制改变了某个变量，并通过某个观测指标表现出来。这样的句子比单独背诵术语更长，但它包含了自学、复现和引用时真正需要的信息。把本章用于自学时，可以按三遍阅读。第一遍只追踪对象和定义，确认每个术语到底指向文本、token、张量、样本、请求还是评估记录。第二遍追踪公式和伪代码，确认每一步改变了哪些状态。第三遍追踪实验和 caveat，确认哪些结论只在特定规模、数据或硬件条件下成立。把本章用于复习时，则应反过来操作：先从一个失败案例或异常指标出发，再回到相应机制。loss 曲线异常可能指向数据、优化或模型容量；吞吐异常可能指向 kernel、通信或调度；benchmark 异常可能指向 contamination、prompt protocol 或采样设置。能够从异常反推机制，是掌握本章的实用标准。

8 例题：把概念变成可计算检查

8.1 Triton row-wise softmax 的稳定实现

一行 logits 需要做 softmax。直接 $\exp(x)$ 可能溢出，分多个 kernel 又会多次访问 HBM。

- 先在 block 内求 $m=\max(x)$ ，计算 $\exp(x-m)$ 。
- 再求和并归一化，把读、reduction、写尽量放在一个 kernel 中。
- mask 必须正确处理越界元素，否则会产生 silent numerical bugs。

结论。 Triton 的价值在于把数值稳定性和内存访问模式放在同一个可读程序里。这个例题的目的不是替代完整工程实现，而是给出一种最小推理形式：把问题转成变量，写出近似公式或伪代码，再说明近似何时失效。

9 实验设计：从小例子到可复现结论

学习 Kernel、Triton 与 XLA 不能停在概念层。一个教材化结论至少需要四件事：可构造的输入、可观测的中间量、可比较的指标，以及清楚的失败条件。下面的实验设计不是要求训练大模型，而是要求读者用小规模程序复现本章机制。

9.1 最小输入

先构造只触发本章核心机制的小输入。例如 tokenizer 章节可以用几个包含 rare words 和 Unicode 字符的字符串；GPU 章节可以用一个 elementwise kernel 和一个 GEMM；data 章节可以用十几篇近重复文档。小输入的价值在于它让错误可见。

9.2 资源账本

对同一输入写下 FLOPs、bytes moved、显存峰值、通信量或样本数量的估算。估算不需要一开始就精确，但必须说明单位和近似假设。随后用 profiler、benchmark、validation loss 或人工检查去验证估算是否同量级。

9.3 单变量消融

一次只改变一个因素，例如 vocabulary size、head 数、batch size、filter threshold、reward scale、decoding temperature 或 GPU 数量。若多个变量同时变化，实验只能说明系统变了，不能说明机制是什么。

9.4 失败条件

最后要刻意触发失败：极端 context length、错误 dtype、过小 batch、污染 benchmark、低质量数据或不可靠 verifier。失败条件常常比成功曲线更能说明一个方法的边界。

9.5 记录与报告

实验记录应能让另一个读者复现同一结论。最少要写清楚输入构造、代码版本、随机种子、硬件、batch shape、数据版本、评价指标和异常处理方式。报告结论时不要只写“更快”或“更好”，而要说明快在哪里、好在哪个指标上，以及是否牺牲了内存、稳定性、数据质量或可解释性。常见报告错误是把局部现象写成普遍规律。例如一次小模型实验里的 loss 改善，不等于更大模型或不同数据分布上也会改善；一次 kernel benchmark 的吞吐提升，不等于端到端 serving latency 一定降低。教材中的实验报告应始终把结论限定在可复现条件内。

9.6 从小实验到大结论

小实验的作用不是替代课程中的完整训练或系统实现，而是建立一条可检查的推理链。先在小输入上确认变量方向，再在中等规模上确认数量级，最后才讨论是否能外推到更大模型、更多 token 或更复杂 workload。缺少中间层级时，大结论通常只是在复述直觉。因此，实验报告最好同时包含正例和反例：正例说明机制在受控条件下怎样工作，反例说明条件稍变时哪里失效。对 CS336 这样的课程，反例尤其重要，因为 language model 的错误常来自多个层面叠加：数据分布、优化稳定性、硬件瓶颈、评估协议和服务负载都可能同时改变观察结果。还要记录资料版本与复现边界。公开视频、课程页、配套代码和 PDF 可能在不同时间更新；复现实验应写明使用的材料版本，并说明哪些结论来自课程事实，哪些是为了帮助理解而加入的解释性推导。这样才能把学习笔记变成可检查的技术记录。若复现实验与课程结论不一致，首先检查输入、版本和评估协议，而不是立即修改模型假设。

9.7 把本章重新读成一条证据链

完成小实验之后，可以把 Kernel、Triton 与 XLA 重新读成一条证据链：先用“Benchmark/profiling 的基本闭环”确定问题入口，再用“Triton 的 programming model”解释主要机制，最后用“Fusion、tiling 与 compiler”检查边界或外推条件。这种读法比按页背诵更接近教材训练，因为它要求每个结论都能说明前提、

变量和证据。本章反复使用的术语包括 kernel（内核）、Triton、XLA、profiling（性能剖析）、fusion（算子融合）、tiling（分块）。复习时不要只写定义，而要为每个术语补上一句操作性说明：它在课程代码、公式、数据记录或评估协议中对应什么对象；如果这个对象被删除、替换或缩放，哪一个观测量会变化。最后，用一个反例结束复习。反例可以是资源估算不准、数据污染、reward 失真、benchmark 解析失败、长上下文显存爆炸或 scaling 外推失效。能解释反例的章节，才真正具备自学和引用价值；不能解释反例的章节，只是把课程材料换了一种排版。引用本章结论时，还应保留来源边界：哪些说法直接来自视频、课程页、slides 或代码，哪些是为了串联教材叙述而加入的解释性推导。这个边界不会削弱结论，反而让读者知道何处可以复核，何处需要在自己的实验中重新验证。如果后续课程材料更新，也可以沿着同一证据链替换来源，而不必重写整章结构。

10 课程资料与回看路径

教材正文已经把主要概念、公式和实现逻辑组织成连续叙述；本节只提供回到课程材料的阅读路径。读者复习时可以先完成前文练习，再按下面的时间段或资料组核对细节。

10.1 视频回看路径

时间	关键词	复习任务
00:00:05-00:07:20	thread, then, going, there's, memory, threads, think	回看定义、例子、推导或 caveat 在该段中的位置。
00:21:09-00:28:32	then, going, your, time, profiling, it's, think	回看定义、例子、推导或 caveat 在该段中的位置。
00:43:33-00:50:41	going, then, it's, blocksize, block, memory, yeah	回看定义、例子、推导或 caveat 在该段中的位置。
00:58:06-01:05:21	going, then, number, softmax, think, row., that's	回看定义、例子、推导或 caveat 在该段中的位置。
01:19:28-01:26:30	then, going, there's, write, think, time, tile	回看定义、例子、推导或 caveat 在该段中的位置。

这些时间段用于定位视频中的讲解顺序。严肃引用时，应回到视频片段确认讲者表述、板书或幻灯片内容。回看视频时，不必逐字抄录字幕。更有效的方法是把每个时间段判定为定义、例子、推导、代码解释或 caveat，再把它放回本章对应小节。

10.2 课程资料阅读路径

资料组	标题/页块	关键词
official_01	Last lecture: high-level overview of GPUs and performance	cache, shared, memory, size, bandwidth, performance, triton
official_03	TTTT TTTT 	threads, thread, bank, warp, registers, same, multiple
official_05	You can use ...	time, pytorch, recipes, cuda, benchmark, https, html

official_08	- Multiply and accumulate.	tile, shared, memory, arithmetic, intensity, output, accumulate
official_10	Function: triton_row_sum	function, triton_row_sum, row_sum_kernel, triton_matmul_relu_example, naive_matmul_relu, triton_matmul_relu, matmul_relu_kernel

课程资料通常给出更稳定的标题、公式、图或代码结构；视频则补充动机和口头解释。两者合起来构成本章的事实边界。阅读资料时，可以把每个资料组拆成三个对象：一个定义，一个公式或伪代码动作，一个边界条件。这样比逐字翻译页面或脚本更容易形成可用知识。

11 实践说明、历史位置与风险

Kernel、Triton 与 XLA 在 CS336 的课程结构中连接基础机制和规模化问题。基础机制解释方法为什么成立；规模化问题检验它在更大模型、更长上下文、更复杂数据或真实 serving workload 下是否仍成立。历史位置不等于模型年表。更重要的是识别问题如何迁移：早期方法解决小规模建模问题，现代方法常常解决同一问题在大规模数据、GPU 集群、长上下文或人类偏好约束下的新形态。

- 资料版本限制 (version risk): 课程页、公开视频和配套资料可能在学期中更新；引用时应注明访问日期或资料版本。
- 测量口径限制 (measurement risk): FLOPs、tokens/s、benchmark score、reward 等数字只有在协议完整时可比较。
- 规模外推限制 (scaling risk): 小模型或小 batch 的机制不必然外推到 frontier-scale。
- 数据分布限制 (data risk): 数据来源、过滤和去重策略会改变模型行为，也会改变 evaluation contamination 风险。
- 系统风险 (systems risk): 性能剖析结果依赖硬件、driver、kernel、shape 和 batch distribution；脱离 workload 的性能数字不可直接复用。

12 复习要点

下面的问题把本章内容压缩为可反复检查的条目。每个条目都应能回到前文的解释、公式、伪代码或例题。

条目	对象	检查动作
条目 1	kernel (内核)	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。
条目 2	Triton	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。

条目 3	XLA	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。
条目 4	profiling（性能剖析）	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。
条目 5	fusion（算子融合）	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。
条目 6	tiling（分块）	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。

12.1 综合自测

- 我能否不用课程原句，独立解释本章中心问题？
- 我能否把本章至少一个公式改写成可运行的估算函数？
- 我能否指出一个看似改进但实际转移成本的方法？
- 我能否回到视频和课程资料，找到支持本章关键论断的位置？
- 我能否设计一个最小 ablation，验证本章一个 caveat？

13 总结与延伸

13.1 本章小结

- Kernel、Triton 与 XLA 的核心不是记忆术语，而是理解对象、变量和机制之间的关系。
- 视频给出讲解顺序，课程资料提供可核对的公式、图、代码或页面结构。
- 复习时应把每个术语连接到至少一个变量、一个实现动作和一个失败模式。

13.2 延伸解释

本章中的延伸解释用于连接课程材料中的概念，例如说明公式里的 proxy、解释系统 tradeoff，或把多个材料片段串成一个完整推理链。若需要严肃引用，应回到课程视频和配套资料核对原始表述。

13.3 练习

- 定义题：用中英双语解释本章任意五个重要术语，并说明它们属于 compute、memory、data、optimization、evaluation 还是 deployment 账本。
- 推导题：选择本章一个公式，逐个解释符号、单位和近似假设，并说明哪个变量最难在真实系统中测量。
- 实现题：把本章伪代码改写成一个最小 Python sanity check，不要求训练大模型，但要输出可检查的中间量。

- 资料题：从“课程资料与回看路径”中选择一个视频时间段，回到原始视频核对讲者例子，并记录是否存在字幕误差。
- 评估题：为本章方法设计一个 ablation 或 benchmark，说明控制变量、数据版本、指标和 failure criteria。
- 边界题：说明本章一个结论在更长 context、更大 batch、更多 GPU 或更复杂数据分布下可能如何失效。
- 综合题：把本章内容和前一章或后一章连接起来，写出一个端到端训练/推理 pipeline 中的依赖关系。
- 批判题：指出一个容易被营销材料夸大的说法，并用本章的资源账本或评估协议反驳它。

7 07 多 GPU 并行基础

第 07 讲：多 GPU 并行基础

CS336 Spring 2026 public videos and official course materials

目录

1 本章导读	2
1.1 本章知识路线	2
1.2 结构图	2
2 本章主线	2
2.1 本章要解决的核心问题	2
2.2 从机制到资源账本	3
2.3 从课堂讲解到可复现实验	3
3 术语、符号与问题边界	4
3.1 术语之间的关系	4
4 为什么单 GPU 不够	4
4.1 课程目标与问题边界	4
4.2 为什么单 GPU 不够的机制	5
4.3 资源、效率与效果的关系	5
4.4 把课程目标写成检查流程	5
4.5 边界条件与常见误解	5
4.6 与本章其他概念的关系	5
4.7 怎样检验这一课程目标	6
4.8 概念辨析：为什么单 GPU 不够	6
4.9 为什么单 GPU 不够的小结	6
5 Data parallelism 与 all-reduce	6
5.1 为什么要讨论 Data parallelism 与 all-reduce	6
5.2 Data parallelism 与 all-reduce 的机制	7
5.3 Data parallelism 与 all-reduce 的资源账本	7
5.4 Data parallelism 与 all-reduce 在数据管线中的实现	7
5.5 边界条件与常见误解	7

5.6	与本章其他概念的关系	7
5.7	怎样检验 Data parallelism 与 all-reduce	8
5.8	概念辨析: Data parallelism 与 all-reduce	8
5.9	Data parallelism 与 all-reduce 的小结	8
6	Tensor/pipeline parallelism 的切分直觉	8
6.1	为什么要讨论 Tensor/pipeline parallelism 的切分直觉	8
6.2	Tensor/pipeline parallelism 的切分直觉的机制	9
6.3	Tensor/pipeline parallelism 的切分直觉的资源账本	9
6.4	从 Tensor/pipeline parallelism 的切分直觉到程序	9
6.5	边界条件与常见误解	9
6.6	与本章其他概念的关系	9
6.7	怎样检验 Tensor/pipeline parallelism 的切分直觉	10
6.8	概念辨析: Tensor/pipeline parallelism 的切分直觉	10
6.9	Tensor/pipeline parallelism 的切分直觉的小结	10
7	综合讨论: 从局部机制到完整系统	10
7.1	为什么单 GPU 不够在系统中的位置	11
7.2	Data parallelism 与 all-reduce 在系统中的位置	11
7.3	Tensor/pipeline parallelism 的切分直觉在系统中的位置	11
7.4	把本章知识用于读论文和复现实验	11
8	例题: 把概念变成可计算检查	12
8.1	Data parallel all-reduce 的通信账本	12
9	实验设计: 从小例子到可复现结论	12
9.1	最小输入	12
9.2	资源账本	12
9.3	单变量消融	12
9.4	失败条件	12
9.5	记录与报告	13
9.6	从小实验到大结论	13
9.7	把本章重新读成一条证据链	13
10	课程资料与回看路径	13
10.1	视频回看路径	13
10.2	课程资料阅读路径	14
11	实践说明、历史位置与风险	14

12 复习要点

15

12.1 综合自测

15

13 总结与延伸

15

13.1 本章小结

15

13.2 延伸解释

16

13.3 练习

16

1 本章导读

本章讨论 **多 GPU 并行基础**。CS336 的第一层目标不是把现成大模型当作黑箱使用，而是把 language model 从输入文本、训练目标、模型结构、数据、系统和评估这些部件重新拆开。只有拆开之后，读者才能判断一个设计选择究竟改变了什么。设想单张 GPU 已经放不下模型，或者单卡训练时间不可接受。并行训练不是简单把 GPU 数乘上去，而是要决定哪些张量复制、哪些切分、哪些通过 all-reduce、all-gather 或 reduce-scatter 通信。本章对应 CS336 Spring 2026 第 07 个公开视频，并结合课程页提供的可解析资料。视频和课程页给出事实边界；正文把这些材料改写为可自学的教材章节。阅读本章时，先理解问题为什么存在，再看定义、公式和实现形态，随后通过例题和练习检查自己是否能独立复现这条 reasoning chain。

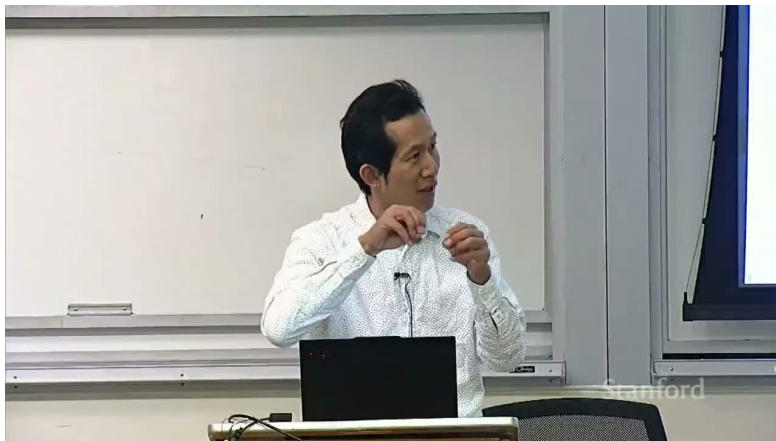


图 1: 第 07 讲的课程图像锚点。

1.1 本章知识路线

- 本章首先说明多 GPU 并行基础为什么是 language modeling pipeline 中不可跳过的一环。
- 其次定义本章反复使用的术语，并说明这些术语对应的计算对象或系统对象。
- 随后用公式和伪代码表达主要机制，让概念能够落到可计算的变量上。
- 最后给出例题、风险讨论和练习，便于读者检查自己是否真正掌握了机制。

1.2 结构图

下面的图用于帮助读者定位本章的核心结构。先看概念之间的依赖，再回到正文中的公式、伪代码和例题。

2 本章主线

多 GPU 并行基础的主线是把 language model 的抽象计算放回真实机器：哪些张量要移动，哪些状态要保存，哪些瓶颈会在训练或服务时显现。

2.1 本章要解决的核心问题

本讲把上一周的单 GPU 层级扩展为多 GPU/多节点层级：NVLink、NVSwitch、InfiniBand、Ethernet 和 RDMA 都会进入通信账本。读完这一段后，应能用一两句话说明多 GPU 并行基础要解决的瓶颈，并指出这

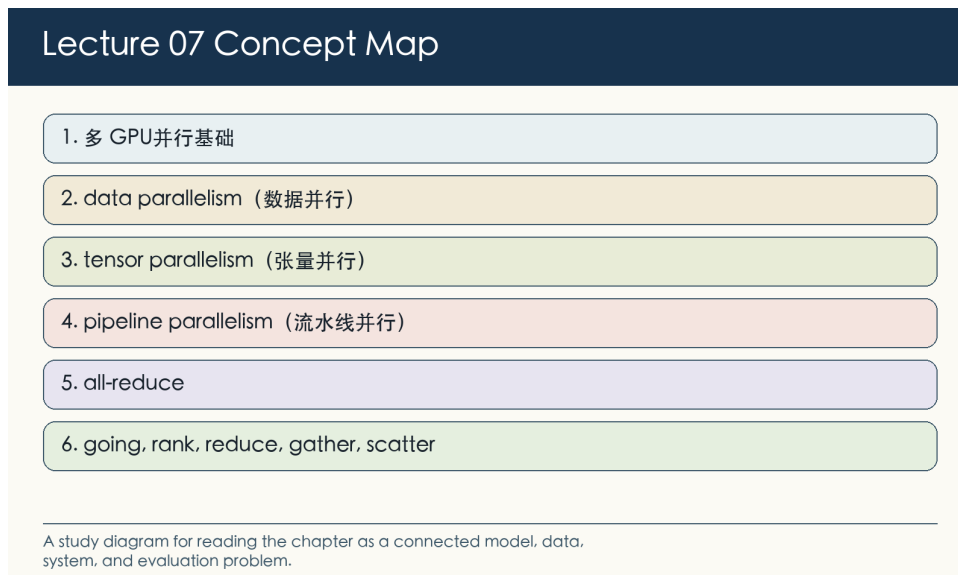


图 2: 第 07 讲概念图: 本章问题、术语和机制的关系。

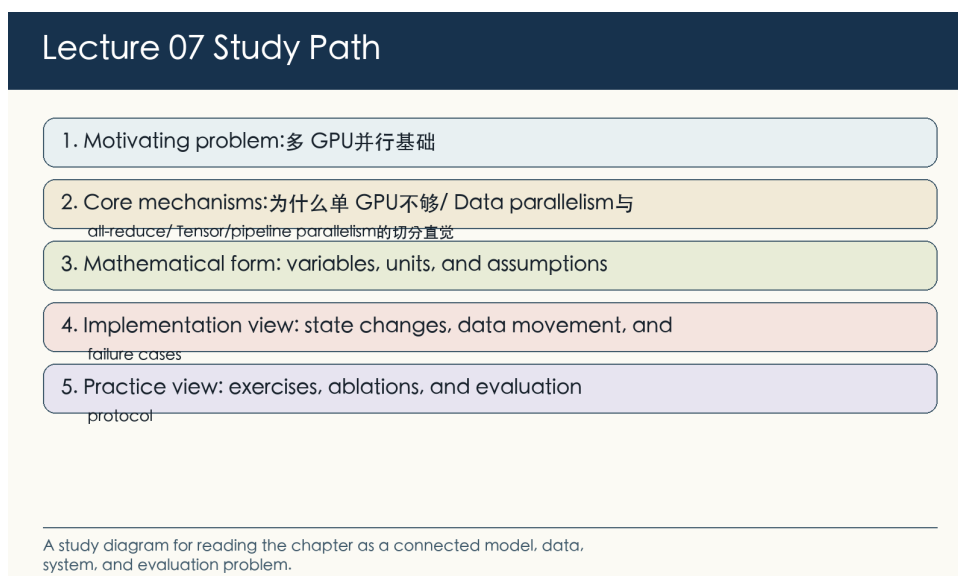


图 3: 第 07 讲学习路径: 从问题定义进入公式、实现、实验和练习。

个瓶颈发生在数据、模型、优化、系统还是评估层。

2.2 从机制到资源账本

collective communication 是并行训练的语言。broadcast、scatter、gather、reduce、all-gather、reduce-scatter、all-reduce、all-to-all 分别对应不同的数据移动模式。随后把机制写成账本: 输入规模是什么, 主要状态是什么, 成本或质量指标怎样随变量改变。这个账本让后面的公式不再只是符号。

2.3 从课堂讲解到可复现实验

data parallel、tensor parallel、pipeline parallel 和 sequence/expert parallel 的选择取决于模型能否放下、互连是否足够快、通信能否和计算重叠。最后把课堂讲解转成可复现实验: 固定协议, 改变一个变量, 记录中间量, 并解释结果为何支持或反驳原来的机制判断。

3 术语、符号与问题边界

本章的术语横跨模型、数据和系统。读者需要先知道每个词指向的对象：有些词描述数学目标，有些词描述张量形状，有些词描述硬件瓶颈，还有一些词描述评估协议。术语边界不清，后面的公式和实现就会变成空洞的缩写。

#	术语	阅读要求
1	data parallelism（数据并行）	说明它如何改变训练样本或 token 分布，并给出一个会改变结论的数据边界条件。
2	tensor parallelism（张量并行）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
3	pipeline parallelism（流水线并行）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
4	all-reduce	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
5	communication cost（通信成本）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。

这些术语之间有依赖关系。例如 tokenization 改变 sequence length，sequence length 又改变 attention FLOPs、KV cache 和 evaluation token budget；parallelism 改变显存占用，也改变通信瓶颈和故障恢复方式。

3.1 术语之间的关系

在本章中，data parallelism（数据并行）给出问题入口，tensor parallelism（张量并行）描述主要机制，pipeline parallelism（流水线并行）则把机制连接到可测量的结果。读教材时应当沿着这个链条追问：输入是什么，中间状态怎样改变，输出指标为什么随之变化。课程材料还会反复出现 data、rank、gpus、parallelism、e.g.、ranks 等词。它们不是一组同义标签，而是提示本章的不同层次：有的指对象，有的指操作，有的指约束或指标。因此，术语表不是背诵清单，而是后文阅读的索引。遇到一个术语时，先定位它属于 representation、optimization、systems、data 还是 evaluation，再判断它改变哪一项账本。

4 为什么单 GPU 不够

本节讨论为什么单 GPU 不够。它先限定多 GPU 并行基础要回答的基本问题：哪些现象需要解释，哪些变量可以测量，哪些结论必须回到吞吐、延迟、显存峰值、通信量和 kernel 利用率中检验。

4.1 课程目标与问题边界

视频从上一周单 GPU kernel 优化过渡到多 GPU：目标是利用多张 GPU 训练更大模型或更快完成训练。并行不是免费加速；参数、activation、optimizer state、gradient 和数据都可能需要复制、切分或通信。选择并行策略要同时满足显存容量、计算吞吐和互连带宽约束。系统部分的核心问题是把数学计算映射到硬件。一个算法的复杂度公式只能给出方向；真正的速度还取决于内存层级、通信拓扑、kernel shape 和调度方式。

4.2 为什么单 GPU 不够的机制

视频从上一周单 GPU kernel 优化过渡到多 GPU：目标是利用多张 GPU 训练更大模型或更快完成训练。在为什么单 GPU 不够中，核心对象是硬件和系统状态：同一算法会因为 memory hierarchy、通信拓扑或 kernel shape 不同而表现出完全不同的速度。并行不是免费加速；参数、activation、optimizer state、gradient 和数据都可能需要复制、切分或通信。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。选择并行策略要同时满足显存容量、计算吞吐和互连带宽约束。可检验性来自 profile trace、roofline 估算和端到端延迟分解；如果这些证据互相矛盾，就要先解释 measurement setup。因此，为什么单 GPU 不够的学习目标是建立一张成本地图：读者应能指出瓶颈在计算、内存、通信还是调度，并能设计一个小 benchmark 去验证。

4.3 资源、效率与效果的关系

$$\text{speedup}(n) = \frac{T_1}{T_n}, \quad \text{efficiency}(n) = \frac{\text{speedup}(n)}{n}$$

符号说明： T_1 是单设备时间， T_n 是 n 设备时间；efficiency 衡量扩展损失。这个关系式应被读成为什么单 GPU 不够的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。在系统实验中，tokens/s、TTFT、显存峰值和 kernel time 都是 proxy。它们各自只观察一部分系统，因此报告时要同时写清 workload shape、warmup、同步方式和硬件型号。

4.4 把课程目标写成检查流程

把为什么单 GPU 不够写成程序时，最重要的是暴露状态变化和数据移动。下面的伪代码保留执行顺序，省略框架样板。

```
if model_fits_each_gpu:
    start_with_data_parallel()
else:
    shard_parameters_and_activations()
```

阅读这段实现时，重点看为什么单 GPU 不够怎样移动张量、占用显存并触发通信。实验记录至少要包含吞吐、延迟、显存峰值、通信量和 kernel 利用率，否则很难判断瓶颈来自算子、内存层级还是调度。

4.5 边界条件与常见误解

- 能跑起来不等于高效；通信可能吞掉全部扩展收益。
- 并行策略常和 batch size、sequence length、optimizer state 绑定。
- 小规模 kernel benchmark 不一定预测端到端训练或 serving latency，因为调度、通信和请求形状会改变瓶颈。
- 硬件规格表和框架 profiler 都需要结合 workload 解释，不能把单个峰值数字当作机制证明。

4.6 与本章其他概念的关系

为什么单 GPU 不够连接本章的数学机制和工程成本：术语表说明对象，公式估算资源，伪代码暴露状态变化，例题则让读者检查一次完整的成本推理。

4.7 怎样检验这一课程目标

检验这一课程目标时，先把抽象说法落到硬件和系统状态上。与 parallelism、multi GPU、speedup、efficiency、communication 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： T_1 是单设备时间， T_n 是 n 设备时间；efficiency 衡量扩展损失。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。系统消融通常一次只改变 batch shape、sequence length、GPU 数、kernel 实现或调度策略。若同时改变硬件和 workload，实验只能说明端到端表现不同，不能定位瓶颈。最后检查失败条件。工程判断往往在反例中变清楚：能跑起来不等于高效；通信可能吞掉全部扩展收益；并行策略常和 batch size、sequence length、optimizer state 绑定。复习时可以把这些限制改写成反例测试，观察它们怎样改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。把实验结论放回多 GPU 并行基础时，要说明为什么单 GPU 不够改变的是训练吞吐、推理延迟、显存峰值还是通信开销。能做出这种归因，才说明读者已经从接口使用进入系统解释。

4.8 概念辨析：为什么单 GPU 不够

parallelism、multi GPU、speedup、efficiency、communication 常一起出现在系统讲解中，但它们分别指硬件对象、程序操作、瓶颈来源或测量指标。读者应先定位每个词在执行路径中的位置。其中，parallelism（并行）把模型、数据、序列或专家切分到多设备；GPU 是 CS336 systems 部分的主要硬件对象。区分这些词时，可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕为什么单 GPU 不够复习时，可以写一个小 benchmark：固定输入形状，改变一个系统变量，记录 profile trace 和端到端时间。

4.9 为什么单 GPU 不够的小结

- 讨论为什么单 GPU 不够时，应同时报告问题规模、实验设置和主要观测变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。
- 如果方法声称更快，要区分计算减少、内存访问减少、通信隐藏和调度改善；这些机制不能用一个 tokens/s 数字互相替代。
- profile trace、roofline 估算和端到端 benchmark 应互相校验，单独一个指标不足以解释系统瓶颈。

5 Data parallelism 与 all-reduce

现在进入 Data parallelism 与 all-reduce。这一节把算法放到硬件和运行时中考察：同一个数学表达式在不同 batch、shape、GPU 拓扑和 kernel 实现下会得到不同的成本账本。

5.1 为什么要讨论 Data parallelism 与 all-reduce

data parallelism 复制模型，把 batch 切给不同 GPU，反向传播后聚合梯度。all-reduce 是核心通信原语：每个设备最终得到全局梯度和。梯度累积可以在显存有限时模拟更大 batch，也可以减少通信频率，但会改变优化动态。数据部分的核心问题是分布控制。采集、过滤、去重和混合并不会只改变文件大小，它们会改变模型看到的语言、知识、偏差和 benchmark contamination 风险。

5.2 Data parallelism 与 all-reduce 的机制

data parallelism 复制模型，把 batch 切给不同 GPU，反向传播后聚合梯度。在 Data parallelism 与 all-reduce 中，核心对象是硬件和系统状态：同一算法会因为 memory hierarchy、通信拓扑或 kernel shape 不同而表现出完全不同的速度。all-reduce 是核心通信原语：每个设备最终得到全局梯度和。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。梯度累积可以在显存有限时模拟更大 batch，也可以减少通信频率，但会改变优化动态。可检验性来自 profile trace、roofline 估算和端到端延迟分解；如果这些证据互相矛盾，就要先解释 measurement setup。因此，Data parallelism 与 all-reduce 的学习目标是建立一张成本地图：读者应能指出瓶颈在计算、内存、通信还是调度，并能设计一个小 benchmark 去验证。

5.3 Data parallelism 与 all-reduce 的资源账本

$$g = \frac{1}{n} \sum_{i=1}^n g_i, \quad \theta \leftarrow \theta - \eta g$$

符号说明： g_i 是第 i 个 worker 的局部梯度；all-reduce 后每个 worker 使用同一个全局梯度更新。这个关系式应被读成 Data parallelism 与 all-reduce 的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。在系统实验中，tokens/s、TTFT、显存峰值和 kernel time 都是 proxy。它们各自只观察一部分系统，因此报告时要同时写清 workload shape、warmup、同步方式和硬件型号。

5.4 Data parallelism 与 all-reduce 在数据管线中的实现

把 Data parallelism 与 all-reduce 写成程序时，最重要的是暴露状态变化和数据移动。下面的伪代码保留执行顺序，省略框架样板。

```
loss.backward()
all_reduce(param.grad, op='sum')
param.grad /= world_size
optimizer.step()
```

阅读这段实现时，重点看 Data parallelism 与 all-reduce 怎样移动张量、占用显存并触发通信。实验记录至少要包含吞吐、延迟、显存峰值、通信量和 kernel 利用率，否则很难判断瓶颈来自算子、内存层级还是调度。

5.5 边界条件与常见误解

- global batch 变大后 learning rate schedule 可能需要调整。
- all-reduce 成本与参数量和网络拓扑有关，不只与 GPU 数量有关。
- 小规模 kernel benchmark 不一定预测端到端训练或 serving latency，因为调度、通信和请求形状会改变瓶颈。
- 硬件规格表和框架 profiler 都需要结合 workload 解释，不能把单个峰值数字当作机制证明。

5.6 与本章其他概念的关系

Data parallelism 与 all-reduce 连接本章的数学机制和工程成本：术语表说明对象，公式估算资源，伪代码暴露状态变化，例题则让读者检查一次完整的成本推理。

5.7 怎样检验 Data parallelism 与 all-reduce

检验 Data parallelism 与 all-reduce 时, 先把抽象说法落到硬件和系统状态上。与 data parallel、all-reduce、gradient、batch、optimizer 相关的对象必须能被构造、打印、计数或评分; 否则实验只能得到描述性观察, 不能得到可复现结论。一个合适的小实验应当保留本节机制, 同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真; 关键是读者能手算或打印关键中间量, 并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明: (g_i) 是第 (i) 个 worker 的局部梯度; all-reduce 后每个 worker 使用同一个全局梯度更新。如果数量只能间接观测, 就必须说明使用了什么 proxy; 如果使用 benchmark 或 reward, 也要说明协议和随机性。系统消融通常一次只改变 batch shape、sequence length、GPU 数、kernel 实现或调度策略。若同时改变硬件和 workload, 实验只能说明端到端表现不同, 不能定位瓶颈。最后检查失败条件。工程判断往往在反例中变清楚: global batch 变大后 learning rate schedule 可能需要调整; all-reduce 成本与参数量和 network 拓扑有关, 不只与 GPU 数量有关。复习时可以把这些限制改写成反例测试, 观察它们怎样改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。把实验结论放回多 GPU 并行基础时, 要说明 Data parallelism 与 all-reduce 改变的是训练吞吐、推理延迟、显存峰值还是通信开销。能做出这种归因, 才说明读者已经从接口使用进入系统解释。

5.8 概念辨析: Data parallelism 与 all-reduce

data parallel、all-reduce、gradient、batch、optimizer 常一起出现在系统讲解中, 但它们分别指硬件对象、程序操作、瓶颈来源或测量指标。读者应先定位每个词在执行路径中的位置。其中, data (数据) 是模型能力、偏差和法律风险的主要来源; all-reduce 聚合各 rank 梯度, 是 data parallel training 的核心 collective。区分这些词时, 可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。一个术语如果不能对应到变量、状态、代码路径或评估协议, 就还没有形成可操作含义。围绕 Data parallelism 与 all-reduce 复习时, 可以写一个小 benchmark: 固定输入形状, 改变一个系统变量, 记录 profile trace 和端到端时间。

5.9 Data parallelism 与 all-reduce 的小结

- 讨论 Data parallelism 与 all-reduce 时, 应同时报告问题规模、实验设置和主要观测变量, 尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。
- 如果方法声称更快, 要区分计算减少、内存访问减少、通信隐藏和调度改善; 这些机制不能用一个 tokens/s 数字互相替代。
- profile trace、roofline 估算和端到端 benchmark 应互相校验, 单独一个指标不足以解释系统瓶颈。

6 Tensor/pipeline parallelism 的切分直觉

现在进入 Tensor/pipeline parallelism 的切分直觉。这一节把算法放到硬件和运行时中考察: 同一个数学表达式在不同 batch、shape、GPU 拓扑和 kernel 实现下会得到不同的成本账本。

6.1 为什么要讨论 Tensor/pipeline parallelism 的切分直觉

tensor parallelism 切矩阵或 head, 让单层计算分布到多设备; pipeline parallelism 切层, 把不同 micro-batches 送入流水线。pipeline bubble 是流水线并行的典型低效来源; micro-batching 是主要缓解手段。真正的训练系统常混合 data、tensor、pipeline、sequence/context parallelism。系统部分的核心问题是把数学计算映射到硬件。一个算法的复杂度公式只能给出方向; 真正的速度还取决于内存层级、通信拓扑、kernel shape 和调度方式。

6.2 Tensor/pipeline parallelism 的切分直觉的机制

tensor parallelism 切矩阵或 head，让单层计算分布到多设备；pipeline parallelism 切层，把不同 micro-batches 送入流水线。在 Tensor/pipeline parallelism 的切分直觉中，核心对象是硬件和系统状态：同一算法会因为 memory hierarchy、通信拓扑或 kernel shape 不同而表现出完全不同的速度。pipeline bubble 是流水线并行的典型低效来源；micro-batching 是主要缓解手段。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。真正的训练系统常混合 data、tensor、pipeline、sequence/context parallelism。可检验性来自 profile trace、roofline 估算和端到端延迟分解；如果这些证据互相矛盾，就要先解释 measurement setup。因此，Tensor/pipeline parallelism 的切分直觉的学习目标是建立一张成本地图：读者应能指出瓶颈在计算、内存、通信还是调度，并能设计一个小 benchmark 去验证。

6.3 Tensor/pipeline parallelism 的切分直觉的资源账本

$$\text{pipeline bubble fraction} \approx \frac{p-1}{m+p-1}$$

符号说明： p 是 pipeline stages， m 是 micro-batches；micro-batches 越多，bubble 比例越低。这个关系式应被读成 Tensor/pipeline parallelism 的切分直觉的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。在系统实验中，tokens/s、TTFT、显存峰值和 kernel time 都是 proxy。它们各自只观察一部分系统，因此报告时要同时写清 workload shape、warmup、同步方式和硬件型号。

6.4 从 Tensor/pipeline parallelism 的切分直觉到程序

把 Tensor/pipeline parallelism 的切分直觉写成程序时，最重要的是暴露状态变化和数据移动。下面的伪代码保留执行顺序，省略框架样板。

```
split_layers_across_stages(model)
for microbatch in schedule:
    forward_stage_then_backward_stage(microbatch)
```

阅读这段实现时，重点看 Tensor/pipeline parallelism 的切分直觉怎样移动张量、占用显存并触发通信。实验记录至少要包含吞吐、延迟、显存峰值、通信量和 kernel 利用率，否则很难判断瓶颈来自算子、内存层级还是调度。

6.5 边界条件与常见误解

- micro-batch 增多会增加 activation 管理复杂度。
- tensor parallelism 需要高带宽互连，否则 matmul 中间结果通信会成为瓶颈。
- 小规模 kernel benchmark 不一定预测端到端训练或 serving latency，因为调度、通信和请求形状会改变瓶颈。
- 硬件规格表和框架 profiler 都需要结合 workload 解释，不能把单个峰值数字当作机制证明。

6.6 与本章其他概念的关系

Tensor/pipeline parallelism 的切分直觉连接本章的数学机制和工程成本：术语表说明对象，公式估算资源，伪代码暴露状态变化，例题则让读者检查一次完整的成本推理。

6.7 怎样检验 Tensor/pipeline parallelism 的切分直觉

检验 Tensor/pipeline parallelism 的切分直觉时，先把抽象说法落到硬件和系统状态上。与 tensor parallel、pipeline、microbatch、bubble、stage 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： $\backslash(p\backslash)$ 是 pipeline stages, $\backslash(m\backslash)$ 是 micro-batches；micro-batches 越多，bubble 比例越低。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。系统消融通常一次只改变 batch shape、sequence length、GPU 数、kernel 实现或调度策略。若同时改变硬件和 workload，实验只能说明端到端表现不同，不能定位瓶颈。最后检查失败条件。工程判断往往在反例中变清楚：micro-batch 增多会增加 activation 管理复杂度；tensor parallelism 需要高带宽互连，否则 matmul 中间结果通信会成为瓶颈。复习时可以把这些限制改写成反例测试，观察它们怎样改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。把实验结论放回多 GPU 并行基础时，要说明 Tensor/pipeline parallelism 的切分直觉改变的是训练吞吐、推理延迟、显存峰值还是通信开销。能做出这种归因，才说明读者已经从接口使用进入系统解释。

6.8 概念辨析：Tensor/pipeline parallelism 的切分直觉

tensor parallel、pipeline、microbatch、bubble、stage 常一起出现在系统讲解中，但它们分别指硬件对象、程序操作、瓶颈来源或测量指标。读者应先定位每个词在执行路径中的位置。其中，tensor（张量）是参数、激活、梯度和数据的共同表示。区分这些词时，可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕 Tensor/pipeline parallelism 的切分直觉复习时，可以写一个小 benchmark：固定输入形状，改变一个系统变量，记录 profile trace 和端到端时间。

6.9 Tensor/pipeline parallelism 的切分直觉的小结

- 讨论 Tensor/pipeline parallelism 的切分直觉时，应同时报告问题规模、实验设置和主要观测变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。
- 如果方法声称更快，要区分计算减少、内存访问减少、通信隐藏和调度改善；这些机制不能用一个 tokens/s 数字互相替代。
- profile trace、roofline 估算和端到端 benchmark 应互相校验，单独一个指标不足以解释系统瓶颈。

7 综合讨论：从局部机制到完整系统

本章的主题是多 GPU 并行基础，但它不应被读成几个相邻知识点的拼接。为什么单 GPU 不够、Data parallelism 与 all-reduce、Tensor/pipeline parallelism 的切分直觉共同回答同一个问题：语言模型系统中哪些对象可以被定义，哪些成本可以被估算，哪些结论必须通过实验或评估来确认。这些对象包括 data parallelism（数据并行）、tensor parallelism（张量并行）、pipeline parallelism（流水线并行）、all-reduce、communication cost（通信成本）。读者可以把它们看成一组接口：有的接口连接文本和 token，有的连接模型结构和张量计算，有的连接数据样本和训练目标，有的连接离线评估和在线服务。接口一旦改变，后面的成本、错误类型和优化空间也会随之改变。

7.1 为什么单 GPU 不够在系统中的位置

为什么单 GPU 不够首先提供一个局部解释：视频从上一周单 GPU kernel 优化过渡到多 GPU：目标是利用多张 GPU 训练更大模型或更快完成训练。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，为什么单 GPU 不够会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，为什么单 GPU 不够不能脱离边界条件理解。一个关键 caveat 是：能跑起来不等于高效；通信可能吞掉全部扩展收益。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.2 Data parallelism 与 all-reduce 在系统中的位置

Data parallelism 与 all-reduce 首先提供一个局部解释：data parallelism 复制模型，把 batch 切给不同 GPU，反向传播后聚合梯度。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，Data parallelism 与 all-reduce 会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，Data parallelism 与 all-reduce 不能脱离边界条件理解。一个关键 caveat 是：global batch 变大后 learning rate schedule 可能需要调整。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.3 Tensor/pipeline parallelism 的切分直觉在系统中的位置

Tensor/pipeline parallelism 的切分直觉首先提供一个局部解释：tensor parallelism 切矩阵或 head，让单层计算分布到多设备；pipeline parallelism 切层，把不同 micro-batches 送入流水线。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，Tensor/pipeline parallelism 的切分直觉会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，Tensor/pipeline parallelism 的切分直觉不能脱离边界条件理解。一个关键 caveat 是：micro-batch 增多会增加 activation 管理复杂度。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.4 把本章知识用于读论文和复现实验

读相关论文时，可以把本章变成一个检查表。第一，论文是否清楚定义了输入规模、模型规模、数据来源和评估协议；第二，论文声称的改进落在哪个变量上；第三，作者是否报告了会让结论失效的设置；第四，实验是否能分离模型结构、数据质量、优化设置和系统实现的影响。复现实验时，也应避免直接追求大规模。先用最小程序复现一个公式、一个伪代码分支或一个 profile 现象，再扩大输入规模。这样做的原因很简单：如果小规模程序无法解释中间量，大规模训练只会把错误隐藏在日志和随机波动里。把这些检查合在一起，本章给出的不是一个固定答案，而是一种阅读 CS336 的方法：每个模型机制都要落到变量，每个变量都要落到测量，每个测量都要说明协议，每个协议都要承认边界。因此，本章中的任何一句结论都可以被改写成一个可引用的完整句式：在给定数据、模型、硬件和评估协议下，某个机制改变了某个变量，并通过某个观测指标表现出来。这样的句子比单独背诵术语更长，但它包含了自学、复现和引用时真正需要的信息。把本章用于自学时，可以按三遍阅读。第一遍只追踪对象和定义，确认每个术语到底指向文本、token、张量、样本、请求还是评

估记录。第二遍追踪公式和伪代码，确认每一步改变了哪些状态。第三遍追踪实验和 caveat，确认哪些结论只在特定规模、数据或硬件条件下成立。把本章用于复习时，则应反过来操作：先从一个失败案例或异常指标出发，再回到相应机制。loss 曲线异常可能指向数据、优化或模型容量；吞吐异常可能指向 kernel、通信或调度；benchmark 异常可能指向 contamination、prompt protocol 或采样设置。能够从异常反推机制，是掌握本章的实用标准。

8 例题：把概念变成可计算检查

8.1 Data parallel all-reduce 的通信账本

每张 GPU 得到局部梯度 `'g_i'`，训练需要全局平均梯度。

- all-reduce 让每个 rank 都得到 `'sum_i g_i'`。
- 再除以 world size，得到全局 batch 的平均梯度。
- 参数越大、通信网络越慢，扩展效率越容易下降。

结论。数据并行的数学等价很简单，系统代价集中在梯度通信和 batch/LR 调整。这个例题的目的不是替代完整工程实现，而是给出一种最小推理形式：把问题转成变量，写出近似公式或伪代码，再说明近似何时失效。

9 实验设计：从小例子到可复现结论

学习多 GPU 并行基础不能停在概念层。一个教材化结论至少需要四件事：可构造的输入、可观测的中间量、可比较的指标，以及清楚的失败条件。下面的实验设计不是要求训练大模型，而是要求读者用小规模程序复现本章机制。

9.1 最小输入

先构造只触发本章核心机制的小输入。例如 tokenizer 章节可以用几个包含 rare words 和 Unicode 字符的字符串；GPU 章节可以用一个 elementwise kernel 和一个 GEMM；data 章节可以用十几篇近重复文档。小输入的价值在于它让错误可见。

9.2 资源账本

对同一输入写下 FLOPs、bytes moved、显存峰值、通信量或样本数量的估算。估算不需要一开始就精确，但必须说明单位和近似假设。随后用 profiler、benchmark、validation loss 或人工检查去验证估算是否同量级。

9.3 单变量消融

一次只改变一个因素，例如 vocabulary size、head 数、batch size、filter threshold、reward scale、decoding temperature 或 GPU 数量。若多个变量同时变化，实验只能说明系统变了，不能说明机制是什么。

9.4 失败条件

最后要刻意触发失败：极端 context length、错误 dtype、过小 batch、污染 benchmark、低质量数据或不可靠 verifier。失败条件常常比成功曲线更能说明一个方法的边界。

9.5 记录与报告

实验记录应能让另一个读者复现同一结论。最少要写清楚输入构造、代码版本、随机种子、硬件、batch shape、数据版本、评价指标和异常处理方式。报告结论时不要只写“更快”或“更好”，而要说明快在哪里、好在哪个指标上，以及是否牺牲了内存、稳定性、数据质量或可解释性。常见报告错误是把局部现象写成普遍规律。例如一次小模型实验里的 loss 改善，不等于更大模型或不同数据分布上也会改善；一次 kernel benchmark 的吞吐提升，不等于端到端 serving latency 一定降低。教材中的实验报告应始终把结论限定在可复现条件内。

9.6 从小实验到大结论

小实验的作用不是替代课程中的完整训练或系统实现，而是建立一条可检查的推理链。先在小输入上确认变量方向，再在中等规模上确认数量级，最后才讨论是否能外推到更大模型、更多 token 或更复杂 workload。缺少中间层级时，大结论通常只是在复述直觉。因此，实验报告最好同时包含正例和反例：正例说明机制在受控条件下怎样工作，反例说明条件稍变时哪里失效。对 CS336 这样的课程，反例尤其重要，因为 language model 的错误常来自多个层面叠加：数据分布、优化稳定性、硬件瓶颈、评估协议和服务负载都可能同时改变观察结果。还要记录资料版本与复现边界。公开视频、课程页、配套代码和 PDF 可能在不同时间更新；复现实验应写明使用的材料版本，并说明哪些结论来自课程事实，哪些是为了帮助理解而加入的解释性推导。这样才能把学习笔记变成可检查的技术记录。若复现实验与课程结论不一致，首先检查输入、版本和评估协议，而不是立即修改模型假设。

9.7 把本章重新读成一条证据链

完成小实验之后，可以把多 GPU 并行基础重新读成一条证据链：先用“为什么单 GPU 不够”确定问题入口，再用“Data parallelism 与 all-reduce”解释主要机制，最后用“Tensor/pipeline parallelism 的切分直觉”检查边界或外推条件。这种读法比按页背诵更接近教材训练，因为它要求每个结论都能说明前提、变量和证据。本章反复使用的术语包括 data parallelism（数据并行）、tensor parallelism（张量并行）、pipeline parallelism（流水线并行）、all-reduce、communication cost（通信成本）。复习时不要只写定义，而要为每个术语补上一句操作性说明：它在课程代码、公式、数据记录或评估协议中对应什么对象；如果这个对象被删除、替换或缩放，哪一个观测量会变化。最后，用一个反例结束复习。反例可以是资源估算不准、数据污染、reward 失真、benchmark 解析失败、长上下文显存爆炸或 scaling 外推失效。能解释反例的章节，才真正具备自学和引用价值；不能解释反例的章节，只是把课程材料换了一种排版。引用本章结论时，还应保留来源边界：哪些说法直接来自视频、课程页、slides 或代码，哪些是为了串联教材叙述而加入的解释性推导。这个边界不会削弱结论，反而让读者知道何处可以复核，何处需要在自己的实验中重新验证。如果后续课程材料更新，也可以沿着同一证据链替换来源，而不必重写整章结构。

10 课程资料与回看路径

教材正文已经把主要概念、公式和实现逻辑组织成连续叙述；本节只提供回到课程材料的阅读路径。读者复习时可以先完成前文练习，再按下面的时间段或资料组核对细节。

10.1 视频回看路径

时间	关键词	复习任务
----	-----	------

00:00:05-00:05:42	going, single, we're, gpu, your, talk, then	回看定义、例子、推导或 caveat 在该段中的位置。
00:19:02-00:25:39	gpu, then, inaudible, rank, connected, particular, very	回看定义、例子、推导或 caveat 在该段中的位置。
00:38:59-00:46:07	going, then, reduce, it's, processes, rank, input	回看定义、例子、推导或 caveat 在该段中的位置。
00:53:43-01:00:16	going, data, then, it's, cuda, data., batch	回看定义、例子、推导或 caveat 在该段中的位置。
01:14:14-01:20:44	parallelism, then, data, tensor, pipeline, parallel, we'll	回看定义、例子、推导或 caveat 在该段中的位置。

这些时间段用于定位视频中的讲解顺序。严肃引用时，应回到视频片段确认讲者表述、板书或幻灯片内容。回看视频时，不必逐字抄录字幕。更有效的方法是把每个时间段判定为定义、例子、推导、代码解释或 caveat，再把它放回本章对应小节。

10.2 课程资料阅读路径

资料组	标题/页块	关键词
official_01	Lecture 7: parallelism	single, week, parallelism, node, multi-gpu, last, across
official_03	- Data parallelism: DDP (all-reduce), FSDP/ZeRO (all-gather + reduce-scatter)	parallelism, interconnects, pipeline, work, reduce, store, memory
official_05	Use case: after backward pass, sum gradients from different data shards, but ...	data, rank, splits, case, backward, pass, gradients
official_08	Notes:	across, ranks, data, same, sharding, strategy, distributed
official_10	Function: pipeline_parallelism	function, pipeline_parallelism, pipeline_parallelism_main, setup, cleanup, spawn, get_init_params

课程资料通常给出更稳定的标题、公式、图或代码结构；视频则补充动机和口头解释。两者合起来构成本章的事实边界。阅读资料时，可以把每个资料组拆成三个对象：一个定义，一个公式或伪代码动作，一个边界条件。这样比逐字翻译页面或脚本更容易形成可用知识。

11 实践说明、历史位置与风险

多 GPU 并行基础在 CS336 的课程结构中连接基础机制和规模化问题。基础机制解释方法为什么成立；规模化问题检验它在更大模型、更长上下文、更复杂数据或真实 serving workload 下是否仍成立。历史位置不等于模型年表。更重要的是识别问题如何迁移：早期方法解决小规模建模问题，现代方法常常解决同一问题在大规模数据、GPU 集群、长上下文或人类偏好约束下的新形态。

- 资料版本限制 (version risk): 课程页、公开视频和配套资料可能在学期中更新；引用时应注明访问日期或资料版本。

- 测量口径限制 (measurement risk): FLOPs、tokens/s、benchmark score、reward 等数字只有在协议完整时可比较。
- 规模外推限制 (scaling risk): 小模型或小 batch 的机制不必然外推到 frontier-scale。
- 数据分布限制 (data risk): 数据来源、过滤和去重策略会改变模型行为, 也会改变 evaluation contamination 风险。
- 系统风险 (systems risk): 性能剖析结果依赖硬件、driver、kernel、shape 和 batch distribution; 脱离 workload 的性能数字不可直接复用。

12 复习要点

下面的问题把本章内容压缩为可反复检查的条目。每个条目都应能回到前文的解释、公式、伪代码或例题。

条目	对象	检查动作
条目 1	data parallelism (数据并行)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 2	tensor parallelism (张量并行)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 3	pipeline parallelism (流水线并行)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 4	all-reduce	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 5	communication cost (通信成本)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。

12.1 综合自测

- 我能否不用课程原句, 独立解释本章中心问题?
- 我能否把本章至少一个公式改写成可运行的估算函数?
- 我能否指出一个看似改进但实际转移成本的方法?
- 我能否回到视频和课程资料, 找到支持本章关键论断的位置?
- 我能否设计一个最小 ablation, 验证本章一个 caveat?

13 总结与延伸

13.1 本章小结

- 多 GPU 并行基础的核心不是记忆术语, 而是理解对象、变量和机制之间的关系。

- 视频给出讲解顺序，课程资料提供可核对的公式、图、代码或页面结构。
- 复习时应把每个术语连接到至少一个变量、一个实现动作和一个失败模式。

13.2 延伸解释

本章中的延伸解释用于连接课程材料中的概念，例如说明公式里的 proxy、解释系统 tradeoff，或把多个材料片段串成一个完整推理链。若需要严肃引用，应回到课程视频和配套资料核对原始表述。

13.3 练习

- 定义题：用中英双语解释本章任意五个重要术语，并说明它们属于 compute、memory、data、optimization、evaluation 还是 deployment 账本。
- 推导题：选择本章一个公式，逐个解释符号、单位和近似假设，并说明哪个变量最难在真实系统中测量。
- 实现题：把本章伪代码改写成一个最小 Python sanity check，不要求训练大模型，但要输出可检查的中间量。
- 资料题：从“课程资料与回看路径”中选择一个视频时间段，回到原始视频核对讲者例子，并记录是否存在字幕误差。
- 评估题：为本章方法设计一个 ablation 或 benchmark，说明控制变量、数据版本、指标和 failure criteria。
- 边界题：说明本章一个结论在更长 context、更大 batch、更多 GPU 或更复杂数据分布下可能如何失效。
- 综合题：把本章内容和前一章或后一章连接起来，写出一个端到端训练/推理 pipeline 中的依赖关系。
- 批判题：指出一个容易被营销材料夸大的说法，并用本章的资源账本或评估协议反驳它。

第 08 讲：现代训练并行栈

CS336 Spring 2026 public videos and official course materials

目录

1 本章导读	2
1.1 本章知识路线	2
1.2 结构图	2
2 本章主线	2
2.1 本章要解决的核心问题	2
2.2 从机制到资源账本	3
2.3 从课堂讲解到可复现实验	3
3 术语、符号与问题边界	4
3.1 术语之间的关系	4
4 从机制到现代 recipe	4
4.1 为什么要讨论从机制到现代 recipe	4
4.2 从机制到现代 recipe 的机制	5
4.3 从机制到现代 recipe 的数学表达	5
4.4 从从机制到现代 recipe 到程序	5
4.5 边界条件与常见误解	5
4.6 与本章其他概念的关系	5
4.7 怎样检验从机制到现代 recipe	6
4.8 概念辨析：从机制到现代 recipe	6
4.9 从机制到现代 recipe 的小结	6
5 通信拓扑与 collectives	6
5.1 为什么要讨论通信拓扑与 collectives	6
5.2 通信拓扑与 collectives 的机制	7
5.3 通信拓扑与 collectives 的数学表达	7
5.4 从通信拓扑与 collectives 到程序	7
5.5 边界条件与常见误解	7

5.6	与本章其他概念的关系	7
5.7	怎样检验通信拓扑与 collectives	8
5.8	概念辨析：通信拓扑与 collectives	8
5.9	通信拓扑与 collectives 的小结	8
6	MoE 与长上下文下的并行特殊问题	8
6.1	为什么要讨论 MoE 与长上下文下的并行特殊问题	8
6.2	MoE 与长上下文下的并行特殊问题的机制	9
6.3	MoE 与长上下文下的并行特殊问题的形式化表示	9
6.4	从 MoE 与长上下文下的并行特殊问题到程序	9
6.5	边界条件与常见误解	9
6.6	与本章其他概念的关系	9
6.7	怎样检验 MoE 与长上下文下的并行特殊问题	10
6.8	概念辨析：MoE 与长上下文下的并行特殊问题	10
6.9	MoE 与长上下文下的并行特殊问题的小结	10
7	综合讨论：从局部机制到完整系统	10
7.1	从机制到现代 recipe 在系统中的位置	11
7.2	通信拓扑与 collectives 在系统中的位置	11
7.3	MoE 与长上下文下的并行特殊问题在系统中的位置	11
7.4	把本章知识用于读论文和复现实验	11
8	例题：把概念变成可计算检查	12
8.1	FSDP 为什么省显存但增加通信	12
9	实验设计：从小例子到可复现结论	12
9.1	最小输入	12
9.2	资源账本	12
9.3	单变量消融	12
9.4	失败条件	12
9.5	记录与报告	13
9.6	从小实验到大结论	13
9.7	把本章重新读成一条证据链	13
10	课程资料与回看路径	13
10.1	视频回看路径	13
10.2	课程资料阅读路径	14
11	实践说明、历史位置与风险	14

12 复习要点 **15**

12.1 综合自测 15

13 总结与延伸 **16**

13.1 本章小结 16

13.2 延伸解释 16

13.3 练习 16

1 本章导读

本章讨论 **现代训练并行栈**。CS336 的第一层目标不是把现成大模型当作黑箱使用，而是把 language model 从输入文本、训练目标、模型结构、数据、系统和评估这些部件重新拆开。只有拆开之后，读者才能判断一个设计选择究竟改变了什么。设想你要训练一个模型，其参数、optimizer state 和 activation 都超过单机容量。现代训练 recipe 会组合 FSDP/ZeRO、tensor parallel、pipeline parallel、expert parallel 和 checkpointing；本章把它们放入同一张显存-通信账本。本章对应 CS336 Spring 2026 第 08 个公开视频，并结合课程页提供的可解析资料。视频和课程页给出事实边界；正文把这些材料改写为可自学的教材章节。阅读本章时，先理解问题为什么存在，再看定义、公式和实现形态，随后通过例题和练习检查自己是否能独立复现这条 reasoning chain。

Lecture 8

PARALLELISM BASICS

CS336

Tatsu H



图 1: 第 08 讲的课程图像锚点。

1.1 本章知识路线

- 本章首先说明现代训练并行栈为什么是 language modeling pipeline 中不可跳过的一环。
- 其次定义本章反复使用的术语，并说明这些术语对应的计算对象或系统对象。
- 随后用公式和伪代码表达主要机制，让概念能够落到可计算的变量上。
- 最后给出例题、风险讨论和练习，便于读者检查自己是否真正掌握了机制。

1.2 结构图

下面的图用于帮助读者定位本章的核心结构。先看概念之间的依赖，再回到正文中的公式、伪代码和例题。

2 本章主线

现代训练并行栈的主线是把 language model 的抽象计算放回真实机器：哪些张量要移动，哪些状态要保存，哪些瓶颈会在训练或服务时显现。

2.1 本章要解决的核心问题

本讲把并行机制提升到现代大模型训练 recipe。真实训练通常组合 ZeRO/FSDP、tensor parallel、pipeline parallel、expert parallel 和 activation checkpointing。读完这一段后，应能用一两句话说明现代训练并行栈要

Outline and goals

- Understand the systems complexities of training huge models
- Different parallelization paradigms and why people use multiple at once
- What large scale training runs often look like

图 2: 官方材料图页: 现代训练并行栈的课程页/PDF/script 图像锚点。

Lecture 8

PARALLELISM BASICS

CS336

Tatsu H

图 3: 官方材料图页: 现代训练并行栈的课程页/PDF/script 图像锚点。

解决的瓶颈，并指出这个瓶颈发生在数据、模型、优化、系统还是评估层。

2.2 从机制到资源账本

并行策略的本质是在显存、FLOPs 和通信之间换资源。sharding 降低每卡状态，但增加 all-gather/reduce-scatter; checkpointing 降低 activation memory，但增加 recompute。随后把机制写成账本：输入规模是什么，主要状态是什么，成本或质量指标怎样随变量改变。这个账本让后面的公式不再只是符号。

2.3 从课堂讲解到可复现实验

训练系统还要考虑拓扑、rank placement、failure recovery、checkpoint format 和不同 parallel dimensions 的组合，而不是只写一个 all-reduce。最后把课堂讲解转成可复现实验：固定协议，改变一个变量，记录中间量，并解释结果为何支持或反驳原来的机制判断。

3 术语、符号与问题边界

本章的术语横跨模型、数据和系统。读者需要先知道每个词指向的对象：有些词描述数学目标，有些词描述张量形状，有些词描述硬件瓶颈，还有一些词描述评估协议。术语边界不清，后面的公式和实现就会变成空洞的缩写。

#	术语	阅读要求
1	FSDP (fully sharded data parallel)	说明它如何改变训练样本或 token 分布，并给出一个会改变结论的数据边界条件。
2	ZeRO	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
3	expert parallelism (专家并行)	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
4	sequence parallelism (序列并行)	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
5	activation checkpointing (激活检查点)	说明它在计算图中的位置、输入输出形状，以及改变它会影响的训练或推理成本。

这些术语之间有依赖关系。例如 tokenization 改变 sequence length, sequence length 又改变 attention FLOPs、KV cache 和 evaluation token budget; parallelism 改变显存占用，也改变通信瓶颈和故障恢复方式。

3.1 术语之间的关系

在本章中，FSDP (fully sharded data parallel) 给出问题入口，ZeRO 描述主要机制，expert parallelism (专家并行) 则把机制连接到可测量的结果。读教材时应当沿着这个链条追问：输入是什么，中间状态怎样改变，输出指标为什么随之变化。课程材料还会反复出现 parallel、parallelism、zero、memory、stage、tensor 等词。它们不是一组同义标签，而是提示本章的不同层次：有的指对象，有的指操作，有的指约束或指标。因此，术语表不是背诵清单，而是后文阅读的索引。遇到一个术语时，先定位它属于 representation、optimization、systems、data 还是 evaluation，再判断它改变哪一项账本。

4 从机制到现代 recipe

现在进入从机制到现代 recipe。这一节把算法放到硬件和运行时中考察：同一个数学表达式在不同 batch、shape、GPU 拓扑和 kernel 实现下会得到不同的成本账本。

4.1 为什么要讨论从机制到现代 recipe

视频延续 parallelism 主题，但更强调现代大模型训练中的细节、trivia 和复杂性。训练大模型通常不是选择一种并行，而是组合数据并行、FSDP/ZeRO、tensor parallel、pipeline parallel 和 expert parallel。并行 recipe 的正确性包括数值等价、随机性控制、checkpoint 恢复和故障处理。从机制到现代 recipe 把现代训练并行栈放进资源账本。接下来要看的不是单个 API，而是张量形状、内存层级、通信和调度怎样共同决定成本。

4.2 从机制到现代 recipe 的机制

视频延续 parallelism 主题，但更强调现代大模型训练中的细节、trivia 和复杂性。在从机制到现代 recipe 中，核心对象是硬件和系统状态：同一算法会因为 memory hierarchy、通信拓扑或 kernel shape 不同而表现出完全不同的速度。训练大模型通常不是选择一种并行，而是组合数据并行、FSDP/ZeRO、tensor parallel、pipeline parallel 和 expert parallel。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。并行 recipe 的正确性包括数值等价、随机性控制、checkpoint 恢复和故障处理。可检验性来自 profile trace、roofline 估算和端到端延迟分解；如果这些证据互相矛盾，就要先解释 measurement setup。因此，从机制到现代 recipe 的学习目标是建立一张成本地图：读者应能指出瓶颈在计算、内存、通信还是调度，并能设计一个小 benchmark 去验证。

4.3 从机制到现代 recipe 的数学表达

$$\text{memory per GPU} \approx \frac{\text{parameters} + \text{gradients} + \text{optimizer states}}{\text{shard count}} + \text{activations}$$

符号说明：shard count 表示状态被切到多少设备；activations 是否能切分取决于并行策略和 checkpointing。这个关系式应被读成从机制到现代 recipe 的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。在系统实验中，tokens/s、TTFT、显存峰值和 kernel time 都是 proxy。它们各自只观察一部分系统，因此报告时要同时写清 workload shape、warmup、同步方式和硬件型号。

4.4 从从机制到现代 recipe 到程序

把从机制到现代 recipe 写成程序时，最重要的是暴露状态变化和数据移动。下面的伪代码保留执行顺序，省略框架样板。

```
for layer in model:
    all_gather_shard(layer.params)
    y = layer(x)
    free_full_params(layer)
reduce_scatter_gradients()
```

阅读这段实现时，重点看从机制到现代 recipe 怎样移动张量、占用显存并触发通信。实验记录至少要包含吞吐、延迟、显存峰值、通信量和 kernel 利用率，否则很难判断瓶颈来自算子、内存层级还是调度。

4.5 边界条件与常见误解

- FSDP 减显存但增加 all-gather/reduce-scatter 通信。
- checkpointing 降 activation memory，但增加 recomputation FLOPs。
- 小规模 kernel benchmark 不一定预测端到端训练或 serving latency，因为调度、通信和请求形状会改变瓶颈。
- 硬件规格表和框架 profiler 都需要结合 workload 解释，不能把单个峰值数字当作机制证明。

4.6 与本章其他概念的关系

从机制到现代 recipe 连接本章的数学机制和工程成本：术语表说明对象，公式估算资源，伪代码暴露状态变化，例题则让读者检查一次完整的成本推理。对这一类主题，最常见的误解是只列方法名。更有用的做法是比较每个方法改变的成本项、依赖的假设和可能的失败模式。

4.7 怎样检验从机制到现代 recipe

检验从机制到现代 recipe 时,先把抽象说法落到硬件和系统状态上。与 FSDP、ZeRO、shard、checkpointing、all-gather 相关的对象必须能被构造、打印、计数或评分;否则实验只能得到描述性观察,不能得到可复现结论。一个合适的小实验应当保留本节机制,同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真;关键是读者能手算或打印关键中间量,并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明: shard count 表示状态被切到多少设备; activations 是否能切分取决于并行策略和 checkpointing。如果数量只能间接观测,就必须说明使用了什么 proxy;如果使用 benchmark 或 reward,也要说明协议和随机性。系统消融通常一次只改变 batch shape、sequence length、GPU 数、kernel 实现或调度策略。若同时改变硬件和 workload,实验只能说明端到端表现不同,不能定位瓶颈。最后检查失败条件。工程判断往往在反例中变清楚: FSDP 减显存但增加 all-gather/reduce-scatter 通信; checkpointing 降 activation memory,但增加 recomputation FLOPs。复习时可以把这些限制改写成反例测试,观察它们怎样改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。把实验结论放回现代训练并行栈时,要说明从机制到现代 recipe 改变的是训练吞吐、推理延迟、显存峰值还是通信开销。能做出这种归因,才说明读者已经从接口使用进入系统解释。

4.8 概念辨析: 从机制到现代 recipe

FSDP、ZeRO、shard、checkpointing、all-gather 常一起出现在系统讲解中,但它们分别指硬件对象、程序操作、瓶颈来源或测量指标。读者应先定位每个词在执行路径中的位置。区分这些词时,可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。一个术语如果不能对应到变量、状态、代码路径或评估协议,就还没有形成可操作含义。围绕从机制到现代 recipe 复习时,可以写一个小 benchmark: 固定输入形状,改变一个系统变量,记录 profile trace 和端到端时间。

4.9 从机制到现代 recipe 的小结

- 讨论从机制到现代 recipe 时,应同时报告问题规模、实验设置和主要观测变量,尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。
- 如果方法声称更快,要区分计算减少、内存访问减少、通信隐藏和调度改善;这些机制不能用一个 tokens/s 数字互相替代。
- profile trace、roofline 估算和端到端 benchmark 应互相校验,单独一个指标不足以解释系统瓶颈。

5 通信拓扑与 collectives

现在进入通信拓扑与 collectives。这一节把算法放到硬件和运行时中考察:同一个数学表达式在不同 batch、shape、GPU 拓扑和 kernel 实现下会得到不同的成本账本。

5.1 为什么要讨论通信拓扑与 collectives

现代 GPU 集群有 node 内 NVLink/NVSwitch 和 node 间 InfiniBand/RDMA 等不同带宽层级。collective 选择和 rank placement 会直接影响扩展效率。ring、tree、hierarchical all-reduce/all-gather 的选择取决于消息大小和拓扑。通信拓扑与 collectives 把现代训练并行栈放进资源账本。接下来要看的不是单个 API,而是张量形状、内存层级、通信和调度怎样共同决定成本。

5.2 通信拓扑与 collectives 的机制

现代 GPU 集群有 node 内 NVLink/NVSwitch 和 node 间 InfiniBand/RDMA 等不同带宽层级。在通信拓扑与 collectives 中，核心对象是硬件和系统状态：同一算法会因为 memory hierarchy、通信拓扑或 kernel shape 不同而表现出完全不同的速度。collective 选择和 rank placement 会直接影响扩展效率。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。ring、tree、hierarchical all-reduce/all-gather 的选择取决于消息大小和拓扑。可检验性来自 profile trace、roofline 估算和端到端延迟分解；如果这些证据互相矛盾，就要先解释 measurement setup。因此，通信拓扑与 collectives 的学习目标是建立一张成本地图：读者应能指出瓶颈在计算、内存、通信还是调度，并能设计一个小 benchmark 去验证。

5.3 通信拓扑与 collectives 的数学表达

$$T_{\text{comm}} \approx \alpha \cdot \text{\#messages} + \beta \cdot \text{\#bytes}$$

符号说明： α 是 latency 项， β 是带宽倒数；小消息受 latency，大消息受 bandwidth。这个关系式应被读成通信拓扑与 collectives 的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。在系统实验中，tokens/s、TTFT、显存峰值和 kernel time 都是 proxy。它们各自只观察一部分系统，因此报告时要同时写清 workload shape、warmup、同步方式和硬件型号。

5.4 从通信拓扑与 collectives 到程序

把通信拓扑与 collectives 写成程序时，最重要的是暴露状态变化和数据移动。下面的伪代码保留执行顺序，省略框架样板。

```
place_tensor_parallel_ranks_within_node()
place_data_parallel_ranks_across_nodes()
benchmark_collectives()
```

阅读这段实现时，重点看通信拓扑与 collectives 怎样移动张量、占用显存并触发通信。实验记录至少要包含吞吐、延迟、显存峰值、通信量和 kernel 利用率，否则很难判断瓶颈来自算子、内存层级还是调度。

5.5 边界条件与常见误解

- 拓扑错误会让理论上合理的并行策略实际很慢。
- 通信 overlap 需要 runtime 和 kernel schedule 配合，不能只靠公式。
- 小规模 kernel benchmark 不一定预测端到端训练或 serving latency，因为调度、通信和请求形状会改变瓶颈。
- 硬件规格表和框架 profiler 都需要结合 workload 解释，不能把单个峰值数字当作机制证明。

5.6 与本章其他概念的关系

通信拓扑与 collectives 连接本章的数学机制和工程成本：术语表说明对象，公式估算资源，伪代码暴露状态变化，例题则让读者检查一次完整的成本推理。对这一类主题，最常见的误解是只列方法名。更有用的做法是比较每个方法改变的成本项、依赖的假设和可能的失败模式。

5.7 怎样检验通信拓扑与 collectives

检验通信拓扑与 collectives 时，先把抽象说法落到硬件和系统状态上。与 NVLink、InfiniBand、collective、latency、bandwidth 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： α 是 latency 项， β 是带宽倒数；小消息受 latency，大消息受 bandwidth。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。系统消融通常一次只改变 batch shape、sequence length、GPU 数、kernel 实现或调度策略。若同时改变硬件和 workload，实验只能说明端到端表现不同，不能定位瓶颈。最后检查失败条件。工程判断往往在反例中变清楚：拓扑错误会让理论上合理的并行策略实际很慢；通信 overlap 需要 runtime 和 kernel schedule 配合，不能只靠公式。复习时可以把这些限制改写成反例测试，观察它们怎样改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。把实验结论放回现代训练并行栈时，要说明通信拓扑与 collectives 改变的是训练吞吐、推理延迟、显存峰值还是通信开销。能做出这种归因，才说明读者已经从接口使用进入系统解释。

5.8 概念辨析：通信拓扑与 collectives

NVLink、InfiniBand、collective、latency、bandwidth 常一起出现在系统讲解中，但它们分别指硬件对象、程序操作、瓶颈来源或测量指标。读者应先定位每个词在执行路径中的位置。区分这些词时，可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕通信拓扑与 collectives 复习时，可以写一个小 benchmark：固定输入形状，改变一个系统变量，记录 profile trace 和端到端时间。

5.9 通信拓扑与 collectives 的小结

- 讨论通信拓扑与 collectives 时，应同时报告问题规模、实验设置和主要观测变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。
- 如果方法声称更快，要区分计算减少、内存访问减少、通信隐藏和调度改善；这些机制不能用一个 tokens/s 数字互相替代。
- profile trace、roofline 估算和端到端 benchmark 应互相校验，单独一个指标不足以解释系统瓶颈。

6 MoE 与长上下文下的并行特殊问题

现在进入 MoE 与长上下文下的并行特殊问题。这一节把算法放到硬件和运行时中考察：同一个数学表达式在不同 batch、shape、GPU 拓扑和 kernel 实现下会得到不同的成本账本。

6.1 为什么要讨论 MoE 与长上下文下的并行特殊问题

MoE 引入 expert parallelism：token dispatch 和 combine 需要跨设备通信。长上下文引入 sequence/context parallelism：activation 与 attention 的序列维切分变得重要。本讲把并行系统看成 constraints satisfaction：模型结构、batch、sequence、hardware topology 和 failure tolerance 必须同时满足。MoE 与长上下文下的并行特殊问题把现代训练并行栈放进资源账本。接下来要看的不是单个 API，而是张量形状、内存层级、通信和调度怎样共同决定成本。

6.2 MoE 与长上下文下的并行特殊问题的机制

MoE 引入 expert parallelism: token dispatch 和 combine 需要跨设备通信。在 MoE 与长上下文下的并行特殊问题中，核心对象是硬件和系统状态：同一算法会因为 memory hierarchy、通信拓扑或 kernel shape 不同而表现出完全不同的速度。长上下文引入 sequence/context parallelism: activation 与 attention 的序列维切分变得重要。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。本讲把并行系统看成 constraints satisfaction: 模型结构、batch、sequence、hardware topology 和 failure tolerance 必须同时满足。可检验性来自 profile trace、roofline 估算和端到端延迟分解；如果这些证据互相矛盾，就要先解释 measurement setup。因此，MoE 与长上下文下的并行特殊问题的学习目标是建立一张成本地图：读者应能指出瓶颈在计算、内存、通信还是调度，并能设计一个小 benchmark 去验证。

6.3 MoE 与长上下文下的并行特殊问题的形式化表示

$$\text{active FLOPs/token} \ll \text{total parameters} \quad \text{in sparse MoE}$$

符号说明：MoE 的 total parameters 很大，但每个 token 只走少数 experts，因此 active compute 与总参数量脱钩。这个关系式应被读成 MoE 与长上下文下的并行特殊问题的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。在系统实验中，tokens/s、TTFT、显存峰值和 kernel time 都是 proxy。它们各自只观察一部分系统，因此报告时要同时写清 workload shape、warmup、同步方式和硬件型号。

6.4 从 MoE 与长上下文下的并行特殊问题到程序

把 MoE 与长上下文下的并行特殊问题写成程序时，最重要的是暴露状态变化和数据移动。下面的伪代码保留执行顺序，省略框架样板。

```
tokens_by_expert = route(tokens)
all_to_all(tokens_by_expert)
run_local_experts()
all_to_all(combine_outputs)
```

阅读这段实现时，重点看 MoE 与长上下文下的并行特殊问题怎样移动张量、占用显存并触发通信。实验记录至少要包含吞吐、延迟、显存峰值、通信量和 kernel 利用率，否则很难判断瓶颈来自算子、内存层级还是调度。

6.5 边界条件与常见误解

- all-to-all 是 MoE 的核心瓶颈之一。
- 负载均衡 loss 解决的是吞吐问题，同时可能影响表示学习。
- 小规模 kernel benchmark 不一定预测端到端训练或 serving latency，因为调度、通信和请求形状会改变瓶颈。
- 硬件规格表和框架 profiler 都需要结合 workload 解释，不能把单个峰值数字当作机制证明。

6.6 与本章其他概念的关系

MoE 与长上下文下的并行特殊问题连接本章的数学机制和工程成本：术语表说明对象，公式估算资源，伪代码暴露状态变化，例题则让读者检查一次完整的成本推理。对这一类主题，最常见的误解是只列方法名。更

有用的做法是比较每个方法改变的成本项、依赖的假设和可能的失败模式。

6.7 怎样检验 MoE 与长上下文下的并行特殊问题

检验 MoE 与长上下文下的并行特殊问题时，先把抽象说法落到硬件和系统状态上。与 MoE、expert parallel、all-to-all、long context、sequence parallel 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明：MoE 的 total parameters 很大，但每个 token 只走少数 experts，因此 active compute 与总参数量脱钩。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。系统消融通常一次只改变 batch shape、sequence length、GPU 数、kernel 实现或调度策略。若同时改变硬件和 workload，实验只能说明端到端表现不同，不能定位瓶颈。最后检查失败条件。工程判断往往在反例中变清楚：all-to-all 是 MoE 的核心瓶颈之一；负载均衡 loss 解决的是吞吐问题，同时可能影响表示学习。复习时可以把这些限制改写成反例测试，观察它们怎样改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。把实验结论放回现代训练并行栈时，要说明 MoE 与长上下文下的并行特殊问题改变的是训练吞吐、推理延迟、显存峰值还是通信开销。能做出这种归因，才说明读者已经从接口使用进入系统解释。

6.8 概念辨析：MoE 与长上下文下的并行特殊问题

MoE、expert parallel、all-to-all、long context、sequence parallel 常一起出现在系统讲解中，但它们分别指硬件对象、程序操作、瓶颈来源或测量指标。读者应先定位每个词在执行路径中的位置。其中，Mixture of Experts (MoE) 用稀疏激活扩展参数容量；expert（专家）是 MoE 中由 router 选择的子网络。区分这些词时，可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕 MoE 与长上下文下的并行特殊问题复习时，可以写一个小 benchmark：固定输入形状，改变一个系统变量，记录 profile trace 和端到端时间。

6.9 MoE 与长上下文下的并行特殊问题的小结

- 讨论 MoE 与长上下文下的并行特殊问题时，应同时报告问题规模、实验设置和主要观测变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。
- 如果方法声称更快，要区分计算减少、内存访问减少、通信隐藏和调度改善；这些机制不能用一个 tokens/s 数字互相替代。
- profile trace、roofline 估算和端到端 benchmark 应互相校验，单独一个指标不足以解释系统瓶颈。

7 综合讨论：从局部机制到完整系统

本章的主题是现代训练并行栈，但它不应被读成几个相邻知识点的拼接。从机制到现代 recipe、通信拓扑与 collectives、MoE 与长上下文下的并行特殊问题共同回答同一个问题：语言模型系统中哪些对象可以被定义，哪些成本可以被估算，哪些结论必须通过实验或评估来确认。这些对象包括 FSDP (fully sharded data parallel)、ZeRO、expert parallelism (专家并行)、sequence parallelism (序列并行)、activation checkpointing (激活检查点)。读者可以把它们看成一组接口：有的接口连接文本和 token，有的连接模型结构和张量计算，有的连接数据样本和训练目标，有的连接离线评估和在线服务。接口一旦改变，后面的成本、错误类型和优化空间也会随之改变。

7.1 从机制到现代 recipe 在系统中的位置

从机制到现代 recipe 首先提供一个局部解释：视频延续 parallelism 主题，但更强调现代大模型训练中的细节、trivia 和复杂性。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，从机制到现代 recipe 会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，从机制到现代 recipe 不能脱离边界条件理解。一个关键 caveat 是：FSDP 减显存但增加 all-gather/reduce-scatter 通信。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.2 通信拓扑与 collectives 在系统中的位置

通信拓扑与 collectives 首先提供一个局部解释：现代 GPU 集群有 node 内 NVLink/NVSwitch 和 node 间 InfiniBand/RDMA 等不同带宽层级。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，通信拓扑与 collectives 会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，通信拓扑与 collectives 不能脱离边界条件理解。一个关键 caveat 是：拓扑错误会让理论上合理的并行策略实际很慢。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.3 MoE 与长上下文下的并行特殊问题在系统中的位置

MoE 与长上下文下的并行特殊问题首先提供一个局部解释：MoE 引入 expert parallelism：token dispatch 和 combine 需要跨设备通信。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，MoE 与长上下文下的并行特殊问题会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，MoE 与长上下文下的并行特殊问题不能脱离边界条件理解。一个关键 caveat 是：all-to-all 是 MoE 的核心瓶颈之一。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.4 把本章知识用于读论文和复现实验

读相关论文时，可以把本章变成一个检查表。第一，论文是否清楚定义了输入规模、模型规模、数据来源和评估协议；第二，论文声称的改进落在哪个变量上；第三，作者是否报告了会让结论失效的设置；第四，实验是否能分离模型结构、数据质量、优化设置和系统实现的影响。复现实验时，也应避免直接追求大规模。先用最小程序复现一个公式、一个伪代码分支或一个 profile 现象，再扩大输入规模。这样做的原因很简单：如果小规模程序无法解释中间量，大规模训练只会把错误隐藏在日志和随机波动里。把这些检查合在一起，本章给出的不是一个固定答案，而是一种阅读 CS336 的方法：每个模型机制都要落到变量，每个变量都要落到测量，每个测量都要说明协议，每个协议都要承认边界。因此，本章中的任何一句结论都可以被改写成一个可引用的完整句式：在给定数据、模型、硬件和评估协议下，某个机制改变了某个变量，并通过某个观测指标表现出来。这样的句子比单独背诵术语更长，但它包含了自学、复现和引用时真正需要的信息。把本章用于自学时，可以按三遍阅读。第一遍只追踪对象和定义，确认每个术语到底指向文本、token、张量、样本、请求还是评估记录。第二遍追踪公式和伪代码，确认每一步改变了哪些状态。第三遍追踪实验和 caveat，确认哪些结论只

在特定规模、数据或硬件条件下成立。把本章用于复习时，则应反过来操作：先从一个失败案例或异常指标出发，再回到相应机制。loss 曲线异常可能指向数据、优化或模型容量；吞吐异常可能指向 kernel、通信或调度；benchmark 异常可能指向 contamination、prompt protocol 或采样设置。能够从异常反推机制，是掌握本章的实用标准。

8 例题：把概念变成可计算检查

8.1 FSDP 为什么省显存但增加通信

把参数、梯度和 optimizer state 按 rank shard。每个 layer 计算前需要临时 all-gather 完整参数。

- 静态存储近似除以 shard count。
- 前向/反向时需要 all-gather 参数，反向后 reduce-scatter 梯度。
- 如果通信不能与计算重叠，显存节省会换来 step time 上升。

结论。现代并行栈的每个“省显存”技巧都必须问它把成本转移到了哪里。这个例题的目的不是替代完整工程实现，而是给出一种最小推理形式：把问题转成变量，写出近似公式或伪代码，再说明近似何时失效。

9 实验设计：从小例子到可复现结论

学习现代训练并行栈不能停在概念层。一个教材化结论至少需要四件事：可构造的输入、可观测的中间量、可比较的指标，以及清楚的失败条件。下面的实验设计不是要求训练大模型，而是要求读者用小规模程序复现本章机制。

9.1 最小输入

先构造只触发本章核心机制的小输入。例如 tokenizer 章节可以用几个包含 rare words 和 Unicode 字符的字符串；GPU 章节可以用一个 elementwise kernel 和一个 GEMM；data 章节可以用十几篇近重复文档。小输入的价值在于它让错误可见。

9.2 资源账本

对同一输入写下 FLOPs、bytes moved、显存峰值、通信量或样本数量的估算。估算不需要一开始就精确，但必须说明单位和近似假设。随后用 profiler、benchmark、validation loss 或人工检查去验证估算是否同量级。

9.3 单变量消融

一次只改变一个因素，例如 vocabulary size、head 数、batch size、filter threshold、reward scale、decoding temperature 或 GPU 数量。若多个变量同时变化，实验只能说明系统变了，不能说明机制是什么。

9.4 失败条件

最后要刻意触发失败：极端 context length、错误 dtype、过小 batch、污染 benchmark、低质量数据或不可靠 verifier。失败条件常常比成功曲线更能说明一个方法的边界。

9.5 记录与报告

实验记录应能让另一个读者复现同一结论。最少要写清楚输入构造、代码版本、随机种子、硬件、batch shape、数据版本、评价指标和异常处理方式。报告结论时不要只写“更快”或“更好”，而要说明快在哪里、好在哪个指标上，以及是否牺牲了内存、稳定性、数据质量或可解释性。常见报告错误是把局部现象写成普遍规律。例如一次小模型实验里的 loss 改善，不等于更大模型或不同数据分布上也会改善；一次 kernel benchmark 的吞吐提升，不等于端到端 serving latency 一定降低。教材中的实验报告应始终把结论限定在可复现条件内。

9.6 从小实验到大结论

小实验的作用不是替代课程中的完整训练或系统实现，而是建立一条可检查的推理链。先在小输入上确认变量方向，再在中等规模上确认数量级，最后才讨论是否能外推到更大模型、更多 token 或更复杂 workload。缺少中间层级时，大结论通常只是在复述直觉。因此，实验报告最好同时包含正例和反例：正例说明机制在受控条件下怎样工作，反例说明条件稍变时哪里失效。对 CS336 这样的课程，反例尤其重要，因为 language model 的错误常来自多个层面叠加：数据分布、优化稳定性、硬件瓶颈、评估协议和服务负载都可能同时改变观察结果。还要记录资料版本与复现边界。公开视频、课程页、配套代码和 PDF 可能在不同时间更新；复现实验应写明使用的材料版本，并说明哪些结论来自课程事实，哪些是为了帮助理解而加入的解释性推导。这样才能把学习笔记变成可检查的技术记录。若复现实验与课程结论不一致，首先检查输入、版本和评估协议，而不是立即修改模型假设。

9.7 把本章重新读成一条证据链

完成小实验之后，可以把现代训练并行栈重新读成一条证据链：先用“从机制到现代 recipe”确定问题入口，再用“通信拓扑与 collectives”解释主要机制，最后用“MoE 与长上下文下的并行特殊问题”检查边界或外推条件。这种读法比按页背诵更接近教材训练，因为它要求每个结论都能说明前提、变量和证据。本章反复使用的术语包括 FSDP (fully sharded data parallel)、ZeRO、expert parallelism (专家并行)、sequence parallelism (序列并行)、activation checkpointing (激活检查点)。复习时不要只写定义，而要为每个术语补上一句操作性说明：它在课程代码、公式、数据记录或评估协议中对应什么对象；如果这个对象被删除、替换或缩放，哪一个观测量会变化。最后，用一个反例结束复习。反例可以是资源估算不准、数据污染、reward 失真、benchmark 解析失败、长上下文显存爆炸或 scaling 外推失效。能解释反例的章节，才真正具备自学和引用价值；不能解释反例的章节，只是把课程材料换了一种排版。引用本章结论时，还应保留来源边界：哪些说法直接来自视频、课程页、slides 或代码，哪些是为了串联教材叙述而加入的解释性推导。这个边界不会削弱结论，反而让读者知道何处可以复核，何处需要在自己的实验中重新验证。如果后续课程材料更新，也可以沿着同一证据链替换来源，而不必重写整章结构。

10 课程资料与回看路径

教材正文已经把主要概念、公式和实现逻辑组织成连续叙述；本节只提供回到课程材料的阅读路径。读者复习时可以先完成前文练习，再按下面的时间段或资料组核对细节。

10.1 视频回看路径

时间	关键词	复习任务
----	-----	------

00:00:05-00:06:18	going, very, it's, different, many, talk, we're	回看定义、例子、推导或 caveat 在该段中的位置。
00:19:06-00:25:30	going, right., gradients, communication, then, computation, zero	回看定义、例子、推导或 caveat 在该段中的位置。
00:38:19-00:44:53	going, parallel, then, tensor, very, your, communication	回看定义、例子、推导或 caveat 在该段中的位置。
00:51:40-00:59:00	parallel, your, expert, parallelism, tensor, going, very	回看定义、例子、推导或 caveat 在该段中的位置。
01:12:26-01:19:39	parallel, expert, tensor, data, pipeline, very, training	回看定义、例子、推导或 caveat 在该段中的位置。

这些时间段用于定位视频中的讲解顺序。严肃引用时，应回到视频片段确认讲者表述、板书或幻灯片内容。回看视频时，不必逐字抄录字幕。更有效的方法是把每个时间段判定为定义、例子、推导、代码解释或 caveat，再把它放回本章对应小节。

10.2 课程资料阅读路径

资料组	标题/页块	关键词
official_01	Lecture 8 PA R A L L E L I S M B A S I C S CS336 Tatsu H	parallelism, scaling, parallel, memory, compute, bytes, part
official_02	ZeRO stage 1. optimizer state sharding High level idea: • Split up the optimizer ...	zero, stage, parallel, param, params, communication, gradients
official_03	Trading communication bandwidth for utilization Some more crazy pipeline patterns can ...	parallel, tensor, attention, memory, pipeline, activations, activation
official_05	Careful ‘3D’ parallelism gives linear gains More GPUS, same, flat utilization!	tensor, parallel, zero, stage, parallelism, activation, recomputation
official_06	Llama3 405B (Stage 1, small bsz training, Stage 2 pretraining, Stage 3 long-context)	parallelism, stage, megatron, zero1, llama3, long-context, gemma

课程资料通常给出更稳定的标题、公式、图或代码结构；视频则补充动机和口头解释。两者合起来构成本章的事实边界。阅读资料时，可以把每个资料组拆成三个对象：一个定义，一个公式或伪代码动作，一个边界条件。这样比逐字翻译页面或脚本更容易形成可用知识。

11 实践说明、历史位置与风险

现代训练并行栈在 CS336 的课程结构中连接基础机制和规模化问题。基础机制解释方法为什么成立；规模化问题检验它在更大模型、更长上下文、更复杂数据或真实 serving workload 下是否仍成立。历史位置不等于模型年表。更重要的是识别问题如何迁移：早期方法解决小规模建模问题，现代方法常常解决同一问题在大规模数据、GPU 集群、长上下文或人类偏好约束下的新形态。

- 资料版本限制 (version risk): 课程页、公开视频和配套资料可能在学期中更新; 引用时应注明访问日期或资料版本。
- 测量口径限制 (measurement risk): FLOPs、tokens/s、benchmark score、reward 等数字只有在协议完整时可比较。
- 规模外推限制 (scaling risk): 小模型或小 batch 的机制不必然外推到 frontier-scale。
- 数据分布限制 (data risk): 数据来源、过滤和去重策略会改变模型行为, 也会改变 evaluation contamination 风险。
- 系统风险 (systems risk): 性能剖析结果依赖硬件、driver、kernel、shape 和 batch distribution; 脱离 workload 的性能数字不可直接复用。

12 复习要点

下面的问题把本章内容压缩为可反复检查的条目。每个条目都应能回到前文的解释、公式、伪代码或例题。

条目	对象	检查动作
条目 1	FSDP (fully sharded data parallel)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 2	ZeRO	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 3	expert parallelism (专家并行)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 4	sequence parallelism (序列并行)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 5	activation checkpointing (激活检查点)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。

12.1 综合自测

- 我能否不用课程原句, 独立解释本章中心问题?
- 我能否把本章至少一个公式改写成可运行的估算函数?
- 我能否指出一个看似改进但实际转移成本的方法?
- 我能否回到视频和课程资料, 找到支持本章关键论断的位置?
- 我能否设计一个最小 ablation, 验证本章一个 caveat?

13 总结与延伸

13.1 本章小结

- 现代训练并行栈的核心不是记忆术语，而是理解对象、变量和机制之间的关系。
- 视频给出讲解顺序，课程资料提供可核对的公式、图、代码或页面结构。
- 复习时应把每个术语连接到至少一个变量、一个实现动作和一个失败模式。

13.2 延伸解释

本章中的延伸解释用于连接课程材料中的概念，例如说明公式里的 proxy、解释系统 tradeoff，或把多个材料片段串成一个完整推理链。若需要严肃引用，应回到课程视频和配套资料核对原始表述。

13.3 练习

- 定义题：用中英双语解释本章任意五个重要术语，并说明它们属于 compute、memory、data、optimization、evaluation 还是 deployment 账本。
- 推导题：选择本章一个公式，逐个解释符号、单位和近似假设，并说明哪个变量最难在真实系统中测量。
- 实现题：把本章伪代码改写成一个最小 Python sanity check，不要求训练大模型，但要输出可检查的中间量。
- 资料题：从“课程资料与回看路径”中选择一个视频时间段，回到原始视频核对讲者例子，并记录是否存在字幕误差。
- 评估题：为本章方法设计一个 ablation 或 benchmark，说明控制变量、数据版本、指标和 failure criteria。
- 边界题：说明本章一个结论在更长 context、更大 batch、更多 GPU 或更复杂数据分布下可能如何失效。
- 综合题：把本章内容和前一章或后一章连接起来，写出一个端到端训练/推理 pipeline 中的依赖关系。
- 批判题：指出一个容易被营销材料夸大的说法，并用本章的资源账本或评估协议反驳它。

9 09 Scaling laws 基础

第 09 讲: Scaling laws 基础

CS336 Spring 2026 public videos and official course materials

目录

1 本章导读	2
1.1 本章知识路线	2
1.2 结构图	2
2 本章主线	2
2.1 本章要解决的核心问题	2
2.2 从机制到资源账本	3
2.3 从课堂讲解到可复现实验	3
3 术语、符号与问题边界	4
3.1 术语之间的关系	4
4 为什么 scaling laws 有用	4
4.1 课程目标与问题边界	4
4.2 为什么 scaling laws 有用的机制	5
4.3 资源、效率与效果的关系	5
4.4 把课程目标写成检查流程	5
4.5 边界条件与常见误解	5
4.6 与本章其他概念的关系	5
4.7 怎样检验这一课程目标	6
4.8 概念辨析: 为什么 scaling laws 有用	6
4.9 为什么 scaling laws 有用的小结	6
5 IsoFLOPs 与 compute-optimal tradeoff	6
5.1 为什么要讨论 IsoFLOPs 与 compute-optimal tradeoff	6
5.2 IsoFLOPs 与 compute-optimal tradeoff 的机制	7
5.3 IsoFLOPs 与 compute-optimal tradeoff 的数学表达	7
5.4 从 IsoFLOPs 与 compute-optimal tradeoff 到程序	7
5.5 边界条件与常见误解	7

5.6	与本章其他概念的关系	7
5.7	怎样检验 IsoFLOPs 与 compute-optimal tradeoff	8
5.8	概念辨析: IsoFLOPs 与 compute-optimal tradeoff	8
5.9	IsoFLOPs 与 compute-optimal tradeoff 的小结	8
6	从小实验到大训练的风险控制	8
6.1	为什么要讨论从小实验到大训练的风险控制	8
6.2	从小实验到大训练的风险控制的机制	9
6.3	从小实验到大训练的风险控制的数学表达	9
6.4	从从小实验到大训练的风险控制到程序	9
6.5	边界条件与常见误解	9
6.6	与本章其他概念的关系	9
6.7	怎样检验从小实验到大训练的风险控制	10
6.8	概念辨析: 从小实验到大训练的风险控制	10
6.9	从小实验到大训练的风险控制的小结	10
7	综合讨论: 从局部机制到完整系统	10
7.1	为什么 scaling laws 有用在系统中的位置	10
7.2	IsoFLOPs 与 compute-optimal tradeoff 在系统中的位置	11
7.3	从小实验到大训练的风险控制在系统中的位置	11
7.4	把本章知识用于读论文和复现实验	11
8	例题: 把概念变成可计算检查	12
8.1	固定 compute 下选 N 与 D	12
9	实验设计: 从小例子到可复现结论	12
9.1	最小输入	12
9.2	资源账本	12
9.3	单变量消融	12
9.4	失败条件	12
9.5	记录与报告	13
9.6	从小实验到大结论	13
9.7	把本章重新读成一条证据链	13
10	课程资料与回看路径	13
10.1	视频回看路径	13
10.2	课程资料阅读路径	14
11	实践说明、历史位置与风险	14

12 复习要点	15
12.1 综合自测	15
13 总结与延伸	15
13.1 本章小结	15
13.2 延伸解释	16
13.3 练习	16

1 本章导读

本章讨论 **Scaling laws 基础**。CS336 的第一层目标不是把现成大模型当作黑箱使用，而是把 language model 从输入文本、训练目标、模型结构、数据、系统和评估这些部件重新拆开。只有拆开之后，读者才能判断一个设计选择究竟改变了什么。设想你只有预算训练一次大模型。Scaling laws（缩放律）的作用，是用一组小实验预测大训练，并在固定 compute 下选择参数量和 token 数，而不是用昂贵试错赌一个配置。本章对应 CS336 Spring 2026 第 09 个公开视频，并结合课程页提供的可解析资料。视频和课程页给出事实边界；正文把这些材料改写为可自学的教材章节。阅读本章时，先理解问题为什么存在，再看定义、公式和实现形态，随后通过例题和练习检查自己是否能独立复现这条 reasoning chain。

Lecture 9

SCALING LAWS - BASICS

CS336



图 1: 第 09 讲的课程图像锚点。

1.1 本章知识路线

- 本章首先说明 Scaling laws 基础为什么是 language modeling pipeline 中不可跳过的一环。
- 其次定义本章反复使用的术语，并说明这些术语对应的计算对象或系统对象。
- 随后用公式和伪代码表达主要机制，让概念能够落到可计算的变量上。
- 最后给出例题、风险讨论和练习，便于读者检查自己是否真正掌握了机制。

1.2 结构图

下面的图用于帮助读者定位本章的核心结构。先看概念之间的依赖，再回到正文中的公式、伪代码和例题。

2 本章主线

Scaling laws 基础的主线是从小规模实验走向大规模决策。读者需要同时追踪参数量、token 数、compute budget、loss 和 benchmark 之间的关系。

2.1 本章要解决的核心问题

本讲介绍 scaling laws 的基础动机：用小规模实验预测大规模训练，而不是在昂贵大模型上盲目调参。读完这一段后，应能用一两句话说明 Scaling laws 基础要解决的瓶颈，并指出这个瓶颈发生在数据、模型、优化、

Taking scaling seriously

Imagine the following scenario..

Your friend has given you ten thousand B200s for a month,
and asked you to build a good open source LM.

What do you do?

- Put together your infra team and distributed training framework (A2)
- Put together a great pretraining dataset (A4)
- Run a big model (but which one??) <- we are here.

图 2: 官方材料图页: Scaling laws 基础的课程页/PDF/script 图像锚点。

Lecture 9

SCALING LAWS - BASICS

CS336

图 3: 官方材料图页: Scaling laws 基础的课程页/PDF/script 图像锚点。

系统还是评估层。

2.2 从机制到资源账本

核心问题是固定 compute budget 时如何选择参数量 N 和训练 token 数 D 。Chinchilla 式分析把模型大小、数据量和 loss 连接到一个可决策的曲面。随后把机制写成账本: 输入规模是什么, 主要状态是什么, 成本或质量指标怎样随变量改变。这个账本让后面的公式不再只是符号。

2.3 从课堂讲解到可复现实验

scaling law 是经验预测工具, 不是自然定律。数据分布变化、优化失败、architecture 改动和评估污染都会让外推失效。最后把课堂讲解转成可复现实验: 固定协议, 改变一个变量, 记录中间量, 并解释结果为何支持或反驳原来的机制判断。

3 术语、符号与问题边界

本章的术语横跨模型、数据和系统。读者需要先知道每个词指向的对象：有些词描述数学目标，有些词描述张量形状，有些词描述硬件瓶颈，还有一些词描述评估协议。术语边界不清，后面的公式和实现就会变成空洞的缩写。

#	术语	阅读要求
1	scaling law（缩放律）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
2	loss（损失）	说明它测量的对象、协议依赖和可能的 contamination 或 prompt 敏感性。
3	compute-optimal（计算最优）	把它放入资源账本：单位是什么，随输入规模怎样变化，最容易被哪类 benchmark 误读。
4	Chinchilla	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
5	IsoFLOP curve（等 FLOPs 曲线）	把它放入资源账本：单位是什么，随输入规模怎样变化，最容易被哪类 benchmark 误读。

这些术语之间有依赖关系。例如 tokenization 改变 sequence length，sequence length 又改变 attention FLOPs、KV cache 和 evaluation token budget；parallelism 改变显存占用，也改变通信瓶颈和故障恢复方式。

3.1 术语之间的关系

在本章中，scaling law（缩放律）给出问题入口，loss（损失）描述主要机制，compute-optimal（计算最优）则把机制连接到可测量的结果。读教材时应当沿着这个链条追问：输入是什么，中间状态怎样改变，输出指标为什么随之变化。课程材料还会反复出现 scaling、data、laws、size、batch、performance 等词。它们不是一组同义标签，而是提示本章的不同层次：有的指对象，有的指操作，有的指约束或指标。因此，术语表不是背诵清单，而是后文阅读的索引。遇到一个术语时，先定位它属于 representation、optimization、systems、data 还是 evaluation，再判断它改变哪一项账本。

4 为什么 scaling laws 有用

本节讨论为什么 scaling laws 有用。它先限定 Scaling laws 基础要回答的基本问题：哪些现象需要解释，哪些变量可以测量，哪些结论必须回到参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系中检验。

4.1 课程目标与问题边界

视频把本讲称为离开 systems 后回到 deep learning 的 scaling law 基础。scaling law 让大规模训练从盲目试错变成可预测实验：用小 runs 估计大 run 的 loss。真正的用途是决策：给定 compute/data budget，应选多大模型、多少 tokens、什么训练长度。为什么 scaling laws 有用把 Scaling laws 基础连接到大规模决策。接下来要看的不是曲线是否平滑，而是实验点、预算变量和外推假设是否一致。

4.2 为什么 scaling laws 有用的机制

视频把本讲称为离开 systems 后回到 deep learning 的 scaling law 基础。在为什么 scaling laws 有用中，核心对象是模型规模、token 数和训练预算：小实验只有在预算、数据和优化条件清楚时，才可能支持大规模外推。scaling law 让大规模训练从盲目试错变成可预测实验：用小 runs 估计大 run 的 loss。因而同一个方法在不同规模下可能改变不同的变量，尤其是参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。比较方法时，必须同时给出问题规模和实验条件。真正的用途是决策：给定 compute/data budget，应选多大模型、多少 tokens、什么训练长度。可检验性来自多个实验点、残差分析和外推误差估计；单条曲线若没有不确定性，就不能支撑大规模决策。因此，为什么 scaling laws 有用的学习目标是建立一套外推判断：读者应能说明小实验为何能或不能指导 Scaling laws 基础中的大训练配置。

4.3 资源、效率与效果的关系

$$L(N, D) = L_{\infty} + AN^{-\alpha} + BD^{-\beta}$$

符号说明： L 是 validation loss， N 是参数量， D 是训练 tokens， α, β 控制随规模下降的速度。这个关系式给出为什么 scaling laws 有用的缩放假设。它把参数量、训练 token 数和 compute budget 放到同一张账本中，重点是判断小实验能否可靠外推到大训练。在 scaling 实验中，validation loss、benchmark transfer 和训练吞吐都只是不同侧面的 proxy。外推时要说明哪个 proxy 被拟合，哪个 proxy 只是辅助诊断。

4.4 把课程目标写成检查流程

把为什么 scaling laws 有用写成程序后，抽象机制会变成输入、状态和输出之间的变换。下面的伪代码保留核心计算顺序。

```
fit_loss_surface(small_runs)
for budget in budgets:
    choose_N_D_that_minimizes_predicted_loss(budget)
```

阅读这段流程时，重点看小规模实验怎样产生可外推曲线。参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系需要在每个实验点上同时记录，缺少其中一项都会改变 scaling 解释。

4.5 边界条件与常见误解

- 缩放律是经验模型，不是自然定律。
- 分布变化、优化失败、数据质量变化会破坏外推。
- 小规模 scaling 曲线不一定覆盖大训练的优化不稳定、数据瓶颈和系统瓶颈。
- compute-optimal 结论依赖目标指标；以 loss 最优并不自动意味着 benchmark、latency 或成本最优。

4.6 与本章其他概念的关系

为什么 scaling laws 有用连接本章的小实验和大规模决策：术语表标出预算变量，公式给出外推结构，伪代码组织实验点，例题检查 compute-optimal 判断是否稳健。

4.7 怎样检验这一课程目标

检验这一课程目标时，先把抽象说法落到模型规模、token 数和训练预算上。与 scaling laws、loss、parameters、tokens、predict 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： $\backslash(L\backslash)$ 是 validation loss， $\backslash(N\backslash)$ 是参数量， $\backslash(D\backslash)$ 是训练 tokens， $\backslash(\text{lpha},\text{beta}\backslash)$ 控制随规模下降的速度。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。scaling 消融通常一次只改变参数量、token 数、数据 pipeline 或优化 schedule。若多个轴同时移动，曲线拟合会混合不同机制。最后检查失败条件。工程判断往往在反例中变清楚：缩放律是经验模型，不是自然定律；分布变化、优化失败、数据质量变化会破坏外推。复习时可以把这些限制改写成反例测试，观察它们怎样改变参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。把实验结论放回 Scaling laws 基础时，要说明为什么 scaling laws 有用改变的是预算轴、拟合曲线、外推误差还是目标指标。能拆开这些因素，才说明读者真正理解 scaling。

4.8 概念辨析：为什么 scaling laws 有用

scaling laws、loss、parameters、tokens、predict 常一起出现在 scaling 讨论中，但它们分别指预算轴、拟合对象、外推假设或风险来源。读者应先确定曲线中的自变量和因变量。其中，scaling laws（缩放律）用小实验预测大模型 loss 或能力趋势。区分这些词时，可以先问它们是否改变参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕为什么 scaling laws 有用复习时，可以画三到五个小实验点：记录参数量、token 数、预算和 loss，再检查外推是否合理。

4.9 为什么 scaling laws 有用的小结

- 讨论为什么 scaling laws 有用时，应同时报告问题规模、实验设置和主要观测变量，尤其是参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。
- 如果方法声称可以外推，要说明实验点覆盖范围、拟合形式和数据管线是否一致；曲线本身不是自然定律。
- scaling 结论需要把 loss、benchmark 和成本之间的转换关系说清楚，否则 compute-optimal 只是一条局部经验曲线。

5 IsoFLOPs 与 compute-optimal tradeoff

现在进入 IsoFLOPs 与 compute-optimal tradeoff。这一节把模型大小、训练 token、compute budget 和 loss 放在同一坐标系里，讨论何时可以从小实验推断大训练。

5.1 为什么要讨论 IsoFLOPs 与 compute-optimal tradeoff

IsoFLOP curve 固定训练 compute，在不同 $\backslash(N,D\backslash)$ 组合中找最小 loss。Chinchilla 结论推动了对 under-trained large models 的反思：大模型不一定优于更小但训练更充分的模型。课程强调这种分析应服务于资源配置，而不是把某条论文曲线当作永恒配方。IsoFLOPs 与 compute-optimal tradeoff 把 Scaling laws 基础连接到大规模决策。接下来要看的不是曲线是否平滑，而是实验点、预算变量和外推假设是否一致。

5.2 IsoFLOPs 与 compute-optimal tradeoff 的机制

IsoFLOP curve 固定训练 compute, 在不同 (N, D) 组合中找最小 loss。在 IsoFLOPs 与 compute-optimal tradeoff 中, 核心对象是模型规模、token 数和训练预算: 小实验只有在预算、数据和优化条件清楚时, 才可能支持大规模外推。Chinchilla 结论推动了对 undertrained large models 的反思: 大模型不一定优于更小但训练更充分的模型。因而同一个方法在不同规模下可能改变不同的变量, 尤其是参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。比较方法时, 必须同时给出问题规模和实验条件。课程强调这种分析应服务于资源配置, 而不是把某条论文曲线当作永恒配方。可检验性来自多个实验点、残差分析和外推误差估计; 单条曲线若没有不确定性, 就不能支撑大规模决策。因此, IsoFLOPs 与 compute-optimal tradeoff 的学习目标是建立一套外推判断: 读者应能说明小实验为何能或不能指导 Scaling laws 基础中的大训练配置。

5.3 IsoFLOPs 与 compute-optimal tradeoff 的数学表达

$$C \approx 6ND, \quad (N^*, D^*) = \arg \min_{6ND=C} L(N, D)$$

符号说明: C 是总训练 FLOPs; N^*, D^* 是固定 compute 下预测最优的参数数量和 token 数。这个关系式给出 IsoFLOPs 与 compute-optimal tradeoff 的缩放假设。它把参数量、训练 token 数和 compute budget 放到同一张账本中, 重点是判断小实验能否可靠外推到大训练。在 scaling 实验中, validation loss、benchmark transfer 和训练吞吐都只是不同侧面的 proxy。外推时要说明哪个 proxy 被拟合, 哪个 proxy 只是辅助诊断。

5.4 从 IsoFLOPs 与 compute-optimal tradeoff 到程序

把 IsoFLOPs 与 compute-optimal tradeoff 写成程序后, 抽象机制会变成输入、状态和输出之间的变换。下面的伪代码保留核心计算顺序。

```
for N in candidate_model_sizes:
    D = compute_budget / (6 * N)
    loss = scaling_law(N, D)
select_min_loss_configuration()
```

阅读这段流程时, 重点看小规模实验怎样产生可外推曲线。参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系需要在每个实验点上同时记录, 缺少其中一项都会改变 scaling 解释。

5.5 边界条件与常见误解

- 公式里的 $6ND$ 是近似; architecture 和 sequence length 会改变常数。
- compute-optimal 不等于 latency-optimal 或 serving-cost-optimal。
- 小规模 scaling 曲线不一定覆盖大训练的优化不稳定、数据瓶颈和系统瓶颈。
- compute-optimal 结论依赖目标指标; 以 loss 最优并不自动意味着 benchmark、latency 或成本最优。

5.6 与本章其他概念的关系

IsoFLOPs 与 compute-optimal tradeoff 连接本章的小实验和大规模决策: 术语表标出预算变量, 公式给出外推结构, 伪代码组织实验点, 例题检查 compute-optimal 判断是否稳健。

5.7 怎样检验 IsoFLOPs 与 compute-optimal tradeoff

检验 IsoFLOPs 与 compute-optimal tradeoff 时，先把抽象说法落到模型规模、token 数和训练预算上。与 IsoFLOPs、compute-optimal、Chinchilla、6ND、budget 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： $\backslash(C\backslash)$ 是总训练 FLOPs； $\backslash(N^{\star}, D^{\star}\backslash)$ 是固定 compute 下预测最优的参数量和 token 数。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。scaling 消融通常一次只改变参数量、token 数、数据 pipeline 或优化 schedule。若多个轴同时移动，曲线拟合会混合不同机制。最后检查失败条件。工程判断往往在反例中变清楚：公式里的 6ND 是近似；architecture 和 sequence length 会改变常数；compute-optimal 不等于 latency-optimal 或 serving-cost-optimal。复习时可以把这些限制改写成反例测试，观察它们怎样改变参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。把实验结论放回 Scaling laws 基础时，要说明 IsoFLOPs 与 compute-optimal tradeoff 改变的是预算轴、拟合曲线、外推误差还是目标指标。能拆开这些因素，才说明读者真正理解 scaling。

5.8 概念辨析：IsoFLOPs 与 compute-optimal tradeoff

IsoFLOPs、compute-optimal、Chinchilla、6ND、budget 常一起出现在 scaling 讨论中，但它们分别指预算轴、拟合对象、外推假设或风险来源。读者应先确定曲线中的自变量和因变量。其中，FLOPs (floating-point operations) 是训练和推理成本账本的核心单位；Chinchilla 风格分析把参数量、token 数和 compute budget 联系起来。区分这些词时，可以先问它们是否改变参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕 IsoFLOPs 与 compute-optimal tradeoff 复习时，可以画三到五个小实验点：记录参数量、token 数、预算和 loss，再检查外推是否合理。

5.9 IsoFLOPs 与 compute-optimal tradeoff 的小结

- 讨论 IsoFLOPs 与 compute-optimal tradeoff 时，应同时报告问题规模、实验设置和主要观测变量，尤其是参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。
- 如果方法声称可以外推，要说明实验点覆盖范围、拟合形式和数据管线是否一致；曲线本身不是自然定律。
- scaling 结论需要把 loss、benchmark 和成本之间的转换关系说清楚，否则 compute-optimal 只是一条局部经验曲线。

6 从小实验到大训练的风险控制

现在进入从小实验到大训练的风险控制。这一节把模型大小、训练 token、compute budget 和 loss 放在同一坐标系里，讨论何时可以从小实验推断大训练。

6.1 为什么要讨论从小实验到大训练的风险控制

scaling law 实验要覆盖足够的模型大小、数据量和训练长度，否则拟合不稳定。validation loss 的测量要保持数据分布一致，并避免污染和重复。当预测大 run 时，置信区间、残差分析和 sanity checks 比单个点预测更重要。从小实验到大训练的风险控制把 Scaling laws 基础连接到大规模决策。接下来要看的不是曲线是否平滑，而是实验点、预算变量和外推假设是否一致。

6.2 从小实验到大训练的风险控制的机制

scaling law 实验要覆盖足够的模型大小、数据量和训练长度，否则拟合不稳定。在从小实验到大训练的风险控制中，核心对象是模型规模、token 数和训练预算：小实验只有在预算、数据和优化条件清楚时，才可能支持大规模外推。validation loss 的测量要保持数据分布一致，并避免污染和重复。因而同一个方法在不同规模下可能改变不同的变量，尤其是参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。比较方法时，必须同时给出问题规模和实验条件。当预测大 run 时，置信区间、残差分析和 sanity checks 比单个点预测更重要。可检验性来自多个实验点、残差分析和外推误差估计；单条曲线若没有不确定性，就不能支撑大规模决策。因此，从小实验到大训练的风险控制的学习目标是建立一套外推判断：读者应能说明小实验为何能或不能指导 Scaling laws 基础中的大训练配置。

6.3 从小实验到大训练的风险控制的数学表达

$$\hat{L}_{\text{large}} \pm z\sigma_{\text{residual}}$$

符号说明： $\hat{L}_{\text{large}}$ 是预测 loss， σ_{residual} 是小实验拟合残差的尺度。这个关系式给出从小实验到大训练的风险控制的缩放假设。它把参数量、训练 token 数和 compute budget 放到同一张账本中，重点是判断小实验能否可靠外推到大训练。在 scaling 实验中，validation loss、benchmark transfer 和训练吞吐都只是不同侧面的 proxy。外推时要说明哪个 proxy 被拟合，哪个 proxy 只是辅助诊断。

6.4 从从小实验到大训练的风险控制到程序

把从小实验到大训练的风险控制写成程序后，抽象机制会变成输入、状态和输出之间的变换。下面的伪代码保留核心计算顺序。

```
runs = launch_grid(model_sizes, token_counts)
fit = robust_fit(runs)
plot_residuals_and_refuse_bad_extrapolation(fit)
```

阅读这段流程时，重点看小规模实验怎样产生可外推曲线。参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系需要在每个实验点上同时记录，缺少其中一项都会改变 scaling 解释。

6.5 边界条件与常见误解

- 过度外推会把小模型 regime 的现象误用于大模型。
- 数据 pipeline 改变后，旧 scaling law 需要重新校准。
- 小规模 scaling 曲线不一定覆盖大训练的优化不稳定、数据瓶颈和系统瓶颈。
- compute-optimal 结论依赖目标指标；以 loss 最优并不自动意味着 benchmark、latency 或成本最优。

6.6 与本章其他概念的关系

从小实验到大训练的风险控制连接本章的小实验和大规模决策：术语表标出预算变量，公式给出外推结构，伪代码组织实验点，例题检查 compute-optimal 判断是否稳健。

6.7 怎样检验从小实验到大训练的风险控制

检验从小实验到大训练的风险控制时，先把抽象说法落到模型规模、token 数和训练预算上。与 extrapolation、residual、validation、grid、forecast 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： $\hat{L}_{\text{ext}\{\text{large}\}}$ 是预测 loss， $\sigma_{\text{ext}\{\text{residual}\}}$ 是小实验拟合残差的尺度。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。scaling 消融通常一次只改变参数量、token 数、数据 pipeline 或优化 schedule。若多个轴同时移动，曲线拟合会混合不同机制。最后检查失败条件。工程判断往往在反例中变清楚：过度外推会把小模型 regime 的现象误用于大模型；数据 pipeline 改变后，旧 scaling law 需要重新校准。复习时可以把这些限制改写成反例测试，观察它们怎样改变参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。把实验结论放回 Scaling laws 基础时，要说明从小实验到大训练的风险控制改变的是预算轴、拟合曲线、外推误差还是目标指标。能拆开这些因素，才说明读者真正理解 scaling。

6.8 概念辨析：从小实验到大训练的风险控制

extrapolation、residual、validation、grid、forecast 常一起出现在 scaling 讨论中，但它们分别指预算轴、拟合对象、外推假设或风险来源。读者应先确定曲线中的自变量和因变量。区分这些词时，可以先问它们是否改变参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕从小实验到大训练的风险控制复习时，可以画三到五个小实验点：记录参数量、token 数、预算和 loss，再检查外推是否合理。

6.9 从小实验到大训练的风险控制的小结

- 讨论从小实验到大训练的风险控制时，应同时报告问题规模、实验设置和主要观测变量，尤其是参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。
- 如果方法声称可以外推，要说明实验点覆盖范围、拟合形式和数据管线是否一致；曲线本身不是自然定律。
- scaling 结论需要把 loss、benchmark 和成本之间的转换关系说清楚，否则 compute-optimal 只是一条局部经验曲线。

7 综合讨论：从局部机制到完整系统

本章的主题是 Scaling laws 基础，但它不应被读成几个相邻知识点的拼接。为什么 scaling laws 有用、IsoFLOPs 与 compute-optimal tradeoff、从小实验到大训练的风险控制共同回答同一个问题：语言模型系统中哪些对象可以被定义，哪些成本可以被估算，哪些结论必须通过实验或评估来确认。这些对象包括 scaling law (缩放律)、loss (损失)、compute-optimal (计算最优)、Chinchilla、IsoFLOP curve (等 FLOPs 曲线)。读者可以把它们看成一组接口：有的接口连接文本和 token，有的连接模型结构和张量计算，有的连接数据样本和训练目标，有的连接离线评估和在线服务。接口一旦改变，后面的成本、错误类型和优化空间也会随之改变。

7.1 为什么 scaling laws 有用在系统中的位置

为什么 scaling laws 有用首先提供一个局部解释：视频把本讲称为离开 systems 后回到 deep learning 的 scaling law 基础。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、

输出、中间状态和失败条件，而不是只记住方法名。其次，为什么 scaling laws 有用会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，为什么 scaling laws 有用不能脱离边界条件理解。一个关键 caveat 是：缩放律是经验模型，不是自然定律。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.2 IsoFLOPs 与 compute-optimal tradeoff 在系统中的位置

IsoFLOPs 与 compute-optimal tradeoff 首先提供一个局部解释：IsoFLOP curve 固定训练 compute，在不同 (N,D) 组合中找最小 loss。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，IsoFLOPs 与 compute-optimal tradeoff 会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，IsoFLOPs 与 compute-optimal tradeoff 不能脱离边界条件理解。一个关键 caveat 是：公式里的 $6ND$ 是近似；architecture 和 sequence length 会改变常数。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.3 从小实验到大训练的风险控制在系统中的位置

从小实验到大训练的风险控制首先提供一个局部解释：scaling law 实验要覆盖足够的模型大小、数据量和训练长度，否则拟合不稳定。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，从小实验到大训练的风险控制会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，从小实验到大训练的风险控制不能脱离边界条件理解。一个关键 caveat 是：过度外推会把小模型 regime 的现象误用于大模型。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.4 把本章知识用于读论文和复现实验

读相关论文时，可以把本章变成一个检查表。第一，论文是否清楚定义了输入规模、模型规模、数据来源和评估协议；第二，论文声称的改进落在哪个变量上；第三，作者是否报告了会让结论失效的设置；第四，实验是否能分离模型结构、数据质量、优化设置和系统实现的影响。复现实验时，也应避免直接追求大规模。先用最小程序复现一个公式、一个伪代码分支或一个 profile 现象，再扩大输入规模。这样做的原因很简单：如果小规模程序无法解释中间量，大规模训练只会把错误隐藏在日志和随机波动里。把这些检查合在一起，本章给出的不是一个固定答案，而是一种阅读 CS336 的方法：每个模型机制都要落到变量，每个变量都要落到测量，每个测量都要说明协议，每个协议都要承认边界。因此，本章中的任何一句结论都可以被改写成一个可引用的完整句式：在给定数据、模型、硬件和评估协议下，某个机制改变了某个变量，并通过某个观测指标表现出来。这样的句子比单独背诵术语更长，但它包含了自学、复现和引用时真正需要的信息。把本章用于自学时，可以按三遍阅读。第一遍只追踪对象和定义，确认每个术语到底指向文本、token、张量、样本、请求还是评估记录。第二遍追踪公式和伪代码，确认每一步改变了哪些状态。第三遍追踪实验和 caveat，确认哪些结论只在特定规模、数据或硬件条件下成立。把本章用于复习时，则应反过来操作：先从一个失败案例或异常指标出发，再回到相应机制。loss 曲线异常可能指向数据、优化或模型容量；吞吐异常可能指向 kernel、通信或调度；

benchmark 异常可能指向 contamination、prompt protocol 或采样设置。能够从异常反推机制，是掌握本章的实用标准。

8 例题：把概念变成可计算检查

8.1 固定 compute 下选 N 与 D

给定 ‘C 6ND’，如果模型参数 ‘N’ 翻倍，而 compute 固定，则训练 tokens ‘D’ 约减半。

- 大模型 token 不足会 undertrained。
- 小模型 token 充足但容量可能限制 loss。
- IsoFLOP 实验通过多个 ‘N,D’ 点拟合 loss 曲面。

结论。 compute-optimal 是一个受数据、模型和目标指标共同影响的选择问题。这个例题的目的不是替代完整工程实现，而是给出一种最小推理形式：把问题转成变量，写出近似公式或伪代码，再说明近似何时失效。

9 实验设计：从小例子到可复现结论

学习 Scaling laws 基础不能停在概念层。一个教材化结论至少需要四件事：可构造的输入、可观测的中间量、可比较的指标，以及清楚的失败条件。下面的实验设计不是要求训练大模型，而是要求读者用小规模程序复现本章机制。

9.1 最小输入

先构造只触发本章核心机制的小输入。例如 tokenizer 章节可以用几个包含 rare words 和 Unicode 字符的字符串；GPU 章节可以用一个 elementwise kernel 和一个 GEMM；data 章节可以用十几篇近重复文档。小输入的价值在于它让错误可见。

9.2 资源账本

对同一输入写下 FLOPs、bytes moved、显存峰值、通信量或样本数量的估算。估算不需要一开始就精确，但必须说明单位和近似假设。随后用 profiler、benchmark、validation loss 或人工检查去验证估算是否同量级。

9.3 单变量消融

一次只改变一个因素，例如 vocabulary size、head 数、batch size、filter threshold、reward scale、decoding temperature 或 GPU 数量。若多个变量同时变化，实验只能说明系统变了，不能说明机制是什么。

9.4 失败条件

最后要刻意触发失败：极端 context length、错误 dtype、过小 batch、污染 benchmark、低质量数据或不可靠 verifier。失败条件常常比成功曲线更能说明一个方法的边界。

9.5 记录与报告

实验记录应能让另一个读者复现同一结论。最少要写清楚输入构造、代码版本、随机种子、硬件、batch shape、数据版本、评价指标和异常处理方式。报告结论时不要只写“更快”或“更好”，而要说明快在哪里、好在哪个指标上，以及是否牺牲了内存、稳定性、数据质量或可解释性。常见报告错误是把局部现象写成普遍规律。例如一次小模型实验里的 loss 改善，不等于更大模型或不同数据分布上也会改善；一次 kernel benchmark 的吞吐提升，不等于端到端 serving latency 一定降低。教材中的实验报告应始终把结论限定在可复现条件内。

9.6 从小实验到大结论

小实验的作用不是替代课程中的完整训练或系统实现，而是建立一条可检查的推理链。先在小输入上确认变量方向，再在中等规模上确认数量级，最后才讨论是否能外推到更大模型、更多 token 或更复杂 workload。缺少中间层级时，大结论通常只是在复述直觉。因此，实验报告最好同时包含正例和反例：正例说明机制在受控条件下怎样工作，反例说明条件稍变时哪里失效。对 CS336 这样的课程，反例尤其重要，因为 language model 的错误常来自多个层面叠加：数据分布、优化稳定性、硬件瓶颈、评估协议和服务负载都可能同时改变观察结果。还要记录资料版本与复现边界。公开视频、课程页、配套代码和 PDF 可能在不同时间更新；复现实验应写明使用的材料版本，并说明哪些结论来自课程事实，哪些是为了帮助理解而加入的解释性推导。这样才能把学习笔记变成可检查的技术记录。若复现实验与课程结论不一致，首先检查输入、版本和评估协议，而不是立即修改模型假设。

9.7 把本章重新读成一条证据链

完成小实验之后，可以把 Scaling laws 基础重新读成一条证据链：先用“为什么 scaling laws 有用”确定问题入口，再用“IsoFLOPs 与 compute-optimal tradeoff”解释主要机制，最后用“从小实验到大训练的风险控制”检查边界或外推条件。这种读法比按页背诵更接近教材训练，因为它要求每个结论都能说明前提、变量和证据。本章反复使用的术语包括 scaling law (缩放律)、loss (损失)、compute-optimal (计算最优)、Chinchilla、IsoFLOP curve (等 FLOPs 曲线)。复习时不要只写定义，而要为每个术语补上一句操作性说明：它在课程代码、公式、数据记录或评估协议中对应什么对象；如果这个对象被删除、替换或缩放，哪一个观测量会变化。最后，用一个反例结束复习。反例可以是资源估算不准、数据污染、reward 失真、benchmark 解析失败、长上下文显存爆炸或 scaling 外推失效。能解释反例的章节，才真正具备自学和引用价值；不能解释反例的章节，只是把课程材料换了一种排版。引用本章结论时，还应保留来源边界：哪些说法直接来自视频、课程页、slides 或代码，哪些是为了串联教材叙述而加入的解释性推导。这个边界不会削弱结论，反而让读者知道何处可以复核，何处需要在自己的实验中重新验证。如果后续课程材料更新，也可以沿着同一证据链替换来源，而不必重写整章结构。

10 课程资料与回看路径

教材正文已经把主要概念、公式和实现逻辑组织成连续叙述；本节只提供回到课程材料的阅读路径。读者复习时可以先完成前文练习，再按下面的时间段或资料组核对细节。

10.1 视频回看路径

时间	关键词	复习任务
----	-----	------

00:00:06-00:06:49	scaling, very, going, that's, laws, scale, your	回看定义、例子、推导或 caveat 在该段中的位置。
00:20:24-00:26:39	data, scaling, than, small, that's, mixture, laws	回看定义、例子、推导或 caveat 在该段中的位置。
00:39:11-00:45:44	batch, number, going, size, your, parameters, point	回看定义、例子、推导或 caveat 在该段中的位置。
00:52:07-00:58:35	very, going, think, scaling, data, it's, your	回看定义、例子、推导或 caveat 在该段中的位置。
01:11:20-01:17:53	think, scaling, chinchilla, compute, very, training, scale	回看定义、例子、推导或 caveat 在该段中的位置。

这些时间段用于定位视频中的讲解顺序。严肃引用时，应回到视频片段确认讲者表述、板书或幻灯片内容。回看视频时，不必逐字抄录字幕。更有效的方法是把每个时间段判定为定义、例子、推导、代码解释或 caveat，再把它放回本章对应小节。

10.2 课程资料阅读路径

资料组	标题/页块	关键词
official_01	Lecture 9 S C A L I N G L A W S - B A S I C S CS336	scaling, laws, your, together, simple, predictive, tune
official_02	Earliest (data) scaling law paper –1993	scaling, data, early, earliest, history, laws, hestness
official_03	Hestness II Very ahead of its time.. “Emergence” Scaling by compute Speed = accuracy	scaling, data, laws, performance, dataset, size, error
official_05	Side note – ‘Value of parameters’ With MoEs, we expect parameter scaling to change	scaling, batch, size, critical, data, joint, over
official_06	Explanation 2 Non-embedding vs total param choice + small nonlinearities	param, tokens, data, scaling, compute, chinchilla, small

课程资料通常给出更稳定的标题、公式、图或代码结构；视频则补充动机和口头解释。两者合起来构成本章的事实边界。阅读资料时，可以把每个资料组拆成三个对象：一个定义，一个公式或伪代码动作，一个边界条件。这样比逐字翻译页面或脚本更容易形成可用知识。

11 实践说明、历史位置与风险

Scaling laws 基础在 CS336 的课程结构中连接基础机制和规模化问题。基础机制解释方法为什么成立；规模化问题检验它在更大模型、更长上下文、更复杂数据或真实 serving workload 下是否仍成立。历史位置不等于模型年表。更重要的是识别问题如何迁移：早期方法解决小规模建模问题，现代方法常常解决同一问题在大规模数据、GPU 集群、长上下文或人类偏好约束下的新形态。

- 资料版本限制 (version risk): 课程页、公开视频和配套资料可能在学期中更新；引用时应注明访问日期或资料版本。
- 测量口径限制 (measurement risk): FLOPs、tokens/s、benchmark score、reward 等数字只有在协议完整时可比较。

- 规模外推限制 (scaling risk): 小模型或小 batch 的机制不必然外推到 frontier-scale。
- 数据分布限制 (data risk): 数据来源、过滤和去重策略会改变模型行为, 也会改变 evaluation contamination 风险。

12 复习要点

下面的问题把本章内容压缩为可反复检查的条目。每个条目都应能回到前文的解释、公式、伪代码或例题。

条目	对象	检查动作
条目 1	scaling law (缩放律)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 2	loss (损失)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 3	compute-optimal (计算最优)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 4	Chinchilla	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 5	IsoFLOP curve (等 FLOPs 曲线)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。

12.1 综合自测

- 我能否不用课程原句, 独立解释本章中心问题?
- 我能否把本章至少一个公式改写成可运行的估算函数?
- 我能否指出一个看似改进但实际转移成本的方法?
- 我能否回到视频和课程资料, 找到支持本章关键论断的位置?
- 我能否设计一个最小 ablation, 验证本章一个 caveat?

13 总结与延伸

13.1 本章小结

- Scaling laws 基础的核心不是记忆术语, 而是理解对象、变量和机制之间的关系。
- 视频给出讲解顺序, 课程资料提供可核对的公式、图、代码或页面结构。
- 复习时应把每个术语连接到至少一个变量、一个实现动作和一个失败模式。

13.2 延伸解释

本章中的延伸解释用于连接课程材料中的概念，例如说明公式里的 proxy、解释系统 tradeoff，或把多个材料片段串成一个完整推理链。若需要严肃引用，应回到课程视频和配套资料核对原始表述。

13.3 练习

- 定义题：用中英双语解释本章任意五个重要术语，并说明它们属于 compute、memory、data、optimization、evaluation 还是 deployment 账本。
- 推导题：选择本章一个公式，逐个解释符号、单位和近似假设，并说明哪个变量最难在真实系统中测量。
- 实现题：把本章伪代码改写成一个最小 Python sanity check，不要求训练大模型，但要输出可检查的中间量。
- 资料题：从“课程资料与回看路径”中选择一个视频时间段，回到原始视频核对讲者例子，并记录是否存在字幕误差。
- 评估题：为本章方法设计一个 ablation 或 benchmark，说明控制变量、数据版本、指标和 failure criteria。
- 边界题：说明本章一个结论在更长 context、更大 batch、更多 GPU 或更复杂数据分布下可能如何失效。
- 综合题：把本章内容和前一章或后一章连接起来，写出一个端到端训练/推理 pipeline 中的依赖关系。
- 批判题：指出一个容易被营销材料夸大的说法，并用本章的资源账本或评估协议反驳它。

10 10 推理系统与 decoding

第 10 讲：推理系统与 decoding

CS336 Spring 2026 public videos and official course materials

目录

1 本章导读	2
1.1 本章知识路线	2
1.2 结构图	2
2 本章主线	2
2.1 本章要解决的核心问题	2
2.2 从机制到资源账本	3
2.3 从课堂讲解到可复现实验	3
3 术语、符号与问题边界	4
3.1 术语之间的关系	4
4 Inference 的问题形态	4
4.1 为什么要讨论 Inference 的问题形态	4
4.2 Inference 的问题形态的机制	5
4.3 Inference 的问题形态的资源账本	5
4.4 Inference 的问题形态在系统中的实现	5
4.5 边界条件与常见误解	5
4.6 与本章其他概念的关系	5
4.7 怎样检验 Inference 的问题形态	6
4.8 概念辨析：Inference 的问题形态	6
4.9 Inference 的问题形态的小结	6
5 KV cache、batching 与显存压力	6
5.1 为什么要讨论 KV cache、batching 与显存压力	6
5.2 KV cache、batching 与显存压力的机制	7
5.3 KV cache、batching 与显存压力的资源账本	7
5.4 从 KV cache、batching 与显存压力到程序	7
5.5 边界条件与常见误解	7

5.6	与本章其他概念的关系	7
5.7	怎样检验 KV cache、batching 与显存压力	8
5.8	概念辨析：KV cache、batching 与显存压力	8
5.9	KV cache、batching 与显存压力的小结	8
6	Sampling、quantization 与加速技巧	8
6.1	为什么要讨论 Sampling、quantization 与加速技巧	8
6.2	Sampling、quantization 与加速技巧的机制	9
6.3	Sampling、quantization 与加速技巧的数学表达	9
6.4	从 Sampling、quantization 与加速技巧到程序	9
6.5	边界条件与常见误解	9
6.6	与本章其他概念的关系	10
6.7	怎样检验 Sampling、quantization 与加速技巧	10
6.8	概念辨析：Sampling、quantization 与加速技巧	10
6.9	Sampling、quantization 与加速技巧的小结	10
7	综合讨论：从局部机制到完整系统	10
7.1	Inference 的问题形态在系统中的位置	11
7.2	KV cache、batching 与显存压力在系统中的位置	11
7.3	Sampling、quantization 与加速技巧在系统中的位置	11
7.4	把本章知识用于读论文和复现实验	11
8	例题：把概念变成可计算检查	12
8.1	Prefill 与 decode 的瓶颈分离	12
9	实验设计：从小例子到可复现结论	12
9.1	最小输入	12
9.2	资源账本	12
9.3	单变量消融	12
9.4	失败条件	13
9.5	记录与报告	13
9.6	从小实验到大结论	13
9.7	把本章重新读成一条证据链	13
10	课程资料与回看路径	13
10.1	视频回看路径	14
10.2	课程资料阅读路径	14
11	实践说明、历史位置与风险	14

12 复习要点	15
12.1 综合自测	15
13 总结与延伸	16
13.1 本章小结	16
13.2 延伸解释	16
13.3 练习	16

1 本章导读

本章讨论 **推理系统与 decoding**。CS336 的第一层目标不是把现成大模型当作黑箱使用，而是把 language model 从输入文本、训练目标、模型结构、数据、系统和评估这些部件重新拆开。只有拆开之后，读者才能判断一个设计选择究竟改变了什么。设想模型已经训练好，真正成本开始出现在每个用户请求上。Prefill 可以并行，decode 必须逐 token；KV cache 节省重复计算，却把长上下文变成显存和带宽问题。本章对应 CS336 Spring 2026 第 10 个公开视频，并结合课程页提供的可解析资料。视频和课程页给出事实边界；正文把这些材料改写为可自学的教材章节。阅读本章时，先理解问题为什么存在，再看定义、公式和实现形态，随后通过例题和练习检查自己是否能独立复现这条 reasoning chain。



图 1: 第 10 讲的课程图像锚点。

1.1 本章知识路线

- 本章首先说明推理系统与 decoding 为什么是 language modeling pipeline 中不可跳过的一环。
- 其次定义本章反复使用的术语，并说明这些术语对应的计算对象或系统对象。
- 随后用公式和伪代码表达主要机制，让概念能够落到可计算的变量上。
- 最后给出例题、风险讨论和练习，便于读者检查自己是否真正掌握了机制。

1.2 结构图

下面的图用于帮助读者定位本章的核心结构。先看概念之间的依赖，再回到正文中的公式、伪代码和例题。

2 本章主线

推理系统与 decoding 的主线是把 language model 的抽象计算放回真实机器：哪些张量要移动，哪些状态要保存，哪些瓶颈会在训练或服务时显现。

2.1 本章要解决的核心问题

本讲讲 inference：训练成本是一次性的，推理成本会在每个用户请求上重复发生，因此 TTFT、latency、throughput、KV cache 和 batching 都是核心指标。读完这一段后，应能用一两句话说明推理系统与 decoding

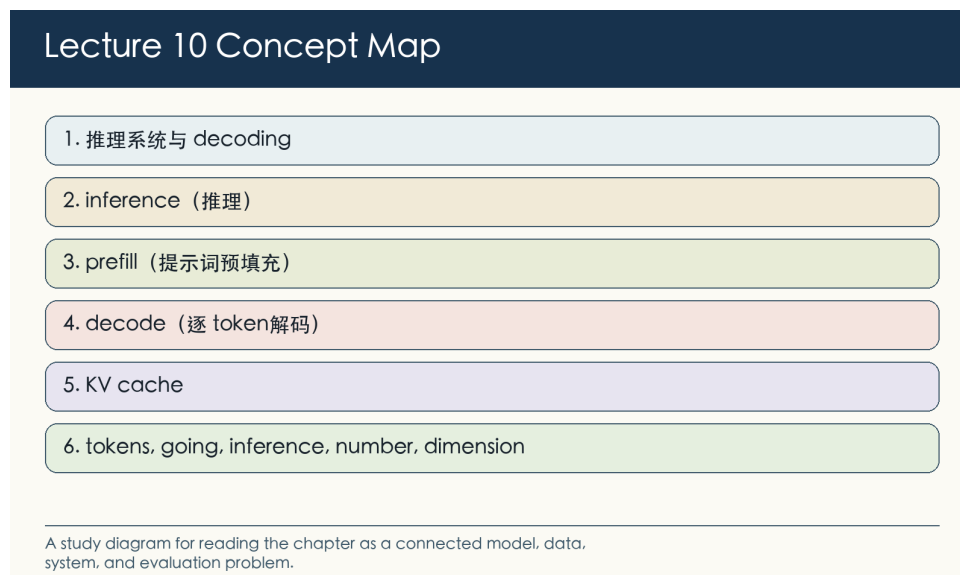


图 2: 第 10 讲概念图: 本章问题、术语和机制的关系。

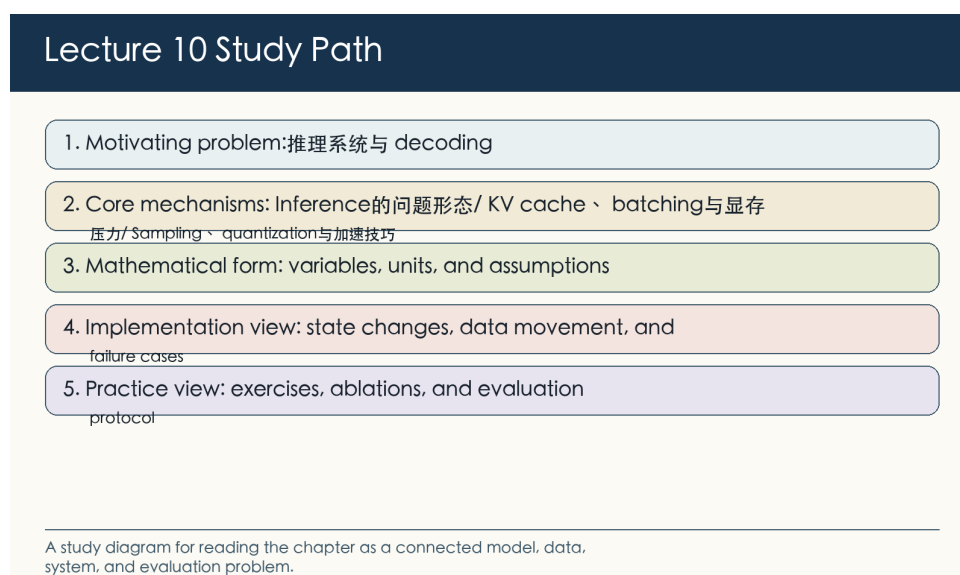


图 3: 第 10 讲学习路径: 从问题定义进入公式、实现、实验和练习。

要解决的瓶颈, 并指出这个瓶颈发生在数据、模型、优化、系统还是评估层。

2.2 从机制到资源账本

prefill 阶段可以并行处理 prompt, decode 阶段逐 token 生成, 常常变成 memory bandwidth bound。KV cache 省掉重复计算, 但把长上下文变成显存压力。随后把机制写成账本: 输入规模是什么, 主要状态是什么, 成本或质量指标怎样随变量改变。这个账本让后面的公式不再只是符号。

2.3 从课堂讲解到可复现实验

推理优化包括 GQA/MLA 等降低 KV cache 的结构、quantization、pruning/distillation、speculative sampling、prefix sharing、paging 和动态 batching。最后把课堂讲解转成可复现实验: 固定协议, 改变一个变量, 记录中间量, 并解释结果为何支持或反驳原来的机制判断。

3 术语、符号与问题边界

本章的术语横跨模型、数据和系统。读者需要先知道每个词指向的对象：有些词描述数学目标，有些词描述张量形状，有些词描述硬件瓶颈，还有一些词描述评估协议。术语边界不清，后面的公式和实现就会变成空洞的缩写。

#	术语	阅读要求
1	inference（推理）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
2	prefill（提示词预填充）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
3	decode（逐 token 解码）	说明它如何改变训练样本或 token 分布，并给出一个会改变结论的数据边界条件。
4	KV cache	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
5	batching（批处理）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
6	speculative decoding（投机解码）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。

这些术语之间有依赖关系。例如 tokenization 改变 sequence length，sequence length 又改变 attention FLOPs、KV cache 和 evaluation token budget；parallelism 改变显存占用，也改变通信瓶颈和故障恢复方式。

3.1 术语之间的关系

在本章中，inference（推理）给出问题入口，prefill（提示词预填充）描述主要机制，decode（逐 token 解码）则把机制连接到可测量的结果。读教材时应当沿着这个链条追问：输入是什么，中间状态怎样改变，输出指标为什么随之变化。课程材料还会反复出现 tokens、attention、inference、throughput、latency、cache 等词。它们不是一组同义标签，而是提示本章的不同层次：有的指对象，有的指操作，有的指约束或指标。因此，术语表不是背诵清单，而是后文阅读的索引。遇到一个术语时，先定位它属于 representation、optimization、systems、data 还是 evaluation，再判断它改变哪一项账本。

4 Inference 的问题形态

现在进入 Inference 的问题形态。这一节把算法放到硬件和运行时中考察：同一个数学表达式在不同 batch、shape、GPU 拓扑和 kernel 实现下会得到不同的成本账本。

4.1 为什么要讨论 Inference 的问题形态

视频把 inference 定义为：模型已训练好，给定 prompt，尽量准确且快速地产生 response。推理只有一讲，但重要性上升，因为 serving 成本、latency 和用户体验在真实部署中占主导。prefill 阶段并行处理 prompt，decode 阶段每次生成一个新 token；两者瓶颈不同。推理阶段的关键区别在于请求是在线到达的，输出又是逐 token 生成的。训练时可以用大 batch 摊薄成本；服务时还要同时考虑首 token 延迟、吞吐、显存占用和长尾请求。

4.2 Inference 的问题形态的机制

视频把 inference 定义为：模型已训练好，给定 prompt，尽量准确且快速地产生产 response。在 Inference 的问题形态中，核心对象是硬件和系统状态：同一算法会因为 memory hierarchy、通信拓扑或 kernel shape 不同而表现出完全不同的速度。推理只有一讲，但重要性上升，因为 serving 成本、latency 和用户体验在真实部署中占主导。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。prefill 阶段并行处理 prompt，decode 阶段每次生成一个新 token；两者瓶颈不同。可检验性来自 profile trace、roofline 估算和端到端延迟分解；如果这些证据互相矛盾，就要先解释 measurement setup。因此，Inference 的问题形态的学习目标是建立一张成本地图：读者应能指出瓶颈在计算、内存、通信还是调度，并能设计一个小 benchmark 去验证。

4.3 Inference 的问题形态的资源账本

$$p(y_t \mid x, y_{<t}) = \text{softmax}(f_\theta(x, y_{<t}))_{y_t}$$

符号说明： x 是 prompt， y_t 是第 t 个输出 token， f_θ 输出 logits。这个关系式应被读成 Inference 的问题形态的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。在系统实验中，tokens/s、TTFT、显存峰值和 kernel time 都是 proxy。它们各自只观察一部分系统，因此报告时要同时写清 workload shape、warmup、同步方式和硬件型号。

4.4 Inference 的问题形态在系统中的实现

把 Inference 的问题形态写成程序时，最重要的是暴露状态变化和数据移动。下面的伪代码保留执行顺序，省略框架样板。

```
kv_cache = prefill(prompt)
while not stop:
    logits, kv_cache = decode_one_token(last_token, kv_cache)
    last_token = sample(logits)
```

阅读这段实现时，重点看 Inference 的问题形态怎样移动张量、占用显存并触发通信。实验记录至少要包含吞吐、延迟、显存峰值、通信量和 kernel 利用率，否则很难判断瓶颈来自算子、内存层级还是调度。

4.5 边界条件与常见误解

- 高吞吐和低延迟常有冲突：大 batch 提高利用率但可能增加等待。
- 推理优化不能只看 tokens/s，也要看 time-to-first-token 和 tail latency。
- 小规模 kernel benchmark 不一定预测端到端训练或 serving latency，因为调度、通信和请求形状会改变瓶颈。
- 硬件规格表和框架 profiler 都需要结合 workload 解释，不能把单个峰值数字当作机制证明。

4.6 与本章其他概念的关系

Inference 的问题形态连接本章的数学机制和工程成本：术语表说明对象，公式估算资源，伪代码暴露状态变化，例题则让读者检查一次完整的成本推理。对这一类主题，最常见的误解是只列方法名。更有用的做法是比较每个方法改变的成本项、依赖的假设和可能的失败模式。

4.7 怎样检验 Inference 的问题形态

检验 Inference 的问题形态时, 先把抽象说法落到硬件和系统状态上。与 inference、prefill、decode、latency、throughput 相关的对象必须能被构造、打印、计数或评分; 否则实验只能得到描述性观察, 不能得到可复现结论。一个合适的小实验应当保留本节机制, 同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真; 关键是读者能手算或打印关键中间量, 并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明: (x) 是 prompt, (y_t) 是第 (t) 个输出 token, (f_{heta}) 输出 logits。如果数量只能间接观测, 就必须说明使用了什么 proxy; 如果使用 benchmark 或 reward, 也要说明协议和随机性。系统消融通常一次只改变 batch shape、sequence length、GPU 数、kernel 实现或调度策略。若同时改变硬件和 workload, 实验只能说明端到端表现不同, 不能定位瓶颈。最后检查失败条件。工程判断往往在反例中变清楚: 高吞吐和低延迟常有冲突: 大 batch 提高利用率但可能增加等待; 推理优化不能只看 tokens/s, 也要看 time-to-first-token 和 tail latency。复习时可以把这些限制改写成反例测试, 观察它们怎样改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。把实验结论放回推理系统与 decoding 时, 要说明 Inference 的问题形态改变的是训练吞吐、推理延迟、显存峰值还是通信开销。能做出这种归因, 才说明读者已经从接口使用进入系统解释。

4.8 概念辨析: Inference 的问题形态

inference、prefill、decode、latency、throughput 常一起出现在系统讲解中, 但它们分别指硬件对象、程序操作、瓶颈来源或测量指标。读者应先定位每个词在执行路径中的位置。其中, inference (推理) 是已训练模型服务请求并生成 token 的过程; prefill (预填充) 并行处理 prompt, 是推理的第一阶段。区分这些词时, 可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。一个术语如果不能对应到变量、状态、代码路径或评估协议, 就还没有形成可操作含义。围绕 Inference 的问题形态复习时, 可以写一个小 benchmark: 固定输入形状, 改变一个系统变量, 记录 profile trace 和端到端时间。

4.9 Inference 的问题形态的小结

- 讨论 Inference 的问题形态时, 应同时报告问题规模、实验设置和主要观测变量, 尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。
- 如果方法声称更快, 要区分计算减少、内存访问减少、通信隐藏和调度改善; 这些机制不能用一个 tokens/s 数字互相替代。
- profile trace、roofline 估算和端到端 benchmark 应互相校验, 单独一个指标不足以解释系统瓶颈。

5 KV cache、batching 与显存压力

现在进入 KV cache、batching 与显存压力。这一节把算法放到硬件和运行时中考察: 同一个数学表达式在不同 batch、shape、GPU 拓扑和 kernel 实现下会得到不同的成本账本。

5.1 为什么要讨论 KV cache、batching 与显存压力

KV cache 避免每步重复计算历史 token 的 key/value, 但把长上下文转化为显存压力。continuous batching、paged attention 和 cache 管理是现代 serving engine 的关键。不同请求的 prompt/output length 分布会导致调度复杂性, 不能用单一固定 shape benchmark 代表生产流量。推理阶段的关键区别在于请求是在线到达的, 输出又是逐 token 生成的。训练时可以用大 batch 摊薄成本; 服务时还要同时考虑首 token 延迟、吞吐、显存占用和长尾请求。

5.2 KV cache、batching 与显存压力的机制

KV cache 避免每步重复计算历史 token 的 key/value，但把长上下文转化为显存压力。在 KV cache、batching 与显存压力中，核心对象是硬件和系统状态：同一算法会因为 memory hierarchy、通信拓扑或 kernel shape 不同而表现出完全不同的速度。continuous batching、paged attention 和 cache 管理是现代 serving engine 的关键。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。不同请求的 prompt/output length 分布会导致调度复杂性，不能用单一固定 shape benchmark 代表生产流量。可检验性来自 profile trace、roofline 估算和端到端延迟分解；如果这些证据互相矛盾，就要先解释 measurement setup。因此，KV cache、batching 与显存压力的学习目标是建立一张成本地图：读者应能指出瓶颈在计算、内存、通信还是调度，并能设计一个小 benchmark 去验证。

5.3 KV cache、batching 与显存压力的资源账本

$$\text{cache size} \propto L \cdot B \cdot T \cdot H_{kv} \cdot d_{\text{head}}$$

符号说明： B 是并发 batch， T 是已缓存 token 长度；长上下文和高并发会线性放大 cache。这个关系式应被读成 KV cache、batching 与显存压力的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。在系统实验中，tokens/s、TTFT、显存峰值和 kernel time 都是 proxy。它们各自只观察一部分系统，因此报告时要同时写清 workload shape、warmup、同步方式和硬件型号。

5.4 从 KV cache、batching 与显存压力到程序

把 KV cache、batching 与显存压力写成程序时，最重要的是暴露状态变化和数据移动。下面的伪代码保留执行顺序，省略框架样板。

```
while requests_active:
    batch = scheduler.pack_ready_requests()
    run_decode_step(batch)
    evict_or_page_kv_cache(batch)
```

阅读这段实现时，重点看 KV cache、batching 与显存压力怎样移动张量、占用显存并触发通信。实验记录至少要包含吞吐、延迟、显存峰值、通信量和 kernel 利用率，否则很难判断瓶颈来自算子、内存层级还是调度。

5.5 边界条件与常见误解

- cache fragmentation 会让显存利用率低于理论容量。
- 长 prompt 与长输出对系统瓶颈的影响不同。
- 小规模 kernel benchmark 不一定预测端到端训练或 serving latency，因为调度、通信和请求形状会改变瓶颈。
- 硬件规格表和框架 profiler 都需要结合 workload 解释，不能把单个峰值数字当作机制证明。

5.6 与本章其他概念的关系

KV cache、batching 与显存压力连接本章的数学机制和工程成本：术语表说明对象，公式估算资源，伪代码暴露状态变化，例题则让读者检查一次完整的成本推理。对这一类主题，最常见的误解是只列方法名。更有用的做法是比较每个方法改变的成本项、依赖的假设和可能的失败模式。

5.7 怎样检验 KV cache、batching 与显存压力

检验 KV cache、batching 与显存压力时，先把抽象说法落到硬件和系统状态上。与 KV cache、batching、paged、scheduler、memory 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： $\backslash(B\backslash)$ 是并发 batch， $\backslash(T\backslash)$ 是已缓存 token 长度；长上下文和高并发会线性放大 cache。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。系统消融通常一次只改变 batch shape、sequence length、GPU 数、kernel 实现或调度策略。若同时改变硬件和 workload，实验只能说明端到端表现不同，不能定位瓶颈。最后检查失败条件。工程判断往往在反例中变清楚：cache fragmentation 会让显存利用率低于理论容量；长 prompt 与长输出对系统瓶颈的影响不同。复习时可以把这些限制改写成反例测试，观察它们怎样改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。把实验结论放回推理系统与 decoding 时，要说明 KV cache、batching 与显存压力改变的是训练吞吐、推理延迟、显存峰值还是通信开销。能做出这种归因，才说明读者已经从接口使用进入系统解释。

5.8 概念辨析：KV cache、batching 与显存压力

KV cache、batching、paged、scheduler、memory 常一起出现在系统讲解中，但它们分别指硬件对象、程序操作、瓶颈来源或测量指标。读者应先定位每个词在执行路径中的位置。其中，KV cache 缓存 key/value，避免重复计算历史上下文；memory（内存/显存）约束决定 batch、sequence length、optimizer state 和 KV cache 能否放下。区分这些词时，可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕 KV cache、batching 与显存压力复习时，可以写一个小 benchmark：固定输入形状，改变一个系统变量，记录 profile trace 和端到端时间。

5.9 KV cache、batching 与显存压力的小结

- 讨论 KV cache、batching 与显存压力时，应同时报告问题规模、实验设置和主要观测变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。
- 如果方法声称更快，要区分计算减少、内存访问减少、通信隐藏和调度改善；这些机制不能用一个 tokens/s 数字互相替代。
- profile trace、roofline 估算和端到端 benchmark 应互相校验，单独一个指标不足以解释系统瓶颈。

6 Sampling、quantization 与加速技巧

现在进入 Sampling、quantization 与加速技巧。这一节把算法放到硬件和运行时中考察：同一个数学表达式在不同 batch、shape、GPU 拓扑和 kernel 实现下会得到不同的成本账本。

6.1 为什么要讨论 Sampling、quantization 与加速技巧

推理质量由 decoding 策略影响：greedy、temperature、top-k、top-p、beam 等会改变输出分布。quantization 降低权重/activation/cache 的内存和带宽，但可能带来精度或校准问题。speculative decoding 用小模型提议多个 token，再由大模型验证，目标是减少大模型调用次数。Sampling、quantization 与加速技巧把推理系统

与 decoding 放进资源账本。接下来要看的不是单个 API，而是张量形状、内存层级、通信和调度怎样共同决定成本。

6.2 Sampling、quantization 与加速技巧的机制

推理质量由 decoding 策略影响：greedy、temperature、top-k、top-p、beam 等会改变输出分布。在 Sampling、quantization 与加速技巧中，核心对象是硬件和系统状态：同一算法会因为 memory hierarchy、通信拓扑或 kernel shape 不同而表现出完全不同的速度。quantization 降低权重/activation/cache 的内存和带宽，但可能带来精度或校准问题。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。speculative decoding 用小模型提议多个 token，再由大模型验证，目标是减少大模型调用次数。可检验性来自 profile trace、roofline 估算和端到端延迟分解；如果这些证据互相矛盾，就要先解释 measurement setup。因此，Sampling、quantization 与加速技巧的学习目标是建立一张成本地图：读者应能指出瓶颈在计算、内存、通信还是调度，并能设计一个小 benchmark 去验证。

6.3 Sampling、quantization 与加速技巧的数学表达

$$\Pr(y = i) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}$$

符号说明： z_i 是 logits， τ 是 temperature； τ 越低分布越尖锐。这个关系式应被读成 Sampling、quantization 与加速技巧的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。在系统实验中，tokens/s、TTFT、显存峰值和 kernel time 都是 proxy。它们各自只观察一部分系统，因此报告时要同时写清 workload shape、warmup、同步方式和硬件型号。

6.4 从 Sampling、quantization 与加速技巧到程序

把 Sampling、quantization 与加速技巧写成程序时，最重要的是暴露状态变化和数据移动。下面的伪代码保留执行顺序，省略框架样板。

```
draft = small_model.propose(k_tokens)
accepted = large_model.verify(draft)
commit_prefix(accepted)
```

阅读这段实现时，重点看 Sampling、quantization 与加速技巧怎样移动张量、占用显存并触发通信。实验记录至少要包含吞吐、延迟、显存峰值、通信量和 kernel 利用率，否则很难判断瓶颈来自算子、内存层级还是调度。

6.5 边界条件与常见误解

- 采样策略影响安全、重复、幻觉和创造性，不能只用平均 accuracy 评价。
- 量化收益取决于硬件支持和 kernel 实现。
- 小规模 kernel benchmark 不一定预测端到端训练或 serving latency，因为调度、通信和请求形状会改变瓶颈。
- 硬件规格表和框架 profiler 都需要结合 workload 解释，不能把单个峰值数字当作机制证明。

6.6 与本章其他概念的关系

Sampling、quantization 与加速技巧连接本章的数学机制和工程成本：术语表说明对象，公式估算资源，伪代码暴露状态变化，例题则让读者检查一次完整的成本推理。对这一类主题，最常见的误解是只列方法名。更有用的做法是比较每个方法改变的成本项、依赖的假设和可能的失败模式。

6.7 怎样检验 Sampling、quantization 与加速技巧

检验 Sampling、quantization 与加速技巧时，先把抽象说法落到硬件和系统状态上。与 temperature、top-p、quantization、speculative、sampling 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： $\logits$ 是 z_i ， $temperature$ 是 τ ； τ 越低分布越尖锐。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。系统消融通常一次只改变 batch shape、sequence length、GPU 数、kernel 实现或调度策略。若同时改变硬件和 workload，实验只能说明端到端表现不同，不能定位瓶颈。最后检查失败条件。工程判断往往在反例中变清楚：采样策略影响安全、重复、幻觉和创造性，不能只用平均 accuracy 评价；量化收益取决于硬件支持和 kernel 实现。复习时可以把这些限制改写成反例测试，观察它们怎样改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。把实验结论放回推理系统与 decoding 时，要说明 Sampling、quantization 与加速技巧改变的是训练吞吐、推理延迟、显存峰值还是通信开销。能做出这种归因，才说明读者已经从接口使用进入系统解释。

6.8 概念辨析：Sampling、quantization 与加速技巧

temperature、top-p、quantization、speculative、sampling 常一起出现在系统讲解中，但它们分别指硬件对象、程序操作、瓶颈来源或测量指标。读者应先定位每个词在执行路径中的位置。区分这些词时，可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕 Sampling、quantization 与加速技巧复习时，可以写一个小 benchmark：固定输入形状，改变一个系统变量，记录 profile trace 和端到端时间。

6.9 Sampling、quantization 与加速技巧的小结

- 讨论 Sampling、quantization 与加速技巧时，应同时报告问题规模、实验设置和主要观测变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。
- 如果方法声称更快，要区分计算减少、内存访问减少、通信隐藏和调度改善；这些机制不能用一个 tokens/s 数字互相替代。
- profile trace、roofline 估算和端到端 benchmark 应互相校验，单独一个指标不足以解释系统瓶颈。

7 综合讨论：从局部机制到完整系统

本章的主题是推理系统与 decoding，但它不应被读成几个相邻知识点的拼接。Inference 的问题形态、KV cache、batching 与显存压力、Sampling、quantization 与加速技巧共同回答同一个问题：语言模型系统中哪些对象可以被定义，哪些成本可以被估算，哪些结论必须通过实验或评估来确认。这些对象包括 inference（推理）、prefill（提示词预填充）、decode（逐 token 解码）、KV cache、batching（批处理）、speculative decoding（投机解码）。读者可以把它们看成一组接口：有的接口连接文本和 token，有的连接模型结构和张量计算，有

的连接数据样本和训练目标，有的连接离线评估和在线服务。接口一旦改变，后面的成本、错误类型和优化空间也会随之改变。

7.1 Inference 的问题形态在系统中的位置

Inference 的问题形态首先提供一个局部解释：视频把 inference 定义为：模型已训练好，给定 prompt，尽量准确且快速地产生产 response。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，Inference 的问题形态会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，Inference 的问题形态不能脱离边界条件理解。一个关键 caveat 是：高吞吐和低延迟常有冲突：大 batch 提高利用率但可能增加等待。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.2 KV cache、batching 与显存压力在系统中的位置

KV cache、batching 与显存压力首先提供一个局部解释：KV cache 避免每步重复计算历史 token 的 key/value，但把长上下文转化为显存压力。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，KV cache、batching 与显存压力会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，KV cache、batching 与显存压力不能脱离边界条件理解。一个关键 caveat 是：cache fragmentation 会让显存利用率低于理论容量。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.3 Sampling、quantization 与加速技巧在系统中的位置

Sampling、quantization 与加速技巧首先提供一个局部解释：推理质量由 decoding 策略影响：greedy、temperature、top-k、top-p、beam 等会改变输出分布。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，Sampling、quantization 与加速技巧会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，Sampling、quantization 与加速技巧不能脱离边界条件理解。一个关键 caveat 是：采样策略影响安全、重复、幻觉和创造性，不能只用平均 accuracy 评价。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.4 把本章知识用于读论文和复现实验

读相关论文时，可以把本章变成一个检查表。第一，论文是否清楚定义了输入规模、模型规模、数据来源和评估协议；第二，论文声称的改进落在哪个变量上；第三，作者是否报告了会让结论失效的设置；第四，实验是否能分离模型结构、数据质量、优化设置和系统实现的影响。复现实验时，也应避免直接追求大规模。先用小程序复现一个公式、一个伪代码分支或一个 profile 现象，再扩大输入规模。这样做的原因很简单：如果小规模程序无法解释中间量，大规模训练只会把错误隐藏在日志和随机波动里。把这些检查合在一起，本章给出的不是一个固定答案，而是一种阅读 CS336 的方法：每个模型机制都要落到变量，每个变量都要落到测量，每个测量都要说明协议，每个协议都要承认边界。因此，本章中的任何一句结论都可以被改写成一个可引

用的完整句式：在给定数据、模型、硬件和评估协议下，某个机制改变了某个变量，并通过某个观测指标表现出来。这样的句子比单独背诵术语更长，但它包含了自学、复现和引用时真正需要的信息。把本章用于自学时，可以按三遍阅读。第一遍只追踪对象和定义，确认每个术语到底指向文本、token、张量、样本、请求还是评估记录。第二遍追踪公式和伪代码，确认每一步改变了哪些状态。第三遍追踪实验和 caveat，确认哪些结论只在特定规模、数据或硬件条件下成立。把本章用于复习时，则应反过来操作：先从一个失败案例或异常指标出发，再回到相应机制。loss 曲线异常可能指向数据、优化或模型容量；吞吐异常可能指向 kernel、通信或调度；benchmark 异常可能指向 contamination、prompt protocol 或采样设置。能够从异常反推机制，是掌握本章的实用标准。

8 例题：把概念变成可计算检查

8.1 Prefill 与 decode 的瓶颈分离

一个请求包含长 prompt 和短回答，另一个请求包含短 prompt 和长回答。

- 长 prompt 主要增加 prefill 计算，可并行处理。
- 长回答增加 decode 步数，每步都要读权重和 KV cache。
- 调度器必须同时优化 TTFT、吞吐和 tail latency。

结论。推理系统不能只报告平均 tokens/s；不同 workload 形状对应不同瓶颈。这个例题的目的不是替代完整工程实现，而是给出一种最小推理形式：把问题转成变量，写出近似公式或伪代码，再说明近似何时失效。

9 实验设计：从小例子到可复现结论

学习推理系统与 decoding 不能停在概念层。一个教材化结论至少需要四件事：可构造的输入、可观测的中间量、可比较的指标，以及清楚的失败条件。下面的实验设计不是要求训练大模型，而是要求读者用小规模程序复现本章机制。

9.1 最小输入

先构造只触发本章核心机制的小输入。例如 tokenizer 章节可以用几个包含 rare words 和 Unicode 字符的字符串；GPU 章节可以用一个 elementwise kernel 和一个 GEMM；data 章节可以用十几篇近重复文档。小输入的价值在于它让错误可见。

9.2 资源账本

对同一输入写下 FLOPs、bytes moved、显存峰值、通信量或样本数量的估算。估算不需要一开始就精确，但必须说明单位和近似假设。随后用 profiler、benchmark、validation loss 或人工检查去验证估算是否同量级。

9.3 单变量消融

一次只改变一个因素，例如 vocabulary size、head 数、batch size、filter threshold、reward scale、decoding temperature 或 GPU 数量。若多个变量同时变化，实验只能说明系统变了，不能说明机制是什么。

9.4 失败条件

最后要刻意触发失败：极端 context length、错误 dtype、过小 batch、污染 benchmark、低质量数据或不可靠 verifier。失败条件常常比成功曲线更能说明一个方法的边界。

9.5 记录与报告

实验记录应能让另一个读者复现同一结论。最少要写清楚输入构造、代码版本、随机种子、硬件、batch shape、数据版本、评价指标和异常处理方式。报告结论时不要只写“更快”或“更好”，而要说明快在哪里、好在哪个指标上，以及是否牺牲了内存、稳定性、数据质量或可解释性。常见报告错误是把局部现象写成普遍规律。例如一次小模型实验里的 loss 改善，不等于更大模型或不同数据分布上也会改善；一次 kernel benchmark 的吞吐提升，不等于端到端 serving latency 一定降低。教材中的实验报告应始终把结论限定在可复现条件内。

9.6 从小实验到大结论

小实验的作用不是替代课程中的完整训练或系统实现，而是建立一条可检查的推理链。先在小输入上确认变量方向，再在中等规模上确认数量级，最后才讨论是否能外推到更大模型、更多 token 或更复杂 workload。缺少中间层级时，大结论通常只是在复述直觉。因此，实验报告最好同时包含正例和反例：正例说明机制在受控条件下怎样工作，反例说明条件稍变时哪里失效。对 CS336 这样的课程，反例尤其重要，因为 language model 的错误常来自多个层面叠加：数据分布、优化稳定性、硬件瓶颈、评估协议和服务负载都可能同时改变观察结果。还要记录资料版本与复现边界。公开视频、课程页、配套代码和 PDF 可能在不同时间更新；复现实验应写明使用的材料版本，并说明哪些结论来自课程事实，哪些是为了帮助理解而加入的解释性推导。这样才能把学习笔记变成可检查的技术记录。若复现实验与课程结论不一致，首先检查输入、版本和评估协议，而不是立即修改模型假设。

9.7 把本章重新读成一条证据链

完成小实验之后，可以把推理系统与 decoding 重新读成一条证据链：先用“Inference 的问题形态”确定问题入口，再用“KV cache、batching 与显存压力”解释主要机制，最后用“Sampling、quantization 与加速技巧”检查边界或外推条件。这种读法比按页背诵更接近教材训练，因为它要求每个结论都能说明前提、变量和证据。本章反复使用的术语包括 inference（推理）、prefill（提示词预填充）、decode（逐 token 解码）、KV cache、batching（批处理）、speculative decoding（投机解码）。复习时不要只写定义，而要为每个术语补上一句操作性说明：它在课程代码、公式、数据记录或评估协议中对应什么对象；如果这个对象被删除、替换或缩放，哪一个观测量会变化。最后，用一个反例结束复习。反例可以是资源估算不准、数据污染、reward 失真、benchmark 解析失败、长上下文显存爆炸或 scaling 外推失效。能解释反例的章节，才真正具备自学和引用价值；不能解释反例的章节，只是把课程材料换了一种排版。引用本章结论时，还应保留来源边界：哪些说法直接来自视频、课程页、slides 或代码，哪些是为了串联教材叙述而加入的解释性推导。这个边界不会削弱结论，反而让读者知道何处可以复核，何处需要在自己的实验中重新验证。如果后续课程材料更新，也可以沿着同一证据链替换来源，而不必重写整章结构。

10 课程资料与回看路径

教材正文已经把主要概念、公式和实现逻辑组织成连续叙述；本节只提供回到课程材料的阅读路径。读者复习时可以先完成前文练习，再按下面的时间段或资料组核对细节。

10.1 视频回看路径

时间	关键词	复习任务
00:00:05-00:06:34	inference, tokens, going, it's, fast, there's, very	回看定义、例子、推导或 caveat 在该段中的位置。
00:20:09-00:27:17	then, tokens, token, cache, going, number, generate	回看定义、例子、推导或 caveat 在该段中的位置。
00:42:53-00:50:45	batch, throughput, latency, size, your, memory, memory.	回看定义、例子、推导或 caveat 在该段中的位置。
00:58:07-01:04:29	attention, sliding, linear, window, there's, then, maybe	回看定义、例子、推导或 caveat 在该段中的位置。
01:18:05-01:24:54	then, same, cache, there's, it's, different, going	回看定义、例子、推导或 caveat 在该段中的位置。

这些时间段用于定位视频中的讲解顺序。严肃引用时，应回到视频片段确认讲者表述、板书或幻灯片内容。回看视频时，不必逐字抄录字幕。更有效的方法是把每个时间段判定为定义、例子、推导、代码解释或 caveat，再把它放回本章对应小节。

10.2 课程资料阅读路径

资料组	标题/页块	关键词
official_01	Lecture 10: inference	inference, faster, different, architecture, distillation, sequences, many
official_03	2. Read W ($D \times F$) from HBM	intensity, compute, read, tokens, accelerator, flops, inference
official_05	Let us now compute the theoretical maximum latency and throughput of a single request.	throughput, latency, worse, better, result, memory, batch
official_08	Compute the probabilities of speculatively sampling a token:	draft, sampling, target, parameters, length, training, problem
official_10	- fp8 (1 byte) [-240, 240] for e4m3 on H100s: can train if you dare	function, byte, less, accurate, cheaper, quantization, paper

课程资料通常给出更稳定的标题、公式、图或代码结构；视频则补充动机和口头解释。两者合起来构成本章的事实边界。阅读资料时，可以把每个资料组拆成三个对象：一个定义，一个公式或伪代码动作，一个边界条件。这样比逐字翻译页面或脚本更容易形成可用知识。

11 实践说明、历史位置与风险

推理系统与 decoding 在 CS336 的课程结构中连接基础机制和规模化问题。基础机制解释方法为什么成立；规模化问题检验它在更大模型、更长上下文、更复杂数据或真实 serving workload 下是否仍成立。历史位置不等于模型年表。更重要的是识别问题如何迁移：早期方法解决小规模建模问题，现代方法常常解决同一问题在大规模数据、GPU 集群、长上下文或人类偏好约束下的新形态。

- 资料版本限制 (version risk): 课程页、公开视频和配套资料可能在学期中更新; 引用时应注明访问日期或资料版本。
- 测量口径限制 (measurement risk): FLOPs、tokens/s、benchmark score、reward 等数字只有在协议完整时可比较。
- 规模外推限制 (scaling risk): 小模型或小 batch 的机制不必然外推到 frontier-scale。
- 数据分布限制 (data risk): 数据来源、过滤和去重策略会改变模型行为, 也会改变 evaluation contamination 风险。
- 系统风险 (systems risk): 性能剖析结果依赖硬件、driver、kernel、shape 和 batch distribution; 脱离 workload 的性能数字不可直接复用。

12 复习要点

下面的问题把本章内容压缩为可反复检查的条目。每个条目都应能回到前文的解释、公式、伪代码或例题。

条目	对象	检查动作
条目 1	inference (推理)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 2	prefill (提示词预填充)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 3	decode (逐 token 解码)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 4	KV cache	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 5	batching (批处理)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 6	speculative decoding (投机解码)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。

12.1 综合自测

- 我能否不用课程原句, 独立解释本章中心问题?
- 我能否把本章至少一个公式改写成可运行的估算函数?
- 我能否指出一个看似改进但实际转移成本的方法?
- 我能否回到视频和课程资料, 找到支持本章关键论断的位置?
- 我能否设计一个最小 ablation, 验证本章一个 caveat?

13 总结与延伸

13.1 本章小结

- 推理系统与 decoding 的核心不是记忆术语，而是理解对象、变量和机制之间的关系。
- 视频给出讲解顺序，课程资料提供可核对的公式、图、代码或页面结构。
- 复习时应把每个术语连接到至少一个变量、一个实现动作和一个失败模式。

13.2 延伸解释

本章中的延伸解释用于连接课程材料中的概念，例如说明公式里的 proxy、解释系统 tradeoff，或把多个材料片段串成一个完整推理链。若需要严肃引用，应回到课程视频和配套资料核对原始表述。

13.3 练习

- 定义题：用中英双语解释本章任意五个重要术语，并说明它们属于 compute、memory、data、optimization、evaluation 还是 deployment 账本。
- 推导题：选择本章一个公式，逐个解释符号、单位和近似假设，并说明哪个变量最难在真实系统中测量。
- 实现题：把本章伪代码改写成一个最小 Python sanity check，不要求训练大模型，但要输出可检查的中间量。
- 资料题：从“课程资料与回看路径”中选择一个视频时间段，回到原始视频核对讲者例子，并记录是否存在字幕误差。
- 评估题：为本章方法设计一个 ablation 或 benchmark，说明控制变量、数据版本、指标和 failure criteria。
- 边界题：说明本章一个结论在更长 context、更大 batch、更多 GPU 或更复杂数据分布下可能如何失效。
- 综合题：把本章内容和前一章或后一章连接起来，写出一个端到端训练/推理 pipeline 中的依赖关系。
- 批判题：指出一个容易被营销材料夸大的说法，并用本章的资源账本或评估协议反驳它。

11 11 Scaling laws 进阶与实践

第 11 讲: Scaling laws 进阶与实践

CS336 Spring 2026 public videos and official course materials

目录

1 本章导读	2
1.1 本章知识路线	2
1.2 结构图	2
2 本章主线	2
2.1 本章要解决的核心问题	2
2.2 从机制到资源账本	3
2.3 从课堂讲解到可复现实验	3
3 术语、符号与问题边界	4
3.1 术语之间的关系	4
4 更复杂的 scaling 问题	4
4.1 为什么要讨论更复杂的 scaling 问题	4
4.2 更复杂的 scaling 问题的机制	5
4.3 更复杂的 scaling 问题的数学表达	5
4.4 从更复杂的 scaling 问题到程序	5
4.5 边界条件与常见误解	5
4.6 与本章其他概念的关系	5
4.7 怎样检验更复杂的 scaling 问题	6
4.8 概念辨析: 更复杂的 scaling 问题	6
4.9 更复杂的 scaling 问题的小结	6
5 数据受限与重复数据	6
5.1 为什么要讨论数据受限与重复数据	6
5.2 数据受限与重复数据的机制	7
5.3 数据受限与重复数据的数学表达	7
5.4 从数据受限与重复数据到程序	7
5.5 边界条件与常见误解	7

5.6	与本章其他概念的关系	7
5.7	怎样检验数据受限与重复数据	7
5.8	概念辨析：数据受限与重复数据	8
5.9	数据受限与重复数据的小结	8
6	从 loss 到能力指标	8
6.1	为什么要讨论从 loss 到能力指标	8
6.2	从 loss 到能力指标的机制	8
6.3	从 loss 到能力指标的数学表达	9
6.4	从从 loss 到能力指标到程序	9
6.5	边界条件与常见误解	9
6.6	与本章其他概念的关系	9
6.7	怎样检验从 loss 到能力指标	9
6.8	概念辨析：从 loss 到能力指标	10
6.9	从 loss 到能力指标的小结	10
7	综合讨论：从局部机制到完整系统	10
7.1	更复杂的 scaling 问题在系统中的位置	10
7.2	数据受限与重复数据在系统中的位置	11
7.3	从 loss 到能力指标在系统中的位置	11
7.4	把本章知识用于读论文和复现实验	11
8	例题：把概念变成可计算检查	12
8.1	为什么数据 pipeline 改变后要重拟合 scaling law	12
9	实验设计：从小例子到可复现结论	12
9.1	最小输入	12
9.2	资源账本	12
9.3	单变量消融	12
9.4	失败条件	12
9.5	记录与报告	12
9.6	从小实验到大结论	13
9.7	把本章重新读成一条证据链	13
10	课程资料与回看路径	13
10.1	视频回看路径	13
10.2	课程资料阅读路径	14
11	实践说明、历史位置与风险	14

12 复习要点	15
12.1 综合自测	15
13 总结与延伸	15
13.1 本章小结	15
13.2 延伸解释	15
13.3 练习	16

1 本章导读

本章讨论 **Scaling laws 进阶与实践**。CS336 的第一层目标不是把现成大模型当作黑箱使用，而是把 language model 从输入文本、训练目标、模型结构、数据、系统和评估这些部件重新拆开。只有拆开之后，读者才能判断一个设计选择究竟改变了什么。设想基础 scaling law 给出一条漂亮曲线，但你的数据有限、训练 schedule 改了、benchmark 也不只看 loss。进阶 scaling 的问题是：哪些外推仍可信，哪些必须重新校准？本章对应 CS336 Spring 2026 第 11 个公开视频，并结合课程页提供的可解析资料。视频和课程页给出事实边界；正文把这些材料改写为可自学的教材章节。阅读本章时，先理解问题为什么存在，再看定义、公式和实现形态，随后通过例题和练习检查自己是否能独立复现这条 reasoning chain。

Lecture 11

SCALING – CASE STUDY AND DETAILS

CS336



图 1: 第 11 讲的课程图像锚点。

1.1 本章知识路线

- 本章首先说明 Scaling laws 进阶与实践为什么是 language modeling pipeline 中不可跳过的一环。
- 其次定义本章反复使用的术语，并说明这些术语对应的计算对象或系统对象。
- 随后用公式和伪代码表达主要机制，让概念能够落到可计算的变量上。
- 最后给出例题、风险讨论和练习，便于读者检查自己是否真正掌握了机制。

1.2 结构图

下面的图用于帮助读者定位本章的核心结构。先看概念之间的依赖，再回到正文中的公式、伪代码和例题。

2 本章主线

Scaling laws 进阶与实践的主线是从小规模实验走向大规模决策。读者需要同时追踪参数量、token 数、compute budget、loss 和 benchmark 之间的关系。

2.1 本章要解决的核心问题

本讲继续 scaling laws，重点转向实践细节：muP、WSD learning rate、batch/LR scaling、MiniCPM 和 DeepSeek 等公开案例。读完这一段后，应能用一两句话说明 Scaling laws 进阶与实践要解决的瓶颈，并指出

Motivation today

What is the best practices for scaling and hparam tuning LMs?

- Does chinchilla's approach to scaling actually work?
- Can we save compute when training and fitting these things?
- Should we be picking particular architectures / parametrizations to scale nicely?

图 2: 官方材料图页: Scaling laws 进阶与实践的课程页/PDF/script 图像锚点。

Lecture 11

SCALING – CASE STUDY AND DETAILS

CS336

图 3: 官方材料图页: Scaling laws 进阶与实践的课程页/PDF/script 图像锚点。

这个瓶颈发生在数据、模型、优化、系统还是评估层。

2.2 从机制到资源账本

进阶 scaling 的难点是如何降低拟合 scaling law 本身的成本。WSD 等 schedule 允许从稳定阶段重启 decay, 以便更便宜地估计数据-模型 tradeoff。随后把机制写成账本: 输入规模是什么, 主要状态是什么, 成本或质量指标怎样随变量改变。这个账本让后面的公式不再只是符号。

2.3 从课堂讲解到可复现实验

实践中要同时预测 loss、选择 batch、学习率、模型宽深比和训练 tokens; 任何 recipe 改变都需要重新验证外推可靠性。最后把课堂讲解转成可复现实验: 固定协议, 改变一个变量, 记录中间量, 并解释结果为何支持或反驳原来的机制判断。

3 术语、符号与问题边界

本章的术语横跨模型、数据和系统。读者需要先知道每个词指向的对象：有些词描述数学目标，有些词描述张量形状，有些词描述硬件瓶颈，还有一些词描述评估协议。术语边界不清，后面的公式和实现就会变成空洞的缩写。

#	术语	阅读要求
1	forecasting（预测）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
2	overtraining（过训练/超额训练）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
3	data-constrained scaling（数据受限缩放）	说明它如何改变训练样本或 token 分布，并给出一个会改变结论的数据边界条件。
4	muP	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
5	emergence（涌现）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。

这些术语之间有依赖关系。例如 tokenization 改变 sequence length，sequence length 又改变 attention FLOPs、KV cache 和 evaluation token budget；parallelism 改变显存占用，也改变通信瓶颈和故障恢复方式。

3.1 术语之间的关系

在本章中，forecasting（预测）给出问题入口，overtraining（过训练/超额训练）描述主要机制，data-constrained scaling（数据受限缩放）则把机制连接到可测量的结果。读教材时应当沿着这个链条追问：输入是什么，中间状态怎样改变，输出指标为什么随之变化。课程材料还会反复出现 scaling、batch、optimal、scale、learning、chinchilla 等词。它们不是一组同义标签，而是提示本章的不同层次：有的指对象，有的指操作，有的指约束或指标。因此，术语表不是背诵清单，而是后文阅读的索引。遇到一个术语时，先定位它属于 representation、optimization、systems、data 还是 evaluation，再判断它改变哪一项账本。

4 更复杂的 scaling 问题

现在进入更复杂的 scaling 问题。这一节把模型大小、训练 token、compute budget 和 loss 放在同一坐标系里，讨论何时可以从小实验推断大训练。

4.1 为什么要讨论更复杂的 scaling 问题

视频开场称本讲是 scaling journey 的继续，覆盖更高级细节和实践中扩展模型的重要问题。实践中不只问 loss 怎么随 (N,D) 变，还要问数据受限、训练长度、architecture 改变、optimizer 改变时预测是否仍成立。scaling law 的价值在于减少大 run 风险，而不是替代工程监控。更复杂的 scaling 问题把 Scaling laws 进阶与实践连接到大规模决策。接下来要看的不是曲线是否平滑，而是实验点、预算变量和外推假设是否一致。

4.2 更复杂的 scaling 问题的机制

视频开场称本讲是 scaling journey 的继续，覆盖更高级细节和实践中扩展模型的重要问题。在更复杂的 scaling 问题中，核心对象是模型规模、token 数和训练预算：小实验只有在预算、数据和优化条件清楚时，才可能支持大规模外推。实践中不只问 loss 怎么随 (N, D) 变，还要问数据受限、训练长度、architecture 改变、optimizer 改变时预测是否仍成立。因而同一个方法在不同规模下可能改变不同的变量，尤其是参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。比较方法时，必须同时给出问题规模和实验条件。scaling law 的价值在于减少大 run 风险，而不是替代工程监控。可检验性来自多个实验点、残差分析和外推误差估计；单条曲线若没有不确定性，就不能支撑大规模决策。因此，更复杂的 scaling 问题的学习目标是建立一套外推判断：读者应能说明小实验为何能或不能指导 Scaling laws 进阶与实践中的大训练配置。

4.3 更复杂的 scaling 问题的数学表达

$$L(C) = L_{\infty} + aC^{-\gamma}$$

符号说明： C 是总 compute；这个一维形式常用于简化预测，但会隐藏 N, D tradeoff。这个关系式给出更复杂的 scaling 问题的缩放假设。它把参数量、训练 token 数和 compute budget 放到同一张账本中，重点是判断小实验能否可靠外推到大训练。在 scaling 实验中，validation loss、benchmark transfer 和训练吞吐都只是不同侧面的 proxy。外推时要说明哪个 proxy 被拟合，哪个 proxy 只是辅助诊断。

4.4 从更复杂的 scaling 问题到程序

把更复杂的 scaling 问题写成程序后，抽象机制会变成输入、状态和输出之间的变换。下面的伪代码保留核心计算顺序。

```
fit_compute_curve(runs)
check_breakpoints_for_new_regime()
forecast_only_with_errorBars()
```

阅读这段流程时，重点看小规模实验怎样产生可外推曲线。参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系需要在每个实验点上同时记录，缺少其中一项都会改变 scaling 解释。

4.5 边界条件与常见误解

- 单变量 compute curve 可能掩盖模型大小和数据量的不同失败模式。
- architecture 改变后，把旧曲线直接外推很危险。
- 小规模 scaling 曲线不一定覆盖大训练的优化不稳定、数据瓶颈和系统瓶颈。
- compute-optimal 结论依赖目标指标；以 loss 最优并不自动意味着 benchmark、latency 或成本最优。

4.6 与本章其他概念的关系

更复杂的 scaling 问题连接本章的小实验和大规模决策：术语表标出预算变量，公式给出外推结构，伪代码组织实验点，例题检查 compute-optimal 判断是否稳健。

4.7 怎样检验更复杂的 scaling 问题

检验更复杂的 scaling 问题时，先把抽象说法落到模型规模、token 数和训练预算上。与 forecasting、compute、regime、loss、practice 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： $\backslash(C\backslash)$ 是总 compute；这个一维形式常用于简化预测，但会隐藏 $\backslash(N,D\backslash)$ tradeoff。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。scaling 消融通常一次只改变参数量、token 数、数据 pipeline 或优化 schedule。若多个轴同时移动，曲线拟合会混合不同机制。最后检查失败条件。工程判断往往在反例中变清楚：单变量 compute curve 可能掩盖模型大小和数据量的不同失败模式；architecture 改变后，把旧曲线直接外推很危险。复习时可以把这些限制改写成反例测试，观察它们怎样改变参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。把实验结论放回 Scaling laws 进阶与实践时，要说明更复杂的 scaling 问题改变的是预算轴、拟合曲线、外推误差还是目标指标。能拆开这些因素，才说明读者真正理解 scaling。

4.8 概念辨析：更复杂的 scaling 问题

forecasting、compute、regime、loss、practice 常一起出现在 scaling 讨论中，但它们分别指预算轴、拟合对象、外推假设或风险来源。读者应先确定曲线中的自变量和因变量。区分这些词时，可以先问它们是否改变参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕更复杂的 scaling 问题复习时，可以画三到五个小实验点：记录参数量、token 数、预算和 loss，再检查外推是否合理。

4.9 更复杂的 scaling 问题的小结

- 讨论更复杂的 scaling 问题时，应同时报告问题规模、实验设置和主要观测变量，尤其是参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。
- 如果方法声称可以外推，要说明实验点覆盖范围、拟合形式和数据管线是否一致；曲线本身不是自然定律。
- scaling 结论需要把 loss、benchmark 和成本之间的转换关系说清楚，否则 compute-optimal 只是一条局部经验曲线。

5 数据受限与重复数据

现在进入数据受限与重复数据。这一节关心训练样本如何形成、被筛选并进入混合分布；它直接决定模型看到什么，也决定 evaluation 是否可能被污染。

5.1 为什么要讨论数据受限与重复数据

当高质量 token 有限时，简单增加训练 tokens 可能意味着重复数据或低质量数据。重复训练同一数据会改变 scaling 行为：短期可能降低 loss，长期可能过拟合或降低泛化。这为后续 data lectures 铺垫：scaling law 必须和数据来源、过滤、去重、混合策略一起讨论。数据受限与重复数据把 Scaling laws 进阶与实践连接到训练分布。接下来要看的不是语料名称，而是样本来源、处理顺序和版本记录怎样改变模型可学习的证据。

5.2 数据受限与重复数据的机制

当高质量 token 有限时，简单增加训练 tokens 可能意味着重复数据或低质量数据。在数据受限与重复数据中，核心对象是样本、来源和分布状态：数据管线每改变一次样本集合，模型实际学习的分布也随之改变。重复训练同一数据会改变 scaling 行为：短期可能降低 loss，长期可能过拟合或降低泛化。因而同一个方法在不同规模下可能改变不同的变量，尤其是样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。比较方法时，必须同时给出问题规模和实验条件。这为后续 data lectures 铺垫：scaling law 必须和数据来源、过滤、去重、混合策略一起讨论。可检验性来自 provenance 记录、过滤前后计数、抽样审计和 held-out contamination 检查；没有这些记录，数据结论无法复现。因此，数据受限与重复数据的学习目标是建立一条数据谱系：读者应能从原始来源追到训练 token，并说明每个处理步骤改变了什么分布。

5.3 数据受限与重复数据的数学表达

$$D_{\text{effective}} \leq D_{\text{raw}} \quad \text{when duplicates and low-quality tokens are present}$$

符号说明： D_{raw} 是原始 token 数， $D_{\text{effective}}$ 是对泛化真正有贡献的有效 token 数。这个关系式刻画数据受限与重复数据对数据分布的影响。读者应关心哪些变量来自采集过程，哪些变量来自过滤或混合策略，哪些变量只能通过抽样审计估计。在数据实验中，去重率、过滤通过率、语言比例和抽样人工判读都是 proxy。它们不能单独代表数据质量，必须和来源记录、版本号以及污染检查一起解释。

5.4 从数据受限与重复数据到程序

把数据受限与重复数据写成程序时，要明确每一步怎样改写样本集合。下面的伪代码强调输入、过滤、统计和输出之间的关系。

```
effective_tokens = estimate_after_dedup_and_filter(raw_tokens)
refit_scaling_law_if_data_pipeline_changes()
```

阅读这段流程时，重点看数据受限与重复数据怎样改变样本集合。每个过滤、去重或混合步骤都应留下计数和抽样检查，使样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险能够被复核。

5.5 边界条件与常见误解

- 有效 token 数不可直接从文件大小读取。
- 低质量数据可能增加 token count 但恶化 downstream behavior。
- 小样本抽样只能发现部分数据问题，不能证明语料整体无污染或无偏差。
- 数据版本、许可状态和处理脚本如果缺失，后续 evaluation 与 scaling 结论都会失去可复现基础。

5.6 与本章其他概念的关系

数据受限与重复数据连接本章的数据来源和模型行为：定义说明样本如何进入语料，公式刻画分布变化，伪代码展示处理顺序，例题检验数据决策怎样影响训练结论。

5.7 怎样检验数据受限与重复数据

检验数据受限与重复数据时，先把抽象说法落到样本、来源和分布状态上。与 data、duplicates、effective tokens、overtraining、quality 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不

能得到可复现结论。一个合适的小实验应当保留本节机制，同时让样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： (D_{raw}) 是原始 token 数， $(D_{\text{effective}})$ 是对泛化真正有贡献的有效 token 数。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。数据消融通常一次只改变过滤阈值、去重策略、混合权重或数据版本。若同时更换 tokenizer、语料和评估集，就无法知道改进来自哪一步。最后检查失败条件。工程判断往往在反例中变清楚：有效 token 数不可直接从文件大小读取；低质量数据可能增加 token count 但恶化 downstream behavior。复习时可以把这些限制改写成反例测试，观察它们怎样改变样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。把实验结论放回 Scaling laws 进阶与实践时，要说明数据受限与重复数据改变的是样本分布、数据质量、污染风险还是可复现性。能追到这些来源，才说明读者真正理解数据机制。

5.8 概念辨析：数据受限与重复数据

data、duplicates、effective tokens、overtraining、quality 常一起出现在数据管线中，但它们分别指来源、处理动作、版本记录或风险标记。读者应先判断它们改变的是样本集合还是审计证据。其中，data（数据）是模型能力、偏差和法律风险的主要来源。区分这些词时，可以先问它们是否改变样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕数据受限与重复数据复习时，可以构造一小批文档：保留来源和版本，运行一个处理步骤，再比较样本计数和抽样质量。

5.9 数据受限与重复数据的小结

- 讨论数据受限与重复数据时，应同时报告问题规模、实验设置和主要观测变量，尤其是样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。
- 如果方法声称数据更好，要说明过滤、去重、混合或合成步骤改变了哪一部分分布，并检查 benchmark contamination。
- 数据结论需要抽样审计和版本记录；没有 provenance 的语料变化不能支持可引用结论。

6 从 loss 到能力指标

现在进入从 loss 到能力指标。这一节把模型大小、训练 token、compute budget 和 loss 放在同一坐标系里，讨论何时可以从小实验推断大训练。

6.1 为什么要讨论从 loss 到能力指标

loss 平滑、可测、适合拟合，但用户关心的是 reasoning、coding、instruction following 等能力。能力指标可能出现阈值效应或评估噪声，使其比 loss 更难预测。进阶 scaling 分析要把 validation loss、benchmark score、推理成本和安全约束一起纳入。从 loss 到能力指标把 Scaling laws 进阶与实践连接到大规模决策。接下来要看的不是曲线是否平滑，而是实验点、预算变量和外推假设是否一致。

6.2 从 loss 到能力指标的机制

loss 平滑、可测、适合拟合，但用户关心的是 reasoning、coding、instruction following 等能力。在从 loss 到能力指标中，核心对象是模型规模、token 数和训练预算：小实验只有在预算、数据和优化条件清楚时，才

可能支持大规模外推。能力指标可能出现阈值效应或评估噪声，使其比 loss 更难预测。因而同一个方法在不同规模下可能改变不同的变量，尤其是参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。比较方法时，必须同时给出问题规模和实验条件。进阶 scaling 分析要把 validation loss、benchmark score、推理成本和安全约束一起纳入。可检验性来自多个实验点、残差分析和外推误差估计；单条曲线若没有不确定性，就不能支撑大规模决策。因此，从 loss 到能力指标的学习目标是建立一套外推判断：读者应能说明小实验为何能或不能指导 Scaling laws 进阶与实践中的大训练配置。

6.3 从 loss 到能力指标的数学表达

$$\mathbb{E}[\text{score}] \approx h(L, \text{prompt}, \text{eval distribution})$$

符号说明： h 表示从 loss 到 benchmark score 的经验映射；它依赖评估任务和 prompting。这个关系式给出从 loss 到能力指标的缩放假设。它把参数量、训练 token 数和 compute budget 放到同一张账本中，重点是判断小实验能否可靠外推到大训练。在 scaling 实验中，validation loss、benchmark transfer 和训练吞吐都只是不同侧面的 proxy。外推时要说明哪个 proxy 被拟合，哪个 proxy 只是辅助诊断。

6.4 从 loss 到能力指标到程序

把从 loss 到能力指标写成程序后，抽象机制会变成输入、状态和输出之间的变换。下面的伪代码保留核心计算顺序。

```
for checkpoint in training_curve:
    measure_loss(checkpoint)
    measure_benchmarks(checkpoint)
    fit_loss_to_score_mapping()
```

阅读这段流程时，重点看小规模实验怎样产生可外推曲线。参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系需要在每个实验点上同时记录，缺少其中一项都会改变 scaling 解释。

6.5 边界条件与常见误解

- benchmark score 的方差和 contamination 会扭曲趋势。
- 能力不是只由 pretraining loss 决定，post-training 也会改变可见行为。
- 小规模 scaling 曲线不一定覆盖大训练的优化不稳定、数据瓶颈和系统瓶颈。
- compute-optimal 结论依赖目标指标；以 loss 最优并不自动意味着 benchmark、latency 或成本最优。

6.6 与本章其他概念的关系

从 loss 到能力指标连接本章的小实验和大规模决策：术语表标出预算变量，公式给出外推结构，伪代码组织实验点，例题检查 compute-optimal 判断是否稳健。

6.7 怎样检验从 loss 到能力指标

检验从 loss 到能力指标时，先把抽象说法落到模型规模、token 数和训练预算上。与 benchmark、emergence、score、loss、post-training 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让参数量、训练 token 数、compute budget、loss

外推误差和 benchmark 转化关系中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： $\backslash(h\backslash)$ 表示从 loss 到 benchmark score 的经验映射；它依赖评估任务和 prompting。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。scaling 消融通常一次只改变参数量、token 数、数据 pipeline 或优化 schedule。若多个轴同时移动，曲线拟合会混合不同机制。最后检查失败条件。工程判断往往在反例中变清楚：benchmark score 的方差和 contamination 会扭曲趋势；能力不是只由 pretraining loss 决定，post-training 也会改变可见行为。复习时可以把这些限制改写成反例测试，观察它们怎样改变参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。把实验结论放回 Scaling laws 进阶与实践时，要说明从 loss 到能力指标改变的是预算轴、拟合曲线、外推误差还是目标指标。能拆开这些因素，才说明读者真正理解 scaling。

6.8 概念辨析：从 loss 到能力指标

benchmark、emergence、score、loss、post-training 常一起出现在 scaling 讨论中，但它们分别指预算轴、拟合对象、外推假设或风险来源。读者应先确定曲线中的自变量和因变量。区分这些词时，可以先问它们是否改变参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕从 loss 到能力指标复习时，可以画三到五个小实验点：记录参数量、token 数、预算和 loss，再检查外推是否合理。

6.9 从 loss 到能力指标的小结

- 讨论从 loss 到能力指标时，应同时报告问题规模、实验设置和主要观测变量，尤其是参数量、训练 token 数、compute budget、loss 外推误差和 benchmark 转化关系。
- 如果方法声称可以外推，要说明实验点覆盖范围、拟合形式和数据管线是否一致；曲线本身不是自然定律。
- scaling 结论需要把 loss、benchmark 和成本之间的转换关系说清楚，否则 compute-optimal 只是一条局部经验曲线。

7 综合讨论：从局部机制到完整系统

本章的主题是 Scaling laws 进阶与实践，但它不应被读成几个相邻知识点的拼接。更复杂的 scaling 问题、数据受限与重复数据、从 loss 到能力指标共同回答同一个问题：语言模型系统中哪些对象可以被定义，哪些成本可以被估算，哪些结论必须通过实验或评估来确认。这些对象包括 forecasting（预测）、overtraining（过训练/超额训练）、data-constrained scaling（数据受限缩放）、muP、emergence（涌现）。读者可以把它们看成一组接口：有的接口连接文本和 token，有的连接模型结构和张量计算，有的连接数据样本和训练目标，有的连接离线评估和在线服务。接口一旦改变，后面的成本、错误类型和优化空间也会随之改变。

7.1 更复杂的 scaling 问题在系统中的位置

更复杂的 scaling 问题首先提供一个局部解释：视频开场称本讲是 scaling journey 的继续，覆盖更高级细节和实践中扩展模型的重要问题。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，更复杂的 scaling 问题会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，更复杂的 scaling 问题不能脱离边

界条件理解。一个关键 caveat 是：单变量 compute curve 可能掩盖模型大小和数据量的不同失败模式。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.2 数据受限与重复数据在系统中的位置

数据受限与重复数据首先提供一个局部解释：当高质量 token 有限时，简单增加训练 tokens 可能意味着重复数据或低质量数据。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，数据受限与重复数据会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，数据受限与重复数据不能脱离边界条件理解。一个关键 caveat 是：有效 token 数不可直接从文件大小读取。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.3 从 loss 到能力指标在系统中的位置

从 loss 到能力指标首先提供一个局部解释：loss 平滑、可测、适合拟合，但用户关心的是 reasoning、coding、instruction following 等能力。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，从 loss 到能力指标会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，从 loss 到能力指标不能脱离边界条件理解。一个关键 caveat 是：benchmark score 的方差和 contamination 会扭曲趋势。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.4 把本章知识用于读论文和复现实验

读相关论文时，可以把本章变成一个检查表。第一，论文是否清楚定义了输入规模、模型规模、数据来源和评估协议；第二，论文声称的改进落在哪个变量上；第三，作者是否报告了会让结论失效的设置；第四，实验是否能分离模型结构、数据质量、优化设置和系统实现的影响。复现实验时，也应避免直接追求大规模。先用最小程序复现一个公式、一个伪代码分支或一个 profile 现象，再扩大输入规模。这样做的原因很简单：如果小规模程序无法解释中间量，大规模训练只会把错误隐藏在日志和随机波动里。把这些检查合在一起，本章给出的不是一个固定答案，而是一种阅读 CS336 的方法：每个模型机制都要落到变量，每个变量都要落到测量，每个测量都要说明协议，每个协议都要承认边界。因此，本章中的任何一句结论都可以被改写成一个可引用的完整句式：在给定数据、模型、硬件和评估协议下，某个机制改变了某个变量，并通过某个观测指标表现出来。这样的句子比单独背诵术语更长，但它包含了自学、复现和引用时真正需要的信息。把本章用于自学时，可以按三遍阅读。第一遍只追踪对象和定义，确认每个术语到底指向文本、token、张量、样本、请求还是评估记录。第二遍追踪公式和伪代码，确认每一步改变了哪些状态。第三遍追踪实验和 caveat，确认哪些结论只在特定规模、数据或硬件条件下成立。把本章用于复习时，则应反过来操作：先从一个失败案例或异常指标出发，再回到相应机制。loss 曲线异常可能指向数据、优化或模型容量；吞吐异常可能指向 kernel、通信或调度；benchmark 异常可能指向 contamination、prompt protocol 或采样设置。能够从异常反推机制，是掌握本章的实用标准。

8 例题：把概念变成可计算检查

8.1 为什么数据 pipeline 改变后要重拟合 scaling law

同一模型大小和 token 数，换用更强过滤后的数据。

- validation loss 曲线可能整体下移，因为有效 token 质量提高。
- 不同 benchmark 的改善幅度不一定一致。
- 旧曲线的外推误差不再代表新数据分布。

结论。 scaling law 不是模型大小的自然属性，而是模型、数据、优化和评估分布的联合经验规律。这个例题的目的不是替代完整工程实现，而是给出一种最小推理形式：把问题转成变量，写出近似公式或伪代码，再说明近似何时失效。

9 实验设计：从小例子到可复现结论

学习 Scaling laws 进阶与实践不能停在概念层。一个教材化结论至少需要四件事：可构造的输入、可观测的中间量、可比较的指标，以及清楚的失败条件。下面的实验设计不是要求训练大模型，而是要求读者用小规模程序复现本章机制。

9.1 最小输入

先构造只触发本章核心机制的小输入。例如 tokenizer 章节可以用几个包含 rare words 和 Unicode 字符的字符串；GPU 章节可以用一个 elementwise kernel 和一个 GEMM；data 章节可以用十几篇近重复文档。小输入的价值在于它让错误可见。

9.2 资源账本

对同一输入写下 FLOPs、bytes moved、显存峰值、通信量或样本数量的估算。估算不需要一开始就精确，但必须说明单位和近似假设。随后用 profiler、benchmark、validation loss 或人工检查去验证估算是否同量级。

9.3 单变量消融

一次只改变一个因素，例如 vocabulary size、head 数、batch size、filter threshold、reward scale、decoding temperature 或 GPU 数量。若多个变量同时变化，实验只能说明系统变了，不能说明机制是什么。

9.4 失败条件

最后要刻意触发失败：极端 context length、错误 dtype、过小 batch、污染 benchmark、低质量数据或不可靠 verifier。失败条件常常比成功曲线更能说明一个方法的边界。

9.5 记录与报告

实验记录应能让另一个读者复现同一结论。最少要写清楚输入构造、代码版本、随机种子、硬件、batch shape、数据版本、评价指标和异常处理方式。报告结论时不要只写“更快”或“更好”，而要说明快在哪里、好

在哪个指标上，以及是否牺牲了内存、稳定性、数据质量或可解释性。常见报告错误是把局部现象写成普遍规律。例如一次小模型实验里的 loss 改善，不等于更大模型或不同数据分布上也会改善；一次 kernel benchmark 的吞吐提升，不等于端到端 serving latency 一定降低。教材中的实验报告应始终把结论限定在可复现条件内。

9.6 从小实验到大结论

小实验的作用不是替代课程中的完整训练或系统实现，而是建立一条可检查的推理链。先在小输入上确认变量方向，再在中等规模上确认数量级，最后才讨论是否能外推到更大模型、更多 token 或更复杂 workload。缺少中间层级时，大结论通常只是在复述直觉。因此，实验报告最好同时包含正例和反例：正例说明机制在受控条件下怎样工作，反例说明条件稍变时哪里失效。对 CS336 这样的课程，反例尤其重要，因为 language model 的错误常来自多个层面叠加：数据分布、优化稳定性、硬件瓶颈、评估协议和服务负载都可能同时改变观察结果。还要记录资料版本与复现边界。公开视频、课程页、配套代码和 PDF 可能在不同时间更新；复现实验应写明使用的材料版本，并说明哪些结论来自课程事实，哪些是为了帮助理解而加入的解释性推导。这样才能把学习笔记变成可检查的技术记录。若复现实验与课程结论不一致，首先检查输入、版本和评估协议，而不是立即修改模型假设。

9.7 把本章重新读成一条证据链

完成小实验之后，可以把 Scaling laws 进阶与实践重新读成一条证据链：先用“更复杂的 scaling 问题”确定问题入口，再用“数据受限与重复数据”解释主要机制，最后用“从 loss 到能力指标”检查边界或外推条件。这种读法比按页背诵更接近教材训练，因为它要求每个结论都能说明前提、变量和证据。本章反复使用的术语包括 forecasting（预测）、overtraining（过训练/超额训练）、data-constrained scaling（数据受限缩放）、muP、emergence（涌现）。复习时不要只写定义，而要为每个术语补上一句操作性说明：它在课程代码、公式、数据记录或评估协议中对应什么对象；如果这个对象被删除、替换或缩放，哪一个观测量会变化。最后，用一个反例结束复习。反例可以是资源估算不准、数据污染、reward 失真、benchmark 解析失败、长上下文显存爆炸或 scaling 外推失效。能解释反例的章节，才真正具备自学和引用价值；不能解释反例的章节，只是把课程材料换了一种排版。引用本章结论时，还应保留来源边界：哪些说法直接来自视频、课程页、slides 或代码，哪些是为了串联教材叙述而加入的解释性推导。这个边界不会削弱结论，反而让读者知道何处可以复核，何处需要在自己的实验中重新验证。如果后续课程材料更新，也可以沿着同一证据链替换来源，而不必重写整章结构。

10 课程资料与回看路径

教材正文已经把主要概念、公式和实现逻辑组织成连续叙述；本节只提供回到课程材料的阅读路径。读者复习时可以先完成前文练习，再按下面的时间段或资料组核对细节。

10.1 视频回看路径

时间	关键词	复习任务
00:00:05-00:06:25	going, talk, scale, scaling, think, we're, learning	回看定义、例子、推导或 caveat 在该段中的位置。
00:19:14-00:26:00	scaling, they're, then, going, sparsity, learning, different	回看定义、例子、推导或 caveat 在该段中的位置。
00:39:25-00:45:53	think, maybe, scaling, then, scale., scale, over	回看定义、例子、推导或 caveat 在该段中的位置。

00:51:54-00:57:45	muon, think, then, it's, scale, matrix, very	回看定义、例子、推导或 caveat 在该段中的位置。
01:10:16-01:16:41	going, scaling, learning, very, it's, over, different	回看定义、例子、推导或 caveat 在该段中的位置。

这些时间段用于定位视频中的讲解顺序。严肃引用时，应回到视频片段确认讲者表述、板书或幻灯片内容。回看视频时，不必逐字抄录字幕。更有效的方法是把每个时间段判定为定义、例子、推导、代码解释或 caveat，再把它放回本章对应小节。

10.2 课程资料阅读路径

资料组	标题/页块	关键词
official_01	Lecture 11 S C A L I N G -C A S E S T U D Y A N D D E T A I L S CS336	scaling, scale, initialization, optimizers, cs336, motivation, best
official_02	Working through some detailed, public scaling recipes 1. MiniCPM 2. DeepSeek	scaling, minicpm, stabilize, many, working, detailed, public
official_03	Optimal LR According to muP –optimal learning rate should be (roughly) stable. Is it?	scaling, batch, learning, size, optimal, analysis, rate
official_05	Observation 3 (?) robustness Generalization to MoEs Generalization to other datasets	scaling, scale, update, parametrization, muon, gains, recall
official_06	What is muP not robust to? Exotic optimizers There are other, exotic optimizers based ...	scaling, robust, exotic, optimizers, strong, weight, decay

课程资料通常给出更稳定的标题、公式、图或代码结构；视频则补充动机和口头解释。两者合起来构成本章的事实边界。阅读资料时，可以把每个资料组拆成三个对象：一个定义，一个公式或伪代码动作，一个边界条件。这样比逐字翻译页面或脚本更容易形成可用知识。

11 实践说明、历史位置与风险

Scaling laws 进阶与实践在 CS336 的课程结构中连接基础机制和规模化问题。基础机制解释方法为什么成立；规模化问题检验它在更大模型、更长上下文、更复杂数据或真实 serving workload 下是否仍成立。历史位置不等于模型年表。更重要的是识别问题如何迁移：早期方法解决小规模建模问题，现代方法常常解决同一问题在大规模数据、GPU 集群、长上下文或人类偏好约束下的新形态。

- 资料版本限制 (version risk): 课程页、公开视频和配套资料可能在学期中更新；引用时应注明访问日期或资料版本。
- 测量口径限制 (measurement risk): FLOPs、tokens/s、benchmark score、reward 等数字只有在协议完整时可比较。
- 规模外推限制 (scaling risk): 小模型或小 batch 的机制不必然外推到 frontier-scale。
- 数据分布限制 (data risk): 数据来源、过滤和去重策略会改变模型行为，也会改变 evaluation contamination 风险。

12 复习要点

下面的问题把本章内容压缩为可反复检查的条目。每个条目都应能回到前文的解释、公式、伪代码或例题。

条目	对象	检查动作
条目 1	forecasting (预测)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 2	overtraining (过训练/超额训练)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 3	data-constrained scaling (数据受限缩放)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 4	muP	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 5	emergence (涌现)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。

12.1 综合自测

- 我能否不用课程原句，独立解释本章中心问题？
- 我能否把本章至少一个公式改写成可运行的估算函数？
- 我能否指出一个看似改进但实际转移成本的方法？
- 我能否回到视频和课程资料，找到支持本章关键论断的位置？
- 我能否设计一个最小 ablation，验证本章一个 caveat？

13 总结与延伸

13.1 本章小结

- Scaling laws 进阶与实践的核心不是记忆术语，而是理解对象、变量和机制之间的关系。
- 视频给出讲解顺序，课程资料提供可核对的公式、图、代码或页面结构。
- 复习时应把每个术语连接到至少一个变量、一个实现动作和一个失败模式。

13.2 延伸解释

本章中的延伸解释用于连接课程材料中的概念，例如说明公式里的 proxy、解释系统 tradeoff，或把多个材料片段串成一个完整推理链。若需要严肃引用，应回到课程视频和配套资料核对原始表述。

13.3 练习

- 定义题：用中英双语解释本章任意五个重要术语，并说明它们属于 compute、memory、data、optimization、evaluation 还是 deployment 账本。
- 推导题：选择本章一个公式，逐个解释符号、单位和近似假设，并说明哪个变量最难在真实系统中测量。
- 实现题：把本章伪代码改写成一个最小 Python sanity check，不要求训练大模型，但要输出可检查的中间量。
- 资料题：从“课程资料与回看路径”中选择一个视频时间段，回到原始视频核对讲者例子，并记录是否存在字幕误差。
- 评估题：为本章方法设计一个 ablation 或 benchmark，说明控制变量、数据版本、指标和 failure criteria。
- 边界题：说明本章一个结论在更长 context、更大 batch、更多 GPU 或更复杂数据分布下可能如何失效。
- 综合题：把本章内容和前一章或后一章连接起来，写出一个端到端训练/推理 pipeline 中的依赖关系。
- 批判题：指出一个容易被营销材料夸大的说法，并用本章的资源账本或评估协议反驳它。

12 12 Evaluation 方法论

第 12 讲: Evaluation 方法论

CS336 Spring 2026 public videos and official course materials

目录

1 本章导读	2
1.1 本章知识路线	2
1.2 结构图	2
2 本章主线	2
2.1 本章要解决的核心问题	2
2.2 从机制到资源账本	3
2.3 从课堂讲解到可复现实验	3
3 术语、符号与问题边界	4
3.1 术语之间的关系	4
4 评估在 LM 生命周期中的位置	4
4.1 为什么要讨论评估在 LM 生命周期中的位置	4
4.2 评估在 LM 生命周期中的位置的机制	5
4.3 评估在 LM 生命周期中的位置的数学表达	5
4.4 从评估在 LM 生命周期中的位置到程序	5
4.5 边界条件与常见误解	5
4.6 与本章其他概念的关系	5
4.7 怎样检验评估在 LM 生命周期中的位置	6
4.8 概念辨析: 评估在 LM 生命周期中的位置	6
4.9 评估在 LM 生命周期中的位置的小结	6
5 Benchmark 设计与污染	6
5.1 为什么要讨论 Benchmark 设计与污染	6
5.2 Benchmark 设计与污染的机制	7
5.3 Benchmark 设计与污染的数学表达	7
5.4 从 Benchmark 设计与污染到程序	7
5.5 边界条件与常见误解	7

5.6	与本章其他概念的关系	7
5.7	怎样检验 Benchmark 设计与污染	8
5.8	概念辨析: Benchmark 设计与污染	8
5.9	Benchmark 设计与污染的小结	8
6	从离线评估到 deployment 指标	8
6.1	为什么要讨论从离线评估到 deployment 指标	8
6.2	从离线评估到 deployment 指标的机制	9
6.3	从离线评估到 deployment 指标的数学表达	9
6.4	从从离线评估到 deployment 指标到程序	9
6.5	边界条件与常见误解	9
6.6	与本章其他概念的关系	9
6.7	怎样检验从离线评估到 deployment 指标	10
6.8	概念辨析: 从离线评估到 deployment 指标	10
6.9	从离线评估到 deployment 指标的小结	10
7	综合讨论: 从局部机制到完整系统	10
7.1	评估在 LM 生命周期中的位置在系统中的位置	11
7.2	Benchmark 设计与污染在系统中的位置	11
7.3	从离线评估到 deployment 指标在系统中的位置	11
7.4	把本章知识用于读论文和复现实验	11
8	例题: 把概念变成可计算检查	12
8.1	同一 benchmark 的可复现评分	12
9	实验设计: 从小例子到可复现结论	12
9.1	最小输入	12
9.2	资源账本	12
9.3	单变量消融	12
9.4	失败条件	13
9.5	记录与报告	13
9.6	从小实验到大结论	13
9.7	把本章重新读成一条证据链	13
10	课程资料与回看路径	13
10.1	视频回看路径	14
10.2	课程资料阅读路径	14
11	实践说明、历史位置与风险	14

12 复习要点	15
12.1 综合自测	15
13 总结与延伸	16
13.1 本章小结	16
13.2 延伸解释	16
13.3 练习	16

1 本章导读

本章讨论 **Evaluation 方法论**。CS336 的第一层目标不是把现成大模型当作黑箱使用，而是把 language model 从输入文本、训练目标、模型结构、数据、系统和评估这些部件重新拆开。只有拆开之后，读者才能判断一个设计选择究竟改变了什么。设想两个 checkpoint 的 validation loss 很接近，但一个在 coding benchmark 上好、另一个更安全。Evaluation（评估）不是给模型打一个总分，而是建立一套可复现、抗污染、能服务决策的测量系统。本章对应 CS336 Spring 2026 第 12 个公开视频，并结合课程页提供的可解析资料。视频和课程页给出事实边界；正文把这些材料改写为可自学的教材章节。阅读本章时，先理解问题为什么存在，再看定义、公式和实现形态，随后通过例题和练习检查自己是否能独立复现这条 reasoning chain。



图 1: 第 12 讲的课程图像锚点。

1.1 本章知识路线

- 本章首先说明 Evaluation 方法论为什么是 language modeling pipeline 中不可跳过的一环。
- 其次定义本章反复使用的术语，并说明这些术语对应的计算对象或系统对象。
- 随后用公式和伪代码表达主要机制，让概念能够落到可计算的变量上。
- 最后给出例题、风险讨论和练习，便于读者检查自己是否真正掌握了机制。

1.2 结构图

下面的图用于帮助读者定位本章的核心结构。先看概念之间的依赖，再回到正文中的公式、伪代码和例题。

2 本章主线

Evaluation 方法论的主线是把能力声明变成测量协议。一个分数只有在数据版本、prompt、采样、答案解析和误差分析都清楚时才可引用。

2.1 本章要解决的核心问题

本讲讲 evaluation：训练出模型后，必须用 loss、perplexity、benchmark、human preference、安全测试和部署指标分别回答不同问题。读完这一段后，应能用一两句话说明 Evaluation 方法论要解决的瓶颈，并指

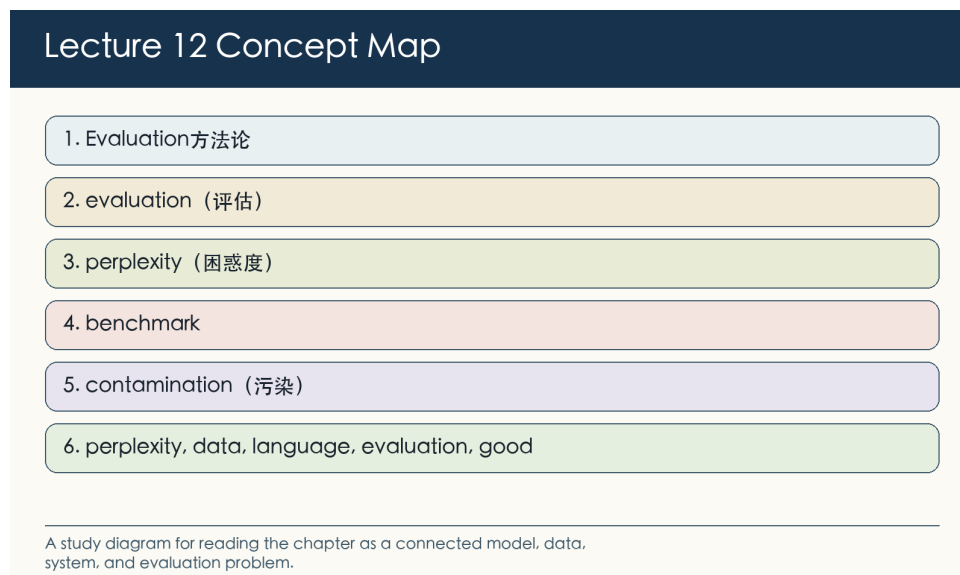


图 2: 第 12 讲概念图: 本章问题、术语和机制的关系。

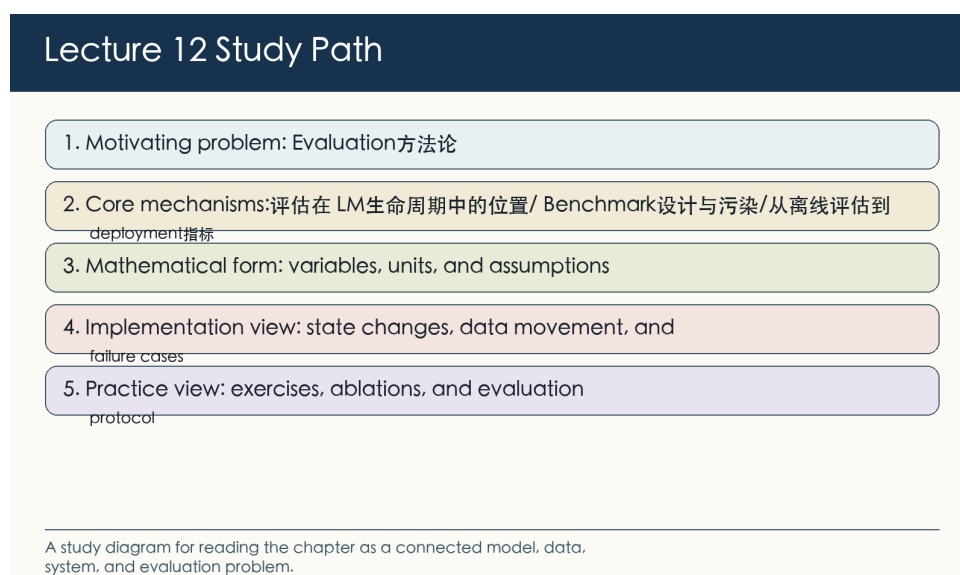


图 3: 第 12 讲学习路径: 从问题定义进入公式、实现、实验和练习。

出这个瓶颈发生在数据、模型、优化、系统还是评估层。

2.2 从机制到资源账本

benchmark 不是一个数字, 而是一套 prompt、scoring rule、aggregation、decoding setting 和 contamination control。测试集污染会让分数失去泛化意义。随后把机制写成账本: 输入规模是什么, 主要状态是什么, 成本或质量指标怎样随变量改变。这个账本让后面的公式不再只是符号。

2.3 从课堂讲解到可复现实验

评估要服务于决策: 选 checkpoint、选 data mixture、判断 post-training 是否改善、判断 serving tradeoff 是否可接受。最后把课堂讲解转成可复现实验: 固定协议, 改变一个变量, 记录中间量, 并解释结果为何支持或反驳原来的机制判断。

3 术语、符号与问题边界

本章的术语横跨模型、数据和系统。读者需要先知道每个词指向的对象：有些词描述数学目标，有些词描述张量形状，有些词描述硬件瓶颈，还有一些词描述评估协议。术语边界不清，后面的公式和实现就会变成空洞的缩写。

#	术语	阅读要求
1	evaluation（评估）	说明它测量的对象、协议依赖和可能的 contamination 或 prompt 敏感性。
2	perplexity（困惑度）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
3	benchmark	说明它测量的对象、协议依赖和可能的 contamination 或 prompt 敏感性。
4	contamination（污染）	说明它如何改变训练样本或 token 分布，并给出一个会改变结论的数据边界条件。
5	calibration（校准）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
6	human evaluation（人工评估）	说明它测量的对象、协议依赖和可能的 contamination 或 prompt 敏感性。

这些术语之间有依赖关系。例如 tokenization 改变 sequence length，sequence length 又改变 attention FLOPs、KV cache 和 evaluation token budget；parallelism 改变显存占用，也改变通信瓶颈和故障恢复方式。

3.1 术语之间的关系

在本章中，evaluation（评估）给出问题入口，perplexity（困惑度）描述主要机制，benchmark 则把机制连接到可测量的结果。读教材时应当沿着这个链条追问：输入是什么，中间状态怎样改变，输出指标为什么随之变化。课程材料还会反复出现 perplexity、evaluation、language、tasks、people、data 等词。它们不是一组同义标签，而是提示本章的不同层次：有的指对象，有的指操作，有的指约束或指标。因此，术语表不是背诵清单，而是后文阅读的索引。遇到一个术语时，先定位它属于 representation、optimization、systems、data 还是 evaluation，再判断它改变哪一项账本。

4 评估在 LM 生命周期中的位置

现在进入评估在 LM 生命周期中的位置。这一节把算法放到硬件和运行时中考察：同一个数学表达式在不同 batch、shape、GPU 拓扑和 kernel 实现下会得到不同的成本账本。

4.1 为什么要讨论评估在 LM 生命周期中的位置

视频指出课程至此已覆盖 architecture、optimizer、training loop、systems、scaling 和 inference，剩下的关键是如何判断模型好坏。评估连接训练目标、数据选择、模型比较和部署决策。好的评估应区分 intrinsic loss、task benchmark、用户偏好、安全性和系统指标。评估在 LM 生命周期中的位置把 Evaluation 方法论放进资源账本。接下来要看的不是单个 API，而是张量形状、内存层级、通信和调度怎样共同决定成本。

4.2 评估在 LM 生命周期中的位置的机制

视频指出课程至此已覆盖 architecture、optimizer、training loop、systems、scaling 和 inference，剩下的关键是如何判断模型好坏。在评估在 LM 生命周期中的位置中，核心对象是硬件和系统状态：同一算法会因为 memory hierarchy、通信拓扑或 kernel shape 不同而表现出完全不同的速度。评估连接训练目标、数据选择、模型比较和部署决策。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。好的评估应区分 intrinsic loss、task benchmark、用户偏好、安全性和系统指标。可检验性来自 profile trace、roofline 估算和端到端延迟分解；如果这些证据互相矛盾，就要先解释 measurement setup。因此，评估在 LM 生命周期中的位置的学习目标是建立一张成本地图：读者应能指出瓶颈在计算、内存、通信还是调度，并能设计一个小 benchmark 去验证。

4.3 评估在 LM 生命周期中的位置的数学表达

$$\text{PPL} = \exp \left(-\frac{1}{T} \sum_{t=1}^T \log p_{\theta}(x_t | x_{<t}) \right)$$

符号说明：PPL 是 token-level cross entropy 的指数形式；越低通常表示语言建模越好。这个关系式应被读成评估在 LM 生命周期中的位置的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。在系统实验中，tokens/s、TTFT、显存峰值和 kernel time 都是 proxy。它们各自只观察一部分系统，因此报告时要同时写清 workload shape、warmup、同步方式和硬件型号。

4.4 从评估在 LM 生命周期中的位置到程序

把评估在 LM 生命周期中的位置写成程序时，最重要的是暴露状态变化和数据移动。下面的伪代码保留执行顺序，省略框架样板。

```
evaluate_checkpoints = [loss, perplexity, benchmark_suite, safety_suite]
compare_with_confidence_intervals(evaluate_checkpoints)
```

阅读这段实现时，重点看评估在 LM 生命周期中的位置怎样移动张量、占用显存并触发通信。实验记录至少要包含吞吐、延迟、显存峰值、通信量和 kernel 利用率，否则很难判断瓶颈来自算子、内存层级还是调度。

4.5 边界条件与常见误解

- perplexity 不直接等于 instruction-following 能力。
- 不同 tokenizer 下的 perplexity 不可直接比较。
- 小规模 kernel benchmark 不一定预测端到端训练或 serving latency，因为调度、通信和请求形状会改变瓶颈。
- 硬件规格表和框架 profiler 都需要结合 workload 解释，不能把单个峰值数字当作机制证明。

4.6 与本章其他概念的关系

评估在 LM 生命周期中的位置连接本章的数学机制和工程成本：术语表说明对象，公式估算资源，伪代码暴露状态变化，例题则让读者检查一次完整的成本推理。

4.7 怎样检验评估在 LM 生命周期中的位置

检验评估在 LM 生命周期中的位置时，先把抽象说法落到硬件和系统状态上。与 evaluation、perplexity、loss、benchmark、checkpoint 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明：PPL 是 token-level cross entropy 的指数形式；越低通常表示语言建模越好。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。系统消融通常一次只改变 batch shape、sequence length、GPU 数、kernel 实现或调度策略。若同时改变硬件和 workload，实验只能说明端到端表现不同，不能定位瓶颈。最后检查失败条件。工程判断往往在反例中变清楚：perplexity 不直接等于 instruction-following 能力；不同 tokenizer 下的 perplexity 不可直接比较。复习时可以把这些限制改写成反例测试，观察它们怎样改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。把实验结论放回 Evaluation 方法论时，要说明评估在 LM 生命周期中的位置改变的是训练吞吐、推理延迟、显存峰值还是通信开销。能做出这种归因，才说明读者已经从接口使用进入系统解释。

4.8 概念辨析：评估在 LM 生命周期中的位置

evaluation、perplexity、loss、benchmark、checkpoint 常一起出现在系统讲解中，但它们分别指硬件对象、程序操作、瓶颈来源或测量指标。读者应先定位每个词在执行路径中的位置。其中，evaluation（评估）把 loss、benchmark、人类偏好和部署指标区分开。区分这些词时，可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕评估在 LM 生命周期中的位置复习时，可以写一个小 benchmark：固定输入形状，改变一个系统变量，记录 profile trace 和端到端时间。

4.9 评估在 LM 生命周期中的位置的小结

- 讨论评估在 LM 生命周期中的位置时，应同时报告问题规模、实验设置和主要观测变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。
- 如果方法声称更快，要区分计算减少、内存访问减少、通信隐藏和调度改善；这些机制不能用一个 tokens/s 数字互相替代。
- profile trace、roofline 估算和端到端 benchmark 应互相校验，单独一个指标不足以解释系统瓶颈。

5 Benchmark 设计与污染

现在进入 Benchmark 设计与污染。这一节把“模型好不好”拆成可执行的测量协议，避免把单个分数误读为能力本身。

5.1 为什么要讨论 Benchmark 设计与污染

benchmark 要明确定义任务、数据、prompt、few-shot examples、scoring rule 和 aggregation。contamination 是评估大模型时的核心风险：测试集可能进入 pretraining 或 post-training 数据。评估应记录 exact prompt、decoding parameters、model checkpoint 和版本，保证可复现。Benchmark 设计与污染把 Evaluation 方法论连接到测量协议。接下来要看的不是分数本身，而是题目、prompt、解析规则和误差分析怎样支撑这个分数。

5.2 Benchmark 设计与污染的机制

benchmark 要明确定义任务、数据、prompt、few-shot examples、scoring rule 和 aggregation。在 Benchmark 设计与污染中，核心对象是评估样本、答案解析和评分状态：分数只有在题目、提示、解析规则和采样协议都明确时才有解释力。contamination 是评估大模型时的核心风险：测试集可能进入 pretraining 或 post-training 数据。因而同一个方法在不同规模下可能改变不同的变量，尤其是 prompt 模板、采样参数、答案解析规则、置信区间和污染检查。比较方法时，必须同时给出问题规模和实验条件。评估应记录 exact prompt、decoding parameters、model checkpoint 和版本，保证可复现。可检验性来自固定协议、置信区间、错误分类和污染排查；分数的含义取决于这些测量条件。因此，Benchmark 设计与污染的学习目标是建立一套证据规则：读者应能说明某个分数测量了什么、没有测量什么，以及哪些设置会改变结论。

5.3 Benchmark 设计与污染的数学表达

$$\hat{s} = \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{f_{\theta}(q_i) = a_i\}$$

符号说明： $\hat{s}$ 是平均准确率； q_i, a_i 是第 i 个样本的问题和答案。这个关系式给出 Benchmark 设计与污染的测量框架。分数只有在 prompt、答案解析、采样和数据版本固定时才具有可比性。在评估实验中，accuracy、pass@k、win rate 和 rubric score 都是 proxy。只有题目版本、prompt、decoding 和解析规则固定时，分数才具有比较意义。

5.4 从 Benchmark 设计与污染到程序

把 Benchmark 设计与污染写成评估流程时，要先固定协议，再运行模型，最后解析答案和汇总分数。下面的伪代码强调这个顺序。

```
for item in benchmark:
    prompt = render_prompt(template, item)
    pred = model.generate(prompt, decoding)
    score += grader(pred, item.answer)
```

阅读这段评估流程时，重点检查协议是否在模型运行前固定。prompt 模板、采样参数、答案解析规则、置信区间和污染检查决定分数能否比较，也决定错误分析能否复现。

5.5 边界条件与常见误解

- 自动 grader 可能奖励格式匹配而非真实能力。
- 测试集泄漏会让 score 失去泛化意义。
- benchmark 分数会受 prompt、decoding、答案解析和数据版本影响，跨论文比较必须先核对协议。
- 单个平均分会掩盖错误类型；没有错误分析时，很难判断失败来自知识、推理、格式还是污染。

5.6 与本章其他概念的关系

Benchmark 设计与污染连接本章的能力声明和证据标准：定义限定测量对象，公式给出汇总方式，伪代码固定协议，例题展示分数如何被解释和误解释。

5.7 怎样检验 Benchmark 设计与污染

检验 Benchmark 设计与污染时，先把抽象说法落到评估样本、答案解析和评分状态上。与 benchmark、prompt、grader、contamination、reproducible 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让 prompt 模板、采样参数、答案解析规则、置信区间和污染检查中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： $\hat{s}$ 是平均准确率； (q_i, a_i) 是第 i 个样本的问题和答案。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。评估消融通常一次只改变 prompt 模板、decoding 参数、答案解析或数据版本。若协议整体改变，分数差异不能被解释为模型能力差异。最后检查失败条件。工程判断往往在反例中变清楚：自动 grader 可能奖励格式匹配而非真实能力；测试集泄漏会让 score 失去泛化意义。复习时可以把这些限制改写成反例测试，观察它们怎样改变 prompt 模板、采样参数、答案解析规则、置信区间和污染检查。把实验结论放回 Evaluation 方法论时，要说明 Benchmark 设计与污染改变的是测量协议、答案解析、统计汇总还是错误分布。能解释这些环节，才说明读者真正理解评估。

5.8 概念辨析：Benchmark 设计与污染

benchmark、prompt、grader、contamination、reproducible 常一起出现在评估协议中，但它们分别指题目、模型输出、解析规则或汇总统计。读者应先确认分数由哪一步产生。区分这些词时，可以先问它们是否改变 prompt 模板、采样参数、答案解析规则、置信区间和污染检查。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕 Benchmark 设计与污染复习时，可以固定一个小 benchmark：写下 prompt、decoding、解析规则和错误分类，再解释分数变化。

5.9 Benchmark 设计与污染的小结

- 讨论 Benchmark 设计与污染时，应同时报告问题规模、实验设置和主要观测变量，尤其是 prompt 模板、采样参数、答案解析规则、置信区间和污染检查。
- 如果方法声称分数更高，要同时固定 prompt、decoding、答案解析、数据版本和置信区间；否则比较对象并不相同。
- 评估结论需要错误分析支撑，尤其要区分真实能力、格式适配和数据污染。

6 从离线评估到 deployment 指标

现在进入从离线评估到 deployment 指标。这一节把算法放到硬件和运行时中考察：同一个数学表达式在不同 batch、shape、GPU 拓扑和 kernel 实现下会得到不同的成本账本。

6.1 为什么要讨论从离线评估到 deployment 指标

真实模型上线需要结合 latency、cost、toxicity、helpfulness、truthfulness、robustness 和 refusal behavior。人工评估和 pairwise preference 能捕捉开放式输出质量，但成本高、方差大、标准易漂移。课程后续 post-training 会把 evaluation 与 reward modeling/RL 联系起来。从离线评估到 deployment 指标把 Evaluation 方法论放进资源账本。接下来要看的不是单个 API，而是张量形状、内存层级、通信和调度怎样共同决定成本。

6.2 从离线评估到 deployment 指标的机制

真实模型上线需要结合 latency、cost、toxicity、helpfulness、truthfulness、robustness 和 refusal behavior。在从离线评估到 deployment 指标中，核心对象是硬件和系统状态：同一算法会因为 memory hierarchy、通信拓扑或 kernel shape 不同而表现出完全不同的速度。人工评估和 pairwise preference 能捕捉开放式输出质量，但成本高、方差大、标准易漂移。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。课程后续 post-training 会把 evaluation 与 reward modeling/RL 联系起来。可检验性来自 profile trace、roofline 估算和端到端延迟分解；如果这些证据互相矛盾，就要先解释 measurement setup。因此，从离线评估到 deployment 指标的学习目标是建立一张成本地图：读者应能指出瓶颈在计算、内存、通信还是调度，并能设计一个小 benchmark 去验证。

6.3 从离线评估到 deployment 指标的数学表达

$$\Pr(A \succ B) = \sigma(r_\phi(x, y_A) - r_\phi(x, y_B))$$

符号说明：preference model 用 reward 差预测人类更偏好哪个回答； σ 是 sigmoid。这个关系式应被读成从离线评估到 deployment 指标的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。在系统实验中，tokens/s、TTFT、显存峰值和 kernel time 都是 proxy。它们各自只观察一部分系统，因此报告时要同时写清 workload shape、warmup、同步方式和硬件型号。

6.4 从离线评估到 deployment 指标到程序

把从离线评估到 deployment 指标写成程序时，最重要的是暴露状态变化和数据移动。下面的伪代码保留执行顺序，省略框架样板。

```
collect_pairwise_preferences(outputs)
fit_reward_model(preferences)
validate_against_heldout_human_judgments()
```

阅读这段实现时，重点看从离线评估到 deployment 指标怎样移动张量、占用显存并触发通信。实验记录至少要包含吞吐、延迟、显存峰值、通信量和 kernel 利用率，否则很难判断瓶颈来自算子、内存层级还是调度。

6.5 边界条件与常见误解

- 人工偏好不是绝对真理，会受标注指南和人群影响。
- reward model 可能被优化过程 exploited。
- 小规模 kernel benchmark 不一定预测端到端训练或 serving latency，因为调度、通信和请求形状会改变瓶颈。
- 硬件规格表和框架 profiler 都需要结合 workload 解释，不能把单个峰值数字当作机制证明。

6.6 与本章其他概念的关系

从离线评估到 deployment 指标连接本章的数学机制和工程成本：术语表说明对象，公式估算资源，伪代码暴露状态变化，例题则让读者检查一次完整的成本推理。

6.7 怎样检验从离线评估到 deployment 指标

检验从离线评估到 deployment 指标时，先把抽象说法落到硬件和系统状态上。与 human evaluation、preference、reward、deployment、safety 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明：preference model 用 reward 差预测人类更偏好哪个回答； σ 是 sigmoid。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。系统消融通常一次只改变 batch shape、sequence length、GPU 数、kernel 实现或调度策略。若同时改变硬件和 workload，实验只能说明端到端表现不同，不能定位瓶颈。最后检查失败条件。工程判断往往在反例中变清楚：人工偏好不是绝对真理，会受标注指南和人群影响；reward model 可能被优化过程 exploited。复习时可以把这些限制改写成反例测试，观察它们怎样改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。把实验结论放回 Evaluation 方法论时，要说明从离线评估到 deployment 指标改变的是训练吞吐、推理延迟、显存峰值还是通信开销。能做出这种归因，才说明读者已经从接口使用进入系统解释。

6.8 概念辨析：从离线评估到 deployment 指标

human evaluation、preference、reward、deployment、safety 常一起出现在系统讲解中，但它们分别指硬件对象、程序操作、瓶颈来源或测量指标。读者应先定位每个词在执行路径中的位置。其中，evaluation（评估）把 loss、benchmark、人类偏好和部署指标区分开。区分这些词时，可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕从离线评估到 deployment 指标复习时，可以写一个小 benchmark：固定输入形状，改变一个系统变量，记录 profile trace 和端到端时间。

6.9 从离线评估到 deployment 指标的小结

- 讨论从离线评估到 deployment 指标时，应同时报告问题规模、实验设置和主要观测变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。
- 如果方法声称更快，要区分计算减少、内存访问减少、通信隐藏和调度改善；这些机制不能用一个 tokens/s 数字互相替代。
- profile trace、roofline 估算和端到端 benchmark 应互相校验，单独一个指标不足以解释系统瓶颈。

7 综合讨论：从局部机制到完整系统

本章的主题是 Evaluation 方法论，但它不应被读成几个相邻知识点的拼接。评估在 LM 生命周期中的位置、Benchmark 设计与污染、从离线评估到 deployment 指标共同回答同一个问题：语言模型系统中哪些对象可以被定义，哪些成本可以被估算，哪些结论必须通过实验或评估来确认。这些对象包括 evaluation（评估）、perplexity（困惑度）、benchmark、contamination（污染）、calibration（校准）、human evaluation（人工评估）。读者可以把它们看成一组接口：有的接口连接文本和 token，有的连接模型结构和张量计算，有的连接数据样本和训练目标，有的连接离线评估和在线服务。接口一旦改变，后面的成本、错误类型和优化空间也会随之改变。

7.1 评估在 LM 生命周期中的位置在系统中的位置

评估在 LM 生命周期中的位置首先提供一个局部解释：视频指出课程至此已覆盖 architecture、optimizer、training loop、systems、scaling 和 inference，剩下的关键是如何判断模型好坏。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，评估在 LM 生命周期中的位置会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，评估在 LM 生命周期中的位置不能脱离边界条件理解。一个关键 caveat 是：perplexity 不直接等于 instruction-following 能力。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.2 Benchmark 设计与污染在系统中的位置

Benchmark 设计与污染首先提供一个局部解释：benchmark 要明确定义任务、数据、prompt、few-shot examples、scoring rule 和 aggregation。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，Benchmark 设计与污染会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，Benchmark 设计与污染不能脱离边界条件理解。一个关键 caveat 是：自动 grader 可能奖励格式匹配而非真实能力。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.3 从离线评估到 deployment 指标在系统中的位置

从离线评估到 deployment 指标首先提供一个局部解释：真实模型上线需要结合 latency、cost、toxicity、helpfulness、truthfulness、robustness 和 refusal behavior。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，从离线评估到 deployment 指标会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，从离线评估到 deployment 指标不能脱离边界条件理解。一个关键 caveat 是：人工偏好不是绝对真理，会受标注指南和人群影响。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.4 把本章知识用于读论文和复现实验

读相关论文时，可以把本章变成一个检查表。第一，论文是否清楚定义了输入规模、模型规模、数据来源和评估协议；第二，论文声称的改进落在哪个变量上；第三，作者是否报告了会让结论失效的设置；第四，实验是否能分离模型结构、数据质量、优化设置和系统实现的影响。复现实验时，也应避免直接追求大规模。先用最小程序复现一个公式、一个伪代码分支或一个 profile 现象，再扩大输入规模。这样做的原因很简单：如果小规模程序无法解释中间量，大规模训练只会把错误隐藏在日志和随机波动里。把这些检查合在一起，本章给出的不是一个固定答案，而是一种阅读 CS336 的方法：每个模型机制都要落到变量，每个变量都要落到测量，每个测量都要说明协议，每个协议都要承认边界。因此，本章中的任何一句结论都可以被改写成一个可引用的完整句式：在给定数据、模型、硬件和评估协议下，某个机制改变了某个变量，并通过某个观测指标表现出来。这样的句子比单独背诵术语更长，但它包含了自学、复现和引用时真正需要的信息。把本章用于自学时，

可以按三遍阅读。第一遍只追踪对象和定义，确认每个术语到底指向文本、token、张量、样本、请求还是评估记录。第二遍追踪公式和伪代码，确认每一步改变了哪些状态。第三遍追踪实验和 caveat，确认哪些结论只在特定规模、数据或硬件条件下成立。把本章用于复习时，则应反过来操作：先从一个失败案例或异常指标出发，再回到相应机制。loss 曲线异常可能指向数据、优化或模型容量；吞吐异常可能指向 kernel、通信或调度；benchmark 异常可能指向 contamination、prompt protocol 或采样设置。能够从异常反推机制，是掌握本章的实用标准。

8 例题：把概念变成可计算检查

8.1 同一 benchmark 的可复现评分

评估一个 multiple-choice benchmark。

- 固定 prompt template、few-shot examples、答案解析方式和 decoding 参数。
- 记录 tokenizer、checkpoint、commit hash 和数据版本。
- 报告平均分时给出方差或置信区间，并检查 contamination。

结论。 benchmark 分数只有在评估协议完整时才可引用。这个例题的目的不是替代完整工程实现，而是给出一种最小推理形式：把问题转成变量，写出近似公式或伪代码，再说明近似何时失效。

9 实验设计：从小例子到可复现结论

学习 Evaluation 方法论不能停在概念层。一个教材化结论至少需要四件事：可构造的输入、可观测的中间量、可比较的指标，以及清楚的失败条件。下面的实验设计不是要求训练大模型，而是要求读者用小规模程序复现本章机制。

9.1 最小输入

先构造只触发本章核心机制的小输入。例如 tokenizer 章节可以用几个包含 rare words 和 Unicode 字符的字符串；GPU 章节可以用一个 elementwise kernel 和一个 GEMM；data 章节可以用十几篇近重复文档。小输入的价值在于它让错误可见。

9.2 资源账本

对同一输入写下 FLOPs、bytes moved、显存峰值、通信量或样本数量的估算。估算不需要一开始就精确，但必须说明单位和近似假设。随后用 profiler、benchmark、validation loss 或人工检查去验证估算是否同量级。

9.3 单变量消融

一次只改变一个因素，例如 vocabulary size、head 数、batch size、filter threshold、reward scale、decoding temperature 或 GPU 数量。若多个变量同时变化，实验只能说明系统变了，不能说明机制是什么。

9.4 失败条件

最后要刻意触发失败：极端 context length、错误 dtype、过小 batch、污染 benchmark、低质量数据或不可靠 verifier。失败条件常常比成功曲线更能说明一个方法的边界。

9.5 记录与报告

实验记录应能让另一个读者复现同一结论。最少要写清楚输入构造、代码版本、随机种子、硬件、batch shape、数据版本、评价指标和异常处理方式。报告结论时不要只写“更快”或“更好”，而要说明快在哪里、好在哪个指标上，以及是否牺牲了内存、稳定性、数据质量或可解释性。常见报告错误是把局部现象写成普遍规律。例如一次小模型实验里的 loss 改善，不等于更大模型或不同数据分布上也会改善；一次 kernel benchmark 的吞吐提升，不等于端到端 serving latency 一定降低。教材中的实验报告应始终把结论限定在可复现条件内。

9.6 从小实验到大结论

小实验的作用不是替代课程中的完整训练或系统实现，而是建立一条可检查的推理链。先在小输入上确认变量方向，再在中等规模上确认数量级，最后才讨论是否能外推到更大模型、更多 token 或更复杂 workload。缺少中间层级时，大结论通常只是在复述直觉。因此，实验报告最好同时包含正例和反例：正例说明机制在受控条件下怎样工作，反例说明条件稍变时哪里失效。对 CS336 这样的课程，反例尤其重要，因为 language model 的错误常来自多个层面叠加：数据分布、优化稳定性、硬件瓶颈、评估协议和服务负载都可能同时改变观察结果。还要记录资料版本与复现边界。公开视频、课程页、配套代码和 PDF 可能在不同时间更新；复现实验应写明使用的材料版本，并说明哪些结论来自课程事实，哪些是为了帮助理解而加入的解释性推导。这样才能把学习笔记变成可检查的技术记录。若复现实验与课程结论不一致，首先检查输入、版本和评估协议，而不是立即修改模型假设。

9.7 把本章重新读成一条证据链

完成小实验之后，可以把 Evaluation 方法论重新读成一条证据链：先用“评估在 LM 生命周期中的位置”确定问题入口，再用“Benchmark 设计与污染”解释主要机制，最后用“从离线评估到 deployment 指标”检查边界或外推条件。这种读法比按页背诵更接近教材训练，因为它要求每个结论都能说明前提、变量和证据。本章反复使用的术语包括 evaluation（评估）、perplexity（困惑度）、benchmark、contamination（污染）、calibration（校准）、human evaluation（人工评估）。复习时不要只写定义，而要为每个术语补上一句操作性说明：它在课程代码、公式、数据记录或评估协议中对应什么对象；如果这个对象被删除、替换或缩放，哪一个观测量会变化。最后，用一个反例结束复习。反例可以是资源估算不准、数据污染、reward 失真、benchmark 解析失败、长上下文显存爆炸或 scaling 外推失效。能解释反例的章节，才真正具备自学和引用价值；不能解释反例的章节，只是把课程材料换了一种排版。引用本章结论时，还应保留来源边界：哪些说法直接来自视频、课程页、slides 或代码，哪些是为了串联教材叙述而加入的解释性推导。这个边界不会削弱结论，反而让读者知道何处可以复核，何处需要在自己的实验中重新验证。如果后续课程材料更新，也可以沿着同一证据链替换来源，而不必重写整章结构。

10 课程资料与回看路径

教材正文已经把主要概念、公式和实现逻辑组织成连续叙述；本节只提供回到课程材料的阅读路径。读者复习时可以先完成前文练习，再按下面的时间段或资料组核对细节。

10.1 视频回看路径

时间	关键词	复习任务
00:00:05-00:06:05	good, evaluation, going, people, data, maybe, different	回看定义、例子、推导或 caveat 在该段中的位置。
00:18:38-00:25:01	then, language, think, it's, question, benchmark, question.	回看定义、例子、推导或 caveat 在该段中的位置。
00:37:31-00:44:21	alpacaeval, evaluate, there's, don't, metric, chatbot, arena	回看定义、例子、推导或 caveat 在该段中的位置。
00:51:41-00:58:20	agent, it's, language, then, reasoning, this., context	回看定义、例子、推导或 caveat 在该段中的位置。
01:11:54-01:18:10	there's, evaluation, think, very, benchmarks, always, cases	回看定义、例子、推导或 caveat 在该段中的位置。

这些时间段用于定位视频中的讲解顺序。严肃引用时，应回到视频片段确认讲者表述、板书或幻灯片内容。回看视频时，不必逐字抄录字幕。更有效的方法是把每个时间段判定为定义、例子、推导、代码解释或 caveat，再把它放回本章对应小节。

10.2 课程资料阅读路径

资料组	标题/页块	关键词
official_01	Lecture 12: evaluation	evaluation, good, maybe, training, data, shapes, behavior
official_03	- For downstream tasks, 'response = LM(prompt)' and compute accuracy on 'response'	questions, accuracy, language, response, summary, still, benchmarks
official_05	- Problem: LLM judges favor longer responses, resulted in leaderboard gaming	tasks, evaluate, responses, judge, judges, metric, chatbot
official_08	Route 3: use fresh evals	evaluating, route, evals, aren, either, perplexity, evaluation
official_10	- Fixed up SWE-Bench to produce SWE-Bench Verified	function, swe-bench, benchmarks, problems, agent, fixed, produce

课程资料通常给出更稳定的标题、公式、图或代码结构；视频则补充动机和口头解释。两者合起来构成本章的事实边界。阅读资料时，可以把每个资料组拆成三个对象：一个定义，一个公式或伪代码动作，一个边界条件。这样比逐字翻译页面或脚本更容易形成可用知识。

11 实践说明、历史位置与风险

Evaluation 方法论在 CS336 的课程结构中连接基础机制和规模化问题。基础机制解释方法为什么成立；规模化问题检验它在更大模型、更长上下文、更复杂数据或真实 serving workload 下是否仍成立。历史位置不等于模型年表。更重要的是识别问题如何迁移：早期方法解决小规模建模问题，现代方法常常解决同一问题在大规模数据、GPU 集群、长上下文或人类偏好约束下的新形态。

- 资料版本限制 (version risk): 课程页、公开视频和配套资料可能在学期中更新; 引用时应注明访问日期或资料版本。
- 测量口径限制 (measurement risk): FLOPs、tokens/s、benchmark score、reward 等数字只有在协议完整时可比较。
- 规模外推限制 (scaling risk): 小模型或小 batch 的机制不必然外推到 frontier-scale。
- 数据分布限制 (data risk): 数据来源、过滤和去重策略会改变模型行为, 也会改变 evaluation contamination 风险。

12 复习要点

下面的问题把本章内容压缩为可反复检查的条目。每个条目都应能回到前文的解释、公式、伪代码或例题。

条目	对象	检查动作
条目 1	evaluation (评估)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 2	perplexity (困惑度)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 3	benchmark	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 4	contamination (污染)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 5	calibration (校准)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 6	human evaluation (人工评估)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。

12.1 综合自测

- 我能否不用课程原句, 独立解释本章中心问题?
- 我能否把本章至少一个公式改写成可运行的估算函数?
- 我能否指出一个看似改进但实际转移成本的方法?
- 我能否回到视频和课程资料, 找到支持本章关键论断的位置?
- 我能否设计一个最小 ablation, 验证本章一个 caveat?

13 总结与延伸

13.1 本章小结

- Evaluation 方法论的核心不是记忆术语，而是理解对象、变量和机制之间的关系。
- 视频给出讲解顺序，课程资料提供可核对的公式、图、代码或页面结构。
- 复习时应把每个术语连接到至少一个变量、一个实现动作和一个失败模式。

13.2 延伸解释

本章中的延伸解释用于连接课程材料中的概念，例如说明公式里的 proxy、解释系统 tradeoff，或把多个材料片段串成一个完整推理链。若需要严肃引用，应回到课程视频和配套资料核对原始表述。

13.3 练习

- 定义题：用中英双语解释本章任意五个重要术语，并说明它们属于 compute、memory、data、optimization、evaluation 还是 deployment 账本。
- 推导题：选择本章一个公式，逐个解释符号、单位和近似假设，并说明哪个变量最难在真实系统中测量。
- 实现题：把本章伪代码改写成一个最小 Python sanity check，不要求训练大模型，但要输出可检查的中间量。
- 资料题：从“课程资料与回看路径”中选择一个视频时间段，回到原始视频核对讲者例子，并记录是否存在字幕误差。
- 评估题：为本章方法设计一个 ablation 或 benchmark，说明控制变量、数据版本、指标和 failure criteria。
- 边界题：说明本章一个结论在更长 context、更大 batch、更多 GPU 或更复杂数据分布下可能如何失效。
- 综合题：把本章内容和前一章或后一章连接起来，写出一个端到端训练/推理 pipeline 中的依赖关系。
- 批判题：指出一个容易被营销材料夸大的说法，并用本章的资源账本或评估协议反驳它。

13 13 预训练数据：来源与数据集

第 13 讲：预训练数据：来源与数据集

CS336 Spring 2026 public videos and official course materials

目录

1 本章导读	2
1.1 本章知识路线	2
1.2 结构图	2
2 本章主线	2
2.1 本章要解决的核心问题	2
2.2 从机制到资源账本	3
2.3 从课堂讲解到可复现实验	3
3 术语、符号与问题边界	4
3.1 术语之间的关系	4
4 为什么数据最重要	4
4.1 课程目标与问题边界	4
4.2 为什么数据最重要的机制	5
4.3 资源、效率与效果的关系	5
4.4 把课程目标写成检查流程	5
4.5 边界条件与常见误解	5
4.6 与本章其他概念的关系	5
4.7 怎样检验这一课程目标	6
4.8 概念辨析：为什么数据最重要	6
4.9 为什么数据最重要的小结	6
5 Web、code、books 与 curated datasets	6
5.1 为什么要讨论 Web、code、books 与 curated datasets	6
5.2 Web、code、books 与 curated datasets 的机制	7
5.3 Web、code、books 与 curated datasets 的数据分布视角	7
5.4 Web、code、books 与 curated datasets 在数据管线中的实现	7
5.5 边界条件与常见误解	7

5.6	与本章其他概念的关系	7
5.7	怎样检验 Web、code、books 与 curated datasets	8
5.8	概念辨析: Web、code、books 与 curated datasets	8
5.9	Web、code、books 与 curated datasets 的小结	8
6	数据集记录与可复现性	8
6.1	为什么要讨论数据集记录与可复现性	8
6.2	数据集记录与可复现性的机制	9
6.3	数据集记录与可复现性的数学表达	9
6.4	从数据集记录与可复现性到程序	9
6.5	边界条件与常见误解	9
6.6	与本章其他概念的关系	9
6.7	怎样检验数据集记录与可复现性	10
6.8	概念辨析: 数据集记录与可复现性	10
6.9	数据集记录与可复现性的小结	10
7	综合讨论: 从局部机制到完整系统	10
7.1	为什么数据最重要在系统中的位置	10
7.2	Web、code、books 与 curated datasets 在系统中的位置	11
7.3	数据集记录与可复现性在系统中的位置	11
7.4	把本章知识用于读论文和复现实验	11
8	例题: 把概念变成可计算检查	12
8.1	从网页到训练样本的 provenance	12
9	实验设计: 从小例子到可复现结论	12
9.1	最小输入	12
9.2	资源账本	12
9.3	单变量消融	12
9.4	失败条件	12
9.5	记录与报告	13
9.6	从小实验到大结论	13
9.7	把本章重新读成一条证据链	13
10	课程资料与回看路径	13
10.1	视频回看路径	13
10.2	课程资料阅读路径	14
11	实践说明、历史位置与风险	14

12 复习要点

15

12.1 综合自测

15

13 总结与延伸

15

13.1 本章小结

15

13.2 延伸解释

16

13.3 练习

16

1 本章导读

本章讨论 **预训练数据：来源与数据集**。CS336 的第一层目标不是把现成大模型当作黑箱使用，而是把 language model 从输入文本、训练目标、模型结构、数据、系统和评估这些部件重新拆开。只有拆开之后，读者才能判断一个设计选择究竟改变了什么。设想有人说模型训练在“互联网上”。教材式回答必须拆开这句话：哪些 crawl、dump、licensed corpora、code、books、papers 和 curated datasets 被采集？谁允许使用？经过了哪些过滤？本章对应 CS336 Spring 2026 第 13 个公开视频，并结合课程页提供的可解析资料。视频和课程页给出事实边界；正文把这些材料改写为可自学的教材章节。阅读本章时，先理解问题为什么存在，再看定义、公式和实现形态，随后通过例题和练习检查自己是否能独立复现这条 reasoning chain。

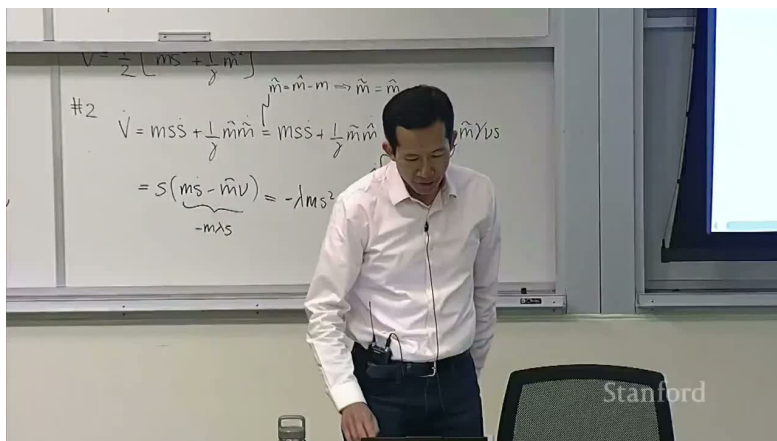


图 1: 第 13 讲的课程图像锚点。

1.1 本章知识路线

- 本章首先说明预训练数据：来源与数据集为什么是 language modeling pipeline 中不可跳过的一环。
- 其次定义本章反复使用的术语，并说明这些术语对应的计算对象或系统对象。
- 随后用公式和伪代码表达主要机制，让概念能够落到可计算的变量上。
- 最后给出例题、风险讨论和练习，便于读者检查自己是否真正掌握了机制。

1.2 结构图

下面的图用于帮助读者定位本章的核心结构。先看概念之间的依赖，再回到正文中的公式、伪代码和例题。

2 本章主线

预训练数据：来源与数据集的主线是把“训练数据”拆成来源、过滤、去重、混合和审计。模型能力不是由参数决定，也由它反复看到的样本分布决定。

2.1 本章要解决的核心问题

本讲进入 data: 语言模型并不是训练在“整个互联网”上, 而是训练在经过 crawl、dump、extraction、filtering、dedup 和 mixture 的数据产物上。读完这一段后, 应能用一两句话说明预训练数据: 来源与数据集

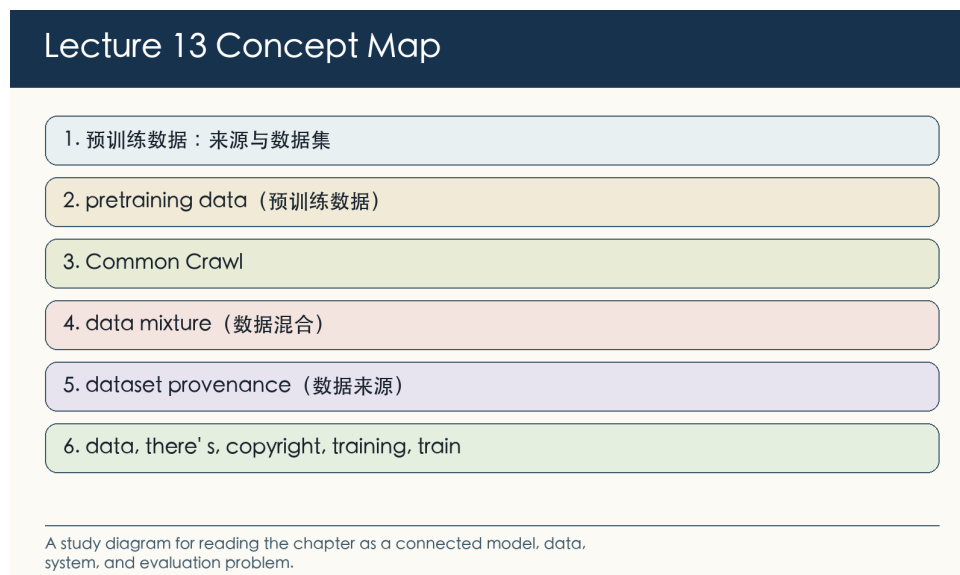


图 2: 第 13 讲概念图：本章问题、术语和机制的关系。

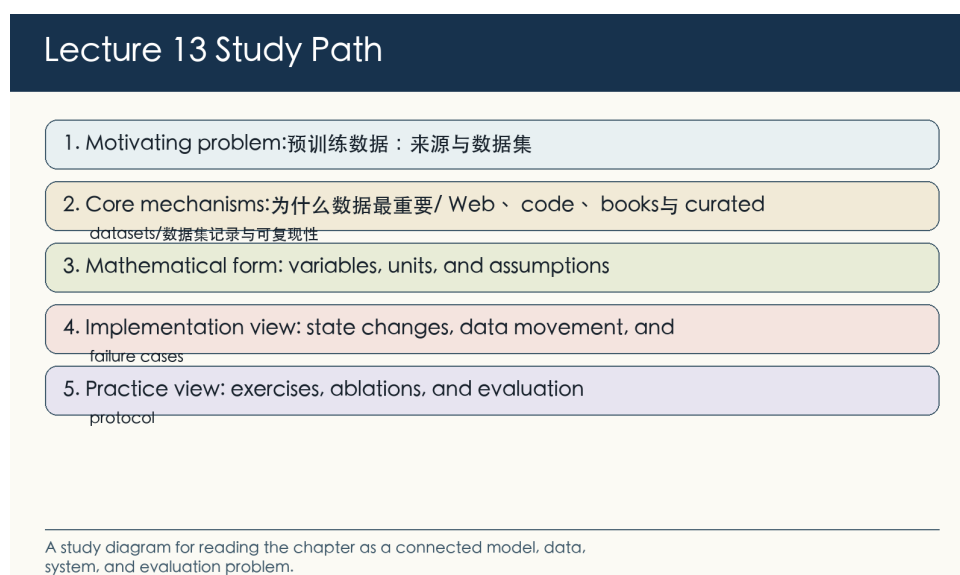


图 3: 第 13 讲学习路径：从问题定义进入公式、实现、实验和练习。

要解决的瓶颈，并指出这个瓶颈发生在数据、模型、优化、系统还是评估层。

2.2 从机制到资源账本

数据来源有技术、法律和伦理约束。robots.txt、ToS、paywall、copyright、privacy、PII、crawler behavior 都会影响可用数据。随后把机制写成账本：输入规模是什么，主要状态是什么，成本或质量指标怎样随变量改变。这个账本让后面的公式不再只是符号。

2.3 从课堂讲解到可复现实验

数据 pipeline 必须版本化。没有 provenance、filter version、dedup strategy 和 mixture weights，就无法复现实验或解释模型行为。最后把课堂讲解转成可复现实验：固定协议，改变一个变量，记录中间量，并解释结果为何支持或反驳原来的机制判断。

3 术语、符号与问题边界

本章的术语横跨模型、数据和系统。读者需要先知道每个词指向的对象：有些词描述数学目标，有些词描述张量形状，有些词描述硬件瓶颈，还有一些词描述评估协议。术语边界不清，后面的公式和实现就会变成空洞的缩写。

#	术语	阅读要求
1	pretraining data（预训练数据）	说明它如何改变训练样本或 token 分布，并给出一个会改变结论的数据边界条件。
2	Common Crawl	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
3	data mixture（数据混合）	说明它如何改变训练样本或 token 分布，并给出一个会改变结论的数据边界条件。
4	dataset provenance（数据来源）	说明它如何改变训练样本或 token 分布，并给出一个会改变结论的数据边界条件。
5	licensing（许可证）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。

这些术语之间有依赖关系。例如 tokenization 改变 sequence length，sequence length 又改变 attention FLOPs、KV cache 和 evaluation token budget；parallelism 改变显存占用，也改变通信瓶颈和故障恢复方式。

3.1 术语之间的关系

在本章中，pretraining data（预训练数据）给出问题入口，Common Crawl 描述主要机制，data mixture（数据混合）则把机制连接到可测量的结果。读教材时应当沿着这个链条追问：输入是什么，中间状态怎样改变，输出指标为什么随之变化。课程材料还会反复出现 data、e.g.、https、copyright、books、filtering 等词。它们不是一组同义标签，而是提示本章的不同层次：有的指对象，有的指操作，有的指约束或指标。因此，术语表不是背诵清单，而是后文阅读的索引。遇到一个术语时，先定位它属于 representation、optimization、systems、data 还是 evaluation，再判断它改变哪一项账本。

4 为什么数据最重要

本节讨论为什么数据最重要。它先限定预训练数据：来源与数据集要回答的基本问题：哪些现象需要解释，哪些变量可以测量，哪些结论必须回到样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险中检验。

4.1 课程目标与问题边界

视频开场说：给定数据后大家已知道如何训练，接下来问题是应该训练在什么数据上。数据决定模型看到的世界、语言、风格、知识和偏见，也决定 downstream 能力的上限。课程把互联网数据看作从 live services 到 dump/crawl 再到 processing pipeline 的产物。为什么数据最重要把预训练数据：来源与数据集连接到训练分布。接下来要看的不是语料名称，而是样本来源、处理顺序和版本记录怎样改变模型可学习的证据。

4.2 为什么数据最重要的机制

视频开场说：给定数据后大家已知道如何训练，接下来问题是应该训练在什么数据上。在为什么数据最重要中，核心对象是样本、来源和分布状态：数据管线每改变一次样本集合，模型实际学习的分布也随之改变。数据决定模型看到的世界、语言、风格、知识和偏见，也决定 downstream 能力的上限。因而同一个方法在不同规模下可能改变不同的变量，尤其是样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。比较方法时，必须同时给出问题规模和实验条件。课程把互联网数据看作从 live services 到 dump/crawl 再到 processing pipeline 的产物。可检验性来自 provenance 记录、过滤前后计数、抽样审计和 held-out contamination 检查；没有这些记录，数据结论无法复现。因此，为什么数据最重要的学习目标是建立一条数据谱系：读者应能从原始来源追到训练 token，并说明每个处理步骤改变了什么分布。

4.3 资源、效率与效果的关系

$$\mathcal{D}_{\text{train}} \sim \sum_{k=1}^K w_k \mathcal{D}_k$$

符号说明： $\mathcal{D}_k$ 是第 k 个数据源， w_k 是混合权重；预训练分布是人为设计的混合。这个关系式刻画为什么数据最重要对数据分布的影响。读者应关心哪些变量来自采集过程，哪些变量来自过滤或混合策略，哪些变量只能通过抽样审计估计。在数据实验中，去重率、过滤通过率、语言比例和抽样人工判读都是 proxy。它们不能单独代表数据质量，必须和来源记录、版本号以及污染检查一起解释。

4.4 把课程目标写成检查流程

把为什么数据最重要写成程序时，要明确每一步怎样改写样本集合。下面的伪代码强调输入、过滤、统计和输出之间的关系。

```
sources = [web, books, code, wikipedia, papers]
for source in sources:
    collect_with_provenance(source)
    normalize_and_filter(source)
```

阅读这段流程时，重点看为什么数据最重要怎样改变样本集合。每个过滤、去重或混合步骤都应留下计数和抽样检查，使样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险能够被复核。

4.5 边界条件与常见误解

- 数据不是从天上掉下来；每个来源都有采集、许可和偏差问题。
- 更多 token 不等于更好，低质量 token 会稀释训练预算。
- 小样本抽样只能发现部分数据问题，不能证明语料整体无污染或无偏差。
- 数据版本、许可状态和处理脚本如果缺失，后续 evaluation 与 scaling 结论都会失去可复现基础。

4.6 与本章其他概念的关系

为什么数据最重要连接本章的数据来源和模型行为：定义说明样本如何进入语料，公式刻画分布变化，伪代码展示处理顺序，例题检验数据决策怎样影响训练结论。

4.7 怎样检验这一课程目标

检验这一课程目标时，先把抽象说法落到样本、来源和分布状态上。与 data、Common Crawl、source、dataset、provenance 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： $\mathcal{D}_k$ 是第 k 个数据源， w_k 是混合权重；预训练分布是人为设计的混合。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。数据消融通常一次只改变过滤阈值、去重策略、混合权重或数据版本。若同时更换 tokenizer、语料和评估集，就无法知道改进来自哪一步。最后检查失败条件。工程判断往往在反例中变清楚：数据不是从天上掉下来；每个来源都有采集、许可和偏差问题；更多 token 不等于更好，低质量 token 会稀释训练预算。复习时可以把这些限制改写成反例测试，观察它们怎样改变样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。把实验结论放回预训练数据：来源与数据集时，要说明为什么数据最重要改变的是样本分布、数据质量、污染风险还是可复现性。能追到这些来源，才说明读者真正理解数据机制。

4.8 概念辨析：为什么数据最重要

data、Common Crawl、source、dataset、provenance 常一起出现在数据管线中，但它们分别指来源、处理动作、版本记录或风险标记。读者应先判断它们改变的是样本集合还是审计证据。其中，data（数据）是模型能力、偏差和法律风险的主要来源；data（数据）是模型能力、偏差和法律风险的主要来源。区分这些词时，可以先问它们是否改变样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕为什么数据最重要复习时，可以构造一小批文档：保留来源和版本，运行一个处理步骤，再比较样本计数和抽样质量。

4.9 为什么数据最重要的小结

- 讨论为什么数据最重要时，应同时报告问题规模、实验设置和主要观测变量，尤其是样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。
- 如果方法声称数据更好，要说明过滤、去重、混合或合成步骤改变了哪一部分分布，并检查 benchmark contamination。
- 数据结论需要抽样审计和版本记录；没有 provenance 的语料变化不能支持可引用结论。

5 Web、code、books 与 curated datasets

现在进入 Web、code、books 与 curated datasets。这一节关心训练样本如何形成、被筛选并进入混合分布；它直接决定模型看到什么，也决定 evaluation 是否可能被污染。

5.1 为什么要讨论 Web、code、books 与 curated datasets

Common Crawl 提供大规模网页原料，但需要 HTML extraction、language ID、质量过滤和去重。code 数据对编程能力关键，但有许可证、重复、生成数据污染和安全风险。books、Wikipedia、论文和问答数据更 curated，但规模、许可和代表性不同。数据部分的核心问题是分布控制。采集、过滤、去重和混合并不会只改变文件大小，它们会改变模型看到的语言、知识、偏差和 benchmark contamination 风险。

5.2 Web、code、books 与 curated datasets 的机制

Common Crawl 提供大规模网页原料，但需要 HTML extraction、language ID、质量过滤和去重。在 Web、code、books 与 curated datasets 中，核心对象是样本、来源和分布状态：数据管线每改变一次样本集合，模型实际学习的分布也随之改变。code 数据对编程能力关键，但有许可证、重复、生成数据污染和安全风险。因而同一个方法在不同规模下可能改变不同的变量，尤其是样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。比较方法时，必须同时给出问题规模和实验条件。books、Wikipedia、论文和问答数据更 curated，但规模、许可和代表性不同。可检验性来自 provenance 记录、过滤前后计数、抽样审计和 held-out contamination 检查；没有这些记录，数据结论无法复现。因此，Web、code、books 与 curated datasets 的学习目标是建立一条数据谱系：读者应能从原始来源追到训练 token，并说明每个处理步骤改变了什么分布。

5.3 Web、code、books 与 curated datasets 的数据分布视角

$$\text{value/source} = \frac{\Delta \text{eval score}}{\text{tokens} \times \text{processing cost}}$$

符号说明：这个比值是延伸解释，用来表达单位 token 和单位处理成本的边际价值，不是官方唯一指标。这个关系式刻画 Web、code、books 与 curated datasets 对数据分布的影响。读者应关心哪些变量来自采集过程，哪些变量来自过滤或混合策略，哪些变量只能通过抽样审计估计。在数据实验中，去重率、过滤通过率、语言比例和抽样人工判读都是 proxy。它们不能单独代表数据质量，必须和来源记录、版本号以及污染检查一起解释。

5.4 Web、code、books 与 curated datasets 在数据管线中的实现

把 Web、code、books 与 curated datasets 写成程序时，要明确每一步怎样改写样本集合。下面的伪代码强调输入、过滤、统计和输出之间的关系。

```
for document in crawl:
    text = extract_main_content(document.html)
    if language_ok(text) and quality_ok(text):
        write_jsonl(text, metadata=document.metadata)
```

阅读这段流程时，重点看 Web、code、books 与 curated datasets 怎样改变样本集合。每个过滤、去重或混合步骤都应留下计数和抽样检查，使样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险能够被复核。

5.5 边界条件与常见误解

- 网页抽取错误会把导航栏、广告、脚本或重复模板带入训练。
- 不同来源的 license 和 consent 不能在技术 pipeline 中被忽略。
- 小样本抽样只能发现部分数据问题，不能证明语料整体无污染或无偏差。
- 数据版本、许可状态和处理脚本如果缺失，后续 evaluation 与 scaling 结论都会失去可复现基础。

5.6 与本章其他概念的关系

Web、code、books 与 curated datasets 连接本章的数据来源和模型行为：定义说明样本如何进入语料，公式刻画分布变化，伪代码展示处理顺序，例题检验数据决策怎样影响训练结论。

5.7 怎样检验 Web、code、books 与 curated datasets

检验 Web、code、books 与 curated datasets 时，先把抽象说法落到样本、来源和分布状态上。与 web、code、books、Wikipedia、license 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明：这个比值是延伸解释，用来表达单位 token 和单位处理成本的边际价值，不是官方唯一指标。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。数据消融通常一次只改变过滤阈值、去重策略、混合权重或数据版本。若同时更换 tokenizer、语料和评估集，就无法知道改进来自哪一步。最后检查失败条件。工程判断往往在反例中变清楚：网页抽取错误会把导航栏、广告、脚本或重复模板带入训练；不同来源的 license 和 consent 不能在技术 pipeline 中被忽略。复习时可以把这些限制改写成反例测试，观察它们怎样改变样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。把实验结论放回预训练数据：来源与数据集时，要说明 Web、code、books 与 curated datasets 改变的是样本分布、数据质量、污染风险还是可复现性。能追到这些来源，才说明读者真正理解数据机制。

5.8 概念辨析：Web、code、books 与 curated datasets

web、code、books、Wikipedia、license 常一起出现在数据管线中，但它们分别指来源、处理动作、版本记录或风险标记。读者应先判断它们改变的是样本集合还是审计证据。其中，decode（解码）逐 token 生成，常受 memory bandwidth 和 KV cache 影响。区分这些词时，可以先问它们是否改变样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕 Web、code、books 与 curated datasets 复习时，可以构造一小批文档：保留来源和版本，运行一个处理步骤，再比较样本计数和抽样质量。

5.9 Web、code、books 与 curated datasets 的小结

- 讨论 Web、code、books 与 curated datasets 时，应同时报告问题规模、实验设置和主要观测变量，尤其是样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。
- 如果方法声称数据更好，要说明过滤、去重、混合或合成步骤改变了哪一部分分布，并检查 benchmark contamination。
- 数据结论需要抽样审计和版本记录；没有 provenance 的语料变化不能支持可引用结论。

6 数据集记录与可复现性

现在进入数据集记录与可复现性。这一节关心训练样本如何形成、被筛选并进入混合分布；它直接决定模型看到什么，也决定 evaluation 是否可能被污染。

6.1 为什么要讨论数据集记录与可复现性

课程强调开放模型与开放数据对信任和创新的作用；数据集 provenance 是可复现训练的基础。每个数据版本应记录来源 URL、抓取时间、过滤器版本、去重策略、tokenizer 和混合权重。数据集本身应当像模型 checkpoint 一样版本化，否则 evaluation 和 scaling 结果难以复现。数据集记录与可复现性把预训练数据：来源与数据集连接到训练分布。接下来要看的不是语料名称，而是样本来源、处理顺序和版本记录怎样改变模型可学习的证据。

6.2 数据集记录与可复现性的机制

课程强调开放模型与开放数据对信任和创新的作用；数据集 provenance 是可复现训练的基础。在数据集记录与可复现性中，核心对象是样本、来源和分布状态：数据管线每改变一次样本集合，模型实际学习的分布也随之改变。每个数据版本应记录来源 URL、抓取时间、过滤器版本、去重策略、tokenizer 和混合权重。因而同一个方法在不同规模下可能改变不同的变量，尤其是样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。比较方法时，必须同时给出问题规模和实验条件。数据集本身应当像模型 checkpoint 一样版本化，否则 evaluation 和 scaling 结果难以复现。可检验性来自 provenance 记录、过滤前后计数、抽样审计和 held-out contamination 检查；没有这些记录，数据结论无法复现。因此，数据集记录与可复现性的学习目标是建立一条数据谱系：读者应能从原始来源追到训练 token，并说明每个处理步骤改变了什么分布。

6.3 数据集记录与可复现性的数学表达

$$\text{dataset version} = f(\text{sources}, \text{filters}, \text{dedup}, \text{tokenizer}, \text{mixture})$$

符号说明：版本不是文件名，而是由完整 pipeline 决定的可审计对象。这个关系式刻画数据集记录与可复现性对数据分布的影响。读者应关心哪些变量来自采集过程，哪些变量来自过滤或混合策略，哪些变量只能通过抽样审计估计。在数据实验中，去重率、过滤通过率、语言比例和抽样人工判读都是 proxy。它们不能单独代表数据质量，必须和来源记录、版本号以及污染检查一起解释。

6.4 从数据集记录与可复现性到程序

把数据集记录与可复现性写成程序时，要明确每一步怎样改写样本集合。下面的伪代码强调输入、过滤、统计和输出之间的关系。

```
manifest = {
    'source': url,
    'crawl_time': timestamp,
    'filters': filter_hash,
    'tokenizer': tokenizer_hash,
}
```

阅读这段流程时，重点看数据集记录与可复现性怎样改变样本集合。每个过滤、去重或混合步骤都应留下计数和抽样检查，使样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险能够被复核。

6.5 边界条件与常见误解

- 只发布 token count 不足以复现数据。
- 数据删除请求、法律要求和安全过滤会让数据版本随时间变化。
- 小样本抽样只能发现部分数据问题，不能证明语料整体无污染或无偏差。
- 数据版本、许可状态和处理脚本如果缺失，后续 evaluation 与 scaling 结论都会失去可复现基础。

6.6 与本章其他概念的关系

数据集记录与可复现性连接本章的数据来源和模型行为：定义说明样本如何进入语料，公式刻画分布变化，伪代码展示处理顺序，例题检验数据决策怎样影响训练结论。

6.7 怎样检验数据集记录与可复现性

检验数据集记录与可复现性时，先把抽象说法落到样本、来源和分布状态上。与 manifest、version、provenance、filter、tokenizer 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明：版本不是文件名，而是由完整 pipeline 决定的可审计对象。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。数据消融通常一次只改变过滤阈值、去重策略、混合权重或数据版本。若同时更换 tokenizer、语料和评估集，就无法知道改进来自哪一步。最后检查失败条件。工程判断往往在反例中变清楚：只发布 token count 不足以复现数据；数据删除请求、法律要求和安全过滤会让数据版本随时间变化。复习时可以把这些限制改写成反例测试，观察它们怎样改变样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。把实验结论放回预训练数据：来源与数据集时，要说明数据集记录与可复现性改变的是样本分布、数据质量、污染风险还是可复现性。能追到这些来源，才说明读者真正理解数据机制。

6.8 概念辨析：数据集记录与可复现性

manifest、version、provenance、filter、tokenizer 常一起出现在数据管线中，但它们分别指来源、处理动作、版本记录或风险标记。读者应先判断它们改变的是样本集合还是审计证据。区分这些词时，可以先问它们是否改变样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕数据集记录与可复现性复习时，可以构造一小批文档：保留来源和版本，运行一个处理步骤，再比较样本计数和抽样质量。

6.9 数据集记录与可复现性的小结

- 讨论数据集记录与可复现性时，应同时报告问题规模、实验设置和主要观测变量，尤其是样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。
- 如果方法声称数据更好，要说明过滤、去重、混合或合成步骤改变了哪一部分分布，并检查 benchmark contamination。
- 数据结论需要抽样审计和版本记录；没有 provenance 的语料变化不能支持可引用结论。

7 综合讨论：从局部机制到完整系统

本章的主题是预训练数据：来源与数据集，但它不应被读成几个相邻知识点的拼接。为什么数据最重要、Web、code、books 与 curated datasets、数据集记录与可复现性共同回答同一个问题：语言模型系统中哪些对象可以被定义，哪些成本可以被估算，哪些结论必须通过实验或评估来确认。这些对象包括 pretraining data（预训练数据）、Common Crawl、data mixture（数据混合）、dataset provenance（数据来源）、licensing（许可证）。读者可以把它们看成一组接口：有的接口连接文本和 token，有的连接模型结构和张量计算，有的连接数据样本和训练目标，有的连接离线评估和在线服务。接口一旦改变，后面的成本、错误类型和优化空间也会随之改变。

7.1 为什么数据最重要在系统中的位置

为什么数据最重要首先提供一个局部解释：视频开场说：给定数据后大家已知道如何训练，接下来问题是应该训练在什么数据上。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它

的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，为什么数据最重要会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，为什么数据最重要不能脱离边界条件理解。一个关键 caveat 是：数据不是从天上掉下来；每个来源都有采集、许可和偏差问题。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.2 Web、code、books 与 curated datasets 在系统中的位置

Web、code、books 与 curated datasets 首先提供一个局部解释：Common Crawl 提供大规模网页原料，但需要 HTML extraction、language ID、质量过滤和去重。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，Web、code、books 与 curated datasets 会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，Web、code、books 与 curated datasets 不能脱离边界条件理解。一个关键 caveat 是：网页抽取错误会把导航栏、广告、脚本或重复模板带入训练。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.3 数据集记录与可复现性在系统中的位置

数据集记录与可复现性首先提供一个局部解释：课程强调开放模型与开放数据对信任和创新的作用；数据集 provenance 是可复现训练的基础。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，数据集记录与可复现性会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，数据集记录与可复现性不能脱离边界条件理解。一个关键 caveat 是：只发布 token count 不足以复现数据。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.4 把本章知识用于读论文和复现实验

读相关论文时，可以把本章变成一个检查表。第一，论文是否清楚定义了输入规模、模型规模、数据来源和评估协议；第二，论文声称的改进落在哪个变量上；第三，作者是否报告了会让结论失效的设置；第四，实验是否能分离模型结构、数据质量、优化设置和系统实现的影响。复现实验时，也应避免直接追求大规模。先用最小程序复现一个公式、一个伪代码分支或一个 profile 现象，再扩大输入规模。这样做的原因很简单：如果小规模程序无法解释中间量，大规模训练只会把错误隐藏在日志和随机波动里。把这些检查合在一起，本章给出的不是一个固定答案，而是一种阅读 CS336 的方法：每个模型机制都要落到变量，每个变量都要落到测量，每个测量都要说明协议，每个协议都要承认边界。因此，本章中的任何一句结论都可以被改写成一个可引用的完整句式：在给定数据、模型、硬件和评估协议下，某个机制改变了某个变量，并通过某个观测指标表现出来。这样的句子比单独背诵术语更长，但它包含了自学、复现和引用时真正需要的信息。把本章用于自学时，可以按三遍阅读。第一遍只追踪对象和定义，确认每个术语到底指向文本、token、张量、样本、请求还是评估记录。第二遍追踪公式和伪代码，确认每一步改变了哪些状态。第三遍追踪实验和 caveat，确认哪些结论只在特定规模、数据或硬件条件下成立。把本章用于复习时，则应反过来操作：先从一个失败案例或异常指标出发，再回到相应机制。loss 曲线异常可能指向数据、优化或模型容量；吞吐异常可能指向 kernel、通信或调度；

benchmark 异常可能指向 contamination、prompt protocol 或采样设置。能够从异常反推机制，是掌握本章的实用标准。

8 例题：把概念变成可计算检查

8.1 从网页到训练样本的 provenance

一个网页进入训练集，要经过 crawl、HTML extraction、language ID、quality filter 和 dedup。

- 每一步都应写入数据清单 (manifest)，而不是只留下最终文本。
- 过滤器版本和阈值会改变数据分布。
- 许可、robots、PII 和删除请求会影响能否持续使用。

结论。数据集是 pipeline 的产物，不是文件夹里若干 ‘jsonl’ 的集合。这个例题的目的不是替代完整工程实现，而是给出一种最小推理形式：把问题转成变量，写出近似公式或伪代码，再说明近似何时失效。

9 实验设计：从小例子到可复现结论

学习预训练数据：来源与数据集不能停在概念层。一个教材化结论至少需要四件事：可构造的输入、可观测的中间量、可比较的指标，以及清楚的失败条件。下面的实验设计不是要求训练大模型，而是要求读者用小规模程序复现本章机制。

9.1 最小输入

先构造只触发本章核心机制的小输入。例如 tokenizer 章节可以用几个包含 rare words 和 Unicode 字符的字符串；GPU 章节可以用一个 elementwise kernel 和一个 GEMM；data 章节可以用十几篇近重复文档。小输入的价值在于它让错误可见。

9.2 资源账本

对同一输入写下 FLOPs、bytes moved、显存峰值、通信量或样本数量的估算。估算不需要一开始就精确，但必须说明单位和近似假设。随后用 profiler、benchmark、validation loss 或人工检查去验证估算是否同量级。

9.3 单变量消融

一次只改变一个因素，例如 vocabulary size、head 数、batch size、filter threshold、reward scale、decoding temperature 或 GPU 数量。若多个变量同时变化，实验只能说明系统变了，不能说明机制是什么。

9.4 失败条件

最后要刻意触发失败：极端 context length、错误 dtype、过小 batch、污染 benchmark、低质量数据或不可靠 verifier。失败条件常常比成功曲线更能说明一个方法的边界。

9.5 记录与报告

实验记录应能让另一个读者复现同一结论。最少要写清楚输入构造、代码版本、随机种子、硬件、batch shape、数据版本、评价指标和异常处理方式。报告结论时不要只写“更快”或“更好”，而要说明快在哪里、好在哪个指标上，以及是否牺牲了内存、稳定性、数据质量或可解释性。常见报告错误是把局部现象写成普遍规律。例如一次小模型实验里的 loss 改善，不等于更大模型或不同数据分布上也会改善；一次 kernel benchmark 的吞吐提升，不等于端到端 serving latency 一定降低。教材中的实验报告应始终把结论限定在可复现条件内。

9.6 从小实验到大结论

小实验的作用不是替代课程中的完整训练或系统实现，而是建立一条可检查的推理链。先在小输入上确认变量方向，再在中等规模上确认数量级，最后才讨论是否能外推到更大模型、更多 token 或更复杂 workload。缺少中间层级时，大结论通常只是在复述直觉。因此，实验报告最好同时包含正例和反例：正例说明机制在受控条件下怎样工作，反例说明条件稍变时哪里失效。对 CS336 这样的课程，反例尤其重要，因为 language model 的错误常来自多个层面叠加：数据分布、优化稳定性、硬件瓶颈、评估协议和服务负载都可能同时改变观察结果。还要记录资料版本与复现边界。公开视频、课程页、配套代码和 PDF 可能在不同时间更新；复现实验应写明使用的材料版本，并说明哪些结论来自课程事实，哪些是为了帮助理解而加入的解释性推导。这样才能把学习笔记变成可检查的技术记录。若复现实验与课程结论不一致，首先检查输入、版本和评估协议，而不是立即修改模型假设。

9.7 把本章重新读成一条证据链

完成小实验之后，可以把预训练数据：来源与数据集重新读成一条证据链：先用“为什么数据最重要”确定问题入口，再用“Web、code、books 与 curated datasets”解释主要机制，最后用“数据集记录与可复现性”检查边界或外推条件。这种读法比按页背诵更接近教材训练，因为它要求每个结论都能说明前提、变量和证据。本章反复使用的术语包括 pretraining data（预训练数据）、Common Crawl、data mixture（数据混合）、dataset provenance（数据来源）、licensing（许可证）。复习时不要只写定义，而要为每个术语补上一句操作性说明：它在课程代码、公式、数据记录或评估协议中对应什么对象；如果这个对象被删除、替换或缩放，哪一个观测量会变化。最后，用一个反例结束复习。反例可以是资源估算不准、数据污染、reward 失真、benchmark 解析失败、长上下文显存爆炸或 scaling 外推失效。能解释反例的章节，才真正具备自学和引用价值；不能解释反例的章节，只是把课程材料换了一种排版。引用本章结论时，还应保留来源边界：哪些说法直接来自视频、课程页、slides 或代码，哪些是为了串联教材叙述而加入的解释性推导。这个边界不会削弱结论，反而让读者知道何处可以复核，何处需要在自己的实验中重新验证。如果后续课程材料更新，也可以沿着同一证据链替换来源，而不必重写整章结构。

10 课程资料与回看路径

教材正文已经把主要概念、公式和实现逻辑组织成连续叙述；本节只提供回到课程材料的阅读路径。读者复习时可以先完成前文练习，再按下面的时间段或资料组核对细节。

10.1 视频回看路径

时间	关键词	复习任务
----	-----	------

00:00:04-00:06:20	data, training, there's, data., that's, talk, train	回看定义、例子、推导或 caveat 在该段中的位置。
00:19:25-00:26:05	fair, copyright, license, creative, it's, that's, don't	回看定义、例子、推导或 caveat 在该段中的位置。
00:40:03-00:46:20	data, it's, github, books, there's, crawl, code	回看定义、例子、推导或 caveat 在该段中的位置。
00:53:24-01:00:55	data, think, books3, wikipedia, books, trained, quality	回看定义、例子、推导或 caveat 在该段中的位置。
01:14:57-01:21:49	data, it's, permissively, think, there's, very, whether	回看定义、例子、推导或 caveat 在该段中的位置。

这些时间段用于定位视频中的讲解顺序。严肃引用时，应回到视频片段确认讲者表述、板书或幻灯片内容。回看视频时，不必逐字抄录字幕。更有效的方法是把每个时间段判定为定义、例子、推导、代码解释或 caveat，再把它放回本章对应小节。

10.2 课程资料阅读路径

资料组	标题/页块	关键词
official_01	Lecture 13: Data I	data, train, training, live, language, pre-training, mid-training
official_03	- Threshold for copyright is extremely low (e.g., your website is copyrighted)	copyright, work, fair, license, training, copyrighted, over
official_05	- Repository: download through git protocol (rather than scraping the GitHub website)	https, github, metadata, arxiv, books, language, crawl
official_08	- ...	data, tokens, trained, classifier, result, remove, permissively
official_10	Reproduced by Together's RedPajama v1	function, refinedweb, dolma, stack, reproduced, together, redpajama

课程资料通常给出更稳定的标题、公式、图或代码结构；视频则补充动机和口头解释。两者合起来构成本章的事实边界。阅读资料时，可以把每个资料组拆成三个对象：一个定义，一个公式或伪代码动作，一个边界条件。这样比逐字翻译页面或脚本更容易形成可用知识。

11 实践说明、历史位置与风险

预训练数据：来源与数据集在 CS336 的课程结构中连接基础机制和规模化问题。基础机制解释方法为什么成立；规模化问题检验它在更大模型、更长上下文、更复杂数据或真实 serving workload 下是否仍成立。历史位置不等于模型年表。更重要的是识别问题如何迁移：早期方法解决小规模建模问题，现代方法常常解决同一问题在大规模数据、GPU 集群、长上下文或人类偏好约束下的新形态。

- 资料版本限制 (version risk)：课程页、公开视频和配套资料可能在学期中更新；引用时应注明访问日期或资料版本。

- 测量口径限制 (measurement risk): FLOPs、tokens/s、benchmark score、reward 等数字只有在协议完整时可比较。
- 规模外推限制 (scaling risk): 小模型或小 batch 的机制不必然外推到 frontier-scale。
- 数据分布限制 (data risk): 数据来源、过滤和去重策略会改变模型行为, 也会改变 evaluation contamination 风险。

12 复习要点

下面的问题把本章内容压缩为可反复检查的条目。每个条目都应能回到前文的解释、公式、伪代码或例题。

条目	对象	检查动作
条目 1	pretraining data (预训练数据)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 2	Common Crawl	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 3	data mixture (数据混合)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 4	dataset provenance (数据来源)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 5	licensing (许可证)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。

12.1 综合自测

- 我能否不用课程原句, 独立解释本章中心问题?
- 我能否把本章至少一个公式改写成可运行的估算函数?
- 我能否指出一个看似改进但实际转移成本的方法?
- 我能否回到视频和课程资料, 找到支持本章关键论断的位置?
- 我能否设计一个最小 ablation, 验证本章一个 caveat?

13 总结与延伸

13.1 本章小结

- 预训练数据: 来源与数据集的核心不是记忆术语, 而是理解对象、变量和机制之间的关系。
- 视频给出讲解顺序, 课程资料提供可核对的公式、图、代码或页面结构。
- 复习时应把每个术语连接到至少一个变量、一个实现动作和一个失败模式。

13.2 延伸解释

本章中的延伸解释用于连接课程材料中的概念，例如说明公式里的 proxy、解释系统 tradeoff，或把多个材料片段串成一个完整推理链。若需要严肃引用，应回到课程视频和配套资料核对原始表述。

13.3 练习

- 定义题：用中英双语解释本章任意五个重要术语，并说明它们属于 compute、memory、data、optimization、evaluation 还是 deployment 账本。
- 推导题：选择本章一个公式，逐个解释符号、单位和近似假设，并说明哪个变量最难在真实系统中测量。
- 实现题：把本章伪代码改写成一个最小 Python sanity check，不要求训练大模型，但要输出可检查的中间量。
- 资料题：从“课程资料与回看路径”中选择一个视频时间段，回到原始视频核对讲者例子，并记录是否存在字幕误差。
- 评估题：为本章方法设计一个 ablation 或 benchmark，说明控制变量、数据版本、指标和 failure criteria。
- 边界题：说明本章一个结论在更长 context、更大 batch、更多 GPU 或更复杂数据分布下可能如何失效。
- 综合题：把本章内容和前一章或后一章连接起来，写出一个端到端训练/推理 pipeline 中的依赖关系。
- 批判题：指出一个容易被营销材料夸大的说法，并用本章的资源账本或评估协议反驳它。

14 14 数据过滤、去重、混合与合成数据

第 14 讲：数据过滤、去重、混合与合成数据

CS336 Spring 2026 public videos and official course materials

目录

1 本章导读	2
1.1 本章知识路线	2
1.2 结构图	2
2 本章主线	2
2.1 本章要解决的核心问题	2
2.2 从机制到资源账本	3
2.3 从课堂讲解到可复现实验	3
3 术语、符号与问题边界	4
3.1 术语之间的关系	4
4 Filtering 的目标与方法	4
4.1 为什么要讨论 Filtering 的目标与方法	4
4.2 Filtering 的目标与方法的机制	5
4.3 Filtering 的目标与方法的数据分布视角	5
4.4 Filtering 的目标与方法在数据管线中的实现	5
4.5 边界条件与常见误解	5
4.6 与本章其他概念的关系	5
4.7 怎样检验 Filtering 的目标与方法	6
4.8 概念辨析：Filtering 的目标与方法	6
4.9 Filtering 的目标与方法的小结	6
5 Deduplication 与 MinHash	6
5.1 为什么要讨论 Deduplication 与 MinHash	6
5.2 Deduplication 与 MinHash 的机制	7
5.3 Deduplication 与 MinHash 的数据分布视角	7
5.4 Deduplication 与 MinHash 在数据管线中的实现	7
5.5 边界条件与常见误解	7

5.6	与本章其他概念的关系	7
5.7	怎样检验 Deduplication 与 MinHash	8
5.8	概念辨析: Deduplication 与 MinHash	8
5.9	Deduplication 与 MinHash 的小结	8
6	Mixture 与 synthetic data	8
6.1	为什么要讨论 Mixture 与 synthetic data	8
6.2	Mixture 与 synthetic data 的机制	9
6.3	Mixture 与 synthetic data 的数据分布视角	9
6.4	Mixture 与 synthetic data 在数据管线中的实现	9
6.5	边界条件与常见误解	9
6.6	与本章其他概念的关系	9
6.7	怎样检验 Mixture 与 synthetic data	10
6.8	概念辨析: Mixture 与 synthetic data	10
6.9	Mixture 与 synthetic data 的小结	10
7	综合讨论: 从局部机制到完整系统	10
7.1	Filtering 的目标与方法在系统中的位置	11
7.2	Deduplication 与 MinHash 在系统中的位置	11
7.3	Mixture 与 synthetic data 在系统中的位置	11
7.4	把本章知识用于读论文和复现实验	11
8	例题: 把概念变成可计算检查	12
8.1	MinHash 近重复检测	12
9	实验设计: 从小例子到可复现结论	12
9.1	最小输入	12
9.2	资源账本	12
9.3	单变量消融	12
9.4	失败条件	12
9.5	记录与报告	13
9.6	从小实验到大结论	13
9.7	把本章重新读成一条证据链	13
10	课程资料与回看路径	13
10.1	视频回看路径	13
10.2	课程资料阅读路径	14
11	实践说明、历史位置与风险	14

12 复习要点	15
12.1 综合自测	15
13 总结与延伸	15
13.1 本章小结	15
13.2 延伸解释	16
13.3 练习	16

1 本章导读

本章讨论 **数据过滤、去重、混合与合成数据**。CS336 的第一层目标不是把现成大模型当作黑箱使用，而是把 language model 从输入文本、训练目标、模型结构、数据、系统和评估这些部件重新拆开。只有拆开之后，读者才能判断一个设计选择究竟改变了什么。设想你有十万亿 raw tokens，但里面有重复网页、模板、PII、低质量文本和 benchmark 泄漏。数据处理不是清洁工序，而是训练目标的一部分；过滤、去重、混合和合成数据共同决定模型行为。本章对应 CS336 Spring 2026 第 14 个公开视频，并结合课程页提供的可解析资料。视频和课程页给出事实边界；正文把这些材料改写为可自学的教材章节。阅读本章时，先理解问题为什么存在，再看定义、公式和实现形态，随后通过例题和练习检查自己是否能独立复现这条 reasoning chain。

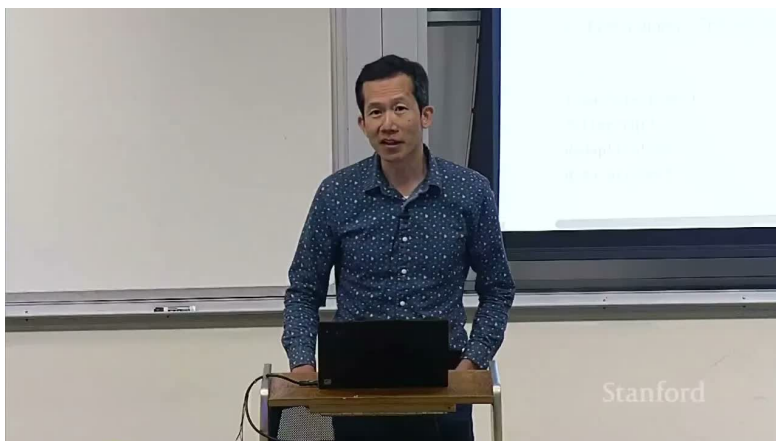


图 1: 第 14 讲的课程图像锚点。

1.1 本章知识路线

- 本章首先说明数据过滤、去重、混合与合成数据为什么是 language modeling pipeline 中不可跳过的一环。
- 其次定义本章反复使用的术语，并说明这些术语对应的计算对象或系统对象。
- 随后用公式和伪代码表达主要机制，让概念能够落到可计算的变量上。
- 最后给出例题、风险讨论和练习，便于读者检查自己是否真正掌握了机制。

1.2 结构图

下面的图用于帮助读者定位本章的核心结构。先看概念之间的依赖，再回到正文中的公式、伪代码和例题。

2 本章主线

数据过滤、去重、混合与合成数据的主线是把“训练数据”拆成来源、过滤、去重、混合和审计。模型能力不是只由参数决定，也由它反复看到的样本分布决定。

2.1 本章要解决的核心问题

本讲继续 data，重点是 filtering、deduplication、mixing 和 synthetic data。数据处理不是清洁步骤，而是模型能力和风险的主要决定因素。读完这一段后，应能用一两句话说明数据过滤、去重、混合与合成数据要

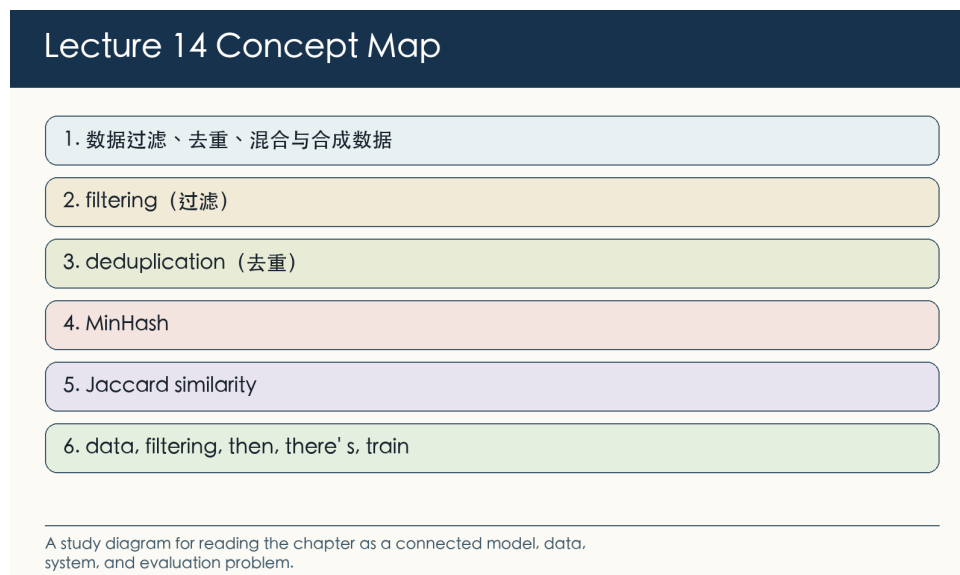


图 2: 第 14 讲概念图: 本章问题、术语和机制的关系。

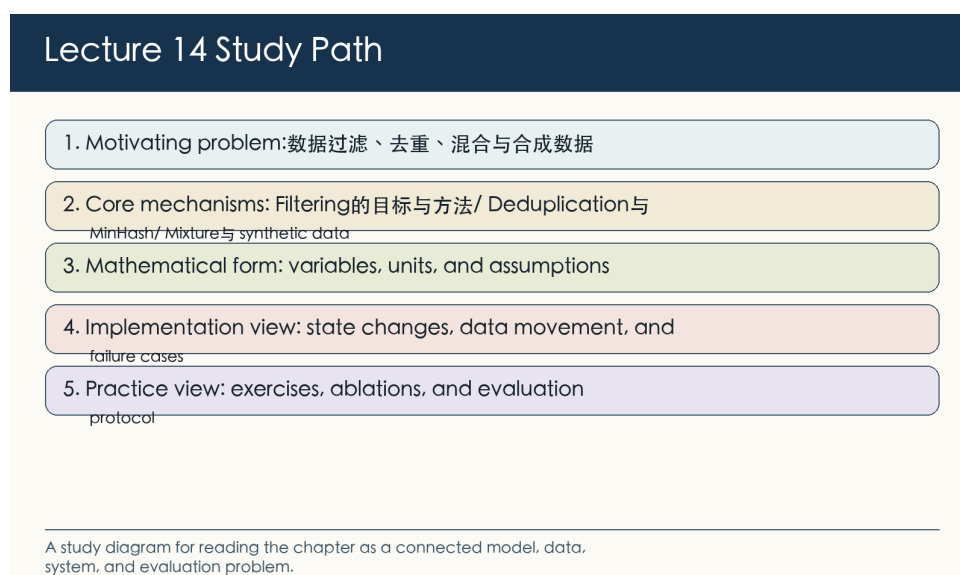


图 3: 第 14 讲学习路径: 从问题定义进入公式、实现、实验和练习。

解决的瓶颈，并指出这个瓶颈发生在数据、模型、优化、系统还是评估层。

2.2 从机制到资源账本

MinHash/Jaccard/LSH 让近重复检测可以扩展到大规模语料; 去重既节省 compute, 也降低 memorization 和 benchmark contamination。随后把机制写成账本: 输入规模是什么, 主要状态是什么, 成本或质量指标怎样随变量改变。这个账本让后面的公式不再只是符号。

2.3 从课堂讲解到可复现实验

data mixture 和 synthetic data 需要评估闭环。提高某类数据比例可能增强对应能力, 也可能损害通用性、安全性或长尾覆盖。最后把课堂讲解转成可复现实验: 固定协议, 改变一个变量, 记录中间量, 并解释结果为何支持或反驳原来的机制判断。

3 术语、符号与问题边界

本章的术语横跨模型、数据和系统。读者需要先知道每个词指向的对象：有些词描述数学目标，有些词描述张量形状，有些词描述硬件瓶颈，还有一些词描述评估协议。术语边界不清，后面的公式和实现就会变成空洞的缩写。

#	术语	阅读要求
1	filtering（过滤）	说明它如何改变训练样本或 token 分布，并给出一个会改变结论的数据边界条件。
2	deduplication（去重）	说明它如何改变训练样本或 token 分布，并给出一个会改变结论的数据边界条件。
3	MinHash	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
4	Jaccard similarity	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
5	data mixing（数据混合）	说明它如何改变训练样本或 token 分布，并给出一个会改变结论的数据边界条件。
6	synthetic data（合成数据）	说明它如何改变训练样本或 token 分布，并给出一个会改变结论的数据边界条件。

这些术语之间有依赖关系。例如 tokenization 改变 sequence length，sequence length 又改变 attention FLOPs、KV cache 和 evaluation token budget；parallelism 改变显存占用，也改变通信瓶颈和故障恢复方式。

3.1 术语之间的关系

在本章中，filtering（过滤）给出问题入口，deduplication（去重）描述主要机制，MinHash 则把机制连接到可测量的结果。读教材时应当沿着这个链条追问：输入是什么，中间状态怎样改变，输出指标为什么随之变化。课程材料还会反复出现 data、filtering、hash、quality、e.g.、train 等词。它们不是一组同义标签，而是提示本章的不同层次：有的指对象，有的指操作，有的指约束或指标。因此，术语表不是背诵清单，而是后文阅读的索引。遇到一个术语时，先定位它属于 representation、optimization、systems、data 还是 evaluation，再判断它改变哪一项账本。

4 Filtering 的目标与方法

现在进入 Filtering 的目标与方法。这一节关心训练样本如何形成、被筛选并进入混合分布；它直接决定模型看到什么，也决定 evaluation 是否可能被污染。

4.1 为什么要讨论 Filtering 的目标与方法

视频开场回顾上一讲：互联网由 live services 构成，数据需要 dump/crawl 和 processing。过滤把原始 crawl 转成可训练数据，常见维度包括语言、长度、质量、毒性、PII、格式、重复模板和 classifier score。过滤器可以是规则、统计模型、分类器或 model-based judge；关键是衡量过滤对最终模型的影响。数据部分的核心问题是分布控制。采集、过滤、去重和混合并不会只改变文件大小，它们会改变模型看到的语言、知识、偏差和 benchmark contamination 风险。

4.2 Filtering 的目标与方法的机制

视频开场回顾上一讲：互联网由 live services 构成，数据需要 dump/crawl 和 processing。在 Filtering 的目标与方法中，核心对象是样本、来源和分布状态：数据管线每改变一次样本集合，模型实际学习的分布也随之改变。过滤把原始 crawl 转成可训练数据，常见维度包括语言、长度、质量、毒性、PII、格式、重复模板和 classifier score。因而同一个方法在不同规模下可能改变不同的变量，尤其是样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。比较方法时，必须同时给出问题规模和实验条件。过滤器可以是规则、统计模型、分类器或 model-based judge；关键是衡量过滤对最终模型的影响。可检验性来自 provenance 记录、过滤前后计数、抽样审计和 held-out contamination 检查；没有这些记录，数据结论无法复现。因此，Filtering 的目标与方法的学习目标是建立一条数据谱系：读者应能从原始来源追到训练 token，并说明每个处理步骤改变了什么分布。

4.3 Filtering 的目标与方法的数据分布视角

$$\mathcal{D}_{\text{filtered}} = \{x \in \mathcal{D} : q(x) \geq \tau\}$$

符号说明： $q(x)$ 是质量函数， τ 是阈值；阈值越高，质量可能升高但覆盖面下降。这个关系式刻画 Filtering 的目标与方法对数据分布的影响。读者应关心哪些变量来自采集过程，哪些变量来自过滤或混合策略，哪些变量只能通过抽样审计估计。在数据实验中，去重率、过滤通过率、语言比例和抽样人工判读都是 proxy。它们不能单独代表数据质量，必须和来源记录、版本号以及污染检查一起解释。

4.4 Filtering 的目标与方法在数据管线中的实现

把 Filtering 的目标与方法写成程序时，要明确每一步怎样改写样本集合。下面的伪代码强调输入、过滤、统计和输出之间的关系。

```
for doc in raw_docs:
    score = quality_model(doc)
    if score >= threshold and not unsafe(doc):
        keep(doc)
```

阅读这段流程时，重点看 Filtering 的目标与方法怎样改变样本集合。每个过滤、去重或混合步骤都应留下计数和抽样检查，使样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险能够被复核。

4.5 边界条件与常见误解

- 过强过滤会删除长尾语言、非标准文本或弱势群体表达。
- 过滤器自身可能被训练数据偏差污染。
- 小样本抽样只能发现部分数据问题，不能证明语料整体无污染或无偏差。
- 数据版本、许可状态和处理脚本如果缺失，后续 evaluation 与 scaling 结论都会失去可复现基础。

4.6 与本章其他概念的关系

Filtering 的目标与方法连接本章的数据来源和模型行为：定义说明样本如何进入语料，公式刻画分布变化，伪代码展示处理顺序，例题检验数据决策怎样影响训练结论。对这一类主题，最常见的误解是只列方法名。更有用的做法是比较每个方法改变的成本项、依赖的假设和可能的失败模式。

4.7 怎样检验 Filtering 的目标与方法

检验 Filtering 的目标与方法时,先把抽象说法落到样本、来源和分布状态上。与 filtering、quality、classifier、threshold、PII 相关的对象必须能被构造、打印、计数或评分;否则实验只能得到描述性观察,不能得到可复现结论。一个合适的小实验应当保留本节机制,同时让样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险中至少一个量发生可解释变化。输入规模不必逼真;关键是读者能手算或打印关键中间量,并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明: $q(x)$ 是质量函数, τ 是阈值;阈值越高,质量可能升高但覆盖面下降。如果数量只能间接观测,就必须说明使用了什么 proxy;如果使用 benchmark 或 reward,也要说明协议和随机性。数据消融通常一次只改变过滤阈值、去重策略、混合权重或数据版本。若同时更换 tokenizer、语料和评估集,就无法知道改进来自哪一步。最后检查失败条件。工程判断往往在反例中变清楚:过强过滤会删除长尾语言、非标准文本或弱势群体表达;过滤器自身可能被训练数据偏差污染。复习时可以把这些限制改写成反例测试,观察它们怎样改变样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。把实验结论放回数据过滤、去重、混合与合成数据时,要说明 Filtering 的目标与方法改变的是样本分布、数据质量、污染风险还是可复现性。能追到这些来源,才说明读者真正理解数据机制。

4.8 概念辨析: Filtering 的目标与方法

filtering、quality、classifier、threshold、PII 常一起出现在数据管线中,但它们分别指来源、处理动作、版本记录或风险标记。读者应先判断它们改变的是样本集合还是审计证据。区分这些词时,可以先问它们是否改变样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。一个术语如果不能对应到变量、状态、代码路径或评估协议,就还没有形成可操作含义。围绕 Filtering 的目标与方法复习时,可以构造一小批文档:保留来源和版本,运行一个处理步骤,再比较样本计数和抽样质量。

4.9 Filtering 的目标与方法的小结

- 讨论 Filtering 的目标与方法时,应同时报告问题规模、实验设置和主要观测变量,尤其是样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。
- 如果方法声称数据更好,要说明过滤、去重、混合或合成步骤改变了哪一部分分布,并检查 benchmark contamination。
- 数据结论需要抽样审计和版本记录;没有 provenance 的语料变化不能支持可引用结论。

5 Deduplication 与 MinHash

现在进入 Deduplication 与 MinHash。这一节关心训练样本如何形成、被筛选并进入混合分布;它直接决定模型看到什么,也决定 evaluation 是否可能被污染。

5.1 为什么要讨论 Deduplication 与 MinHash

去重减少 memorization、污染和无效 token;视频关键词显示本讲重点包括 hash、Jaccard 和 deduplication。exact dedup 处理完全重复, near dedup 处理模板相似或轻微改写文本。MinHash 用随机哈希近似 Jaccard similarity,使大规模 near-duplicate 检索可行。数据部分的核心问题是分布控制。采集、过滤、去重和混合并不会只改变文件大小,它们会改变模型看到的语言、知识、偏差和 benchmark contamination 风险。

5.2 Deduplication 与 MinHash 的机制

去重减少 memorization、污染和无效 token；视频关键词显示本讲重点包括 hash、Jaccard 和 deduplication。在 Deduplication 与 MinHash 中，核心对象是样本、来源和分布状态：数据管线每改变一次样本集合，模型实际学习的分布也随之改变。exact dedup 处理完全重复，near dedup 处理模板相似或轻微改写文本。因而同一个方法在不同规模下可能改变不同的变量，尤其是样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。比较方法时，必须同时给出问题规模和实验条件。MinHash 用随机哈希近似 Jaccard similarity，使大规模 near-duplicate 检索可行。可检验性来自 provenance 记录、过滤前后计数、抽样审计和 held-out contamination 检查；没有这些记录，数据结论无法复现。因此，Deduplication 与 MinHash 的学习目标是建立一条数据谱系：读者应能从原始来源追到训练 token，并说明每个处理步骤改变了什么分布。

5.3 Deduplication 与 MinHash 的数据分布视角

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}, \quad \Pr[h_{\min}(A) = h_{\min}(B)] = J(A, B)$$

符号说明： A, B 是文档的 shingle 集合；MinHash 碰撞概率等于 Jaccard 相似度。这个关系式刻画 Deduplication 与 MinHash 对数据分布的影响。读者应关心哪些变量来自采集过程，哪些变量来自过滤或混合策略，哪些变量只能通过抽样审计估计。在数据实验中，去重率、过滤通过率、语言比例和抽样人工判读都是 proxy。它们不能单独代表数据质量，必须和来源记录、版本号以及污染检查一起解释。

5.4 Deduplication 与 MinHash 在数据管线中的实现

把 Deduplication 与 MinHash 写成程序时，要明确每一步怎样改写样本集合。下面的伪代码强调输入、过滤、统计和输出之间的关系。

```
shingles = make_ngrams(document)
signature = [min_hash(shingles, seed) for seed in seeds]
bucket_by_bands(signature)
```

阅读这段流程时，重点看 Deduplication 与 MinHash 怎样改变样本集合。每个过滤、去重或混合步骤都应留下计数和抽样检查，使样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险能够被复核。

5.5 边界条件与常见误解

- 去重粒度会影响效果：document-level、paragraph-level、line-level 的 tradeoff 不同。
- near dedup 太激进会删除合法引用、模板化代码或多语言平行文本。
- 小样本抽样只能发现部分数据问题，不能证明语料整体无污染或无偏差。
- 数据版本、许可状态和处理脚本如果缺失，后续 evaluation 与 scaling 结论都会失去可复现基础。

5.6 与本章其他概念的关系

Deduplication 与 MinHash 连接本章的数据来源和模型行为：定义说明样本如何进入语料，公式刻画分布变化，伪代码展示处理顺序，例题检验数据决策怎样影响训练结论。对这一类主题，最常见的误解是只列方法名。更有用的做法是比较每个方法改变的成本项、依赖的假设和可能的失败模式。

5.7 怎样检验 Deduplication 与 MinHash

检验 Deduplication 与 MinHash 时, 先把抽象说法落到样本、来源和分布状态上。与 deduplication、MinHash、Jaccard、shingle、hash 相关的对象必须能被构造、打印、计数或评分; 否则实验只能得到描述性观察, 不能得到可复现结论。一个合适的小实验应当保留本节机制, 同时让样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险中至少一个量发生可解释变化。输入规模不必逼真; 关键是读者能手算或打印关键中间量, 并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明: $\backslash(A,B\backslash)$ 是文档的 shingle 集合; MinHash 碰撞概率等于 Jaccard 相似度。如果数量只能间接观测, 就必须说明使用了什么 proxy; 如果使用 benchmark 或 reward, 也要说明协议和随机性。数据消融通常一次只改变过滤阈值、去重策略、混合权重或数据版本。若同时更换 tokenizer、语料和评估集, 就无法知道改进来自哪一步。最后检查失败条件。工程判断往往在反例中变清楚: 去重粒度会影响效果: document-level、paragraph-level、line-level 的 tradeoff 不同; near dedup 太激进会删除合法引用、模板化代码或多语言平行文本。复习时可以把这些限制改写成反例测试, 观察它们怎样改变样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。把实验结论放回数据过滤、去重、混合与合成数据时, 要说明 Deduplication 与 MinHash 改变的是样本分布、数据质量、污染风险还是可复现性。能追到这些来源, 才说明读者真正理解数据机制。

5.8 概念辨析: Deduplication 与 MinHash

deduplication、MinHash、Jaccard、shingle、hash 常一起出现在数据管线中, 但它们分别指来源、处理动作、版本记录或风险标记。读者应先判断它们改变的是样本集合还是审计证据。其中, deduplication (去重) 降低重复、污染和 memorization 风险; MinHash 用哈希签名近似 Jaccard similarity 以做大规模近重复检测。区分这些词时, 可以先问它们是否改变样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。一个术语如果不能对应到变量、状态、代码路径或评估协议, 就还没有形成可操作含义。围绕 Deduplication 与 MinHash 复习时, 可以构造一小批文档: 保留来源和版本, 运行一个处理步骤, 再比较样本计数和抽样质量。

5.9 Deduplication 与 MinHash 的小结

- 讨论 Deduplication 与 MinHash 时, 应同时报告问题规模、实验设置和主要观测变量, 尤其是样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。
- 如果方法声称数据更好, 要说明过滤、去重、混合或合成步骤改变了哪一部分分布, 并检查 benchmark contamination。
- 数据结论需要抽样审计和版本记录; 没有 provenance 的语料变化不能支持可引用结论。

6 Mixture 与 synthetic data

现在进入 Mixture 与 synthetic data。这一节关心训练样本如何形成、被筛选并进入混合分布; 它直接决定模型看到什么, 也决定 evaluation 是否可能被污染。

6.1 为什么要讨论 Mixture 与 synthetic data

数据混合决定不同来源在训练中的采样概率; 它影响能力分布和安全行为。合成数据可以补齐稀缺任务、改善 instruction following 或 reasoning, 但会引入模型偏差和退化风险。课程把 mixing、filtering、dedup 和 synthetic data 放在同一讲, 说明数据 pipeline 是一个联合优化问题。数据部分的核心问题是分布控制。采集、过滤、去重和混合并不会只改变文件大小, 它们会改变模型看到的语言、知识、偏差和 benchmark contamination 风险。

6.2 Mixture 与 synthetic data 的机制

数据混合决定不同来源在训练中的采样概率；它影响能力分布和安全行为。在 Mixture 与 synthetic data 中，核心对象是样本、来源和分布状态：数据管线每改变一次样本集合，模型实际学习的分布也随之改变。合成数据可以补齐稀缺任务、改善 instruction following 或 reasoning，但会引入模型偏差和退化风险。因而同一个方法在不同规模下可能改变不同的变量，尤其是样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。比较方法时，必须同时给出问题规模和实验条件。课程把 mixing、filtering、dedup 和 synthetic data 放在同一讲，说明数据 pipeline 是一个联合优化问题。可检验性来自 provenance 记录、过滤前后计数、抽样审计和 held-out contamination 检查；没有这些记录，数据结论无法复现。因此，Mixture 与 synthetic data 的学习目标是建立一条数据谱系：读者应能从原始来源追到训练 token，并说明每个处理步骤改变了什么分布。

6.3 Mixture 与 synthetic data 的数据分布视角

$$p(x) = \sum_k w_k p_k(x), \quad \sum_k w_k = 1$$

符号说明： p_k 是第 k 个数据源分布， w_k 是采样权重；权重改变等价于改变训练目标分布。这个关系式刻画 Mixture 与 synthetic data 对数据分布的影响。读者应关心哪些变量来自采集过程，哪些变量来自过滤或混合策略，哪些变量只能通过抽样审计估计。在数据实验中，去重率、过滤通过率、语言比例和抽样人工判读都是 proxy。它们不能单独代表数据质量，必须和来源记录、版本号以及污染检查一起解释。

6.4 Mixture 与 synthetic data 在数据管线中的实现

把 Mixture 与 synthetic data 写成程序时，要明确每一步怎样改写样本集合。下面的伪代码强调输入、过滤、统计和输出之间的关系。

```
while training:
    source = sample_source(weights)
    batch = sample_batch(source)
    train_on(batch)
```

阅读这段流程时，重点看 Mixture 与 synthetic data 怎样改变样本集合。每个过滤、去重或混合步骤都应留下计数和抽样检查，使样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险能够被复核。

6.5 边界条件与常见误解

- synthetic data 需要质量控制，否则会放大 teacher model 的错误。
- 混合权重最好通过 ablation、scaling 和 downstream eval 联合确定。
- 小样本抽样只能发现部分数据问题，不能证明语料整体无污染或无偏差。
- 数据版本、许可状态和处理脚本如果缺失，后续 evaluation 与 scaling 结论都会失去可复现基础。

6.6 与本章其他概念的关系

Mixture 与 synthetic data 连接本章的数据来源和模型行为：定义说明样本如何进入语料，公式刻画分布变化，伪代码展示处理顺序，例题检验数据决策怎样影响训练结论。对这一类主题，最常见的误解是只列方法名。更有用的做法是比较每个方法改变的成本项、依赖的假设和可能的失败模式。

6.7 怎样检验 Mixture 与 synthetic data

检验 Mixture 与 synthetic data 时，先把抽象说法落到样本、来源和分布状态上。与 mixing、synthetic data、weights、sampling、ablation 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： $\backslash(p_k)$ 是第 $\backslash(k)$ 个数据源分布， $\backslash(w_k)$ 是采样权重；权重改变等价于改变训练目标分布。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。数据消融通常一次只改变过滤阈值、去重策略、混合权重或数据版本。若同时更换 tokenizer、语料和评估集，就无法知道改进来自哪一步。最后检查失败条件。工程判断往往在反例中变清楚：synthetic data 需要质量控制，否则会放大 teacher model 的错误；混合权重最好通过 ablation、scaling 和 downstream eval 联合确定。复习时可以把这些限制改写成反例测试，观察它们怎样改变样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。把实验结论放回数据过滤、去重、混合与合成数据时，要说明 Mixture 与 synthetic data 改变的是样本分布、数据质量、污染风险还是可复现性。能追到这些来源，才说明读者真正理解数据机制。

6.8 概念辨析：Mixture 与 synthetic data

mixing、synthetic data、weights、sampling、ablation 常一起出现在数据管线中，但它们分别指来源、处理动作、版本记录或风险标记。读者应先判断它们改变的是样本集合还是审计证据。其中，data（数据）是模型能力、偏差和法律风险的主要来源。区分这些词时，可以先问它们是否改变样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕 Mixture 与 synthetic data 复习时，可以构造一小批文档：保留来源和版本，运行一个处理步骤，再比较样本计数和抽样质量。

6.9 Mixture 与 synthetic data 的小结

- 讨论 Mixture 与 synthetic data 时，应同时报告问题规模、实验设置和主要观测变量，尤其是样本数量、去重率、过滤阈值、数据混合比例和 contamination 风险。
- 如果方法声称数据更好，要说明过滤、去重、混合或合成步骤改变了哪一部分分布，并检查 benchmark contamination。
- 数据结论需要抽样审计和版本记录；没有 provenance 的语料变化不能支持可引用结论。

7 综合讨论：从局部机制到完整系统

本章的主题是数据过滤、去重、混合与合成数据，但它不应被读成几个相邻知识点的拼接。Filtering 的目标与方法、Deduplication 与 MinHash、Mixture 与 synthetic data 共同回答同一个问题：语言模型系统中哪些对象可以被定义，哪些成本可以被估算，哪些结论必须通过实验或评估来确认。这些对象包括 filtering（过滤）、deduplication（去重）、MinHash、Jaccard similarity、data mixing（数据混合）、synthetic data（合成数据）。读者可以把它们看成一组接口：有的接口连接文本和 token，有的连接模型结构和张量计算，有的连接数据样本和训练目标，有的连接离线评估和在线服务。接口一旦改变，后面的成本、错误类型和优化空间也会随之改变。

7.1 Filtering 的目标与方法在系统中的位置

Filtering 的目标与方法首先提供一个局部解释：视频开场回顾上一讲：互联网由 live services 构成，数据需要 dump/crawl 和 processing。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，Filtering 的目标与方法会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，Filtering 的目标与方法不能脱离边界条件理解。一个关键 caveat 是：过强过滤会删除长尾语言、非标准文本或弱势群体表达。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.2 Deduplication 与 MinHash 在系统中的位置

Deduplication 与 MinHash 首先提供一个局部解释：去重减少 memorization、污染和无效 token；视频关键词显示本讲重点包括 hash、Jaccard 和 deduplication。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，Deduplication 与 MinHash 会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，Deduplication 与 MinHash 不能脱离边界条件理解。一个关键 caveat 是：去重粒度会影响效果：document-level、paragraph-level、line-level 的 tradeoff 不同。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.3 Mixture 与 synthetic data 在系统中的位置

Mixture 与 synthetic data 首先提供一个局部解释：数据混合决定不同来源在训练中的采样概率；它影响能力分布和安全行为。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，Mixture 与 synthetic data 会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，Mixture 与 synthetic data 不能脱离边界条件理解。一个关键 caveat 是：synthetic data 需要质量控制，否则会放大 teacher model 的错误。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.4 把本章知识用于读论文和复现实验

读相关论文时，可以把本章变成一个检查表。第一，论文是否清楚定义了输入规模、模型规模、数据来源和评估协议；第二，论文声称的改进落在哪个变量上；第三，作者是否报告了会让结论失效的设置；第四，实验是否能分离模型结构、数据质量、优化设置和系统实现的影响。复现实验时，也应避免直接追求大规模。先用最小程序复现一个公式、一个伪代码分支或一个 profile 现象，再扩大输入规模。这样做的原因很简单：如果小规模程序无法解释中间量，大规模训练只会把错误隐藏在日志和随机波动里。把这些检查合在一起，本章给出的不是一个固定答案，而是一种阅读 CS336 的方法：每个模型机制都要落到变量，每个变量都要落到测量，每个测量都要说明协议，每个协议都要承认边界。因此，本章中的任何一句结论都可以被改写成一个可引用的完整句式：在给定数据、模型、硬件和评估协议下，某个机制改变了某个变量，并通过某个观测指标表现出来。这样的句子比单独背诵术语更长，但它包含了自学、复现和引用时真正需要的信息。把本章用于自学时，可以按三遍阅读。第一遍只追踪对象和定义，确认每个术语到底指向文本、token、张量、样本、请求还是评

估记录。第二遍追踪公式和伪代码，确认每一步改变了哪些状态。第三遍追踪实验和 caveat，确认哪些结论只在特定规模、数据或硬件条件下成立。把本章用于复习时，则应反过来操作：先从一个失败案例或异常指标出发，再回到相应机制。loss 曲线异常可能指向数据、优化或模型容量；吞吐异常可能指向 kernel、通信或调度；benchmark 异常可能指向 contamination、prompt protocol 或采样设置。能够从异常反推机制，是掌握本章的实用标准。

8 例题：把概念变成可计算检查

8.1 MinHash 近重复检测

把每篇文档表示为 shingle 集合，用 MinHash 估计 Jaccard similarity。

- 若两文档 shingle 集合相似，MinHash signature 更可能碰撞。
- LSH banding 把候选对数量降到可处理范围。
- 最后仍需阈值和人工/规则策略决定删哪一份。

结论。去重是统计近似、系统扩展和语料政策的组合问题。这个例题的目的不是替代完整工程实现，而是给出一种最小推理形式：把问题转成变量，写出近似公式或伪代码，再说明近似何时失效。

9 实验设计：从小例子到可复现结论

学习数据过滤、去重、混合与合成数据不能停在概念层。一个教材化结论至少需要四件事：可构造的输入、可观测的中间量、可比较的指标，以及清楚的失败条件。下面的实验设计不是要求训练大模型，而是要求读者用小规模程序复现本章机制。

9.1 最小输入

先构造只触发本章核心机制的小输入。例如 tokenizer 章节可以用几个包含 rare words 和 Unicode 字符的字符串；GPU 章节可以用一个 elementwise kernel 和一个 GEMM；data 章节可以用十几篇近重复文档。小输入的价值在于它让错误可见。

9.2 资源账本

对同一输入写下 FLOPs、bytes moved、显存峰值、通信量或样本数量的估算。估算不需要一开始就精确，但必须说明单位和近似假设。随后用 profiler、benchmark、validation loss 或人工检查去验证估算是否同量级。

9.3 单变量消融

一次只改变一个因素，例如 vocabulary size、head 数、batch size、filter threshold、reward scale、decoding temperature 或 GPU 数量。若多个变量同时变化，实验只能说明系统变了，不能说明机制是什么。

9.4 失败条件

最后要刻意触发失败：极端 context length、错误 dtype、过小 batch、污染 benchmark、低质量数据或不可靠 verifier。失败条件常常比成功曲线更能说明一个方法的边界。

9.5 记录与报告

实验记录应能让另一个读者复现同一结论。最少要写清楚输入构造、代码版本、随机种子、硬件、batch shape、数据版本、评价指标和异常处理方式。报告结论时不要只写“更快”或“更好”，而要说明快在哪里、好在哪个指标上，以及是否牺牲了内存、稳定性、数据质量或可解释性。常见报告错误是把局部现象写成普遍规律。例如一次小模型实验里的 loss 改善，不等于更大模型或不同数据分布上也会改善；一次 kernel benchmark 的吞吐提升，不等于端到端 serving latency 一定降低。教材中的实验报告应始终把结论限定在可复现条件内。

9.6 从小实验到大结论

小实验的作用不是替代课程中的完整训练或系统实现，而是建立一条可检查的推理链。先在小输入上确认变量方向，再在中等规模上确认数量级，最后才讨论是否能外推到更大模型、更多 token 或更复杂 workload。缺少中间层级时，大结论通常只是在复述直觉。因此，实验报告最好同时包含正例和反例：正例说明机制在受控条件下怎样工作，反例说明条件稍变时哪里失效。对 CS336 这样的课程，反例尤其重要，因为 language model 的错误常来自多个层面叠加：数据分布、优化稳定性、硬件瓶颈、评估协议和服务负载都可能同时改变观察结果。还要记录资料版本与复现边界。公开视频、课程页、配套代码和 PDF 可能在不同时间更新；复现实验应写明使用的材料版本，并说明哪些结论来自课程事实，哪些是为了帮助理解而加入的解释性推导。这样才能把学习笔记变成可检查的技术记录。若复现实验与课程结论不一致，首先检查输入、版本和评估协议，而不是立即修改模型假设。

9.7 把本章重新读成一条证据链

完成小实验之后，可以把数据过滤、去重、混合与合成数据重新读成一条证据链：先用“Filtering 的目标与方法”确定问题入口，再用“Deduplication 与 MinHash”解释主要机制，最后用“Mixture 与 synthetic data”检查边界或外推条件。这种读法比按页背诵更接近教材训练，因为它要求每个结论都能说明前提、变量和证据。本章反复使用的术语包括 filtering（过滤）、deduplication（去重）、MinHash、Jaccard similarity、data mixing（数据混合）、synthetic data（合成数据）。复习时不要只写定义，而要为每个术语补上一句操作性说明：它在课程代码、公式、数据记录或评估协议中对应什么对象；如果这个对象被删除、替换或缩放，哪一个观测量会变化。最后，用一个反例结束复习。反例可以是资源估算不准、数据污染、reward 失真、benchmark 解析失败、长上下文显存爆炸或 scaling 外推失效。能解释反例的章节，才真正具备自学和引用价值；不能解释反例的章节，只是把课程材料换了一种排版。引用本章结论时，还应保留来源边界：哪些说法直接来自视频、课程页、slides 或代码，哪些是为了串联教材叙述而加入的解释性推导。这个边界不会削弱结论，反而让读者知道何处可以复核，何处需要在自己的实验中重新验证。如果后续课程材料更新，也可以沿着同一证据链替换来源，而不必重写整章结构。

10 课程资料与回看路径

教材正文已经把主要概念、公式和实现逻辑组织成连续叙述；本节只提供回到课程材料的阅读路径。读者复习时可以先完成前文练习，再按下面的时间段或资料组核对细节。

10.1 视频回看路径

时间	关键词	复习任务
----	-----	------

00:00:05-00:07:15	data, there's, pdfs, then, text, html, data.	回看定义、例子、推导或 caveat 在该段中的位置。
00:20:31-00:26:58	data, there's, training, good, train, probably, then	回看定义、例子、推导或 caveat 在该段中的位置。
00:41:19-00:49:49	probability, then, band, it's, threshold, data, similarity	回看定义、例子、推导或 caveat 在该段中的位置。
00:56:52-01:03:27	then, mixture, train, data, very, mixing, training	回看定义、例子、推导或 caveat 在该段中的位置。
01:17:25-01:24:24	then, data, swe-zero, there's, agent, generate, tasks	回看定义、例子、推导或 caveat 在该段中的位置。

这些时间段用于定位视频中的讲解顺序。严肃引用时，应回到视频片段确认讲者表述、板书或幻灯片内容。回看视频时，不必逐字抄录字幕。更有效的方法是把每个时间段判定为定义、例子、推导、代码解释或 caveat，再把它放回本章对应小节。

10.2 课程资料阅读路径

资料组	标题/页块	关键词
official_01	Lecture 14: Data II	data, filtering, service, github, crawl, deduplication, mixing
official_03	- Result: produced 14.7B tokens, used to train 1.4B models that do better than models ...	data, train, samples, wikipedia, quality, filtering, duplicates
official_05	Let's now look at approximate set membership.	first, jaccard, hash, similarity, items, item, threshold
official_08	- Questions come from 27 human and synthetic sources (e.g., StackExchange, ...	tasks, github, better, agent, sources, execution, trajectories
official_10	Simulated epoching	function, simulated, epoching, openthoughts, swe-smith, swe-zero, swe-rebench

课程资料通常给出更稳定的标题、公式、图或代码结构；视频则补充动机和口头解释。两者合起来构成本章的事实边界。阅读资料时，可以把每个资料组拆成三个对象：一个定义，一个公式或伪代码动作，一个边界条件。这样比逐字翻译页面或脚本更容易形成可用知识。

11 实践说明、历史位置与风险

数据过滤、去重、混合与合成数据在 CS336 的课程结构中连接基础机制和规模化问题。基础机制解释方法为什么成立；规模化问题检验它在更大模型、更长上下文、更复杂数据或真实 serving workload 下是否仍成立。历史位置不等于模型年表。更重要的是识别问题如何迁移：早期方法解决小规模建模问题，现代方法常常解决同一问题在大规模数据、GPU 集群、长上下文或人类偏好约束下的新形态。

- 资料版本限制 (version risk): 课程页、公开视频和配套资料可能在学期中更新；引用时应注明访问日期或资料版本。

- 测量口径限制 (measurement risk): FLOPs、tokens/s、benchmark score、reward 等数字只有在协议完整时可比较。
- 规模外推限制 (scaling risk): 小模型或小 batch 的机制不必然外推到 frontier-scale。
- 数据分布限制 (data risk): 数据来源、过滤和去重策略会改变模型行为, 也会改变 evaluation contamination 风险。

12 复习要点

下面的问题把本章内容压缩为可反复检查的条目。每个条目都应能回到前文的解释、公式、伪代码或例题。

条目	对象	检查动作
条目 1	filtering (过滤)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 2	deduplication (去重)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 3	MinHash	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 4	Jaccard similarity	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 5	data mixing (数据混合)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 6	synthetic data (合成数据)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。

12.1 综合自测

- 我能否不用课程原句，独立解释本章中心问题？
- 我能否把本章至少一个公式改写成可运行的估算函数？
- 我能否指出一个看似改进但实际转移成本的方法？
- 我能否回到视频和课程资料，找到支持本章关键论断的位置？
- 我能否设计一个最小 ablation，验证本章一个 caveat？

13 总结与延伸

13.1 本章小结

- 数据过滤、去重、混合与合成数据的核心不是记忆术语，而是理解对象、变量和机制之间的关系。

- 视频给出讲解顺序，课程资料提供可核对的公式、图、代码或页面结构。
- 复习时应把每个术语连接到至少一个变量、一个实现动作和一个失败模式。

13.2 延伸解释

本章中的延伸解释用于连接课程材料中的概念，例如说明公式里的 proxy、解释系统 tradeoff，或把多个材料片段串成一个完整推理链。若需要严肃引用，应回到课程视频和配套资料核对原始表述。

13.3 练习

- 定义题：用中英双语解释本章任意五个重要术语，并说明它们属于 compute、memory、data、optimization、evaluation 还是 deployment 账本。
- 推导题：选择本章一个公式，逐个解释符号、单位和近似假设，并说明哪个变量最难在真实系统中测量。
- 实现题：把本章伪代码改写成一个最小 Python sanity check，不要求训练大模型，但要输出可检查的中间量。
- 资料题：从“课程资料与回看路径”中选择一个视频时间段，回到原始视频核对讲者例子，并记录是否存在字幕误差。
- 评估题：为本章方法设计一个 ablation 或 benchmark，说明控制变量、数据版本、指标和 failure criteria。
- 边界题：说明本章一个结论在更长 context、更大 batch、更多 GPU 或更复杂数据分布下可能如何失效。
- 综合题：把本章内容和前一章或后一章连接起来，写出一个端到端训练/推理 pipeline 中的依赖关系。
- 批判题：指出一个容易被营销材料夸大的说法，并用本章的资源账本或评估协议反驳它。

15 15 Mid/Post-training: SFT 与 RLHF

第 15 讲: Mid/Post-training: SFT 与 RLHF

CS336 Spring 2026 public videos and official course materials

目录

1 本章导读	2
1.1 本章知识路线	2
1.2 结构图	2
2 本章主线	2
2.1 本章要解决的核心问题	2
2.2 从机制到资源账本	3
2.3 从课堂讲解到可复现实验	3
3 术语、符号与问题边界	4
3.1 术语之间的关系	4
4 从 pretraining 到 assistant 行为	4
4.1 为什么要讨论从 pretraining 到 assistant 行为	4
4.2 从 pretraining 到 assistant 行为的机制	5
4.3 从 pretraining 到 assistant 行为的数学表达	5
4.4 从从 pretraining 到 assistant 行为到程序	5
4.5 边界条件与常见误解	5
4.6 与本章其他概念的关系	5
4.7 怎样检验从 pretraining 到 assistant 行为	6
4.8 概念辨析: 从 pretraining 到 assistant 行为	6
4.9 从 pretraining 到 assistant 行为的小结	6
5 Reward modeling 与 RLHF	6
5.1 为什么要讨论 Reward modeling 与 RLHF	6
5.2 Reward modeling 与 RLHF 的机制	7
5.3 Reward modeling 与 RLHF 的目标函数	7
5.4 Reward modeling 与 RLHF 的训练流程	7
5.5 边界条件与常见误解	7

5.6	与本章其他概念的关系	7
5.7	怎样检验 Reward modeling 与 RLHF	8
5.8	概念辨析: Reward modeling 与 RLHF	8
5.9	Reward modeling 与 RLHF 的小结	8
6	DPO 与直接偏好优化	8
6.1	为什么要讨论 DPO 与直接偏好优化	8
6.2	DPO 与直接偏好优化的机制	9
6.3	DPO 与直接偏好优化的目标函数	9
6.4	DPO 与直接偏好优化的训练流程	9
6.5	边界条件与常见误解	9
6.6	与本章其他概念的关系	9
6.7	怎样检验 DPO 与直接偏好优化	10
6.8	概念辨析: DPO 与直接偏好优化	10
6.9	DPO 与直接偏好优化的小结	10
7	综合讨论: 从局部机制到完整系统	10
7.1	从 pretraining 到 assistant 行为在系统中的位置	11
7.2	Reward modeling 与 RLHF 在系统中的位置	11
7.3	DPO 与直接偏好优化在系统中的位置	11
7.4	把本章知识用于读论文和复现实验	11
8	例题: 把概念变成可计算检查	12
8.1	DPO 目标如何偏向 chosen response	12
9	实验设计: 从小例子到可复现结论	12
9.1	最小输入	12
9.2	资源账本	12
9.3	单变量消融	12
9.4	失败条件	12
9.5	记录与报告	13
9.6	从小实验到大结论	13
9.7	把本章重新读成一条证据链	13
10	课程资料与回看路径	13
10.1	视频回看路径	13
10.2	课程资料阅读路径	14
11	实践说明、历史位置与风险	14

12 复习要点	15
12.1 综合自测	15
13 总结与延伸	16
13.1 本章小结	16
13.2 延伸解释	16
13.3 练习	16

1 本章导读

本章讨论 **Mid/Post-training: SFT 与 RLHF**。CS336 的第一层目标不是把现成大模型当作黑箱使用，而是把 language model 从输入文本、训练目标、模型结构、数据、系统和评估这些部件重新拆开。只有拆开之后，读者才能判断一个设计选择究竟改变了什么。设想 base model 会续写文本，却不会稳定遵循 instruction。Post-training（后训练）要把语言模型变成 assistant：SFT 提供示范行为，RLHF/DPO 用偏好约束输出风格和安全边界。本章对应 CS336 Spring 2026 第 15 个公开视频，并结合课程页提供的可解析资料。视频和课程页给出事实边界；正文把这些材料改写为可自学的教材章节。阅读本章时，先理解问题为什么存在，再看定义、公式和实现形态，随后通过例题和练习检查自己是否能独立复现这条 reasoning chain。

Lecture 15

“AFTER PRETRAINING” (MID/POSTTRAINING)

CS336



图 1: 第 15 讲的课程图像锚点。

1.1 本章知识路线

- 本章首先说明 Mid/Post-training: SFT 与 RLHF 为什么是 language modeling pipeline 中不可跳过的一环。
- 其次定义本章反复使用的术语，并说明这些术语对应的计算对象或系统对象。
- 随后用公式和伪代码表达主要机制，让概念能够落到可计算的变量上。
- 最后给出例题、风险讨论和练习，便于读者检查自己是否真正掌握了机制。

1.2 结构图

下面的图用于帮助读者定位本章的核心结构。先看概念之间的依赖，再回到正文中的公式、伪代码和例题。

2 本章主线

Mid/Post-training: SFT 与 RLHF 的主线是解释 base model 之后的行为塑形。课程关心的不是单个 post-training 名词，而是示范、偏好、奖励和评估怎样共同约束输出。

2.1 本章要解决的核心问题

本讲从 pretraining 转向 mid/post-training。SFT 用示范回答塑造 instruction-following，RLHF 用偏好数据进一步控制输出。读完这一段后，应能用一两句话说明 Mid/Post-training: SFT 与 RLHF 要解决的瓶颈，

The class thus far

We've now covered pre-training, which gets you to GPT3.. But how do we get to instructGPT?

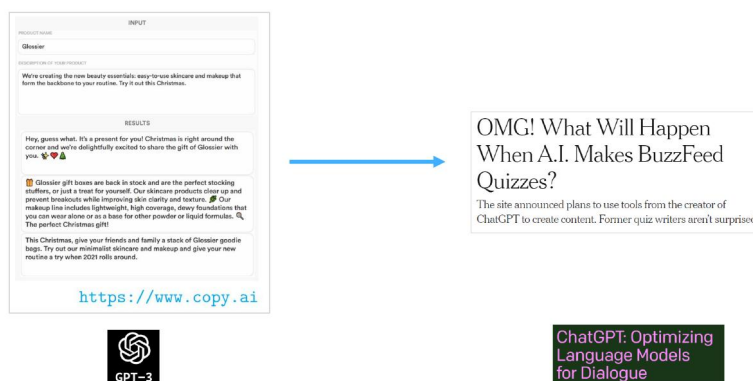


图 2: 官方材料图页: Mid/Post-training: SFT 与 RLHF 的课程页/PDF/script 图像锚点。

Lecture 15

"AFTER PRETRAINING" (MID/POSTTRAINING)

CS336

图 3: 官方材料图页: Mid/Post-training: SFT 与 RLHF 的课程页/PDF/script 图像锚点。

并指出这个瓶颈发生在数据、模型、优化、系统还是评估层。

2.2 从机制到资源账本

post-training 更 artisanal: 数据稀缺、公开细节少、标注规范和 reward model 都会强烈影响模型行为。随后把机制写成账本: 输入规模是什么, 主要状态是什么, 成本或质量指标怎样随变量改变。这个账本让后面的公式不再只是符号。

2.3 从课堂讲解到可复现实验

DPO/PPO/RLHF 等方法的共同主题是让模型偏向更符合人类偏好的回答, 同时用 KL 或 reference policy 控制偏离。最后把课堂讲解转成可复现实验: 固定协议, 改变一个变量, 记录中间量, 并解释结果为何支持或反驳原来的机制判断。

3 术语、符号与问题边界

本章的术语横跨模型、数据和系统。读者需要先知道每个词指向的对象：有些词描述数学目标，有些词描述张量形状，有些词描述硬件瓶颈，还有一些词描述评估协议。术语边界不清，后面的公式和实现就会变成空洞的缩写。

#	术语	阅读要求
1	post-training（后训练）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
2	supervised fine-tuning (SFT)	说明它约束的是模型行为、优化目标还是评估协议，并指出一个奖励或偏好失真的例子。
3	RLHF	说明它约束的是模型行为、优化目标还是评估协议，并指出一个奖励或偏好失真的例子。
4	reward model（奖励模型）	说明它约束的是模型行为、优化目标还是评估协议，并指出一个奖励或偏好失真的例子。
5	DPO	说明它约束的是模型行为、优化目标还是评估协议，并指出一个奖励或偏好失真的例子。
6	PPO	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。

这些术语之间有依赖关系。例如 tokenization 改变 sequence length，sequence length 又改变 attention FLOPs、KV cache 和 evaluation token budget；parallelism 改变显存占用，也改变通信瓶颈和故障恢复方式。

3.1 术语之间的关系

在本章中，post-training（后训练）给出问题入口，supervised fine-tuning (SFT) 描述主要机制，RLHF 则把机制连接到可测量的结果。读教材时应当沿着这个链条追问：输入是什么，中间状态怎样改变，输出指标为什么随之变化。课程材料还会反复出现 data、rlhf、reward、many、instruction、safety 等词。它们不是一组同义标签，而是提示本章的不同层次：有的指对象，有的指操作，有的指约束或指标。因此，术语表不是背诵清单，而是后文阅读的索引。遇到一个术语时，先定位它属于 representation、optimization、systems、data 还是 evaluation，再判断它改变哪一项账本。

4 从 pretraining 到 assistant 行为

现在进入从 pretraining 到 assistant 行为。这一节讨论 base model 之外的行为塑形：示范、偏好、reward 或 verifier 怎样把模型输出推向课程定义的目标行为。

4.1 为什么要讨论从 pretraining 到 assistant 行为

视频开场说 post-training 比 pretraining 更 messy、更 artisanal；它把基础 LM 转成可交互助手。pretraining 学 next-token distribution，SFT 学示范回答格式和任务行为。mid-training/post-training 的核心是改变模型可见行为，而不是重新学习全部世界知识。从 pretraining 到 assistant 行为把 Mid/Post-training：SFT 与 RLHF 连接到行为目标。接下来要看的不是 reward 名称，而是反馈信号、采样分布和约束项怎样改变模型更新。

4.2 从 pretraining 到 assistant 行为的机制

视频开场说 post-training 比 pretraining 更 messy、更 artisanal；它把基础 LM 转成可交互助手。在从 pretraining 到 assistant 行为中，核心对象是行为、奖励和偏好状态：后训练方法并不直接制造知识，而是通过目标和反馈改变模型选择输出的方式。pretraining 学 next-token distribution，SFT 学示范回答格式和任务行为。因而同一个方法在不同规模下可能改变不同的变量，尤其是 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。比较方法时，必须同时给出问题规模和实验条件。mid-training/post-training 的核心是改变模型可见行为，而不是重新学习全部世界知识。可检验性来自训练日志、reward 分布、KL/length 统计和行为评估；只看最终回答样例无法判断更新方向。因此，从 pretraining 到 assistant 行为的学习目标是建立一条行为更新链：读者应能区分示范、偏好、奖励和约束分别给模型提供了什么信号。

4.3 从 pretraining 到 assistant 行为的数学表达

$$\mathcal{L}_{\text{SFT}}(\theta) = - \sum_{(x,y)} \sum_t \log p_{\theta}(y_t \mid x, y_{<t})$$

符号说明： x 是 instruction/context， y 是人工或高质量示范回答；SFT 仍是 token-level 监督学习。这个关系式说明从 pretraining 到 assistant 行为如何把行为偏好写入优化目标。关键不是符号复杂度，而是 reward、reference policy、采样分布和约束强度如何共同决定更新方向。在后训练实验中，reward model score、verifier pass rate、回答长度和人工偏好都是 proxy。它们会被采样策略和格式约束影响，因此不能直接等同于真实有用性。

4.4 从从 pretraining 到 assistant 行为到程序

把从 pretraining 到 assistant 行为写成训练流程时，要区分采样、打分、构造 loss 和更新参数。下面的伪代码保留这些边界。

```
for instruction, response in sft_data:
    loss = cross_entropy(model(instruction, response), response_tokens)
    update(loss)
```

阅读这段训练流程时，重点区分采样、打分和参数更新。只有同时记录 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率，才能判断行为改善来自奖励信号、采样分布还是约束强度。

4.5 边界条件与常见误解

- SFT 数据质量比数量更重要，坏示范会被直接模仿。
- SFT 可能降低 base model 的某些原始能力或校准。
- 后训练指标可能被格式、长度或 verifier 偏差驱动，不能直接等同于真实能力提升。
- 偏好优化可能改善目标行为，也可能牺牲多样性、校准性或某些下游能力。

4.6 与本章其他概念的关系

从 pretraining 到 assistant 行为连接本章的训练目标和输出行为：术语表区分示范、偏好与奖励，公式说明优化方向，伪代码展示采样和更新，例题检查行为约束的边界。对这一类主题，最常见的误解是只列方法名。更有用的做法是比较每个方法改变的成本项、依赖的假设和可能的失败模式。

4.7 怎样检验从 pretraining 到 assistant 行为

检验从 pretraining 到 assistant 行为时，先把抽象说法落到行为、奖励和偏好状态上。与 post-training、SFT、instruction、assistant、demonstration 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： $\backslash(x\backslash)$ 是 instruction/context， $\backslash(y\backslash)$ 是人工或高质量示范回答；SFT 仍是 token-level 监督学习。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。后训练消融通常一次只改变示范数据、reward scale、KL 约束、采样温度或 verifier。若训练目标和解码策略同时变化，行为差异就很难归因。最后检查失败条件。工程判断往往在反例中变清楚：SFT 数据质量比数量更重要，坏示范会被直接模仿；SFT 可能降低 base model 的某些原始能力或校准。复习时可以把这些限制改写成反例测试，观察它们怎样改变 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。把实验结论放回 Mid/Post-training：SFT 与 RLHF 时，要说明从 pretraining 到 assistant 行为改变的是目标函数、采样分布、反馈质量还是行为约束。能区分这些来源，才说明读者真正理解后训练。

4.8 概念辨析：从 pretraining 到 assistant 行为

post-training、SFT、instruction、assistant、demonstration 常一起出现在后训练讨论中，但它们分别指数据、反馈、目标函数或行为指标。读者应先判断每个词给优化过程提供什么信号。其中，supervised fine-tuning (SFT) 用示范回答塑造 assistant 行为。区分这些词时，可以先问它们是否改变 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕从 pretraining 到 assistant 行为复习时，可以构造几条 prompt-response：明确奖励或偏好规则，观察同一模型在更新前后的行为差异。

4.9 从 pretraining 到 assistant 行为的小结

- 讨论从 pretraining 到 assistant 行为时，应同时报告问题规模、实验设置和主要观测变量，尤其是 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。
- 如果方法声称行为更好，要说明示范、偏好、reward 或 verifier 哪一项提供了信号，并检查 reward hacking、length bias 和能力退化。
- 后训练结论需要把训练目标、采样策略和评估协议分开报告，否则无法判断改进来自哪里。

5 Reward modeling 与 RLHF

现在进入 Reward modeling 与 RLHF。这一节讨论 base model 之外的行为塑形：示范、偏好、reward 或 verifier 怎样把模型输出推向课程定义的目标行为。

5.1 为什么要讨论 Reward modeling 与 RLHF

RLHF 用人类偏好训练 reward model，再用 RL 优化策略模型。pairwise preference 更容易标注开放式回答质量，但 reward model 只是偏好的近似。PPO 等 RL 方法需要 KL penalty，防止模型远离 reference policy 而产生 reward hacking。后训练部分的核心问题是行为控制。模型已经学到语言分布，但还需要通过示范、偏好或可验证奖励，把输出推向可用、可评估、可约束的行为。

5.2 Reward modeling 与 RLHF 的机制

RLHF 用人类偏好训练 reward model，再用 RL 优化策略模型。在 Reward modeling 与 RLHF 中，核心对象是行为、奖励和偏好状态：后训练方法并不直接制造知识，而是通过目标和反馈改变模型选择输出的方式。pairwise preference 更容易标注开放式回答质量，但 reward model 只是偏好的近似。因而同一个方法在不同规模下可能改变不同的变量，尤其是 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。比较方法时，必须同时给出问题规模和实验条件。PPO 等 RL 方法需要 KL penalty，防止模型远离 reference policy 而产生 reward hacking。可检验性来自训练日志、reward 分布、KL/length 统计和行为评估；只看最终回答样例无法判断更新方向。因此，Reward modeling 与 RLHF 的学习目标是建立一条行为更新链：读者应能区分示范、偏好、奖励和约束分别给模型提供了什么信号。

5.3 Reward modeling 与 RLHF 的目标函数

$$\mathcal{L}_{\text{RM}} = -\log \sigma(r_{\phi}(x, y^+) - r_{\phi}(x, y^-))$$

符号说明： y^+ 是偏好回答， y^- 是较差回答， r_{ϕ} 是 reward model。这个关系式说明 Reward modeling 与 RLHF 如何把行为偏好写入优化目标。关键不是符号复杂度，而是 reward、reference policy、采样分布和约束强度如何共同决定更新方向。在后训练实验中，reward model score、verifier pass rate、回答长度和人工偏好都是 proxy。它们会被采样策略和格式约束影响，因此不能直接等同于真实有用性。

5.4 Reward modeling 与 RLHF 的训练流程

把 Reward modeling 与 RLHF 写成训练流程时，要区分采样、打分、构造 loss 和更新参数。下面的伪代码保留这些边界。

```
reward = reward_model(prompt, response)
kl = kl_divergence(policy, reference)
objective = reward - beta * kl
policy.update(objective)
```

阅读这段训练流程时，重点区分采样、打分和参数更新。只有同时记录 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率，才能判断行为改善来自奖励信号、采样分布还是约束强度。

5.5 边界条件与常见误解

- reward model 会被策略优化 exploited。
- 偏好数据反映标注规范和人群，不是客观真理。
- 后训练指标可能被格式、长度或 verifier 偏差驱动，不能直接等同于真实能力提升。
- 偏好优化可能改善目标行为，也可能牺牲多样性、校准性或某些下游能力。

5.6 与本章其他概念的关系

Reward modeling 与 RLHF 连接本章的训练目标和输出行为：术语表区分示范、偏好与奖励，公式说明优化方向，伪代码展示采样和更新，例题检查行为约束的边界。对这一类主题，最常见的误解是只列方法名。更有用的做法是比较每个方法改变的成本项、依赖的假设和可能的失败模式。

5.7 怎样检验 Reward modeling 与 RLHF

检验 Reward modeling 与 RLHF 时，先把抽象说法落到行为、奖励和偏好状态上。与 RLHF、reward model、preference、PPO、KL 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： $\langle y^+ \rangle$ 是偏好回答， $\langle y^- \rangle$ 是较差回答， $\langle r_\phi \rangle$ 是 reward model。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。后训练消融通常一次只改变示范数据、reward scale、KL 约束、采样温度或 verifier。若训练目标和解码策略同时变化，行为差异就很难归因。最后检查失败条件。工程判断往往在反例中变清楚：reward model 会被策略优化 exploited；偏好数据反映标注规范和人群，不是客观真理。复习时可以把这些限制改写成反例测试，观察它们怎样改变 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。把实验结论放回 Mid/Post-training：SFT 与 RLHF 时，要说明 Reward modeling 与 RLHF 改变的是目标函数、采样分布、反馈质量还是行为约束。能区分这些来源，才说明读者真正理解后训练。

5.8 概念辨析：Reward modeling 与 RLHF

RLHF、reward model、preference、PPO、KL 常一起出现在后训练讨论中，但它们分别指数据、反馈、目标函数或行为指标。读者应先判断每个词给优化过程提供什么信号。其中，RLHF 用人类偏好 reward 优化模型行为。区分这些词时，可以先问它们是否改变 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕 Reward modeling 与 RLHF 复习时，可以构造几条 prompt-response：明确奖励或偏好规则，观察同一模型在更新前后的行为差异。

5.9 Reward modeling 与 RLHF 的小结

- 讨论 Reward modeling 与 RLHF 时，应同时报告问题规模、实验设置和主要观测变量，尤其是 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。
- 如果方法声称行为更好，要说明示范、偏好、reward 或 verifier 哪一项提供了信号，并检查 reward hacking、length bias 和能力退化。
- 后训练结论需要把训练目标、采样策略和评估协议分开报告，否则无法判断改进来自哪里。

6 DPO 与直接偏好优化

现在进入 DPO 与直接偏好优化。这一节讨论 base model 之外的行为塑形：示范、偏好、reward 或 verifier 怎样把模型输出推向课程定义的目标行为。

6.1 为什么要讨论 DPO 与直接偏好优化

DPO 类方法把偏好优化写成监督式目标，避免显式训练 RL loop。它依然依赖偏好数据和 reference policy，只是把 reward 隐式化。post-training 方法选择应看数据类型、稳定性、compute、目标行为和安全约束。后训练部分的核心问题是行为控制。模型已经学到语言分布，但还需要通过示范、偏好或可验证奖励，把输出推向可用、可评估、可约束的行为。

6.2 DPO 与直接偏好优化的机制

DPO 类方法把偏好优化写成监督式目标，避免显式训练 RL loop。在 DPO 与直接偏好优化中，核心对象是行为、奖励和偏好状态：后训练方法并不直接制造知识，而是通过目标和反馈改变模型选择输出的方式。它依然依赖偏好数据和 reference policy，只是把 reward 隐式化。因而同一个方法在不同规模下可能改变不同的变量，尤其是 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。比较方法时，必须同时给出问题规模和实验条件。post-training 方法选择应看数据类型、稳定性、compute、目标行为和安全约束。可检验性来自训练日志、reward 分布、KL/length 统计和行为评估；只看最终回答样例无法判断更新方向。因此，DPO 与直接偏好优化的学习目标是建立一条行为更新链：读者应能区分示范、偏好、奖励和约束分别给模型提供了什么信号。

6.3 DPO 与直接偏好优化的目标函数

$$\mathcal{L}_{\text{DPO}} = -\log \sigma \left(\beta \log \frac{\pi_{\theta}(y^+ | x)}{\pi_{\text{ref}}(y^+ | x)} - \beta \log \frac{\pi_{\theta}(y^- | x)}{\pi_{\text{ref}}(y^- | x)} \right)$$

符号说明： π_{θ} 是待训练策略， π_{ref} 是参考模型， β 控制偏离参考模型的强度。这个关系式说明 DPO 与直接偏好优化如何把行为偏好写入优化目标。关键不是符号复杂度，而是 reward、reference policy、采样分布和约束强度如何共同决定更新方向。在后训练实验中，reward model score、verifier pass rate、回答长度和人工偏好都是 proxy。它们会被采样策略和格式约束影响，因此不能直接等同于真实有用性。

6.4 DPO 与直接偏好优化的训练流程

把 DPO 与直接偏好优化写成训练流程时，要区分采样、打分、构造 loss 和更新参数。下面的伪代码保留这些边界。

```
chosen_logp = policy.logprob(prompt, chosen)
rejected_logp = policy.logprob(prompt, rejected)
loss = dpo_loss(chosen_logp, rejected_logp, reference_logps)
```

阅读这段训练流程时，重点区分采样、打分和参数更新。只有同时记录 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率，才能判断行为改善来自奖励信号、采样分布还是约束强度。

6.5 边界条件与常见误解

- DPO 不是万能替代 RL；它受偏好数据覆盖范围限制。
- post-training 改善 helpfulness 可能牺牲 calibration 或 diversity。
- 后训练指标可能被格式、长度或 verifier 偏差驱动，不能直接等同于真实能力提升。
- 偏好优化可能改善目标行为，也可能牺牲多样性、校准性或某些下游能力。

6.6 与本章其他概念的关系

DPO 与直接偏好优化连接本章的训练目标和输出行为：术语表区分示范、偏好与奖励，公式说明优化方向，伪代码展示采样和更新，例题检查行为约束的边界。对这一类主题，最常见的误解是只列方法名。更有用的做法是比较每个方法改变的成本项、依赖的假设和可能的失败模式。

6.7 怎样检验 DPO 与直接偏好优化

检验 DPO 与直接偏好优化时，先把抽象说法落到行为、奖励和偏好状态上。与 DPO、preference optimization、reference、beta、alignment 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： π 是待训练策略， π_{ref} 是参考模型， β 控制偏离参考模型的强度。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。后训练消融通常一次只改变示范数据、reward scale、KL 约束、采样温度或 verifier。若训练目标和解码策略同时变化，行为差异就很难归因。最后检查失败条件。工程判断往往在反例中变清楚：DPO 不是万能替代 RL；它受偏好数据覆盖范围限制；post-training 改善 helpfulness 可能牺牲 calibration 或 diversity。复习时可以把这些限制改写成反例测试，观察它们怎样改变 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。把实验结论放回 Mid/Post-training: SFT 与 RLHF 时，要说明 DPO 与直接偏好优化改变的是目标函数、采样分布、反馈质量还是行为约束。能区分这些来源，才说明读者真正理解后训练。

6.8 概念辨析：DPO 与直接偏好优化

DPO、preference optimization、reference、beta、alignment 常一起出现在后训练讨论中，但它们分别指数据、反馈、目标函数或行为指标。读者应先判断每个词给优化过程提供什么信号。其中，DPO 把偏好优化写成直接监督式目标。区分这些词时，可以先问它们是否改变 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕 DPO 与直接偏好优化复习时，可以构造几条 prompt-response：明确奖励或偏好规则，观察同一模型在更新前后的行为差异。

6.9 DPO 与直接偏好优化的小结

- 讨论 DPO 与直接偏好优化时，应同时报告问题规模、实验设置和主要观测变量，尤其是 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。
- 如果方法声称行为更好，要说明示范、偏好、reward 或 verifier 哪一项提供了信号，并检查 reward hacking、length bias 和能力退化。
- 后训练结论需要把训练目标、采样策略和评估协议分开报告，否则无法判断改进来自哪里。

7 综合讨论：从局部机制到完整系统

本章的主题是 Mid/Post-training: SFT 与 RLHF，但它不应被读成几个相邻知识点的拼接。从 pretraining 到 assistant 行为、Reward modeling 与 RLHF、DPO 与直接偏好优化共同回答同一个问题：语言模型系统中哪些对象可以被定义，哪些成本可以被估算，哪些结论必须通过实验或评估来确认。这些对象包括 post-training (后训练)、supervised fine-tuning (SFT)、RLHF、reward model (奖励模型)、DPO、PPO。读者可以把它们看成一组接口：有的接口连接文本和 token，有的连接模型结构和张量计算，有的连接数据样本和训练目标，有的连接离线评估和在线服务。接口一旦改变，后面的成本、错误类型和优化空间也会随之改变。

7.1 从 pretraining 到 assistant 行为在系统中的位置

从 pretraining 到 assistant 行为首先提供一个局部解释：视频开场说 post-training 比 pretraining 更 messy、更 artisanal；它把基础 LM 转成可交互助手。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，从 pretraining 到 assistant 行为会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，从 pretraining 到 assistant 行为不能脱离边界条件理解。一个关键 caveat 是：SFT 数据质量比数量更重要，坏示范会被直接模仿。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.2 Reward modeling 与 RLHF 在系统中的位置

Reward modeling 与 RLHF 首先提供一个局部解释：RLHF 用人类偏好训练 reward model，再用 RL 优化策略模型。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，Reward modeling 与 RLHF 会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，Reward modeling 与 RLHF 不能脱离边界条件理解。一个关键 caveat 是：reward model 会被策略优化 exploited。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.3 DPO 与直接偏好优化在系统中的位置

DPO 与直接偏好优化首先提供一个局部解释：DPO 类方法把偏好优化写成监督式目标，避免显式训练 RL loop。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，DPO 与直接偏好优化会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，DPO 与直接偏好优化不能脱离边界条件理解。一个关键 caveat 是：DPO 不是万能替代 RL；它受偏好数据覆盖范围限制。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.4 把本章知识用于读论文和复现实验

读相关论文时，可以把本章变成一个检查表。第一，论文是否清楚定义了输入规模、模型规模、数据来源和评估协议；第二，论文声称的改进落在哪个变量上；第三，作者是否报告了会让结论失效的设置；第四，实验是否能分离模型结构、数据质量、优化设置和系统实现的影响。复现实验时，也应避免直接追求大规模。先用小程序复现一个公式、一个伪代码分支或一个 profile 现象，再扩大输入规模。这样做的原因很简单：如果小规模程序无法解释中间量，大规模训练只会把错误隐藏在日志和随机波动里。把这些检查合在一起，本章给出的不是一个固定答案，而是一种阅读 CS336 的方法：每个模型机制都要落到变量，每个变量都要落到测量，每个测量都要说明协议，每个协议都要承认边界。因此，本章中的任何一句结论都可以被改写成一个可引用的完整句式：在给定数据、模型、硬件和评估协议下，某个机制改变了某个变量，并通过某个观测指标表现出来。这样的句子比单独背诵术语更长，但它包含了自学、复现和引用时真正需要的信息。把本章用于自学时，可以按三遍阅读。第一遍只追踪对象和定义，确认每个术语到底指向文本、token、张量、样本、请求还是评估记录。第二遍追踪公式和伪代码，确认每一步改变了哪些状态。第三遍追踪实验和 caveat，确认哪些结论只

在特定规模、数据或硬件条件下成立。把本章用于复习时，则应反过来操作：先从一个失败案例或异常指标出发，再回到相应机制。loss 曲线异常可能指向数据、优化或模型容量；吞吐异常可能指向 kernel、通信或调度；benchmark 异常可能指向 contamination、prompt protocol 或采样设置。能够从异常反推机制，是掌握本章的实用标准。

8 例题：把概念变成可计算检查

8.1 DPO 目标如何偏向 chosen response

同一 prompt 有 chosen 和 rejected 两个回答，以及 reference model 的 log probability。

- DPO 比较 policy 相对 reference 对 chosen/rejected 的偏好差。
- 如果 chosen 的相对 log probability 提高，loss 下降。
- 参数 ‘ β ’ 控制偏离 reference 的力度。

结论。 偏好优化仍然依赖数据覆盖和 reference policy，不能把目标函数当成自动 alignment。这个例题的目的不是替代完整工程实现，而是给出一种最小推理形式：把问题转成变量，写出近似公式或伪代码，再说明近似何时失效。

9 实验设计：从小例子到可复现结论

学习 Mid/Post-training：SFT 与 RLHF 不能停在概念层。一个教材化结论至少需要四件事：可构造的输入、可观测的中间量、可比较的指标，以及清楚的失败条件。下面的实验设计不是要求训练大模型，而是要求读者用小规模程序复现本章机制。

9.1 最小输入

先构造只触发本章核心机制的小输入。例如 tokenizer 章节可以用几个包含 rare words 和 Unicode 字符的字符串；GPU 章节可以用一个 elementwise kernel 和一个 GEMM；data 章节可以用十几篇近重复文档。小输入的价值在于它让错误可见。

9.2 资源账本

对同一输入写下 FLOPs、bytes moved、显存峰值、通信量或样本数量的估算。估算不需要一开始就精确，但必须说明单位和近似假设。随后用 profiler、benchmark、validation loss 或人工检查去验证估算是否同量级。

9.3 单变量消融

一次只改变一个因素，例如 vocabulary size、head 数、batch size、filter threshold、reward scale、decoding temperature 或 GPU 数量。若多个变量同时变化，实验只能说明系统变了，不能说明机制是什么。

9.4 失败条件

最后要刻意触发失败：极端 context length、错误 dtype、过小 batch、污染 benchmark、低质量数据或不可靠 verifier。失败条件常常比成功曲线更能说明一个方法的边界。

9.5 记录与报告

实验记录应能让另一个读者复现同一结论。最少要写清楚输入构造、代码版本、随机种子、硬件、batch shape、数据版本、评价指标和异常处理方式。报告结论时不要只写“更快”或“更好”，而要说明快在哪里、好在哪个指标上，以及是否牺牲了内存、稳定性、数据质量或可解释性。常见报告错误是把局部现象写成普遍规律。例如一次小模型实验里的 loss 改善，不等于更大模型或不同数据分布上也会改善；一次 kernel benchmark 的吞吐提升，不等于端到端 serving latency 一定降低。教材中的实验报告应始终把结论限定在可复现条件内。

9.6 从小实验到大结论

小实验的作用不是替代课程中的完整训练或系统实现，而是建立一条可检查的推理链。先在小输入上确认变量方向，再在中等规模上确认数量级，最后才讨论是否能外推到更大模型、更多 token 或更复杂 workload。缺少中间层级时，大结论通常只是在复述直觉。因此，实验报告最好同时包含正例和反例：正例说明机制在受控条件下怎样工作，反例说明条件稍变时哪里失效。对 CS336 这样的课程，反例尤其重要，因为 language model 的错误常来自多个层面叠加：数据分布、优化稳定性、硬件瓶颈、评估协议和服务负载都可能同时改变观察结果。还要记录资料版本与复现边界。公开视频、课程页、配套代码和 PDF 可能在不同时间更新；复现实验应写明使用的材料版本，并说明哪些结论来自课程事实，哪些是为了帮助理解而加入的解释性推导。这样才能把学习笔记变成可检查的技术记录。若复现实验与课程结论不一致，首先检查输入、版本和评估协议，而不是立即修改模型假设。

9.7 把本章重新读成一条证据链

完成小实验之后，可以把 Mid/Post-training: SFT 与 RLHF 重新读成一条证据链：先用“从 pretraining 到 assistant 行为”确定问题入口，再用“Reward modeling 与 RLHF”解释主要机制，最后用“DPO 与直接偏好优化”检查边界或外推条件。这种读法比按页背诵更接近教材训练，因为它要求每个结论都能说明前提、变量和证据。本章反复使用的术语包括 post-training（后训练）、supervised fine-tuning (SFT)、RLHF、reward model（奖励模型）、DPO、PPO。复习时不要只写定义，而要为每个术语补上一句操作性说明：它在课程代码、公式、数据记录或评估协议中对应什么对象；如果这个对象被删除、替换或缩放，哪一个观测量会变化。最后，用一个反例结束复习。反例可以是资源估算不准、数据污染、reward 失真、benchmark 解析失败、长上下文内存爆炸或 scaling 外推失效。能解释反例的章节，才真正具备自学和引用价值；不能解释反例的章节，只是把课程材料换了一种排版。引用本章结论时，还应保留来源边界：哪些说法直接来自视频、课程页、slides 或代码，哪些是为了串联教材叙述而加入的解释性推导。这个边界不会削弱结论，反而让读者知道何处可以复核，何处需要在自己的实验中重新验证。如果后续课程材料更新，也可以沿着同一证据链替换来源，而不必重写整章结构。

10 课程资料与回看路径

教材正文已经把主要概念、公式和实现逻辑组织成连续叙述；本节只提供回到课程材料的阅读路径。读者复习时可以先完成前文练习，再按下面的时间段或资料组核对细节。

10.1 视频回看路径

时间	关键词	复习任务
----	-----	------

00:00:05-00:07:01	going, very, data, think, data., it's, pre-training	回看定义、例子、推导或 caveat 在该段中的位置。
00:20:45-00:27:19	think, knowledge, don't, it's, very, might, your	回看定义、例子、推导或 caveat 在该段中的位置。
00:40:35-00:46:32	good, going, that's, talk, pre-training, your, very	回看定义、例子、推导或 caveat 在该段中的位置。
00:53:43-01:00:14	that's, annotators, think, there's, it's, people, quite	回看定义、例子、推导或 caveat 在该段中的位置。
01:13:09-01:19:48	very, think, rlhf, take, going, good, step	回看定义、例子、推导或 caveat 在该段中的位置。

这些时间段用于定位视频中的讲解顺序。严肃引用时，应回到视频片段确认讲者表述、板书或幻灯片内容。回看视频时，不必逐字抄录字幕。更有效的方法是把每个时间段判定为定义、例子、推导、代码解释或 caveat，再把它放回本章对应小节。

10.2 课程资料阅读路径

资料组	标题/页块	关键词
official_01	Lecture 15 “A F T E R P R E T R A I N I N G ” (M I D / P O S T T R A I N I N G) ...	data, information, annotation, source, part, cs336, class
official_03	Style variations in data and models Models vary a lot in response length.. [Wang+ ...	knowledge, monopsony, market, style, relevant, teaching, extraction
official_05	RLHF Data What types of pairwise feedback. How do we get (good) pairwise feedback?	rlhf, data, pairwise, feedback, standard, types, good
official_06	Modern worker distribution Caveat: many platforms, covers only one (Outlier, ScaleAI)	rlhf, data, distribution, annotators, feedback, crowdsourcing, pairwise
official_08	DPO in LLaMA (and other open models) DPO + Expert iteration for post-training	rlhf, variants, many, watch, pref, lots, overoptimization

课程资料通常给出更稳定的标题、公式、图或代码结构；视频则补充动机和口头解释。两者合起来构成本章的事实边界。阅读资料时，可以把每个资料组拆成三个对象：一个定义，一个公式或伪代码动作，一个边界条件。这样比逐字翻译页面或脚本更容易形成可用知识。

11 实践说明、历史位置与风险

Mid/Post-training: SFT 与 RLHF 在 CS336 的课程结构中连接基础机制和规模化问题。基础机制解释方法为什么成立；规模化问题检验它在更大模型、更长上下文、更复杂数据或真实 serving workload 下是否仍成立。历史位置不等于模型年表。更重要的是识别问题如何迁移：早期方法解决小规模建模问题，现代方法常常解决同一问题在大规模数据、GPU 集群、长上下文或人类偏好约束下的新形态。

- 资料版本限制 (version risk): 课程页、公开视频和配套资料可能在学期中更新；引用时应注明访问日期或资料版本。

- 测量口径限制 (measurement risk): FLOPs、tokens/s、benchmark score、reward 等数字只有在协议完整时可比较。
- 规模外推限制 (scaling risk): 小模型或小 batch 的机制不必然外推到 frontier-scale。
- 数据分布限制 (data risk): 数据来源、过滤和去重策略会改变模型行为, 也会改变 evaluation contamination 风险。
- 对齐风险 (alignment risk): post-training 或 multimodal tuning 的行为变化可能提高 helpfulness, 同时降低校准、透明度或安全边界。

12 复习要点

下面的问题把本章内容压缩为可反复检查的条目。每个条目都应能回到前文的解释、公式、伪代码或例题。

条目	对象	检查动作
条目 1	post-training (后训练)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 2	supervised fine-tuning (SFT)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 3	RLHF	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 4	reward model (奖励模型)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 5	DPO	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 6	PPO	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。

12.1 综合自测

- 我能否不用课程原句, 独立解释本章中心问题?
- 我能否把本章至少一个公式改写成可运行的估算函数?
- 我能否指出一个看似改进但实际转移成本的方法?
- 我能否回到视频和课程资料, 找到支持本章关键论断的位置?
- 我能否设计一个最小 ablation, 验证本章一个 caveat?

13 总结与延伸

13.1 本章小结

- Mid/Post-training: SFT 与 RLHF 的核心不是记忆术语，而是理解对象、变量和机制之间的关系。
- 视频给出讲解顺序，课程资料提供可核对的公式、图、代码或页面结构。
- 复习时应把每个术语连接到至少一个变量、一个实现动作和一个失败模式。

13.2 延伸解释

本章中的延伸解释用于连接课程材料中的概念，例如说明公式里的 proxy、解释系统 tradeoff，或把多个材料片段串成一个完整推理链。若需要严肃引用，应回到课程视频和配套资料核对原始表述。

13.3 练习

- 定义题：用中英双语解释本章任意五个重要术语，并说明它们属于 compute、memory、data、optimization、evaluation 还是 deployment 账本。
- 推导题：选择本章一个公式，逐个解释符号、单位和近似假设，并说明哪个变量最难在真实系统中测量。
- 实现题：把本章伪代码改写成一个最小 Python sanity check，不要求训练大模型，但要输出可检查的中间量。
- 资料题：从“课程资料与回看路径”中选择一个视频时间段，回到原始视频核对讲者例子，并记录是否存在字幕误差。
- 评估题：为本章方法设计一个 ablation 或 benchmark，说明控制变量、数据版本、指标和 failure criteria。
- 边界题：说明本章一个结论在更长 context、更大 batch、更多 GPU 或更复杂数据分布下可能如何失效。
- 综合题：把本章内容和前一章或后一章连接起来，写出一个端到端训练/推理 pipeline 中的依赖关系。
- 批判题：指出一个容易被营销材料夸大的说法，并用本章的资源账本或评估协议反驳它。

16 16 RLVR: 可验证奖励强化学习

第 16 讲：RLVR：可验证奖励强化学习

CS336 Spring 2026 public videos and official course materials

目录

1 本章导读	2
1.1 本章知识路线	2
1.2 结构图	2
2 本章主线	2
2.1 本章要解决的核心问题	2
2.2 从机制到资源账本	3
2.3 从课堂讲解到可复现实验	3
3 术语、符号与问题边界	4
3.1 术语之间的关系	4
4 RLVR 的基本设置	4
4.1 为什么要讨论 RLVR 的基本设置	4
4.2 RLVR 的基本设置的机制	5
4.3 RLVR 的基本设置的目标函数	5
4.4 RLVR 的基本设置的训练流程	5
4.5 边界条件与常见误解	5
4.6 与本章其他概念的关系	5
4.7 怎样检验 RLVR 的基本设置	6
4.8 概念辨析：RLVR 的基本设置	6
4.9 RLVR 的基本设置的小结	6
5 GRPO、优势估计与 length bias	6
5.1 为什么要讨论 GRPO、优势估计与 length bias	6
5.2 GRPO、优势估计与 length bias 的机制	7
5.3 GRPO、优势估计与 length bias 的数学表达	7
5.4 从 GRPO、优势估计与 length bias 到程序	7
5.5 边界条件与常见误解	7

5.6	与本章其他概念的关系	7
5.7	怎样检验 GRPO、优势估计与 length bias	8
5.8	概念辨析：GRPO、优势估计与 length bias	8
5.9	GRPO、优势估计与 length bias 的小结	8
6	从 reasoning gains 到安全边界	8
6.1	为什么要讨论从 reasoning gains 到安全边界	8
6.2	从 reasoning gains 到安全边界的机制	9
6.3	从 reasoning gains 到安全边界的数学表达	9
6.4	从从 reasoning gains 到安全边界到程序	9
6.5	边界条件与常见误解	9
6.6	与本章其他概念的关系	9
6.7	怎样检验从 reasoning gains 到安全边界	10
6.8	概念辨析：从 reasoning gains 到安全边界	10
6.9	从 reasoning gains 到安全边界的小结	10
7	综合讨论：从局部机制到完整系统	10
7.1	RLVR 的基本设置在系统中的位置	10
7.2	GRPO、优势估计与 length bias 在系统中的位置	11
7.3	从 reasoning gains 到安全边界在系统中的位置	11
7.4	把本章知识用于读论文和复现实验	11
8	例题：把概念变成可计算检查	12
8.1	GRPO 的组内优势	12
9	实验设计：从小例子到可复现结论	12
9.1	最小输入	12
9.2	资源账本	12
9.3	单变量消融	12
9.4	失败条件	12
9.5	记录与报告	13
9.6	从小实验到大结论	13
9.7	把本章重新读成一条证据链	13
10	课程资料与回看路径	13
10.1	视频回看路径	13
10.2	课程资料阅读路径	14
11	实践说明、历史位置与风险	14

12 复习要点	15
12.1 综合自测	15
13 总结与延伸	15
13.1 本章小结	15
13.2 延伸解释	16
13.3 练习	16

1 本章导读

本章讨论 **RLVR：可验证奖励强化学习**。CS336 的第一层目标不是把现成大模型当作黑箱使用，而是把 language model 从输入文本、训练目标、模型结构、数据、系统和评估这些部件重新拆开。只有拆开之后，读者才能判断一个设计选择究竟改变了什么。设想数学题或代码题可以自动判定正确性。RLVR (reinforcement learning from verifiable rewards) 利用可验证奖励推动 reasoning，但也引入 verifier overfitting、length bias 和推理成本上升。本章对应 CS336 Spring 2026 第 16 个公开视频，并结合课程页提供的可解析资料。视频和课程页给出事实边界；正文把这些材料改写为可自学的教材章节。阅读本章时，先理解问题为什么存在，再看定义、公式和实现形态，随后通过例题和练习检查自己是否能独立复现这条 reasoning chain。



图 1: 第 16 讲的课程图像锚点。

1.1 本章知识路线

- 本章首先说明 RLVR：可验证奖励强化学习为什么是 language modeling pipeline 中不可跳过的一环。
- 其次定义本章反复使用的术语，并说明这些术语对应的计算对象或系统对象。
- 随后用公式和伪代码表达主要机制，让概念能够落到可计算的变量上。
- 最后给出例题、风险讨论和练习，便于读者检查自己是否真正掌握了机制。

1.2 结构图

下面的图用于帮助读者定位本章的核心结构。先看概念之间的依赖，再回到正文中的公式、伪代码和例题。

2 本章主线

RLVR：可验证奖励强化学习的主线是解释 base model 之后的行为塑形。课程关心的不是单个 post-training 名词，而是示范、偏好、奖励和评估怎样共同约束输出。

2.1 本章要解决的核心问题

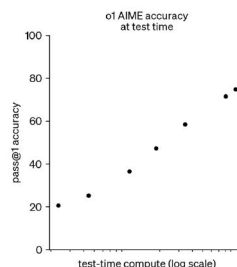
本讲讲 RLVR：在数学、代码、可验证任务中，reward 可以由 verifier 给出，而不是依赖主观偏好标注。读完这一段后，应能用一两句话说明 RLVR：可验证奖励强化学习要解决的瓶颈，并指出这个瓶颈发生在数据、

The class thus far

Pre-training + RLHF gets you to ~ GPT3.5... **Today we'll get all the way to o1/r1**

OMG! What Will Happen
When A.I. Makes BuzzFeed
Quizzes?
The site announced plans to use tools from the creator of
ChatGPT to create content. Former quiz writers aren't surprised.

ChatGPT: Optimizing
Language Models
for Dialogue



o1

图 2: 官方材料图页: RLVR: 可验证奖励强化学习的课程页/PDF/script 图像锚点。

Lecture 16

POST-TRAINING 2

REINFORCEMENT LEARNING FROM VERIFIABLE REWARDS

CS336

图 3: 官方材料图页: RLVR: 可验证奖励强化学习的课程页/PDF/script 图像锚点。

模型、优化、系统还是评估层。

2.2 从机制到资源账本

GRPO 等方法用组内相对 reward/advantage 训练 reasoning policy, 减少 value function 复杂度, 但也带来 length bias 和 baseline 合法性问题。随后把机制写成账本: 输入规模是什么, 主要状态是什么, 成本或质量指标怎样随变量改变。这个账本让后面的公式不再只是符号。

2.3 从课堂讲解到可复现实验

RLVR 的风险是过拟合 verifier、奖励稀疏、泛化不足和推理长度成本上升。评估必须同时看准确率、长度、失败类型和安全性。最后把课堂讲解转成可复现实验: 固定协议, 改变一个变量, 记录中间量, 并解释结果为何支持或反驳原来的机制判断。

3 术语、符号与问题边界

本章的术语横跨模型、数据和系统。读者需要先知道每个词指向的对象：有些词描述数学目标，有些词描述张量形状，有些词描述硬件瓶颈，还有一些词描述评估协议。术语边界不清，后面的公式和实现就会变成空洞的缩写。

#	术语	阅读要求
1	RLVR (reinforcement learning from verifiable rewards)	说明它约束的是模型行为、优化目标还是评估协议，并指出一个奖励或偏好失真的例子。
2	verifiable reward (可验证奖励)	说明它约束的是模型行为、优化目标还是评估协议，并指出一个奖励或偏好失真的例子。
3	GRPO	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
4	reasoning model (推理模型)	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
5	advantage (优势函数)	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。

这些术语之间有依赖关系。例如 tokenization 改变 sequence length, sequence length 又改变 attention FLOPs、KV cache 和 evaluation token budget; parallelism 改变显存占用，也改变通信瓶颈和故障恢复方式。

3.1 术语之间的关系

在本章中，RLVR (reinforcement learning from verifiable rewards) 给出问题入口，verifiable reward (可验证奖励) 描述主要机制，GRPO 则把机制连接到可测量的结果。读教材时应当沿着这个链条追问：输入是什么，中间状态怎样改变，输出指标为什么随之变化。课程材料还会反复出现 grpo、data、rlhf、rewards、reward、reasoning 等词。它们不是一组同义标签，而是提示本章的不同层次：有的指对象，有的指操作，有的指约束或指标。因此，术语表不是背诵清单，而是后文阅读的索引。遇到一个术语时，先定位它属于 representation、optimization、systems、data 还是 evaluation，再判断它改变哪一项账本。

4 RLVR 的基本设置

现在进入 RLVR 的基本设置。这一节讨论 base model 之外的行为塑形：示范、偏好、reward 或 verifier 怎样把模型输出推向课程定义的目标行为。

4.1 为什么要讨论 RLVR 的基本设置

视频把本讲定位为第二节 post-training，聚焦 RLVR 和 reasoning 方向的及时进展。RLVR 不依赖主观偏好 reward，而是利用数学题、代码测试、形式验证等可判定信号。这使 reward 更客观，但任务覆盖范围更窄，并且容易过拟合到 verifier。后训练部分的核心问题是行为控制。模型已经学到语言分布，但还需要通过示范、偏好或可验证奖励，把输出推向可用、可评估、可约束的行为。

4.2 RLVR 的基本设置的机制

视频把本讲定位为第二节 post-training，聚焦 RLVR 和 reasoning 方向的及时进展。在 RLVR 的基本设置中，核心对象是行为、奖励和偏好状态：后训练方法并不直接制造知识，而是通过目标和反馈改变模型选择输出的方式。RLVR 不依赖主观偏好 reward，而是利用数学题、代码测试、形式验证等可判定信号。因而同一个方法在不同规模下可能改变不同的变量，尤其是 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。比较方法时，必须同时给出问题规模和实验条件。这使 reward 更客观，但任务覆盖范围更窄，并且容易过拟合到 verifier。可检验性来自训练日志、reward 分布、KL/length 统计和行为评估；只看最终回答样例无法判断更新方向。因此，RLVR 的基本设置的学习目标是建立一条行为更新链：读者应能区分示范、偏好、奖励和约束分别给模型提供了什么信号。

4.3 RLVR 的基本设置的目标函数

$$R(x, y) = \mathbf{1}\{\text{verify}(x, y) = \text{correct}\}$$

符号说明： x 是问题， y 是模型输出，verify 是外部可验证器；奖励常是稀疏二值信号。这个关系式说明 RLVR 的基本设置如何把行为偏好写入优化目标。关键不是符号复杂度，而是 reward、reference policy、采样分布和约束强度如何共同决定更新方向。在后训练实验中，reward model score、verifier pass rate、回答长度和人工偏好都是 proxy。它们会被采样策略和格式约束影响，因此不能直接等同于真实有用性。

4.4 RLVR 的基本设置的训练流程

把 RLVR 的基本设置写成训练流程时，要区分采样、打分、构造 loss 和更新参数。下面的伪代码保留这些边界。

```
response = policy.generate(problem)
reward = verifier(problem, response)
update_policy_with_rl(reward)
```

阅读这段训练流程时，重点区分采样、打分和参数更新。只有同时记录 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率，才能判断行为改善来自奖励信号、采样分布还是约束强度。

4.5 边界条件与常见误解

- 可验证不等于完整；很多真实任务没有可靠 verifier。
- 稀疏 reward 会让探索和 credit assignment 变难。
- 后训练指标可能被格式、长度或 verifier 偏差驱动，不能直接等同于真实能力提升。
- 偏好优化可能改善目标行为，也可能牺牲多样性、校准性或某些下游能力。

4.6 与本章其他概念的关系

RLVR 的基本设置连接本章的训练目标和输出行为：术语表区分示范、偏好与奖励，公式说明优化方向，伪代码展示采样和更新，例题检查行为约束的边界。对这一类主题，最常见的误解是只列方法名。更有用的做法是比较每个方法改变的成本项、依赖的假设和可能的失败模式。

4.7 怎样检验 RLVR 的基本设置

检验 RLVR 的基本设置时，先把抽象说法落到行为、奖励和偏好状态上。与 RLVR、verifiable、reward、math、code 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： $\backslash(x\backslash)$ 是问题， $\backslash(y\backslash)$ 是模型输出，verify 是外部可验证器；奖励常是稀疏二值信号。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。后训练消融通常一次只改变示范数据、reward scale、KL 约束、采样温度或 verifier。若训练目标和解码策略同时变化，行为差异就很难归因。最后检查失败条件。工程判断往往在反例中变清楚：可验证不等于完整；很多真实任务没有可靠 verifier；稀疏 reward 会让探索和 credit assignment 变难。复习时可以把这些限制改写成反例测试，观察它们怎样改变 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。把实验结论放回 RLVR：可验证奖励强化学习时，要说明 RLVR 的基本设置改变的是目标函数、采样分布、反馈质量还是行为约束。能区分这些来源，才说明读者真正理解后训练。

4.8 概念辨析：RLVR 的基本设置

RLVR、verifiable、reward、math、code 常一起出现在后训练讨论中，但它们分别指数据、反馈、目标函数或行为指标。读者应先判断每个词给优化过程提供什么信号。其中，RLVR (reinforcement learning from verifiable rewards) 使用可验证奖励训练推理能力；decode (解码) 逐 token 生成，常受 memory bandwidth 和 KV cache 影响。区分这些词时，可以先问它们是否改变 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕 RLVR 的基本设置复习时，可以构造几条 prompt-response：明确奖励或偏好规则，观察同一模型在更新前后的行为差异。

4.9 RLVR 的基本设置的小结

- 讨论 RLVR 的基本设置时，应同时报告问题规模、实验设置和主要观测变量，尤其是 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。
- 如果方法声称行为更好，要说明示范、偏好、reward 或 verifier 哪一项提供了信号，并检查 reward hacking、length bias 和能力退化。
- 后训练结论需要把训练目标、采样策略和评估协议分开报告，否则无法判断改进来自哪里。

5 GRPO、优势估计与 length bias

现在进入 GRPO、优势估计与 length bias。这一节讨论 base model 之外的行为塑形：示范、偏好、reward 或 verifier 怎样把模型输出推向课程定义的目标行为。

5.1 为什么要讨论 GRPO、优势估计与 length bias

字幕关键词显示本讲反复讨论 GRPO、reward、advantage、length 和 DeepSeek。group-relative 方法用同一 prompt 的多条采样构造相对优势，避免单独训练 value model 或降低复杂度。reasoning RL 需要处理长输出：更长 chain-of-thought 可能提高正确率，也可能制造 length bias。GRPO、优势估计与 length bias 把 RLVR：可验证奖励强化学习连接到行为目标。接下来要看的不是 reward 名称，而是反馈信号、采样分布和约束项怎样改变模型更新。

5.2 GRPO、优势估计与 length bias 的机制

字幕关键词显示本讲反复讨论 GRPO、reward、advantage、length 和 DeepSeek。在 GRPO、优势估计与 length bias 中，核心对象是行为、奖励和偏好状态：后训练方法并不直接制造知识，而是通过目标和反馈改变模型选择输出的方式。group-relative 方法用同一 prompt 的多条采样构造相对优势，避免单独训练 value model 或降低复杂度。因而同一个方法在不同规模下可能改变不同的变量，尤其是 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。比较方法时，必须同时给出问题规模和实验条件。reasoning RL 需要处理长输出：更长 chain-of-thought 可能提高正确率，也可能制造 length bias。可检验性来自训练日志、reward 分布、KL/length 统计和行为评估；只看最终回答样例无法判断更新方向。因此，GRPO、优势估计与 length bias 的学习目标是建立一条行为更新链：读者应能区分示范、偏好、奖励和约束分别给模型提供了什么信号。

5.3 GRPO、优势估计与 length bias 的数学表达

$$A_i = \frac{R_i - \text{mean}(R_{1:G})}{\text{std}(R_{1:G}) + \epsilon}$$

符号说明：同一问题采样 G 个回答， R_i 是第 i 个回答奖励， A_i 是组内标准化优势。这个关系式说明 GRPO、优势估计与 length bias 如何把行为偏好写入优化目标。关键不是符号复杂度，而是 reward、reference policy、采样分布和约束强度如何共同决定更新方向。在后训练实验中，reward model score、verifier pass rate、回答长度和人工偏好都是 proxy。它们会被采样策略和格式约束影响，因此不能直接等同于真实有用性。

5.4 从 GRPO、优势估计与 length bias 到程序

把 GRPO、优势估计与 length bias 写成训练流程时，要区分采样、打分、构造 loss 和更新参数。下面的伪代码保留这些边界。

```
samples = [policy.generate(x) for _ in range(G)]
rewards = [verify(x, y) for y in samples]
advantages = normalize_within_group(rewards)
policy_gradient_update(samples, advantages)
```

阅读这段训练流程时，重点区分采样、打分和参数更新。只有同时记录 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率，才能判断行为改善来自奖励信号、采样分布还是约束强度。

5.5 边界条件与常见误解

- advantage normalization 会改变 reward scale，影响训练稳定性。
- 如果 verifier 奖励格式技巧，模型可能学习投机输出。
- 后训练指标可能被格式、长度或 verifier 偏差驱动，不能直接等同于真实能力提升。
- 偏好优化可能改善目标行为，也可能牺牲多样性、校准性或某些下游能力。

5.6 与本章其他概念的关系

GRPO、优势估计与 length bias 连接本章的训练目标和输出行为：术语表区分示范、偏好与奖励，公式说明优化方向，伪代码展示采样和更新，例题检查行为约束的边界。对这一类主题，最常见的误解是只列方法名。更有用的做法是比较每个方法改变的成本项、依赖的假设和可能的失败模式。

5.7 怎样检验 GRPO、优势估计与 length bias

检验 GRPO、优势估计与 length bias 时,先把抽象说法落到行为、奖励和偏好状态上。与 GRPO、advantage、DeepSeek、length、thinking 相关的对象必须能被构造、打印、计数或评分;否则实验只能得到描述性观察,不能得到可复现结论。一个合适的小实验应当保留本节机制,同时让 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率中至少一个量发生可解释变化。输入规模不必逼真;关键是读者能手算或打印关键中间量,并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明:同一问题采样 $\backslash(G\backslash)$ 个回答, $\backslash(R_i\backslash)$ 是第 $\backslash(i\backslash)$ 个回答奖励, $\backslash(A_i\backslash)$ 是组内标准化优势。如果数量只能间接观测,就必须说明使用了什么 proxy;如果使用 benchmark 或 reward,也要说明协议和随机性。后训练消融通常一次只改变示范数据、reward scale、KL 约束、采样温度或 verifier。若训练目标和解码策略同时变化,行为差异就很难归因。最后检查失败条件。工程判断往往在反例中变清楚: advantage normalization 会改变 reward scale,影响训练稳定性;如果 verifier 奖励格式技巧,模型可能学习投机输出。复习时可以把这些限制改写成反例测试,观察它们怎样改变 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。把实验结论放回 RLVR: 可验证奖励强化学习时,要说明 GRPO、优势估计与 length bias 改变的是目标函数、采样分布、反馈质量还是行为约束。能区分这些来源,才说明读者真正理解后训练。

5.8 概念辨析: GRPO、优势估计与 length bias

GRPO、advantage、DeepSeek、length、thinking 常一起出现在后训练讨论中,但它们分别指数据、反馈、目标函数或行为指标。读者应先判断每个词给优化过程提供什么信号。其中,GRPO 使用组内相对奖励/优势来训练 reasoning policy。区分这些词时,可以先问它们是否改变 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。一个术语如果不能对应到变量、状态、代码路径或评估协议,就还没有形成可操作含义。围绕 GRPO、优势估计与 length bias 复习时,可以构造几条 prompt-response: 明确奖励或偏好规则,观察同一模型在更新前后的行为差异。

5.9 GRPO、优势估计与 length bias 的小结

- 讨论 GRPO、优势估计与 length bias 时,应同时报告问题规模、实验设置和主要观测变量,尤其是 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。
- 如果方法声称行为更好,要说明示范、偏好、reward 或 verifier 哪一项提供了信号,并检查 reward hacking、length bias 和能力退化。
- 后训练结论需要把训练目标、采样策略和评估协议分开报告,否则无法判断改进来自哪里。

6 从 reasoning gains 到安全边界

现在进入从 reasoning gains 到安全边界。这一节讨论 base model 之外的行为塑形: 示范、偏好、reward 或 verifier 怎样把模型输出推向课程定义的目标行为。

6.1 为什么要讨论从 reasoning gains 到安全边界

RLVR 的吸引力在于把模型推向更强 reasoning,但训练过程也可能增强规避、工具滥用或 reward hacking。可验证任务上的提升不自动泛化到开放式推理、事实性或安全对话。评估 RLVR 模型时要同时看 pass rate、过程长度、失败类型、泛化题集和安全行为。从 reasoning gains 到安全边界把 RLVR: 可验证奖励强化学习连接到行为目标。接下来要看的不是 reward 名称,而是反馈信号、采样分布和约束项怎样改变模型更新。

6.2 从 reasoning gains 到安全边界的机制

RLVR 的吸引力在于把模型推向更强 reasoning, 但训练过程也可能增强规避、工具滥用或 reward hacking。在从 reasoning gains 到安全边界中, 核心对象是行为、奖励和偏好状态: 后训练方法并不直接制造知识, 而是通过目标和反馈改变模型选择输出的方式。可验证任务上的提升不自动泛化到开放式推理、事实性或安全对话。因而同一个方法在不同规模下可能改变不同的变量, 尤其是 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。比较方法时, 必须同时给出问题规模和实验条件。评估 RLVR 模型时要同时看 pass rate、过程长度、失败类型、泛化题集和安全行为。可检验性来自训练日志、reward 分布、KL/length 统计和行为评估; 只看最终回答样例无法判断更新方向。因此, 从 reasoning gains 到安全边界的学习目标是建立一条行为更新链: 读者应能区分示范、偏好、奖励和约束分别给模型提供了什么信号。

6.3 从 reasoning gains 到安全边界的数学表达

$$\max_{\theta} \mathbb{E}_{y \sim \pi_{\theta}(\cdot|x)} [R(x, y)] - \beta \text{KL}(\pi_{\theta} \parallel \pi_{\text{ref}})$$

符号说明: KL 项限制策略偏离参考模型, β 是约束强度。这个关系式说明从 reasoning gains 到安全边界如何把行为偏好写入优化目标。关键不是符号复杂度, 而是 reward、reference policy、采样分布和约束强度如何共同决定更新方向。在后训练实验中, reward model score、verifier pass rate、回答长度和人工偏好都是 proxy。它们会被采样策略和格式约束影响, 因此不能直接等同于真实有用性。

6.4 从 reasoning gains 到安全边界到程序

把从 reasoning gains 到安全边界写成训练流程时, 要区分采样、打分、构造 loss 和更新参数。下面的伪代码保留这些边界。

```
monitor = [accuracy, response_length, verifier_failures, safety_eval]
stop_if_reward_improves_but_generalization_drops(monitor)
```

阅读这段训练流程时, 重点区分采样、打分和参数更新。只有同时记录 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率, 才能判断行为改善来自奖励信号、采样分布还是约束强度。

6.5 边界条件与常见误解

- 训练 reward 和真实目标之间的 gap 是核心风险。
- reasoning 长度增加会显著提高推理成本。
- 后训练指标可能被格式、长度或 verifier 偏差驱动, 不能直接等同于真实能力提升。
- 偏好优化可能改善目标行为, 也可能牺牲多样性、校准性或某些下游能力。

6.6 与本章其他概念的关系

从 reasoning gains 到安全边界连接本章的训练目标和输出行为: 术语表区分示范、偏好与奖励, 公式说明优化方向, 伪代码展示采样和更新, 例题检查行为约束的边界。对这一类主题, 最常见的误解是只列方法名。更有用的做法是比较每个方法改变的成本项、依赖的假设和可能的失败模式。

6.7 怎样检验从 reasoning gains 到安全边界

检验从 reasoning gains 到安全边界时，先把抽象说法落到行为、奖励和偏好状态上。与 reasoning、KL、safety、generalization、reward hacking 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明：KL 项限制策略偏离参考模型， β 是约束强度。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。后训练消融通常一次只改变示范数据、reward scale、KL 约束、采样温度或 verifier。若训练目标和解码策略同时变化，行为差异就很难归因。最后检查失败条件。工程判断往往在反例中变清楚：训练 reward 和真实目标之间的 gap 是核心风险；reasoning 长度增加会显著提高推理成本。复习时可以把这些限制改写成反例测试，观察它们怎样改变 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。把实验结论放回 RLVR：可验证奖励强化学习时，要说明从 reasoning gains 到安全边界改变的是目标函数、采样分布、反馈质量还是行为约束。能区分这些来源，才说明读者真正理解后训练。

6.8 概念辨析：从 reasoning gains 到安全边界

reasoning、KL、safety、generalization、reward hacking 常一起出现在后训练讨论中，但它们分别指数据、反馈、目标函数或行为指标。读者应先判断每个词给优化过程提供什么信号。区分这些词时，可以先问它们是否改变 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕从 reasoning gains 到安全边界复习时，可以构造几条 prompt-response：明确奖励或偏好规则，观察同一模型在更新前后的行为差异。

6.9 从 reasoning gains 到安全边界的小结

- 讨论从 reasoning gains 到安全边界时，应同时报告问题规模、实验设置和主要观测变量，尤其是 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。
- 如果方法声称行为更好，要说明示范、偏好、reward 或 verifier 哪一项提供了信号，并检查 reward hacking、length bias 和能力退化。
- 后训练结论需要把训练目标、采样策略和评估协议分开报告，否则无法判断改进来自哪里。

7 综合讨论：从局部机制到完整系统

本章的主题是 RLVR：可验证奖励强化学习，但它不应被读成几个相邻知识点的拼接。RLVR 的基本设置、GRPO、优势估计与 length bias、从 reasoning gains 到安全边界共同回答同一个问题：语言模型系统中哪些对象可以被定义，哪些成本可以被估算，哪些结论必须通过实验或评估来确认。这些对象包括 RLVR (reinforcement learning from verifiable rewards)、verifiable reward (可验证奖励)、GRPO、reasoning model (推理模型)、advantage (优势函数)。读者可以把它们看成一组接口：有的接口连接文本和 token，有的连接模型结构和张量计算，有的连接数据样本和训练目标，有的连接离线评估和在线服务。接口一旦改变，后面的成本、错误类型和优化空间也会随之改变。

7.1 RLVR 的基本设置在系统中的位置

RLVR 的基本设置首先提供一个局部解释：视频把本讲定位为第二节 post-training，聚焦 RLVR 和 reasoning 方向的及时进展。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它

的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，RLVR 的基本设置会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，RLVR 的基本设置不能脱离边界条件理解。一个关键 caveat 是：可验证不等于完整；很多真实任务没有可靠 verifier。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.2 GRPO、优势估计与 length bias 在系统中的位置

GRPO、优势估计与 length bias 首先提供一个局部解释：字幕关键词显示本讲反复讨论 GRPO、reward、advantage、length 和 DeepSeek。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，GRPO、优势估计与 length bias 会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，GRPO、优势估计与 length bias 不能脱离边界条件理解。一个关键 caveat 是：advantage normalization 会改变 reward scale，影响训练稳定性。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.3 从 reasoning gains 到安全边界在系统中的位置

从 reasoning gains 到安全边界首先提供一个局部解释：RLVR 的吸引力在于把模型推向更强 reasoning，但训练过程也可能增强规避、工具滥用或 reward hacking。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，从 reasoning gains 到安全边界会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，从 reasoning gains 到安全边界不能脱离边界条件理解。一个关键 caveat 是：训练 reward 和真实目标之间的 gap 是核心风险。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.4 把本章知识用于读论文和复现实验

读相关论文时，可以把本章变成一个检查表。第一，论文是否清楚定义了输入规模、模型规模、数据来源和评估协议；第二，论文声称的改进落在哪个变量上；第三，作者是否报告了会让结论失效的设置；第四，实验是否能分离模型结构、数据质量、优化设置和系统实现的影响。复现实验时，也应避免直接追求大规模。先用最小程序复现一个公式、一个伪代码分支或一个 profile 现象，再扩大输入规模。这样做的原因很简单：如果小规模程序无法解释中间量，大规模训练只会把错误隐藏在日志和随机波动里。把这些检查合在一起，本章给出的不是一个固定答案，而是一种阅读 CS336 的方法：每个模型机制都要落到变量，每个变量都要落到测量，每个测量都要说明协议，每个协议都要承认边界。因此，本章中的任何一句结论都可以被改写成一个可引用的完整句式：在给定数据、模型、硬件和评估协议下，某个机制改变了某个变量，并通过某个观测指标表现出来。这样的句子比单独背诵术语更长，但它包含了自学、复现和引用时真正需要的信息。把本章用于自学时，可以按三遍阅读。第一遍只追踪对象和定义，确认每个术语到底指向文本、token、张量、样本、请求还是评估记录。第二遍追踪公式和伪代码，确认每一步改变了哪些状态。第三遍追踪实验和 caveat，确认哪些结论只在特定规模、数据或硬件条件下成立。把本章用于复习时，则应反过来操作：先从一个失败案例或异常指标出发，再回到相应机制。loss 曲线异常可能指向数据、优化或模型容量；吞吐异常可能指向 kernel、通信或调度；

benchmark 异常可能指向 contamination、prompt protocol 或采样设置。能够从异常反推机制，是掌握本章的实用标准。

8 例题：把概念变成可计算检查

8.1 GRPO 的组内优势

同一数学题采样 8 个回答，用 verifier 给出 0/1 奖励。

- 计算组内平均和标准差，把每个回答的 reward 标准化成 advantage。
- 正确回答 advantage 为正，错误回答 advantage 为负。
- 如果所有回答都错或都对，学习信号会变弱。

结论。 RLVR 的训练信号质量取决于采样多样性、verifier 可靠性和奖励分布。这个例题的目的不是替代完整工程实现，而是给出一种最小推理形式：把问题转成变量，写出近似公式或伪代码，再说明近似何时失效。

9 实验设计：从小例子到可复现结论

学习 RLVR：可验证奖励强化学习不能停在概念层。一个教材化结论至少需要四件事：可构造的输入、可观测的中间量、可比较的指标，以及清楚的失败条件。下面的实验设计不是要求训练大模型，而是要求读者用小规模程序复现本章机制。

9.1 最小输入

先构造只触发本章核心机制的小输入。例如 tokenizer 章节可以用几个包含 rare words 和 Unicode 字符的字符串；GPU 章节可以用一个 elementwise kernel 和一个 GEMM；data 章节可以用十几篇近重复文档。小输入的价值在于它让错误可见。

9.2 资源账本

对同一输入写下 FLOPs、bytes moved、显存峰值、通信量或样本数量的估算。估算不需要一开始就精确，但必须说明单位和近似假设。随后用 profiler、benchmark、validation loss 或人工检查去验证估算是否同量级。

9.3 单变量消融

一次只改变一个因素，例如 vocabulary size、head 数、batch size、filter threshold、reward scale、decoding temperature 或 GPU 数量。若多个变量同时变化，实验只能说明系统变了，不能说明机制是什么。

9.4 失败条件

最后要刻意触发失败：极端 context length、错误 dtype、过小 batch、污染 benchmark、低质量数据或不可靠 verifier。失败条件常常比成功曲线更能说明一个方法的边界。

9.5 记录与报告

实验记录应能让另一个读者复现同一结论。最少要写清楚输入构造、代码版本、随机种子、硬件、batch shape、数据版本、评价指标和异常处理方式。报告结论时不要只写“更快”或“更好”，而要说明快在哪里、好在哪个指标上，以及是否牺牲了内存、稳定性、数据质量或可解释性。常见报告错误是把局部现象写成普遍规律。例如一次小模型实验里的 loss 改善，不等于更大模型或不同数据分布上也会改善；一次 kernel benchmark 的吞吐提升，不等于端到端 serving latency 一定降低。教材中的实验报告应始终把结论限定在可复现条件内。

9.6 从小实验到大结论

小实验的作用不是替代课程中的完整训练或系统实现，而是建立一条可检查的推理链。先在小输入上确认变量方向，再在中等规模上确认数量级，最后才讨论是否能外推到更大模型、更多 token 或更复杂 workload。缺少中间层级时，大结论通常只是在复述直觉。因此，实验报告最好同时包含正例和反例：正例说明机制在受控条件下怎样工作，反例说明条件稍变时哪里失效。对 CS336 这样的课程，反例尤其重要，因为 language model 的错误常来自多个层面叠加：数据分布、优化稳定性、硬件瓶颈、评估协议和服务负载都可能同时改变观察结果。还要记录资料版本与复现边界。公开视频、课程页、配套代码和 PDF 可能在不同时间更新；复现实验应写明使用的材料版本，并说明哪些结论来自课程事实，哪些是为了帮助理解而加入的解释性推导。这样才能把学习笔记变成可检查的技术记录。若复现实验与课程结论不一致，首先检查输入、版本和评估协议，而不是立即修改模型假设。

9.7 把本章重新读成一条证据链

完成小实验之后，可以把 RLVR：可验证奖励强化学习重新读成一条证据链：先用“RLVR 的基本设置”确定问题入口，再用“GRPO、优势估计与 length bias”解释主要机制，最后用“从 reasoning gains 到安全边界”检查边界或外推条件。这种读法比按页背诵更接近教材训练，因为它要求每个结论都能说明前提、变量和证据。本章反复使用的术语包括 RLVR (reinforcement learning from verifiable rewards)、verifiable reward (可验证奖励)、GRPO、reasoning model (推理模型)、advantage (优势函数)。复习时不要只写定义，而要为每个术语补上一句操作性说明：它在课程代码、公式、数据记录或评估协议中对应什么对象；如果这个对象被删除、替换或缩放，哪一个观测量会变化。最后，用一个反例结束复习。反例可以是资源估算不准、数据污染、reward 失真、benchmark 解析失败、长上下文内存爆炸或 scaling 外推失效。能解释反例的章节，才真正具备自学和引用价值；不能解释反例的章节，只是把课程材料换了一种排版。引用本章结论时，还应保留来源边界：哪些说法直接来自视频、课程页、slides 或代码，哪些是为了串联教材叙述而加入的解释性推导。这个边界不会削弱结论，反而让读者知道何处可以复核，何处需要在自己的实验中重新验证。如果后续课程材料更新，也可以沿着同一证据链替换来源，而不必重写整章结构。

10 课程资料与回看路径

教材正文已经把主要概念、公式和实现逻辑组织成连续叙述；本节只提供回到课程材料的阅读路径。读者复习时可以先完成前文练习，再按下面的时间段或资料组核对细节。

10.1 视频回看路径

时间	关键词	复习任务
----	-----	------

00:00:05-00:06:23	going, very, we're, that's, reinforcement, openai, we've	回看定义、例子、推导或 caveat 在该段中的位置。
00:18:09-00:24:19	grpo, standard, it's, well, deviation, baseline, going	回看定义、例子、推导或 caveat 在该段中的位置。
00:37:05-00:43:31	very, think, quite, it's, reward, data, talk	回看定义、例子、推导或 caveat 在该段中的位置。
00:50:04-00:56:32	very, training, then, your, inference, think, qwen	回看定义、例子、推导或 caveat 在该段中的位置。
01:09:42-01:15:30	long, then, it's, yeah., training, very, data	回看定义、例子、推导或 caveat 在该段中的位置。

这些时间段用于定位视频中的讲解顺序。严肃引用时，应回到视频片段确认讲者表述、板书或幻灯片内容。回看视频时，不必逐字抄录字幕。更有效的方法是把每个时间段判定为定义、例子、推导、代码解释或 caveat，再把它放回本章对应小节。

10.2 课程资料阅读路径

资料组	标题/页块	关键词
official_01	Lecture 16 P OST- TRAININ G 2 R E INF O R C E M E NT L E A R NIN G F R O M V E R I F ...	practice, policy, reward, grpo, level, rewards, implementation
official_03	Interesting phenomena (?) Longer CoTs during training ‘aha’ moment?	reasoning, rlhf, grpo, language, non-verifiable, data, step
official_04	How well does R1 work? It’ s pretty good (but you knew that)	kimi, distillation, long, policy, curation, reward, length
official_05	Kimi’ s RL setup in detail	data, results, qwen, kimi, scaling, interesting, comes
official_07	Agent RL	agent, recap, overoptimization, problem, narrow, domains, solution

课程资料通常给出更稳定的标题、公式、图或代码结构；视频则补充动机和口头解释。两者合起来构成本章的事实边界。阅读资料时，可以把每个资料组拆成三个对象：一个定义，一个公式或伪代码动作，一个边界条件。这样比逐字翻译页面或脚本更容易形成可用知识。

11 实践说明、历史位置与风险

RLVR：可验证奖励强化学习在 CS336 的课程结构中连接基础机制和规模化问题。基础机制解释方法为什么成立；规模化问题检验它在更大模型、更长上下文、更复杂数据或真实 serving workload 下是否仍成立。历史位置不等于模型年表。更重要的是识别问题如何迁移：早期方法解决小规模建模问题，现代方法常常解决同一问题在大规模数据、GPU 集群、长上下文或人类偏好约束下的新形态。

- 资料版本限制 (version risk)：课程页、公开视频和配套资料可能在学期中更新；引用时应注明访问日期或资料版本。

- 测量口径限制 (measurement risk): FLOPs、tokens/s、benchmark score、reward 等数字只有在协议完整时可比较。
- 规模外推限制 (scaling risk): 小模型或小 batch 的机制不必然外推到 frontier-scale。
- 数据分布限制 (data risk): 数据来源、过滤和去重策略会改变模型行为, 也会改变 evaluation contamination 风险。
- 对齐风险 (alignment risk): post-training 或 multimodal tuning 的行为变化可能提高 helpfulness, 同时降低校准、透明度或安全边界。

12 复习要点

下面的问题把本章内容压缩为可反复检查的条目。每个条目都应能回到前文的解释、公式、伪代码或例题。

条目	对象	检查动作
条目 1	RLVR (reinforcement learning from verifiable rewards)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 2	verifiable reward (可验证奖励)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 3	GRPO	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 4	reasoning model (推理模型)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 5	advantage (优势函数)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。

12.1 综合自测

- 我能否不用课程原句，独立解释本章中心问题？
- 我能否把本章至少一个公式改写成可运行的估算函数？
- 我能否指出一个看似改进但实际转移成本的方法？
- 我能否回到视频和课程资料，找到支持本章关键论断的位置？
- 我能否设计一个最小 ablation，验证本章一个 caveat？

13 总结与延伸

13.1 本章小结

- RLVR：可验证奖励强化学习的核心不是记忆术语，而是理解对象、变量和机制之间的关系。

- 视频给出讲解顺序，课程资料提供可核对的公式、图、代码或页面结构。
- 复习时应把每个术语连接到至少一个变量、一个实现动作和一个失败模式。

13.2 延伸解释

本章中的延伸解释用于连接课程材料中的概念，例如说明公式里的 proxy、解释系统 tradeoff，或把多个材料片段串成一个完整推理链。若需要严肃引用，应回到课程视频和配套资料核对原始表述。

13.3 练习

- 定义题：用中英双语解释本章任意五个重要术语，并说明它们属于 compute、memory、data、optimization、evaluation 还是 deployment 账本。
- 推导题：选择本章一个公式，逐个解释符号、单位和近似假设，并说明哪个变量最难在真实系统中测量。
- 实现题：把本章伪代码改写成一个最小 Python sanity check，不要求训练大模型，但要输出可检查的中间量。
- 资料题：从“课程资料与回看路径”中选择一个视频时间段，回到原始视频核对讲者例子，并记录是否存在字幕误差。
- 评估题：为本章方法设计一个 ablation 或 benchmark，说明控制变量、数据版本、指标和 failure criteria。
- 边界题：说明本章一个结论在更长 context、更大 batch、更多 GPU 或更复杂数据分布下可能如何失效。
- 综合题：把本章内容和前一章或后一章连接起来，写出一个端到端训练/推理 pipeline 中的依赖关系。
- 批判题：指出一个容易被营销材料夸大的说法，并用本章的资源账本或评估协议反驳它。

17 17 Alignment 与 multimodality

第 17 讲: Alignment 与 multimodality

CS336 Spring 2026 public videos and official course materials

目录

1 本章导读	2
1.1 本章知识路线	2
1.2 结构图	2
2 本章主线	2
2.1 本章要解决的核心问题	2
2.2 从机制到资源账本	3
2.3 从课堂讲解到可复现实验	3
3 术语、符号与问题边界	4
3.1 术语之间的关系	4
4 为什么 CS336 需要 multimodality	4
4.1 课程目标与问题边界	4
4.2 为什么 CS336 需要 multimodality 的机制	5
4.3 资源、效率与效果的关系	5
4.4 把课程目标写成检查流程	5
4.5 边界条件与常见误解	5
4.6 与本章其他概念的关系	5
4.7 怎样检验这一课程目标	6
4.8 概念辨析: 为什么 CS336 需要 multimodality	6
4.9 为什么 CS336 需要 multimodality 的小结	6
5 CLIP 式对比学习与表示对齐	6
5.1 为什么要讨论 CLIP 式对比学习与表示对齐	6
5.2 CLIP 式对比学习与表示对齐的机制	7
5.3 CLIP 式对比学习与表示对齐的数学表达	7
5.4 从 CLIP 式对比学习与表示对齐到程序	7
5.5 边界条件与常见误解	7

5.6	与本章其他概念的关系	7
5.7	怎样检验 CLIP 式对比学习与表示对齐	8
5.8	概念辨析: CLIP 式对比学习与表示对齐	8
5.9	CLIP 式对比学习与表示对齐的小结	8
6	VLM instruction tuning 与评估	8
6.1	为什么要讨论 VLM instruction tuning 与评估	8
6.2	VLM instruction tuning 与评估的机制	9
6.3	VLM instruction tuning 与评估的数学表达	9
6.4	从 VLM instruction tuning 与评估到程序	9
6.5	边界条件与常见误解	9
6.6	与本章其他概念的关系	9
6.7	怎样检验 VLM instruction tuning 与评估	10
6.8	概念辨析: VLM instruction tuning 与评估	10
6.9	VLM instruction tuning 与评估的小结	10
7	综合讨论: 从局部机制到完整系统	10
7.1	为什么 CS336 需要 multimodality 在系统中的位置	11
7.2	CLIP 式对比学习与表示对齐在系统中的位置	11
7.3	VLM instruction tuning 与评估在系统中的位置	11
7.4	把本章知识用于读论文和复现实验	11
8	例题: 把概念变成可计算检查	12
8.1	把 CLIP 表示接入 LLM	12
9	实验设计: 从小例子到可复现结论	12
9.1	最小输入	12
9.2	资源账本	12
9.3	单变量消融	12
9.4	失败条件	12
9.5	记录与报告	13
9.6	从小实验到大结论	13
9.7	把本章重新读成一条证据链	13
10	课程资料与回看路径	13
10.1	视频回看路径	13
10.2	课程资料阅读路径	14
11	实践说明、历史位置与风险	14

12 复习要点	15
12.1 综合自测	15
13 总结与延伸	16
13.1 本章小结	16
13.2 延伸解释	16
13.3 练习	16

1 本章导读

本章讨论 **Alignment 与 multimodality**。CS336 的第一层目标不是把现成大模型当作黑箱使用，而是把 language model 从输入文本、训练目标、模型结构、数据、系统和评估这些部件重新拆开。只有拆开之后，读者才能判断一个设计选择究竟改变了什么。设想用户上传一张图并问模型。多模态模型必须把 image、text 甚至 video 接到同一推理接口；问题不只是连接 encoder，而是视觉 grounding、instruction tuning 和 hallucination control。本章对应 CS336 Spring 2026 第 17 个公开视频，并结合课程页提供的可解析资料。视频和课程页给出事实边界；正文把这些材料改写为可自学的教材章节。阅读本章时，先理解问题为什么存在，再看定义、公式和实现形态，随后通过例题和练习检查自己是否能独立复现这条 reasoning chain。

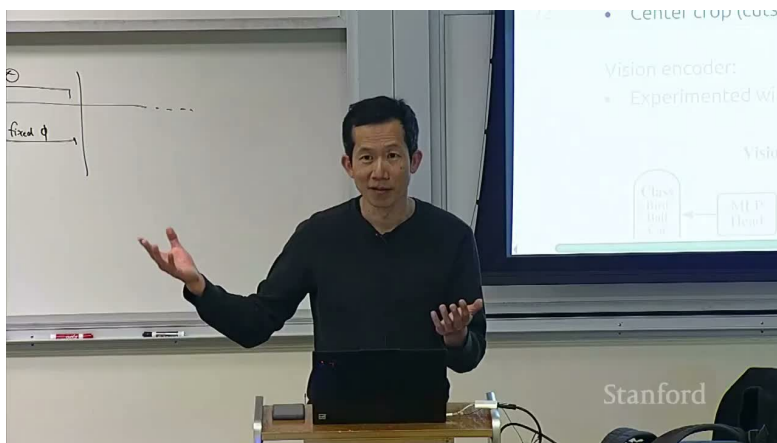


图 1: 第 17 讲的课程图像锚点。

1.1 本章知识路线

- 本章首先说明 Alignment 与 multimodality 为什么是 language modeling pipeline 中不可跳过的一环。
- 其次定义本章反复使用的术语，并说明这些术语对应的计算对象或系统对象。
- 随后用公式和伪代码表达主要机制，让概念能够落到可计算的变量上。
- 最后给出例题、风险讨论和练习，便于读者检查自己是否真正掌握了机制。

1.2 结构图

下面的图用于帮助读者定位本章的核心结构。先看概念之间的依赖，再回到正文中的公式、伪代码和例题。

2 本章主线

Alignment 与 multimodality 的主线是解释 base model 之后的行为塑形。课程关心的不是单个 post-training 名词，而是示范、偏好、奖励和评估怎样共同约束输出。

2.1 本章要解决的核心问题

本讲补上 multimodality。现代 frontier models 不只处理 text-to-text，而要把 image、video、audio 等模态编码成 token 或 embedding。读完这一段后，应能用一两句话说明 Alignment 与 multimodality 要解决的瓶

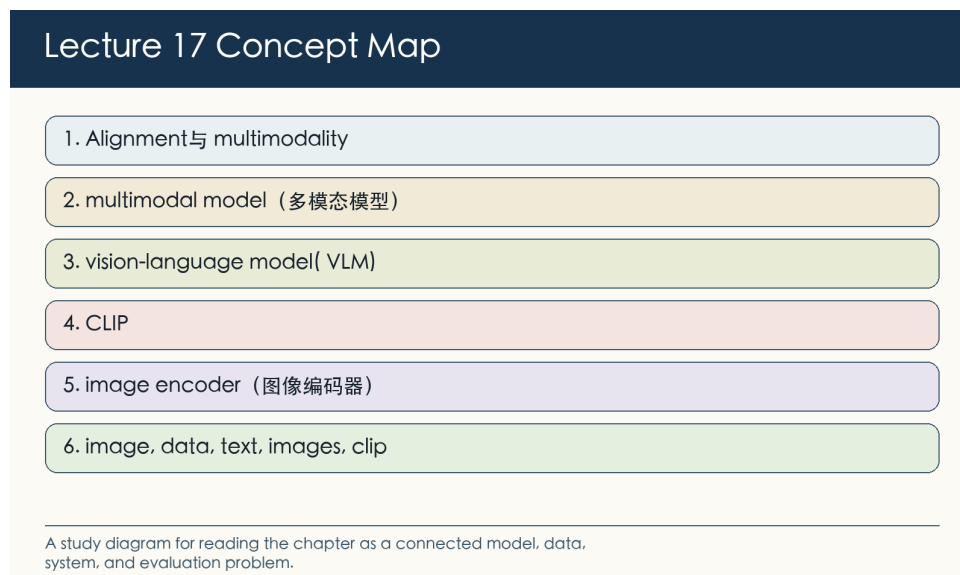


图 2: 第 17 讲概念图: 本章问题、术语和机制的关系。

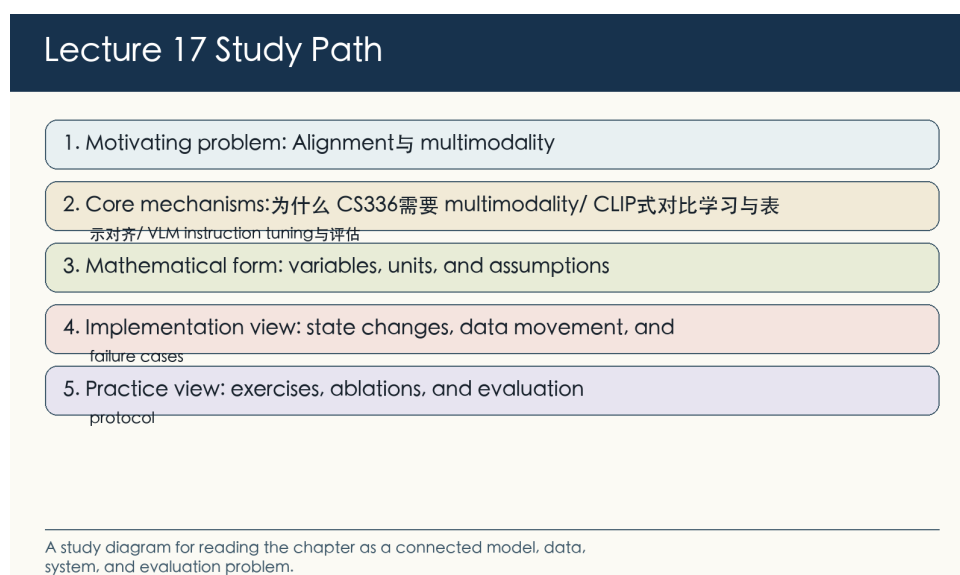


图 3: 第 17 讲学习路径: 从问题定义进入公式、实现、实验和练习。

颈，并指出这个瓶颈发生在数据、模型、优化、系统还是评估层。

2.2 从机制到资源账本

CLIP/SigLIP 用图文对齐学习 image/text representation, LLaVA 等 VLM 再把视觉特征投影到语言模型输入空间并做 instruction tuning。随后把机制写成账本: 输入规模是什么, 主要状态是什么, 成本或质量指标怎样随变量改变。这个账本让后面的公式不再只是符号。

2.3 从课堂讲解到可复现实验

多模态模型的关键 caveat 是视觉 grounding: 模型可能用语言先验回答, 而不是忠实读取图像; OCR、chart、spatial reasoning 和 video 都需要单独评估。最后把课堂讲解转成可复现实验: 固定协议, 改变一个变量, 记录中间量, 并解释结果为何支持或反驳原来的机制判断。

3 术语、符号与问题边界

本章的术语横跨模型、数据和系统。读者需要先知道每个词指向的对象：有些词描述数学目标，有些词描述张量形状，有些词描述硬件瓶颈，还有一些词描述评估协议。术语边界不清，后面的公式和实现就会变成空洞的缩写。

#	术语	阅读要求
1	multimodal model (多模态模型)	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
2	vision-language model (VLM)	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
3	CLIP	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
4	image encoder (图像编码器)	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
5	projector (投影器)	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
6	LLaVA	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。

这些术语之间有依赖关系。例如 tokenization 改变 sequence length, sequence length 又改变 attention FLOPs、KV cache 和 evaluation token budget; parallelism 改变显存占用，也改变通信瓶颈和故障恢复方式。

3.1 术语之间的关系

在本章中，multimodal model (多模态模型) 给出问题入口，vision-language model (VLM) 描述主要机制，CLIP 则把机制连接到可测量的结果。读教材时应当沿着这个链条追问：输入是什么，中间状态怎样改变，输出指标为什么随之变化。课程材料还会反复出现 text、image、data、images、encoder、tokens 等词。它们不是一组同义标签，而是提示本章的不同层次：有的指对象，有的指操作，有的指约束或指标。因此，术语表不是背诵清单，而是后文阅读的索引。遇到一个术语时，先定位它属于 representation、optimization、systems、data 还是 evaluation，再判断它改变哪一项账本。

4 为什么 CS336 需要 multimodality

本节讨论为什么 CS336 需要 multimodality。它先限定 Alignment 与 multimodality 要回答的基本问题：哪些现象需要解释，哪些变量可以测量，哪些结论必须回到 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率中检验。

4.1 课程目标与问题边界

视频说明原计划继续 RL，但课程若完全不讲 multimodality 会不完整，因为主流模型已普遍多模态。多模态不是把图片塞进 prompt 那么简单，而是要把图像、视频、音频等信号变成 LM 可处理的 token/embedding。本讲是 overview，目标是建立结构图和主要训练阶段，而不是覆盖完整 VLM 课程。为什么 CS336 需要 multimodality 把 Alignment 与 multimodality 连接到行为目标。接下来要看的不是 reward 名称，而是反馈信号、采样分布和约束项怎样改变模型更新。

4.2 为什么 CS336 需要 multimodality 的机制

视频说明原计划继续 RL，但课程若完全不讲 multimodality 会不完整，因为主流模型已普遍多模态。在为什么 CS336 需要 multimodality 中，核心对象是行为、奖励和偏好状态：后训练方法并不直接制造知识，而是通过目标和反馈改变模型选择输出的方式。多模态不是把图片塞进 prompt 那么简单，而是要把图像、视频、音频等信号变成 LM 可处理的 token/embedding。因而同一个方法在不同规模下可能改变不同的变量，尤其是 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。比较方法时，必须同时给出问题规模和实验条件。本讲是 overview，目标是建立结构图和主要训练阶段，而不是覆盖完整 VLM 课程。可检验性来自训练日志、reward 分布、KL/length 统计和行为评估；只看最终回答样例无法判断更新方向。因此，为什么 CS336 需要 multimodality 的学习目标是建立一条行为更新链：读者应能区分示范、偏好、奖励和约束分别给模型提供了什么信号。

4.3 资源、效率与效果的关系

$$z_{\text{text}}, z_{\text{image}} \in \mathbb{R}^d$$

符号说明：文本和图像被编码到同一维度的表示空间，后续可用于对齐、检索或作为 LM 输入。这个关系式说明为什么 CS336 需要 multimodality 如何把行为偏好写入优化目标。关键不是符号复杂度，而是 reward、reference policy、采样分布和约束强度如何共同决定更新方向。在后训练实验中，reward model score、verifier pass rate、回答长度和人工偏好都是 proxy。它们会被采样策略和格式约束影响，因此不能直接等同于真实有用性。

4.4 把课程目标写成检查流程

把为什么 CS336 需要 multimodality 写成训练流程时，要区分采样、打分、构造 loss 和更新参数。下面的伪代码保留这些边界。

```
image_features = vision_encoder(image)
image_tokens = projector(image_features)
logits = language_model(concat(image_tokens, text_tokens))
```

阅读这段训练流程时，重点区分采样、打分和参数更新。只有同时记录 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率，才能判断行为改善来自奖励信号、采样分布还是约束强度。

4.5 边界条件与常见误解

- 多模态模型的能力受 vision encoder、数据配对质量和 instruction tuning 共同限制。
- 把 image embedding 接到 LM 不保证 spatial reasoning 或视觉 grounding。
- 后训练指标可能被格式、长度或 verifier 偏差驱动，不能直接等同于真实能力提升。
- 偏好优化可能改善目标行为，也可能牺牲多样性、校准性或某些下游能力。

4.6 与本章其他概念的关系

为什么 CS336 需要 multimodality 连接本章的训练目标和输出行为：术语表区分示范、偏好与奖励，公式说明优化方向，伪代码展示采样和更新，例题检查行为约束的边界。对这一类主题，最常见的误解是只列方法名。更有用的做法是比较每个方法改变的成本项、依赖的假设和可能的失败模式。

4.7 怎样检验这一课程目标

检验这一课程目标时, 先把抽象说法落到行为、奖励和偏好状态上。与 multimodal、image、vision、encoder、tokens 相关的对象必须能被构造、打印、计数或评分; 否则实验只能得到描述性观察, 不能得到可复现结论。一个合适的小实验应当保留本节机制, 同时让 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率中至少一个量发生可解释变化。输入规模不必逼真; 关键是读者能手算或打印关键中间量, 并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明: 文本和图像被编码到同一维度的表示空间, 后续可用于对齐、检索或作为 LM 输入。如果数量只能间接观测, 就必须说明使用了什么 proxy; 如果使用 benchmark 或 reward, 也要说明协议和随机性。后训练消融通常一次只改变示范数据、reward scale、KL 约束、采样温度或 verifier。若训练目标和解码策略同时变化, 行为差异就很难归因。最后检查失败条件。工程判断往往在反例中变清楚: 多模态模型的能力受 vision encoder、数据配对质量和 instruction tuning 共同限制; 把 image embedding 接到 LM 不保证 spatial reasoning 或视觉 grounding。复习时可以把这些限制改写成反例测试, 观察它们怎样改变 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。把实验结论放回 Alignment 与 multimodality 时, 要说明为什么 CS336 需要 multimodality 改变的是目标函数、采样分布、反馈质量还是行为约束。能区分这些来源, 才说明读者真正理解后训练。

4.8 概念辨析: 为什么 CS336 需要 multimodality

multimodal、image、vision、encoder、tokens 常一起出现在后训练讨论中, 但它们分别指数据、反馈、目标函数或行为指标。读者应先判断每个词给优化过程提供什么信号。其中, multimodal model (多模态模型) 把文本、图像、视频等统一到 token/embedding 接口。区分这些词时, 可以先问它们是否改变 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。一个术语如果不能对应到变量、状态、代码路径或评估协议, 就还没有形成可操作含义。围绕为什么 CS336 需要 multimodality 复习时, 可以构造几条 prompt-response: 明确奖励或偏好规则, 观察同一模型在更新前后的行为差异。

4.9 为什么 CS336 需要 multimodality 的小结

- 讨论为什么 CS336 需要 multimodality 时, 应同时报告问题规模、实验设置和主要观测变量, 尤其是 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。
- 如果方法声称行为更好, 要说明示范、偏好、reward 或 verifier 哪一项提供了信号, 并检查 reward hacking、length bias 和能力退化。
- 后训练结论需要把训练目标、采样策略和评估协议分开报告, 否则无法判断改进来自哪里。

5 CLIP 式对比学习与表示对齐

现在进入 CLIP 式对比学习与表示对齐。这一节把算法放到硬件和运行时中考察: 同一个数学表达式在不同 batch、shape、GPU 拓扑和 kernel 实现下会得到不同的成本账本。

5.1 为什么要讨论 CLIP 式对比学习与表示对齐

字幕关键词显示 CLIP、image、text、encoder 反复出现, 是本讲重要主线。CLIP 用图文对比学习把匹配图像和文本拉近, 把不匹配样本推远。这种表示可用于 zero-shot classification、retrieval, 也可作为 VLM 的视觉基础。CLIP 式对比学习与表示对齐把 Alignment 与 multimodality 放进资源账本。接下来要看的不是单个 API, 而是张量形状、内存层级、通信和调度怎样共同决定成本。

5.2 CLIP 式对比学习与表示对齐的机制

字幕关键词显示 CLIP、image、text、encoder 反复出现，是本讲重要主线。在 CLIP 式对比学习与表示对齐中，核心对象是硬件和系统状态：同一算法会因为 memory hierarchy、通信拓扑或 kernel shape 不同而表现出完全不同的速度。CLIP 用图文对比学习把匹配图像和文本拉近，把不匹配样本推远。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。这种表示可用于 zero-shot classification、retrieval，也可作为 VLM 的视觉基础。可检验性来自 profile trace、roofline 估算和端到端延迟分解；如果这些证据互相矛盾，就要先解释 measurement setup。因此，CLIP 式对比学习与表示对齐的学习目标是建立一张成本地图：读者应能指出瓶颈在计算、内存、通信还是调度，并能设计一个小 benchmark 去验证。

5.3 CLIP 式对比学习与表示对齐的数学表达

$$\mathcal{L}_{\text{CLIP}} = -\frac{1}{B} \sum_i \log \frac{\exp(s(z_i^I, z_i^T)/\tau)}{\sum_j \exp(s(z_i^I, z_j^T)/\tau)}$$

符号说明： z_i^I, z_i^T 是第 i 个图像和文本表示， s 是相似度， τ 是 temperature。这个关系式应被读成 CLIP 式对比学习与表示对齐的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。在系统实验中，tokens/s、TTFT、显存峰值和 kernel time 都是 proxy。它们各自只观察一部分系统，因此报告时要同时写清 workload shape、warmup、同步方式和硬件型号。

5.4 从 CLIP 式对比学习与表示对齐到程序

把 CLIP 式对比学习与表示对齐写成程序时，最重要的是暴露状态变化和数据移动。下面的伪代码保留执行顺序，省略框架样板。

```
image_emb = image_encoder(images)
text_emb = text_encoder(captions)
loss = contrastive_loss(image_emb, text_emb)
```

阅读这段实现时，重点看 CLIP 式对比学习与表示对齐怎样移动张量、占用显存并触发通信。实验记录至少要包含吞吐、延迟、显存峰值、通信量和 kernel 利用率，否则很难判断瓶颈来自算子、内存层级还是调度。

5.5 边界条件与常见误解

- contrastive alignment 不等于能生成细粒度视觉描述。
- batch negatives 的质量和规模会影响表示学习。
- 小规模 kernel benchmark 不一定预测端到端训练或 serving latency，因为调度、通信和请求形状会改变瓶颈。
- 硬件规格表和框架 profiler 都需要结合 workload 解释，不能把单个峰值数字当作机制证明。

5.6 与本章其他概念的关系

CLIP 式对比学习与表示对齐连接本章的数学机制和工程成本：术语表说明对象，公式估算资源，伪代码暴露状态变化，例题则让读者检查一次完整的成本推理。对这一类主题，最常见的误解是只列方法名。更有用的做法是比较每个方法改变的成本项、依赖的假设和可能的失败模式。

5.7 怎样检验 CLIP 式对比学习与表示对齐

检验 CLIP 式对比学习与表示对齐时，先把抽象说法落到硬件和系统状态上。与 CLIP、contrastive、image encoder、text encoder、temperature 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明： (z^I_i, z^T_i) 是第 (i) 个图像和文本表示， (s) 是相似度， (au) 是 temperature。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。系统消融通常一次只改变 batch shape、sequence length、GPU 数、kernel 实现或调度策略。若同时改变硬件和 workload，实验只能说明端到端表现不同，不能定位瓶颈。最后检查失败条件。工程判断往往在反例中变清楚：contrastive alignment 不等于能生成细粒度视觉描述；batch negatives 的质量和规模会影响表示学习。复习时可以把这些限制改写成反例测试，观察它们怎样改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。把实验结论放回 Alignment 与 multimodality 时，要说明 CLIP 式对比学习与表示对齐改变的是训练吞吐、推理延迟、显存峰值还是通信开销。能做出这种归因，才说明读者已经从接口使用进入系统解释。

5.8 概念辨析：CLIP 式对比学习与表示对齐

CLIP、contrastive、image encoder、text encoder、temperature 常一起出现在系统讲解中，但它们分别指硬件对象、程序操作、瓶颈来源或测量指标。读者应先定位每个词在执行路径中的位置。其中，CLIP 用图文对比学习对齐 image encoder 和 text encoder。区分这些词时，可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕 CLIP 式对比学习与表示对齐复习时，可以写一个小 benchmark：固定输入形状，改变一个系统变量，记录 profile trace 和端到端时间。

5.9 CLIP 式对比学习与表示对齐的小结

- 讨论 CLIP 式对比学习与表示对齐时，应同时报告问题规模、实验设置和主要观测变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。
- 如果方法声称更快，要区分计算减少、内存访问减少、通信隐藏和调度改善；这些机制不能用一个 tokens/s 数字互相替代。
- profile trace、roofline 估算和端到端 benchmark 应互相校验，单独一个指标不足以解释系统瓶颈。

6 VLM instruction tuning 与评估

现在进入 VLM instruction tuning 与评估。这一节讨论 base model 之外的行为塑形：示范、偏好、reward 或 verifier 怎样把模型输出推向课程定义的目标行为。

6.1 为什么要讨论 VLM instruction tuning 与评估

LLaVA 等模型把视觉特征投影成 LM tokens，再用图文指令数据训练对话能力。多模态 alignment 包含 helpfulness、安全、拒答、视觉事实性和跨模态 grounding。评估 VLM 要覆盖 captioning、VQA、OCR、chart/table、spatial reasoning、video understanding 和 hallucination。VLM instruction tuning 与评估把 Alignment 与 multimodality 连接到行为目标。接下来要看的不是 reward 名称，而是反馈信号、采样分布和约束项怎样改变模型更新。

6.2 VLM instruction tuning 与评估的机制

LLaVA 等模型把视觉特征投影成 LM tokens，再用图文指令数据训练对话能力。在 VLM instruction tuning 与评估中，核心对象是行为、奖励和偏好状态：后训练方法并不直接制造知识，而是通过目标和反馈改变模型选择输出的方式。多模态 alignment 包含 helpfulness、安全、拒答、视觉事实性和跨模态 grounding。因而同一个方法在不同规模下可能改变不同的变量，尤其是 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。比较方法时，必须同时给出问题规模和实验条件。评估 VLM 要覆盖 captioning、VQA、OCR、chart/table、spatial reasoning、video understanding 和 hallucination。可检验性来自训练日志、reward 分布、KL/length 统计和行为评估；只看最终回答样例无法判断更新方向。因此，VLM instruction tuning 与评估的学习目标是建立一条行为更新链：读者应能区分示范、偏好、奖励和约束分别给模型提供了什么信号。

6.3 VLM instruction tuning 与评估的数学表达

$$\mathcal{L}_{\text{VLM}} = - \sum_t \log p_{\theta}(y_t \mid \text{image tokens}, x, y_{<t})$$

符号说明：image tokens 是视觉编码后的连续或离散表示， x 是文本指令， y 是回答。这个关系式说明 VLM instruction tuning 与评估如何把行为偏好写入优化目标。关键不是符号复杂度，而是 reward、reference policy、采样分布和约束强度如何共同决定更新方向。在后训练实验中，reward model score、verifier pass rate、回答长度和人工偏好都是 proxy。它们会被采样策略和格式约束影响，因此不能直接等同于真实有用性。

6.4 从 VLM instruction tuning 与评估到程序

把 VLM instruction tuning 与评估写成训练流程时，要区分采样、打分、构造 loss 和更新参数。下面的伪代码保留这些边界。

```
for image, instruction, answer in vlm_sft_data:
    inputs = build_multimodal_prompt(image, instruction)
    update_cross_entropy(inputs, answer)
```

阅读这段训练流程时，重点区分采样、打分和参数更新。只有同时记录 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率，才能判断行为改善来自奖励信号、采样分布还是约束强度。

6.5 边界条件与常见误解

- VLM hallucination 可能来自语言先验压过视觉证据。
- OCR/图表能力需要专门数据与评估，不能只看通用 caption。
- 后训练指标可能被格式、长度或 verifier 偏差驱动，不能直接等同于真实能力提升。
- 偏好优化可能改善目标行为，也可能牺牲多样性、校准性或某些下游能力。

6.6 与本章其他概念的关系

VLM instruction tuning 与评估连接本章的训练目标和输出行为：术语表区分示范、偏好与奖励，公式说明优化方向，伪代码展示采样和更新，例题检查行为约束的边界。对这一类主题，最常见的误解是只列方法名。更有用的做法是比较每个方法改变的成本项、依赖的假设和可能的失败模式。

6.7 怎样检验 VLM instruction tuning 与评估

检验 VLM instruction tuning 与评估时，先把抽象说法落到行为、奖励和偏好状态上。与 LLaVA、instruction tuning、VQA、OCR、hallucination 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明：image tokens 是视觉编码后的连续或离散表示， $\backslash(x\backslash)$ 是文本指令， $\backslash(y\backslash)$ 是回答。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。后训练消融通常一次只改变示范数据、reward scale、KL 约束、采样温度或 verifier。若训练目标和解码策略同时变化，行为差异就很难归因。最后检查失败条件。工程判断往往在反例中变清楚：VLM hallucination 可能来自语言先验压过视觉证据；OCR/图表能力需要专门数据与评估，不能只看通用 caption。复习时可以把这些限制改写成反例测试，观察它们怎样改变 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。把实验结论放回 Alignment 与 multimodality 时，要说明 VLM instruction tuning 与评估改变的是目标函数、采样分布、反馈质量还是行为约束。能区分这些来源，才说明读者真正理解后训练。

6.8 概念辨析：VLM instruction tuning 与评估

LLaVA、instruction tuning、VQA、OCR、hallucination 常一起出现在后训练讨论中，但它们分别指数据、反馈、目标函数或行为指标。读者应先判断每个词给优化过程提供什么信号。区分这些词时，可以先问它们是否改变 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕 VLM instruction tuning 与评估复习时，可以构造几条 prompt-response：明确奖励或偏好规则，观察同一模型在更新前后的行为差异。

6.9 VLM instruction tuning 与评估的小结

- 讨论 VLM instruction tuning 与评估时，应同时报告问题规模、实验设置和主要观测变量，尤其是 reward 分布、偏好差异、KL 约束、回答长度和 verifier 通过率。
- 如果方法声称行为更好，要说明示范、偏好、reward 或 verifier 哪一项提供了信号，并检查 reward hacking、length bias 和能力退化。
- 后训练结论需要把训练目标、采样策略和评估协议分开报告，否则无法判断改进来自哪里。

7 综合讨论：从局部机制到完整系统

本章的主题是 Alignment 与 multimodality，但它不应被读成几个相邻知识点的拼接。为什么 CS336 需要 multimodality、CLIP 式对比学习与表示对齐、VLM instruction tuning 与评估共同回答同一个问题：语言模型系统中哪些对象可以被定义，哪些成本可以被估算，哪些结论必须通过实验或评估来确认。这些对象包括 multimodal model（多模态模型）、vision-language model (VLM)、CLIP、image encoder（图像编码器）、projector（投影器）、LLaVA。读者可以把它们看成一组接口：有的接口连接文本和 token，有的连接模型结构和张量计算，有的连接数据样本和训练目标，有的连接离线评估和在线服务。接口一旦改变，后面的成本、错误类型和优化空间也会随之改变。

7.1 为什么 CS336 需要 multimodality 在系统中的位置

为什么 CS336 需要 multimodality 首先提供一个局部解释：视频说明原计划继续 RL，但课程若完全不讲 multimodality 会不完整，因为主流模型已普遍多模态。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，为什么 CS336 需要 multimodality 会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，为什么 CS336 需要 multimodality 不能脱离边界条件理解。一个关键 caveat 是：多模态模型的能力受 vision encoder、数据配对质量和 instruction tuning 共同限制。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.2 CLIP 式对比学习与表示对齐在系统中的位置

CLIP 式对比学习与表示对齐首先提供一个局部解释：字幕关键词显示 CLIP、image、text、encoder 反复出现，是本讲重要主线。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，CLIP 式对比学习与表示对齐会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，CLIP 式对比学习与表示对齐不能脱离边界条件理解。一个关键 caveat 是：contrastive alignment 不等于能生成细粒度视觉描述。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.3 VLM instruction tuning 与评估在系统中的位置

VLM instruction tuning 与评估首先提供一个局部解释：LLaVA 等模型把视觉特征投影成 LM tokens，再用图文指令数据训练对话能力。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，VLM instruction tuning 与评估会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，VLM instruction tuning 与评估不能脱离边界条件理解。一个关键 caveat 是：VLM hallucination 可能来自语言先验压过视觉证据。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.4 把本章知识用于读论文和复现实验

读相关论文时，可以把本章变成一个检查表。第一，论文是否清楚定义了输入规模、模型规模、数据来源和评估协议；第二，论文声称的改进落在哪个变量上；第三，作者是否报告了会让结论失效的设置；第四，实验是否能分离模型结构、数据质量、优化设置和系统实现的影响。复现实验时，也应避免直接追求大规模。先用最小程序复现一个公式、一个伪代码分支或一个 profile 现象，再扩大输入规模。这样做的原因很简单：如果小规模程序无法解释中间量，大规模训练只会把错误隐藏在日志和随机波动里。把这些检查合在一起，本章给出的不是一个固定答案，而是一种阅读 CS336 的方法：每个模型机制都要落到变量，每个变量都要落到测量，每个测量都要说明协议，每个协议都要承认边界。因此，本章中的任何一句结论都可以被改写成一个可引用的完整句式：在给定数据、模型、硬件和评估协议下，某个机制改变了某个变量，并通过某个观测指标表现出来。这样的句子比单独背诵术语更长，但它包含了自学、复现和引用时真正需要的信息。把本章用于自学时，可以按三遍阅读。第一遍只追踪对象和定义，确认每个术语到底指向文本、token、张量、样本、请求还是评

估记录。第二遍追踪公式和伪代码，确认每一步改变了哪些状态。第三遍追踪实验和 caveat，确认哪些结论只在特定规模、数据或硬件条件下成立。把本章用于复习时，则应反过来操作：先从一个失败案例或异常指标出发，再回到相应机制。loss 曲线异常可能指向数据、优化或模型容量；吞吐异常可能指向 kernel、通信或调度；benchmark 异常可能指向 contamination、prompt protocol 或采样设置。能够从异常反推机制，是掌握本章的实用标准。

8 例题：把概念变成可计算检查

8.1 把 CLIP 表示接入 LLM

图像经 vision encoder 得到 patch features，再经 projector 映射到语言模型 hidden size。

- CLIP 式对比学习提供图文对齐表示。
- projector 把视觉表示转换成 LM 可消费的 image tokens。
- instruction tuning 教模型在视觉上下文中回答问题。

结论。 VLM 的关键不只是接口形状，而是视觉证据是否真正约束了语言输出。这个例题的目的不是替代完整工程实现，而是给出一种最小推理形式：把问题转成变量，写出近似公式或伪代码，再说明近似何时失效。

9 实验设计：从小例子到可复现结论

学习 Alignment 与 multimodality 不能停在概念层。一个教材化结论至少需要四件事：可构造的输入、可观测的中间量、可比较的指标，以及清楚的失败条件。下面的实验设计不是要求训练大模型，而是要求读者用小规模程序复现本章机制。

9.1 最小输入

先构造只触发本章核心机制的小输入。例如 tokenizer 章节可以用几个包含 rare words 和 Unicode 字符的字符串；GPU 章节可以用一个 elementwise kernel 和一个 GEMM；data 章节可以用十几篇近重复文档。小输入的价值在于它让错误可见。

9.2 资源账本

对同一输入写下 FLOPs、bytes moved、显存峰值、通信量或样本数量的估算。估算不需要一开始就精确，但必须说明单位和近似假设。随后用 profiler、benchmark、validation loss 或人工检查去验证估算是否同量级。

9.3 单变量消融

一次只改变一个因素，例如 vocabulary size、head 数、batch size、filter threshold、reward scale、decoding temperature 或 GPU 数量。若多个变量同时变化，实验只能说明系统变了，不能说明机制是什么。

9.4 失败条件

最后要刻意触发失败：极端 context length、错误 dtype、过小 batch、污染 benchmark、低质量数据或不可靠 verifier。失败条件常常比成功曲线更能说明一个方法的边界。

9.5 记录与报告

实验记录应能让另一个读者复现同一结论。最少要写清楚输入构造、代码版本、随机种子、硬件、batch shape、数据版本、评价指标和异常处理方式。报告结论时不要只写“更快”或“更好”，而要说明快在哪里、好在哪个指标上，以及是否牺牲了内存、稳定性、数据质量或可解释性。常见报告错误是把局部现象写成普遍规律。例如一次小模型实验里的 loss 改善，不等于更大模型或不同数据分布上也会改善；一次 kernel benchmark 的吞吐提升，不等于端到端 serving latency 一定降低。教材中的实验报告应始终把结论限定在可复现条件内。

9.6 从小实验到大结论

小实验的作用不是替代课程中的完整训练或系统实现，而是建立一条可检查的推理链。先在小输入上确认变量方向，再在中等规模上确认数量级，最后才讨论是否能外推到更大模型、更多 token 或更复杂 workload。缺少中间层级时，大结论通常只是在复述直觉。因此，实验报告最好同时包含正例和反例：正例说明机制在受控条件下怎样工作，反例说明条件稍变时哪里失效。对 CS336 这样的课程，反例尤其重要，因为 language model 的错误常来自多个层面叠加：数据分布、优化稳定性、硬件瓶颈、评估协议和服务负载都可能同时改变观察结果。还要记录资料版本与复现边界。公开视频、课程页、配套代码和 PDF 可能在不同时间更新；复现实验应写明使用的材料版本，并说明哪些结论来自课程事实，哪些是为了帮助理解而加入的解释性推导。这样才能把学习笔记变成可检查的技术记录。若复现实验与课程结论不一致，首先检查输入、版本和评估协议，而不是立即修改模型假设。

9.7 把本章重新读成一条证据链

完成小实验之后，可以把 Alignment 与 multimodality 重新读成一条证据链：先用“为什么 CS336 需要 multimodality”确定问题入口，再用“CLIP 式对比学习与表示对齐”解释主要机制，最后用“VLM instruction tuning 与评估”检查边界或外推条件。这种读法比按页背诵更接近教材训练，因为它要求每个结论都能说明前提、变量和证据。本章反复使用的术语包括 multimodal model（多模态模型）、vision-language model (VLM)、CLIP、image encoder（图像编码器）、projector（投影器）、LLaVA。复习时不要只写定义，而要为每个术语补上一句操作性说明：它在课程代码、公式、数据记录或评估协议中对应什么对象；如果这个对象被删除、替换或缩放，哪一个观测量会变化。最后，用一个反例结束复习。反例可以是资源估算不准、数据污染、reward 失真、benchmark 解析失败、长上下文显存爆炸或 scaling 外推失效。能解释反例的章节，才真正具备自学和引用价值；不能解释反例的章节，只是把课程材料换了一种排版。引用本章结论时，还应保留来源边界：哪些说法直接来自视频、课程页、slides 或代码，哪些是为了串联教材叙述而加入的解释性推导。这个边界不会削弱结论，反而让读者知道何处可以复核，何处需要在自己的实验中重新验证。如果后续课程材料更新，也可以沿着同一证据链替换来源，而不必重写整章结构。

10 课程资料与回看路径

教材正文已经把主要概念、公式和实现逻辑组织成连续叙述；本节只提供回到课程材料的阅读路径。读者复习时可以先完成前文练习，再按下面的时间段或资料组核对细节。

10.1 视频回看路径

时间	关键词	复习任务
----	-----	------

00:00:05-00:06:05	language, give, images, image, tokens., class, models.	回看定义、例子、推导或 caveat 在该段中的位置。
00:19:08-00:25:06	it's, text, image, going, images, another, point	回看定义、例子、推导或 caveat 在该段中的位置。
00:38:12-00:44:42	images, then, multiple, data, image, look, single	回看定义、例子、推导或 caveat 在该段中的位置。
00:51:17-00:57:14	then, it's, video, time, model., they're, i'll	回看定义、例子、推导或 caveat 在该段中的位置。
01:11:06-01:17:16	there's, think, training, even, it's, then, your	回看定义、例子、推导或 caveat 在该段中的位置。

这些时间段用于定位视频中的讲解顺序。严肃引用时，应回到视频片段确认讲者表述、板书或幻灯片内容。回看视频时，不必逐字抄录字幕。更有效的方法是把每个时间段判定为定义、例子、推导、代码解释或 caveat，再把它放回本章对应小节。

10.2 课程资料阅读路径

资料组	标题/页块	关键词
official_01	Lecture 17: multimodal models	multimodal, modalities, non-text, text, omni, input, combination
official_03	- Attention pooling: do QKV with query = global average of activations	text, image, images, classification, large, trained, imagenet
official_05	Training:	resolution, image, vision, encoder, multiple, images, video
official_08	- Pre-training has 4 stages (train adapter, train all parameters on 8K, 32K, 256K ...	tokens, image, data, train, images, generate, discrete
official_10	Initialize LM with Qwen2 and vision encoder from DFN	function, vision, encoder, siglip, chameleon, initialize, qwen2

课程资料通常给出更稳定的标题、公式、图或代码结构；视频则补充动机和口头解释。两者合起来构成本章的事实边界。阅读资料时，可以把每个资料组拆成三个对象：一个定义，一个公式或伪代码动作，一个边界条件。这样比逐字翻译页面或脚本更容易形成可用知识。

11 实践说明、历史位置与风险

Alignment 与 multimodality 在 CS336 的课程结构中连接基础机制和规模化问题。基础机制解释方法为什么成立；规模化问题检验它在更大模型、更长上下文、更复杂数据或真实 serving workload 下是否仍成立。历史位置不等于模型年表。更重要的是识别问题如何迁移：早期方法解决小规模建模问题，现代方法常常解决同一问题在大规模数据、GPU 集群、长上下文或人类偏好约束下的新形态。

- 资料版本限制 (version risk): 课程页、公开视频和配套资料可能在学期中更新；引用时应注明访问日期或资料版本。

- 测量口径限制 (measurement risk): FLOPs、tokens/s、benchmark score、reward 等数字只有在协议完整时可比较。
- 规模外推限制 (scaling risk): 小模型或小 batch 的机制不必然外推到 frontier-scale。
- 数据分布限制 (data risk): 数据来源、过滤和去重策略会改变模型行为, 也会改变 evaluation contamination 风险。
- 对齐风险 (alignment risk): post-training 或 multimodal tuning 的行为变化可能提高 helpfulness, 同时降低校准、透明度或安全边界。

12 复习要点

下面的问题把本章内容压缩为可反复检查的条目。每个条目都应能回到前文的解释、公式、伪代码或例题。

条目	对象	检查动作
条目 1	multimodal model (多模态模型)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 2	vision-language model (VLM)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 3	CLIP	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 4	image encoder (图像编码器)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 5	projector (投影器)	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。
条目 6	LLaVA	定义它; 写出一个最小例子; 说明一个 failure mode; 指出它影响哪个资源或评估账本。

12.1 综合自测

- 我能否不用课程原句, 独立解释本章中心问题?
- 我能否把本章至少一个公式改写成可运行的估算函数?
- 我能否指出一个看似改进但实际转移成本的方法?
- 我能否回到视频和课程资料, 找到支持本章关键论断的位置?
- 我能否设计一个最小 ablation, 验证本章一个 caveat?

13 总结与延伸

13.1 本章小结

- Alignment 与 multimodality 的核心不是记忆术语，而是理解对象、变量和机制之间的关系。
- 视频给出讲解顺序，课程资料提供可核对的公式、图、代码或页面结构。
- 复习时应把每个术语连接到至少一个变量、一个实现动作和一个失败模式。

13.2 延伸解释

本章中的延伸解释用于连接课程材料中的概念，例如说明公式里的 proxy、解释系统 tradeoff，或把多个材料片段串成一个完整推理链。若需要严肃引用，应回到课程视频和配套资料核对原始表述。

13.3 练习

- 定义题：用中英双语解释本章任意五个重要术语，并说明它们属于 compute、memory、data、optimization、evaluation 还是 deployment 账本。
- 推导题：选择本章一个公式，逐个解释符号、单位和近似假设，并说明哪个变量最难在真实系统中测量。
- 实现题：把本章伪代码改写成一个最小 Python sanity check，不要求训练大模型，但要输出可检查的中间量。
- 资料题：从“课程资料与回看路径”中选择一个视频时间段，回到原始视频核对讲者例子，并记录是否存在字幕误差。
- 评估题：为本章方法设计一个 ablation 或 benchmark，说明控制变量、数据版本、指标和 failure criteria。
- 边界题：说明本章一个结论在更长 context、更大 batch、更多 GPU 或更复杂数据分布下可能如何失效。
- 综合题：把本章内容和前一章或后一章连接起来，写出一个端到端训练/推理 pipeline 中的依赖关系。
- 批判题：指出一个容易被营销材料夸大的说法，并用本章的资源账本或评估协议反驳它。

18 18 Guest Lecture: Dan Fu 与推理引擎研究

第 18 讲: Guest Lecture: Dan Fu 与推理引擎研究

CS336 Spring 2026 public videos and official course materials

目录

1 本章导读	2
1.1 本章知识路线	2
1.2 结构图	2
2 本章主线	2
2.1 本章要解决的核心问题	3
2.2 从机制到资源账本	3
2.3 从课堂讲解到可复现实验	4
3 术语、符号与问题边界	4
3.1 术语之间的关系	4
4 从训练到 serving 的另一侧	4
4.1 为什么要讨论从训练到 serving 的另一侧	5
4.2 从训练到 serving 的另一侧的机制	5
4.3 从训练到 serving 的另一侧的数学表达	5
4.4 从训练到 serving 的另一侧在系统中的实现	5
4.5 边界条件与常见误解	5
4.6 与本章其他概念的关系	6
4.7 怎样检验从训练到 serving 的另一侧	6
4.8 概念辨析: 从训练到 serving 的另一侧	6
4.9 从训练到 serving 的另一侧的小结	6
5 GPU kernels 与 full-stack innovation	7
5.1 为什么要讨论 GPU kernels 与 full-stack innovation	7
5.2 GPU kernels 与 full-stack innovation 的机制	7
5.3 GPU kernels 与 full-stack innovation 的资源账本	7
5.4 GPU kernels 与 full-stack innovation 在系统中的实现	7
5.5 边界条件与常见误解	8

5.6	与本章其他概念的关系	8
5.7	怎样检验 GPU kernels 与 full-stack innovation	8
5.8	概念辨析: GPU kernels 与 full-stack innovation	8
5.9	GPU kernels 与 full-stack innovation 的小结	8
6	SSM/recurrence 与 inference 研究问题	9
6.1	为什么要讨论 SSM/recurrence 与 inference 研究问题	9
6.2	SSM/recurrence 与 inference 研究问题的机制	9
6.3	SSM/recurrence 与 inference 研究问题的资源账本	9
6.4	SSM/recurrence 与 inference 研究问题在系统中的实现	9
6.5	边界条件与常见误解	10
6.6	与本章其他概念的关系	10
6.7	怎样检验 SSM/recurrence 与 inference 研究问题	10
6.8	概念辨析: SSM/recurrence 与 inference 研究问题	10
6.9	SSM/recurrence 与 inference 研究问题的小结	11
7	综合讨论: 从局部机制到完整系统	11
7.1	从训练到 serving 的另一侧在系统中的位置	11
7.2	GPU kernels 与 full-stack innovation 在系统中的位置	11
7.3	SSM/recurrence 与 inference 研究问题在系统中的位置	11
7.4	把本章知识用于读论文和复现实验	12
8	例题: 把概念变成可计算检查	12
8.1	Serving latency 的分解	12
9	实验设计: 从小例子到可复现结论	12
9.1	最小输入	13
9.2	资源账本	13
9.3	单变量消融	13
9.4	失败条件	13
9.5	记录与报告	13
9.6	从小实验到大结论	13
9.7	把本章重新读成一条证据链	13
10	课程资料与回看路径	14
10.1	视频回看路径	14
10.2	课程资料阅读路径	14
11	实践说明、历史位置与风险	15

12 复习要点	15
12.1 综合自测	15
13 总结与延伸	16
13.1 本章小结	16
13.2 延伸解释	16
13.3 练习	16

1 本章导读

本章讨论 **Guest Lecture: Dan Fu 与推理引擎研究**。CS336 的第一层目标不是把现成大模型当作黑箱使用，而是把 language model 从输入文本、训练目标、模型结构、数据、系统和评估这些部件重新拆开。只有拆开之后，读者才能判断一个设计选择究竟改变了什么。设想训练好的模型要接入真实流量。Serving system (服务系统) 需要调度请求、管理 KV cache、写高效 kernels、处理 prefill/decode，并在 latency、throughput 和成本之间做工程取舍。本章对应 CS336 Spring 2026 第 18 个公开视频，并结合课程页提供的可解析资料。视频和课程页给出事实边界；正文把这些材料改写为可自学的教材章节。阅读本章时，先理解问题为什么存在，再看定义、公式和实现形态，随后通过例题和练习检查自己是否能独立复现这条 reasoning chain。

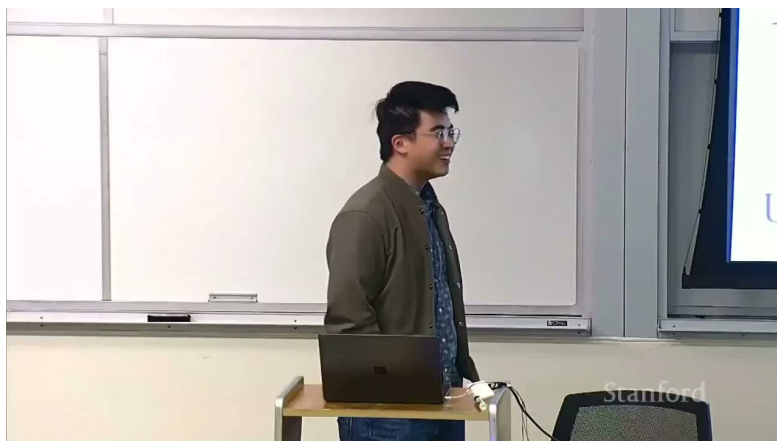


图 1: 第 18 讲的课程图像锚点。

1.1 本章知识路线

- 本章首先说明 Guest Lecture: Dan Fu 与推理引擎研究为什么是 language modeling pipeline 中不可跳过的一环。
- 其次定义本章反复使用的术语，并说明这些术语对应的计算对象或系统对象。
- 随后用公式和伪代码表达主要机制，让概念能够落到可计算的变量上。
- 最后给出例题、风险讨论和练习，便于读者检查自己是否真正掌握了机制。

1.2 结构图

下面的图用于帮助读者定位本章的核心结构。先看概念之间的依赖，再回到正文中的公式、伪代码和例题。

2 本章主线

Guest Lecture: Dan Fu 与推理引擎研究的主线是把 language model 的抽象计算放回真实机器：哪些张量要移动，哪些状态要保存，哪些瓶颈会在训练或服务时显现。

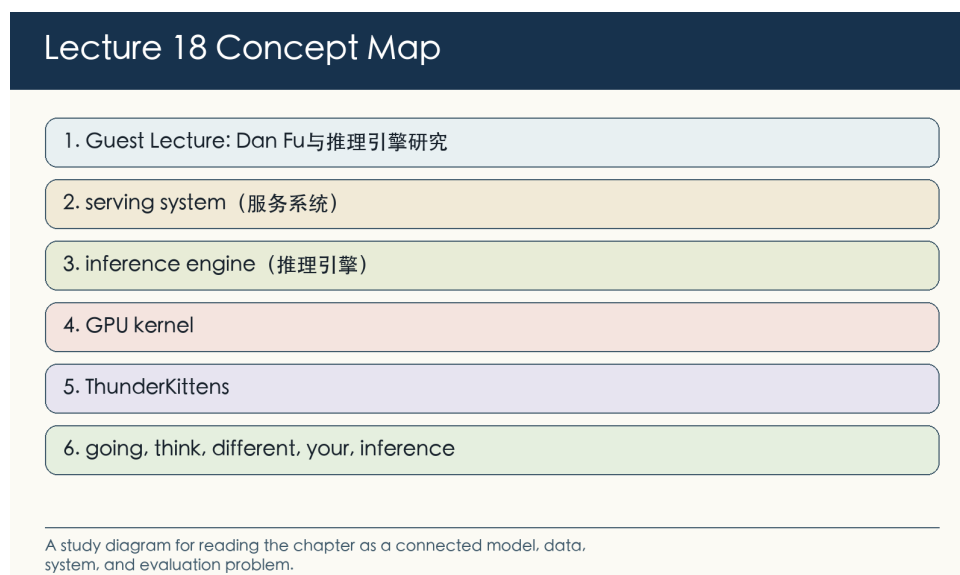


图 2: 第 18 讲概念图: 本章问题、术语和机制的关系。

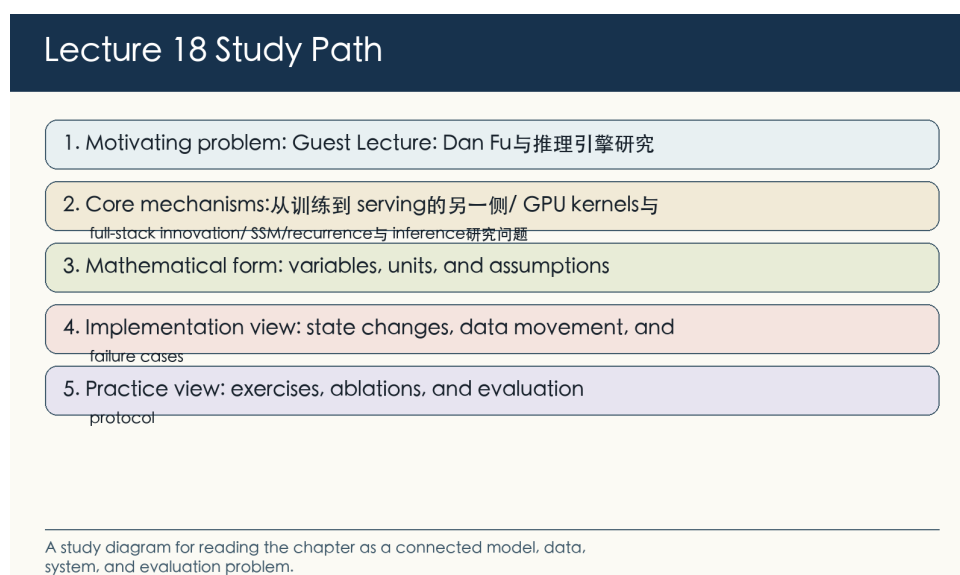


图 3: 第 18 讲学习路径: 从问题定义进入公式、实现、实验和练习。

2.1 本章要解决的核心问题

Dan Fu guest lecture 从 serving 侧看语言模型: 训练好的模型要变成真实服务, 需要 inference engine、GPU kernels、scheduler、KV cache 管理和生产流量处理。读完这一段后, 应能用一两句话说明 Guest Lecture: Dan Fu 与推理引擎研究要解决的瓶颈, 并指出这个瓶颈发生在数据、模型、优化、系统还是评估层。

2.2 从机制到资源账本

推理系统是 full-stack innovation 的位置。算法结构、kernel library、memory bandwidth、batching 和 request distribution 会共同决定用户体验。随后把机制写成账本: 输入规模是什么, 主要状态是什么, 成本或质量指标怎样随变量改变。这个账本让后面的公式不再只是符号。

2.3 从课堂讲解到可复现实验

ThunderKittens、megakernel、SSM/recurrence 等讨论说明 serving 不是模型训练后的附属环节，而是会反过来影响模型架构研究的问题。最后把课堂讲解转成可复现实验：固定协议，改变一个变量，记录中间量，并解释结果为何支持或反驳原来的机制判断。

3 术语、符号与问题边界

本章的术语横跨模型、数据和系统。读者需要先知道每个词指向的对象：有些词描述数学目标，有些词描述张量形状，有些词描述硬件瓶颈，还有一些词描述评估协议。术语边界不清，后面的公式和实现就会变成空洞的缩写。

#	术语	阅读要求
1	serving system（服务系统）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
2	inference engine（推理引擎）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
3	GPU kernel	把它放入资源账本：单位是什么，随输入规模怎样变化，最容易被哪类 benchmark 误读。
4	ThunderKittens	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
5	decode loop（解码循环）	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。
6	state-space model (SSM)	给出定义、一个最小例子、一个关联公式或代码位置，以及一个典型失败情形。

这些术语之间有依赖关系。例如 tokenization 改变 sequence length，sequence length 又改变 attention FLOPs、KV cache 和 evaluation token budget；parallelism 改变显存占用，也改变通信瓶颈和故障恢复方式。

3.1 术语之间的关系

在本章中，serving system（服务系统）给出问题入口，inference engine（推理引擎）描述主要机制，GPU kernel 则把机制连接到可测量的结果。读教材时应当沿着这个链条追问：输入是什么，中间状态怎样改变，输出指标为什么随之变化。课程材料还会反复出现 executable、linked、public、playlist、item、spring 等词。它们不是一组同义标签，而是提示本章的不同层次：有的指对象，有的指操作，有的指约束或指标。因此，术语表不是背诵清单，而是后文阅读的索引。遇到一个术语时，先定位它属于 representation、optimization、systems、data 还是 evaluation，再判断它改变哪一项账本。

4 从训练到 serving 的另一侧

现在进入从训练到 serving 的另一侧。这一节把算法放到硬件和运行时中考察：同一个数学表达式在不同 batch、shape、GPU 拓扑和 kernel 实现下会得到不同的成本账本。

4.1 为什么要讨论从训练到 serving 的另一侧

Dan Fu guest lecture 的视频开场说明：课程大多讲如何训练模型，本讲从拥有模型后的另一侧看 inference 和 serving。推理引擎把抽象的模型计算图映射到具体 GPU kernels、调度器、cache 管理和生产流量。字幕中反复出现 inference、GPU、kernel、decode、cache、request 等词，显示核心主题是 serving stack。从训练到 serving 的另一侧把 Guest Lecture: Dan Fu 与推理引擎研究放进资源账本。接下来要看的不是单个 API，而是张量形状、内存层级、通信和调度怎样共同决定成本。

4.2 从训练到 serving 的另一侧的机制

Dan Fu guest lecture 的视频开场说明：课程大多讲如何训练模型，本讲从拥有模型后的另一侧看 inference 和 serving。在从训练到 serving 的另一侧中，核心对象是硬件和系统状态：同一算法会因为 memory hierarchy、通信拓扑或 kernel shape 不同而表现出完全不同的速度。推理引擎把抽象的模型计算图映射到具体 GPU kernels、调度器、cache 管理和生产流量。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。字幕中反复出现 inference、GPU、kernel、decode、cache、request 等词，显示核心主题是 serving stack。可检验性来自 profile trace、roofline 估算和端到端延迟分解；如果这些证据互相矛盾，就要先解释 measurement setup。因此，从训练到 serving 的另一侧的学习目标是建立一张成本地图：读者应能指出瓶颈在计算、内存、通信还是调度，并能设计一个小 benchmark 去验证。

4.3 从训练到 serving 的另一侧的数学表达

$$\text{user latency} = t_{\text{queue}} + t_{\text{prefill}} + t_{\text{decode}} + t_{\text{postprocess}}$$

符号说明：用户感知延迟由排队、预填充、逐 token 解码和后处理组成；每一项都可能成为瓶颈。这个关系式应被读成从训练到 serving 的另一侧的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。在系统实验中，tokens/s、TTFT、显存峰值和 kernel time 都是 proxy。它们各自只观察一部分系统，因此报告时要同时写清 workload shape、warmup、同步方式和硬件型号。

4.4 从训练到 serving 的另一侧在系统中的实现

把从训练到 serving 的另一侧写成程序时，最重要的是暴露状态变化和数据移动。下面的伪代码保留执行顺序，省略框架样板。

```
request = receive_prompt()
kv = prefill(request.prompt)
for step in decode_loop:
    token, kv = engine.decode(kv)
    stream_to_user(token)
```

阅读这段实现时，重点看从训练到 serving 的另一侧怎样移动张量、占用显存并触发通信。实验记录至少要包含吞吐、延迟、显存峰值、通信量和 kernel 利用率，否则很难判断瓶颈来自算子、内存层级还是调度。

4.5 边界条件与常见误解

- 小规模能工作的 serving code 到大规模流量下会暴露罕见 bug。
- 生产 workload 的输入/输出长度分布和教学 benchmark 差异很大。

- 小规模 kernel benchmark 不一定预测端到端训练或 serving latency，因为调度、通信和请求形状会改变瓶颈。
- 硬件规格表和框架 profiler 都需要结合 workload 解释，不能把单个峰值数字当作机制证明。

4.6 与本章其他概念的关系

从训练到 serving 的另一侧连接本章的数学机制和工程成本：术语表说明对象，公式估算资源，伪代码暴露状态变化，例题则让读者检查一次完整的成本推理。对这一类主题，最常见的误解是只列方法名。更有用的做法是比较每个方法改变的成本项、依赖的假设和可能的失败模式。

4.7 怎样检验从训练到 serving 的另一侧

检验从训练到 serving 的另一侧时，先把抽象说法落到硬件和系统状态上。与 inference、serving、kernel、decode、request 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明：用户感知延迟由排队、预填充、逐 token 解码和后处理组成；每一项都可能成为瓶颈。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。系统消融通常一次只改变 batch shape、sequence length、GPU 数、kernel 实现或调度策略。若同时改变硬件和 workload，实验只能说明端到端表现不同，不能定位瓶颈。最后检查失败条件。工程判断往往在反例中变清楚：小规模能工作的 serving code 到大规模流量下会暴露罕见 bug；生产 workload 的输入/输出长度分布和教学 benchmark 差异很大。复习时可以把这些限制改写成反例测试，观察它们怎样改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。把实验结论放回 Guest Lecture: Dan Fu 与推理引擎研究时，要说明从训练到 serving 的另一侧改变的是训练吞吐、推理延迟、显存峰值还是通信开销。能做出这种归因，才说明读者已经从接口使用进入系统解释。

4.8 概念辨析：从训练到 serving 的另一侧

inference、serving、kernel、decode、request 常一起出现在系统讲解中，但它们分别指硬件对象、程序操作、瓶颈来源或测量指标。读者应先定位每个词在执行路径中的位置。其中，inference（推理）是已训练模型服务请求并生成 token 的过程；serving system（服务系统）把模型计算图变成可承载真实流量的推理引擎。区分这些词时，可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕从训练到 serving 的另一侧复习时，可以写一个小 benchmark：固定输入形状，改变一个系统变量，记录 profile trace 和端到端时间。

4.9 从训练到 serving 的另一侧的小结

- 讨论从训练到 serving 的另一侧时，应同时报告问题规模、实验设置和主要观测变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。
- 如果方法声称更快，要区分计算减少、内存访问减少、通信隐藏和调度改善；这些机制不能用一个 tokens/s 数字互相替代。
- profile trace、roofline 估算和端到端 benchmark 应互相校验，单独一个指标不足以解释系统瓶颈。

5 GPU kernels 与 full-stack innovation

现在进入 GPU kernels 与 full-stack innovation。这一节把算法放到硬件和运行时中考察：同一个数学表达式在不同 batch、shape、GPU 拓扑和 kernel 实现下会得到不同的成本账本。

5.1 为什么要讨论 GPU kernels 与 full-stack innovation

讲者强调理解 inference engines 和 GPU kernels 可以开启 full-stack innovation。ThunderKittens 被作为更低层 kernel-writing library 的例子，用于获得更细粒度控制和高带宽利用。Megakernel、fusion、cache-aware scheduling 等方向说明算法和系统可以共同设计。系统部分的核心问题是把数学计算映射到硬件。一个算法的复杂度公式只能给出方向；真正的速度还取决于内存层级、通信拓扑、kernel shape 和调度方式。

5.2 GPU kernels 与 full-stack innovation 的机制

讲者强调理解 inference engines 和 GPU kernels 可以开启 full-stack innovation。在 GPU kernels 与 full-stack innovation 中，核心对象是硬件和系统状态：同一算法会因为 memory hierarchy、通信拓扑或 kernel shape 不同而表现出完全不同的速度。ThunderKittens 被作为更低层 kernel-writing library 的例子，用于获得更细粒度控制和高带宽利用。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。Megakernel、fusion、cache-aware scheduling 等方向说明算法和系统可以共同设计。可检验性来自 profile trace、roofline 估算和端到端延迟分解；如果这些证据互相矛盾，就要先解释 measurement setup。因此，GPU kernels 与 full-stack innovation 的学习目标是建立一张成本地图：读者应能指出瓶颈在计算、内存、通信还是调度，并能设计一个小 benchmark 去验证。

5.3 GPU kernels 与 full-stack innovation 的资源账本

$$\text{bandwidth utilization} = \frac{\text{bytes served per second}}{\text{peak memory bandwidth}}$$

符号说明：decode 阶段常 memory-bandwidth bound，因此带宽利用率直接影响 tokens/s。这个关系式应被读成 GPU kernels 与 full-stack innovation 的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。在系统实验中，tokens/s、TTFT、显存峰值和 kernel time 都是 proxy。它们各自只观察一部分系统，因此报告时要同时写清 workload shape、warmup、同步方式和硬件型号。

5.4 GPU kernels 与 full-stack innovation 在系统中的实现

把 GPU kernels 与 full-stack innovation 写成程序时，最重要的是暴露状态变化和数据移动。下面的伪代码保留执行顺序，省略框架样板。

```
load_model_tiles()
load_kv_cache_tiles()
run_fused_decode_kernel()
write_next_token_logits()
```

阅读这段实现时，重点看 GPU kernels 与 full-stack innovation 怎样移动张量、占用显存并触发通信。实验记录至少要包含吞吐、延迟、显存峰值、通信量和 kernel 利用率，否则很难判断瓶颈来自算子、内存层级还是调度。

5.5 边界条件与常见误解

- 更低层抽象换来性能，也带来更高 correctness/debug 成本。
- kernel 微小错误在大规模 serving 中可能以极低概率触发，但影响严重。
- 小规模 kernel benchmark 不一定预测端到端训练或 serving latency，因为调度、通信和请求形状会改变瓶颈。
- 硬件规格表和框架 profiler 都需要结合 workload 解释，不能把单个峰值数字当作机制证明。

5.6 与本章其他概念的关系

GPU kernels 与 full-stack innovation 连接本章的数学机制和工程成本：术语表说明对象，公式估算资源，伪代码暴露状态变化，例题则让读者检查一次完整的成本推理。对这一类主题，最常见的误解是只列方法名。更有用的做法是比较每个方法改变的成本项、依赖的假设和可能的失败模式。

5.7 怎样检验 GPU kernels 与 full-stack innovation

检验 GPU kernels 与 full-stack innovation 时，先把抽象说法落到硬件和系统状态上。与 ThunderKittens、megakernel、bandwidth、fusion、GPU 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明：decode 阶段常 memory-bandwidth bound，因此带宽利用率直接影响 tokens/s。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。系统消融通常一次只改变 batch shape、sequence length、GPU 数、kernel 实现或调度策略。若同时改变硬件和 workload，实验只能说明端到端表现不同，不能定位瓶颈。最后检查失败条件。工程判断往往在反例中变清楚：更低层抽象换来性能，也带来更高 correctness/debug 成本；kernel 微小错误在大规模 serving 中可能以极低概率触发，但影响严重。复习时可以把这些限制改写成反例测试，观察它们怎样改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。把实验结论放回 Guest Lecture: Dan Fu 与推理引擎研究时，要说明 GPU kernels 与 full-stack innovation 改变的是训练吞吐、推理延迟、显存峰值还是通信开销。能做出这种归因，才说明读者已经从接口使用进入系统解释。

5.8 概念辨析：GPU kernels 与 full-stack innovation

ThunderKittens、megakernel、bandwidth、fusion、GPU 常一起出现在系统讲解中，但它们分别指硬件对象、程序操作、瓶颈来源或测量指标。读者应先定位每个词在执行路径中的位置。其中，kernel（内核）是高层 tensor 操作落到 GPU 的执行单元；GPU 是 CS336 systems 部分的主要硬件对象。区分这些词时，可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕 GPU kernels 与 full-stack innovation 复习时，可以写一个小 benchmark：固定输入形状，改变一个系统变量，记录 profile trace 和端到端时间。

5.9 GPU kernels 与 full-stack innovation 的小结

- 讨论 GPU kernels 与 full-stack innovation 时，应同时报告问题规模、实验设置和主要观测变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。
- 如果方法声称更快，要区分计算减少、内存访问减少、通信隐藏和调度改善；这些机制不能用一个 tokens/s 数字互相替代。

- profile trace、roofline 估算和端到端 benchmark 应互相校验，单独一个指标不足以解释系统瓶颈。

6 SSM/recurrence 与 inference 研究问题

现在进入 SSM/recurrence 与 inference 研究问题。这一节把算法放到硬件和运行时中考察：同一个数学表达式在不同 batch、shape、GPU 拓扑和 kernel 实现下会得到不同的成本账本。

6.1 为什么要讨论 SSM/recurrence 与 inference 研究问题

字幕中出现 state space、SSM、recurrence 和 scaling laws，说明 guest lecture 还讨论了模型结构如何影响 serving。recurrent/SSM 风格模型有潜力改变长上下文和 decode 的状态管理方式。但新的结构要同时满足训练稳定性、质量、硬件效率和部署复杂度。推理阶段的关键区别在于请求是在线到达的，输出又是逐 token 生成的。训练时可以用大 batch 摊薄成本；服务时还要同时考虑首 token 延迟、吞吐、显存占用和长尾请求。

6.2 SSM/recurrence 与 inference 研究问题的机制

字幕中出现 state space、SSM、recurrence 和 scaling laws，说明 guest lecture 还讨论了模型结构如何影响 serving。在 SSM/recurrence 与 inference 研究问题中，核心对象是硬件和系统状态：同一算法会因为 memory hierarchy、通信拓扑或 kernel shape 不同而表现出完全不同的速度。recurrent/SSM 风格模型有潜力改变长上下文和 decode 的状态管理方式。因而同一个方法在不同规模下可能改变不同的变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。比较方法时，必须同时给出问题规模和实验条件。但新的结构要同时满足训练稳定性、质量、硬件效率和部署复杂度。可检验性来自 profile trace、roofline 估算和端到端延迟分解；如果这些证据互相矛盾，就要先解释 measurement setup。因此，SSM/recurrence 与 inference 研究问题的学习目标是建立一张成本地图：读者应能指出瓶颈在计算、内存、通信还是调度，并能设计一个小 benchmark 去验证。

6.3 SSM/recurrence 与 inference 研究问题的资源账本

$$h_t = Ah_{t-1} + Bx_t, \quad y_t = Ch_t$$

符号说明：这是 SSM/递推模型的简化形式； h_t 是状态， x_t 是输入， y_t 是输出。这个关系式应被读成 SSM/recurrence 与 inference 研究问题的资源账本。它不承诺精确预测每次运行时间，而是帮助判断主要成本来自计算、内存访问、通信还是调度。在系统实验中，tokens/s、TTFT、显存峰值和 kernel time 都是 proxy。它们各自只观察一部分系统，因此报告时要同时写清 workload shape、warmup、同步方式和硬件型号。

6.4 SSM/recurrence 与 inference 研究问题在系统中的实现

把 SSM/recurrence 与 inference 研究问题写成程序时，最重要的是暴露状态变化和数据移动。下面的伪代码保留执行顺序，省略框架样板。

```
state = init_state()
for token in sequence:
    state = recurrent_update(state, token)
    logits = readout(state)
```


阅读这段实现时，重点看 SSM/recurrence 与 inference 研究问题怎样移动张量、占用显存并触发通信。实验记录至少要包含吞吐、延迟、显存峰值、通信量和 kernel 利用率，否则很难判断瓶颈来自算子、内存层级还是调度。

6.5 边界条件与常见误解

- 递推结构的训练并行性和长程记忆质量需要实证验证。
- serving 系统收益只有在端到端 workload 中测量才可信。
- 小规模 kernel benchmark 不一定预测端到端训练或 serving latency，因为调度、通信和请求形状会改变瓶颈。
- 硬件规格表和框架 profiler 都需要结合 workload 解释，不能把单个峰值数字当作机制证明。

6.6 与本章其他概念的关系

SSM/recurrence 与 inference 研究问题连接本章的数学机制和工程成本：术语表说明对象，公式估算资源，伪代码暴露状态变化，例题则让读者检查一次完整的成本推理。对这一类主题，最常见的误解是只列方法名。更有用的做法是比较每个方法改变的成本项、依赖的假设和可能的失败模式。

6.7 怎样检验 SSM/recurrence 与 inference 研究问题

检验 SSM/recurrence 与 inference 研究问题时，先把抽象说法落到硬件和系统状态上。与 SSM、recurrence、state、long context、decode 相关的对象必须能被构造、打印、计数或评分；否则实验只能得到描述性观察，不能得到可复现结论。一个合适的小实验应当保留本节机制，同时让吞吐、延迟、显存峰值、通信量和 kernel 利用率中至少一个量发生可解释变化。输入规模不必逼真；关键是读者能手算或打印关键中间量，并说明哪个变量触发了结果变化。随后把公式说明转成可记录的数量关系。符号说明：这是 SSM/递推模型的简化形式； (h_t) 是状态， (x_t) 是输入， (y_t) 是输出。如果数量只能间接观测，就必须说明使用了什么 proxy；如果使用 benchmark 或 reward，也要说明协议和随机性。系统消融通常一次只改变 batch shape、sequence length、GPU 数、kernel 实现或调度策略。若同时改变硬件和 workload，实验只能说明端到端表现不同，不能定位瓶颈。最后检查失败条件。工程判断往往在反例中变清楚：递推结构的训练并行性和长程记忆质量需要实证验证；serving 系统收益只有在端到端 workload 中测量才可信。复习时可以把这些限制改写成反例测试，观察它们怎样改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。把实验结论放回 Guest Lecture: Dan Fu 与推理引擎研究时，要说明 SSM/recurrence 与 inference 研究问题改变的是训练吞吐、推理延迟、显存峰值还是通信开销。能做出这种归因，才说明读者已经从接口使用进入系统解释。

6.8 概念辨析：SSM/recurrence 与 inference 研究问题

SSM、recurrence、state、long context、decode 常一起出现在系统讲解中，但它们分别指硬件对象、程序操作、瓶颈来源或测量指标。读者应先定位每个词在执行路径中的位置。其中，decode（解码）逐 token 生成，常受 memory bandwidth 和 KV cache 影响。区分这些词时，可以先问它们是否改变吞吐、延迟、显存峰值、通信量和 kernel 利用率。一个术语如果不能对应到变量、状态、代码路径或评估协议，就还没有形成可操作含义。围绕 SSM/recurrence 与 inference 研究问题复习时，可以写一个小 benchmark：固定输入形状，改变一个系统变量，记录 profile trace 和端到端时间。

6.9 SSM/recurrence 与 inference 研究问题的小结

- 讨论 SSM/recurrence 与 inference 研究问题时，应同时报告问题规模、实验设置和主要观测变量，尤其是吞吐、延迟、显存峰值、通信量和 kernel 利用率。
- 如果方法声称更快，要区分计算减少、内存访问减少、通信隐藏和调度改善；这些机制不能用一个 tokens/s 数字互相替代。
- profile trace、roofline 估算和端到端 benchmark 应互相校验，单独一个指标不足以解释系统瓶颈。

7 综合讨论：从局部机制到完整系统

本章的主题是 Guest Lecture: Dan Fu 与推理引擎研究，但它不应被读成几个相邻知识点的拼接。从训练到 serving 的另一侧、GPU kernels 与 full-stack innovation、SSM/recurrence 与 inference 研究问题共同回答同一个问题：语言模型系统中哪些对象可以被定义，哪些成本可以被估算，哪些结论必须通过实验或评估来确认。这些对象包括 serving system（服务系统）、inference engine（推理引擎）、GPU kernel、ThunderKittens、decode loop（解码循环）、state-space model (SSM)。读者可以把它们看成一组接口：有的接口连接文本和 token，有的连接模型结构和张量计算，有的连接数据样本和训练目标，有的连接离线评估和在线服务。接口一旦改变，后面的成本、错误类型和优化空间也会随之改变。

7.1 从训练到 serving 的另一侧在系统中的位置

从训练到 serving 的另一侧首先提供一个局部解释：Dan Fu guest lecture 的视频开场说明：课程大多讲如何训练模型，本讲从拥有模型后的另一侧看 inference 和 serving。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，从训练到 serving 的另一侧会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，从训练到 serving 的另一侧不能脱离边界条件理解。一个关键 caveat 是：小规模能工作的 serving code 到大规模流量下会暴露罕见 bug。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.2 GPU kernels 与 full-stack innovation 在系统中的位置

GPU kernels 与 full-stack innovation 首先提供一个局部解释：讲者强调理解 inference engines 和 GPU kernels 可以开启 full-stack innovation。这类解释的价值在于把直觉压缩成可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，GPU kernels 与 full-stack innovation 会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，GPU kernels 与 full-stack innovation 不能脱离边界条件理解。一个关键 caveat 是：更低层抽象换来性能，也带来更高 correctness/debug 成本。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.3 SSM/recurrence 与 inference 研究问题在系统中的位置

SSM/recurrence 与 inference 研究问题首先提供一个局部解释：字幕中出现 state space、SSM、recurrence 和 scaling laws，说明 guest lecture 还讨论了模型结构如何影响 serving。这类解释的价值在于把直觉压缩成

可操作对象。一旦对象明确，读者就可以追问它的输入、输出、中间状态和失败条件，而不是只记住方法名。其次，SSM/recurrence 与 inference 研究问题会改变至少一个资源或评估变量。它可能改变 sequence length、activation size、memory bandwidth、communication volume、data mixture、reward distribution 或 benchmark protocol。这些变量在小例子里可以手算，在真实训练和服务系统里则需要 profiling、logging 和 held-out evaluation。最后，SSM/recurrence 与 inference 研究问题不能脱离边界条件理解。一个关键 caveat 是：递推结构的训练并行性和长程记忆质量需要实证验证。。把 caveat 写出来不是为了削弱结论，而是为了说明结论在哪些输入、硬件、数据和评估条件下才成立。

7.4 把本章知识用于读论文和复现实验

读相关论文时，可以把本章变成一个检查表。第一，论文是否清楚定义了输入规模、模型规模、数据来源和评估协议；第二，论文声称的改进落在哪个变量上；第三，作者是否报告了会让结论失效的设置；第四，实验是否能分离模型结构、数据质量、优化设置和系统实现的影响。复现实验时，也应避免直接追求大规模。先用最小程序复现一个公式、一个伪代码分支或一个 profile 现象，再扩大输入规模。这样做的原因很简单：如果小规模程序无法解释中间量，大规模训练只会把错误隐藏在日志和随机波动里。把这些检查合在一起，本章给出的不是一个固定答案，而是一种阅读 CS336 的方法：每个模型机制都要落到变量，每个变量都要落到测量，每个测量都要说明协议，每个协议都要承认边界。因此，本章中的任何一句结论都可以被改写成一个可引用的完整句式：在给定数据、模型、硬件和评估协议下，某个机制改变了某个变量，并通过某个观测指标表现出来。这样的句子比单独背诵术语更长，但它包含了自学、复现和引用时真正需要的信息。把本章用于自学时，可以按三遍阅读。第一遍只追踪对象和定义，确认每个术语到底指向文本、token、张量、样本、请求还是评估记录。第二遍追踪公式和伪代码，确认每一步改变了哪些状态。第三遍追踪实验和 caveat，确认哪些结论只在特定规模、数据或硬件条件下成立。把本章用于复习时，则应反过来操作：先从一个失败案例或异常指标出发，再回到相应机制。loss 曲线异常可能指向数据、优化或模型容量；吞吐异常可能指向 kernel、通信或调度；benchmark 异常可能指向 contamination、prompt protocol 或采样设置。能够从异常反推机制，是掌握本章的实用标准。

8 例题：把概念变成可计算检查

8.1 Serving latency 的分解

一个用户请求从进入队列到 token 流式返回。

- 排队时间取决于调度和 batching。
- prefill 处理 prompt，decode 循环逐 token 运行。
- KV cache 管理、kernel 效率和输出长度共同影响 tail latency。

结论。 serving 研究把模型结构、GPU kernels 和系统调度连成一条端到端路径。这个例题的目的不是替代完整工程实现，而是给出一种最小推理形式：把问题转成变量，写出近似公式或伪代码，再说明近似何时失效。

9 实验设计：从小例子到可复现结论

学习 Guest Lecture: Dan Fu 与推理引擎研究不能停在概念层。一个教材化结论至少需要四件事：可构造的输入、可观测的中间量、可比较的指标，以及清楚的失败条件。下面的实验设计不是要求训练大模型，而是要求读者用小规模程序复现本章机制。

9.1 最小输入

先构造只触发本章核心机制的小输入。例如 tokenizer 章节可以用几个包含 rare words 和 Unicode 字符的字符串；GPU 章节可以用一个 elementwise kernel 和一个 GEMM；data 章节可以用十几篇近重复文档。小输入的价值在于它让错误可见。

9.2 资源账本

对同一输入写下 FLOPs、bytes moved、显存峰值、通信量或样本数量的估算。估算不需要一开始就精确，但必须说明单位和近似假设。随后用 profiler、benchmark、validation loss 或人工检查去验证估算是否同量级。

9.3 单变量消融

一次只改变一个因素，例如 vocabulary size、head 数、batch size、filter threshold、reward scale、decoding temperature 或 GPU 数量。若多个变量同时变化，实验只能说明系统变了，不能说明机制是什么。

9.4 失败条件

最后要刻意触发失败：极端 context length、错误 dtype、过小 batch、污染 benchmark、低质量数据或不可靠 verifier。失败条件常常比成功曲线更能说明一个方法的边界。

9.5 记录与报告

实验记录应能让另一个读者复现同一结论。最少要写清楚输入构造、代码版本、随机种子、硬件、batch shape、数据版本、评价指标和异常处理方式。报告结论时不要只写“更快”或“更好”，而要说明快在哪里、好在哪个指标上，以及是否牺牲了内存、稳定性、数据质量或可解释性。常见报告错误是把局部现象写成普遍规律。例如一次小模型实验里的 loss 改善，不等于更大模型或不同数据分布上也会改善；一次 kernel benchmark 的吞吐提升，不等于端到端 serving latency 一定降低。教材中的实验报告应始终把结论限定在可复现条件内。

9.6 从小实验到大结论

小实验的作用不是替代课程中的完整训练或系统实现，而是建立一条可检查的推理链。先在小输入上确认变量方向，再在中等规模上确认数量级，最后才讨论是否能外推到更大模型、更多 token 或更复杂 workload。缺少中间层级时，大结论通常只是在复述直觉。因此，实验报告最好同时包含正例和反例：正例说明机制在受控条件下怎样工作，反例说明条件稍变时哪里失效。对 CS336 这样的课程，反例尤其重要，因为 language model 的错误常来自多个层面叠加：数据分布、优化稳定性、硬件瓶颈、评估协议和服务负载都可能同时改变观察结果。还要记录资料版本与复现边界。公开视频、课程页、配套代码和 PDF 可能在不同时间更新；复现实验应写明使用的材料版本，并说明哪些结论来自课程事实，哪些是为了帮助理解而加入的解释性推导。这样才能把学习笔记变成可检查的技术记录。若复现实验与课程结论不一致，首先检查输入、版本和评估协议，而不是立即修改模型假设。

9.7 把本章重新读成一条证据链

完成小实验之后，可以把 Guest Lecture: Dan Fu 与推理引擎研究重新读成一条证据链：先用“从训练到 serving 的另一侧”确定问题入口，再用“GPU kernels 与 full-stack innovation”解释主要机制，最后用“SSM/recurrence 与 inference 研究问题”检查边界或外推条件。这种读法比按页背诵更接近教材训练，因为

它要求每个结论都能说明前提、变量和证据。本章反复使用的术语包括 serving system (服务系统)、inference engine (推理引擎)、GPU kernel、ThunderKittens、decode loop (解码循环)、state-space model (SSM)。复习时不要只写定义，而要为每个术语补上一句操作性说明：它在课程代码、公式、数据记录或评估协议中对应什么对象；如果这个对象被删除、替换或缩放，哪一个观测量会变化。最后，用一个反例结束复习。反例可以是资源估算不准、数据污染、reward 失真、benchmark 解析失败、长上下文显存爆炸或 scaling 外推失效。能解释反例的章节，才真正具备自学和引用价值；不能解释反例的章节，只是把课程材料换了一种排版。引用本章结论时，还应保留来源边界：哪些说法直接来自视频、课程页、slides 或代码，哪些是为了串联教材叙述而加入的解释性推导。这个边界不会削弱结论，反而让读者知道何处可以复核，何处需要在自己的实验中重新验证。如果后续课程材料更新，也可以沿着同一证据链替换来源，而不必重写整章结构。

10 课程资料与回看路径

教材正文已经把主要概念、公式和实现逻辑组织成连续叙述；本节只提供回到课程材料的阅读路径。读者复习时可以先完成前文练习，再按下面的时间段或资料组核对细节。

10.1 视频回看路径

时间	关键词	复习任务
00:00:05-00:06:23	think, inference, horses, pretty, understand, code, around	回看定义、例子、推导或 caveat 在该段中的位置。
00:17:45-00:23:43	different, very, then, compute, prefill, decode, start	回看定义、例子、推导或 caveat 在该段中的位置。
00:35:36-00:41:54	your, attention, kernel, start, very, then, operation	回看定义、例子、推导或 caveat 在该段中的位置。
00:47:41-00:53:39	going, matrix, there's, look, then, it's, said	回看定义、例子、推导或 caveat 在该段中的位置。
01:05:14-01:11:14	think, then, your, size, cache, going, choices	回看定义、例子、推导或 caveat 在该段中的位置。

这些时间段用于定位视频中的讲解顺序。严肃引用时，应回到视频片段确认讲者表述、板书或幻灯片内容。回看视频时，不必逐字抄录字幕。更有效的方法是把每个时间段判定为定义、例子、推导、代码解释或 caveat，再把它放回本章对应小节。

10.2 课程资料阅读路径

资料组	标题/页块	关键词
official_01	No official slide PDF or executable lecture script is linked for this public playlist ...	official, slide, executable, script, linked, public, playlist

课程资料通常给出更稳定的标题、公式、图或代码结构；视频则补充动机和口头解释。两者合起来构成本章的事实边界。阅读资料时，可以把每个资料组拆成三个对象：一个定义，一个公式或伪代码动作，一个边界条件。这样比逐字翻译页面或脚本更容易形成可用知识。

11 实践说明、历史位置与风险

Guest Lecture: Dan Fu 与推理引擎研究在 CS336 的课程结构中连接基础机制和规模化问题。基础机制解释方法为什么成立；规模化问题检验它在更大模型、更长上下文、更复杂数据或真实 serving workload 下是否仍成立。历史位置不等于模型年表。更重要的是识别问题如何迁移：早期方法解决小规模建模问题，现代方法常常解决同一问题在大规模数据、GPU 集群、长上下文或人类偏好约束下的新形态。

- 资料版本限制 (version risk): 课程页、公开视频和配套资料可能在学期中更新；引用时应注明访问日期或资料版本。
- 测量口径限制 (measurement risk): FLOPs、tokens/s、benchmark score、reward 等数字只有在协议完整时可比较。
- 规模外推限制 (scaling risk): 小模型或小 batch 的机制不必然外推到 frontier-scale。
- 数据分布限制 (data risk): 数据来源、过滤和去重策略会改变模型行为，也会改变 evaluation contamination 风险。
- 系统风险 (systems risk): 性能剖析结果依赖硬件、driver、kernel、shape 和 batch distribution; 脱离 workload 的性能数字不可直接复用。

12 复习要点

下面的问题把本章内容压缩为可反复检查的条目。每个条目都应能回到前文的解释、公式、伪代码或例题。

条目	对象	检查动作
条目 1	serving system (服务系统)	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。
条目 2	inference engine (推理引擎)	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。
条目 3	GPU kernel	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。
条目 4	ThunderKittens	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。
条目 5	decode loop (解码循环)	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。
条目 6	state-space model (SSM)	定义它；写出一个最小例子；说明一个 failure mode；指出它影响哪个资源或评估账本。

12.1 综合自测

- 我能否不用课程原句，独立解释本章中心问题？

- 我能否把本章至少一个公式改写成可运行的估算函数？
- 我能否指出一个看似改进但实际转移成本的方法？
- 我能否回到视频和课程资料，找到支持本章关键论断的位置？
- 我能否设计一个最小 ablation，验证本章一个 caveat？

13 总结与延伸

13.1 本章小结

- Guest Lecture: Dan Fu 与推理引擎研究的核心不是记忆术语，而是理解对象、变量和机制之间的关系。
- 视频给出讲解顺序，课程资料提供可核对的公式、图、代码或页面结构。
- 复习时应把每个术语连接到至少一个变量、一个实现动作和一个失败模式。

13.2 延伸解释

本章中的延伸解释用于连接课程材料中的概念，例如说明公式里的 proxy、解释系统 tradeoff，或把多个材料片段串成一个完整推理链。若需要严肃引用，应回到课程视频和配套资料核对原始表述。

13.3 练习

- 定义题：用中英双语解释本章任意五个重要术语，并说明它们属于 compute、memory、data、optimization、evaluation 还是 deployment 账本。
- 推导题：选择本章一个公式，逐个解释符号、单位和近似假设，并说明哪个变量最难在真实系统中测量。
- 实现题：把本章伪代码改写成一个最小 Python sanity check，不要求训练大模型，但要输出可检查的中间量。
- 资料题：从“课程资料与回看路径”中选择一个视频时间段，回到原始视频核对讲者例子，并记录是否存在字幕误差。
- 评估题：为本章方法设计一个 ablation 或 benchmark，说明控制变量、数据版本、指标和 failure criteria。
- 边界题：说明本章一个结论在更长 context、更大 batch、更多 GPU 或更复杂数据分布下可能如何失效。
- 综合题：把本章内容和前一章或后一章连接起来，写出一个端到端训练/推理 pipeline 中的依赖关系。
- 批判题：指出一个容易被营销材料夸大的说法，并用本章的资源账本或评估协议反驳它。

A 资料缺口说明

以下缺口没有阻塞本书主体完成，但会限制可引用范围：

- Daniel Selsam guest lecture appears in the Spring 2026 course schedule, but no corresponding public video is present in the playlist snapshot used here.
- Dan Fu guest lecture has public video/subtitles but no official slide/script link on the Spring 2026 schedule row; the chapter is grounded primarily in public video subtitles.